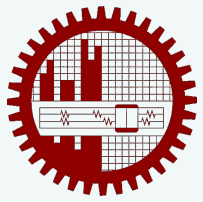


Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

Digital Logic Design Question Bank

CSE 205 · Digital Logic Design

Year Coverage: 2011-12 to 2024–25

Topics: Boolean Algebra, Logic Gates, Minimization, Combinational Logic, Flip-Flops, Sequential Circuits, Registers & Counters, Memory & PLDs

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— LaTeX Typesetting

NotebookLM — Research & Extraction

Claude AI — LaTeX Typesetting

Gemini — Diagram Generation

Table of Contents

Part IV — Registers, Counters & Advanced Topics

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Boolean Algebra, Theorems & Logic Gates

Postulates, Duality, Truth Tables, Gate Implementations

S1 Boolean Algebra, Theorems & Logic Gates

Boolean Algebra & Simplification

Q1. Using laws and theorems of Boolean algebra, prove and simplify the following expression:

$$xy + x'z + yz = xy + x'z$$

Apply relevant laws and theorems to show that the term yz is redundant and obtain the simplified form. **(7 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

Proof of the Consensus Theorem
We are asked to algebraically prove the Consensus Theorem, which states that the term yz is redundant in the expression $xy+x'z+yz$.
Step-by-step Derivation:
We start with the Left Hand Side (LHS) of the equation and apply Boolean algebra postulates and theorems to simplify it to the Right Hand Side (RHS).

$$\begin{aligned}
\text{LHS} &= xy + x'z + yz \\
&= xy + x'z + yz(1) && \text{(Identity Law: } A \cdot 1 = A\text{)} \\
&= xy + x'z + yz(x + x') && \text{(Complementarity: } x + x' = 1\text{)} \\
&= xy + x'z + xyz + x'yz && \text{(Distributive Law: } A(B + C) = AB + AC\text{)} \\
&= xy + xyz + x'z + x'yz && \text{(Commutative Law: Rearranging terms)} \\
&= xy(1 + z) + x'z(1 + y) && \text{(Distributive Law: Factoring out } xy \text{ and } x'z\text{)} \\
&= xy(1) + x'z(1) && \text{(Null/Annihilation Law: } 1 + A = 1\text{)} \\
&= xy + x'z && \text{(Identity Law: } A \cdot 1 = A\text{)} \\
&= \text{RHS}
\end{aligned}$$

Conclusion:
The algebraic manipulation clearly demonstrates that the term yz (known as the consensus term) does not add any new logical conditions to the function, as its conditions are already fully covered by the terms xy and $x'z$.

$$\boxed{xy + x'z + yz = xy + x'z}$$

Q2. Analyze whether the dual and the complement of the following function are the same:

$$F = ABC + A(B + C) + A'B'C'$$

Justify your answer. **(8 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

Analysis of the Function
Given the Boolean function:

$$F = ABC + A(B + C) + A'B'C'$$

To determine if the dual and the complement of the function are the same, we will first simplify the original function F, then derive its complement (F') and its dual (F^D), and finally compare the two results.

Step 1: Simplification of the Original Function (F)
We can simplify F using standard Boolean algebra theorems:

$$\begin{aligned}
F &= ABC + A(B + C) + A'B'C' \\
F &= ABC + AB + AC + A'B'C' && \text{(Distributive Law)} \\
F &= AB(C + 1) + AC + A'B'C' && \text{(Factoring out } AB\text{)} \\
F &= AB(1) + AC + A'B'C' && \text{(Null Law: } C + 1 = 1\text{)} \\
F &= AB + AC + A'B'C' && \text{(Identity Law)}
\end{aligned}$$

Alternatively, we can express the simplified F by factoring out A:

$$F = A(B + C) + A'B'C'$$

Step 2: Derivation of the Complement (F')
The complement of a function is obtained by applying De Morgan's Laws, which states that we invert all variables and swap AND

$(\cdot)$ with OR $(+)$, and vice versa. Using the simplified form $F = A(B+C) + A'B'C'$:

$$\begin{aligned} F' &= (A(B+C) + A'B'C')' \\ F' &= (A(B+C))' \cdot (A'B'C')' && \text{(De Morgan's Law)} \\ F' &= (A' + (B+C)') \cdot (A+B+C) && \text{(De Morgan's Law)} \\ F' &= (A' + B'C') \cdot (A+B+C) && \text{(De Morgan's Law)} \end{aligned}$$

Now, we expand the expression using the Distributive Law:

$$\begin{aligned} F' &= A'A + A'B + A'C + AB'C' + BB'C' + CB'C' \\ F' &= 0 + A'B + A'C + AB'C' + 0 + 0 && \text{(Complement Law: } XX' = 0\text{)} \\ F' &= A'B + A'C + AB'C' \end{aligned}$$

Step 3: Derivation of the Dual (F^D)
The dual of a function is obtained by swapping AND $(\cdot)$ with OR $(+)$, and OR $(+)$ with AND $(\cdot)$, as well as swapping 0s and 1s. **Crucially, the literals themselves are not complemented.** Let us find the dual of the original unsimplified function:

$$F = (A \cdot B \cdot C) + (A \cdot (B+C)) + (A' \cdot B' \cdot C')$$

Applying the duality principle:

$$F^D = (A+B+C) \cdot (A + (B \cdot C)) \cdot (A' + B' + C')$$

Now, we simplify F^D:

$$F^D = [(A+B+C)(A+BC)] \cdot (A' + B' + C')$$

Using the Absorption Law $Y(Y+Z) = Y$, let $Y = A + BC$ and $Z = B + C$. Notice that $A + BC$ is a subset of $A + B + C$ because $BC = 1 \implies B = 1$ and $C = 1 \implies B + C = 1$. Thus, $(A+BC)(A+B+C) = A + BC$.

$$\begin{aligned} F^D &= (A+BC) \cdot (A' + B' + C') \\ F^D &= A(A' + B' + C') + BC(A' + B' + C') && \text{(Distributive Law)} \\ F^D &= AA' + AB' + AC' + A'BC + BB'C + BCC' && \text{(Distributive Law)} \\ F^D &= 0 + AB' + AC' + A'BC + 0 + 0 && \text{(Complement Law: } XX' = 0\text{)} \\ F^D &= AB' + AC' + A'BC \end{aligned}$$

Step 4: Comparison and Verification
Comparing the minimized expressions for F' and F^D:

- **Complement:** $F' = A'B + A'C + AB'C'$
- **Dual:** $F^D = AB' + AC' + A'BC$

The expressions are algebraically different. To conclusively prove this, we can evaluate both functions using a truth table.

A	B	C	$F = AB + AC + A'B'C'$	$F' = A'B + A'C + AB'C'$	$F^D = AB' + AC' + A'BC$
0	0	0	1	**0**	**0**
0	0	1	0	**1**	**0**
0	1	0	0	**1**	**0**
0	1	1	0	**1**	**1**
1	0	0	0	**1**	**1**
1	0	1	1	**0**	**1**
1	1	0	1	**0**	**1**
1	1	1	1	**0**	**0**

Conclusion
As observed in both the derived algebraic expressions and the truth table, the outputs for F' and F^D do not match for multiple input combinations (e.g., for inputs 001, 010, 101, and 110). Therefore, the dual and the complement of the given function are **not the same**.

Q3. Using Boolean algebra, show that $F = xyz' + wx'y' + (x'+z+w)(x'+z+w') + xyz + wx'y$ can be simplified to $F = xy + wx' + x' + z$. **(11 marks)** **[CSE 205 | L-2/T-1 | 2023–24]**

Answer

Simplification of the Boolean Function
Given the original Boolean function:

$$F = xyz' + wx'y' + (x' + z + w)(x' + z + w') + xyz + wx'y$$

We will simplify this step-by-step using standard Boolean algebra theorems and laws to show that it reduces to $F = xy + wx' + x' + z$.

Step 1: Rearrange and Group Terms
First, we use the **Commutative Law** to rearrange the terms, bringing similar products together to make factoring easier:

$$F = (xyz' + xyz) + (wx'y' + wx'y) + (x' + z + w)(x' + z + w')$$

Step 2: Simplify the Sum of Products (SOP) Terms
Apply the **Distributive Law** to factor out the common literals from the grouped SOP terms:

$$F = xy(z' + z) + wx'(y' + y) + (x' + z + w)(x' + z + w')$$

Apply the **Complement Law** $(A + A' = 1)$ to the terms inside the parentheses:

$$F = xy(1) + wx'(1) + (x' + z + w)(x' + z + w')$$

Apply the **Identity Law** $(A \cdot 1 = A)$:

$$F = xy + wx' + (x' + z + w)(x' + z + w')$$

Step 3: Simplify the Product of Sums (POS) Term
Now we focus on the third component: $(x' + z + w)(x' + z + w')$. We can treat the sub-expression $(x' + z)$ as a single variable. Let $A = x' + z$. Using the **Distributive Law** $(A + B)(A + C) = A + BC$ in reverse, we get:

$$(x' + z + w)(x' + z + w') = (x' + z) + (w \cdot w')$$

Apply the **Complement Law** $(w \cdot w' = 0)$:

$$(x' + z + w)(x' + z + w') = (x' + z) + 0$$

Apply the **Identity Law** $(A + 0 = A)$:

$$(x' + z + w)(x' + z + w') = x' + z$$

Step 4: Combine the Simplified Parts
Substitute the simplified POS term back into the main expression for F:

$$F = xy + wx' + (x' + z)$$
$$F = xy + wx' + x' + z$$

Conclusion
Through the step-by-step application of Boolean algebra laws, we have successfully demonstrated that:

$$\mathbf{F = xy + wx' + x' + z}$$

Instructor's Note for Full Minimization:
While we have successfully derived the requested target expression, it is important to recognize that this expression is not yet fully minimized for optimal circuit design.

- Using the **Absorption Law**: $wx' + x' = x'(w + 1) = x'(1) = x'$.
- The function reduces to: $F = xy + x' + z$.
- Using the **Simplification (Redundancy) Law** $(A' + AB = A' + B)$: $x' + xy = x' + y$.
- The ultimately minimized form of this function is $F = x' + y + z$.

Q4. Simplify the following Boolean expressions to a minimum number of literals: $(A + B)'(A' + B')'$. **(10 marks)** **[CSE 205 | L-2/T-1 | 2021–22]**

Answer

Simplification of the Boolean Expression
Given the Boolean expression:

$$F = (A + B)'(A' + B')'$$

We will simplify this expression step-by-step to a minimum number of literals using fundamental Boolean algebra laws.

Step-by-Step Derivation:

$$\begin{aligned}
F &= (A + B)' \cdot (A' + B')' && \\
F &= (A' \cdot B') \cdot (A' + B')' && \text{(De Morgan's Law: } (X + Y)' = X' \cdot Y') \\
F &= (A' \cdot B') \cdot ((A')' \cdot (B')') && \text{(De Morgan's Law: } (X + Y)' = X' \cdot Y') \\
F &= (A' \cdot B') \cdot (A \cdot B) && \text{(Involution Law: } (X')' = X) \\
F &= A' \cdot B' \cdot A \cdot B && \text{(Associative Law)} \\
F &= (A \cdot A') \cdot (B \cdot B') && \text{(Commutative Law)} \\
F &= (0) \cdot (0) && \text{(Complement Law: } X \cdot X' = 0) \\
F &= 0 && \text{(Null Law: } 0 \cdot X = 0)
\end{aligned}$$

Intuitive Explanation:
We can also analyze the function logically. The expression consists of the product (AND) of two terms:

(a) $(A + B)'$: This is the NOR operation, which outputs 1 only when both $A = 0$ and $B = 0$.

(b) $(A' + B')'$: By De Morgan's law, this is equivalent to $A \cdot B$ (the AND operation), which outputs 1 only when both $A = 1$ and $B = 1$.

Because it is impossible for the variables A and B to simultaneously be 0 (to satisfy the NOR term) and 1 (to satisfy the AND term), the intersection of these two conditions is an empty set. Thus, the product of these two terms will always yield 0 regardless of the inputs.

Conclusion:
The Boolean expression logically evaluates to false for all input combinations. It simplifies entirely to the constant logic level **0**. Therefore, the minimum number of literals is zero.

Q5. Simplify the following expression to POS and SOP: $ACD' + C'D + AB' + ABCD$. **(5 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Answer

Simplification of the Boolean Expression
Given the Boolean function:

$$F = ACD' + C'D + AB' + ABCD$$

We will simplify this expression into its minimum Sum of Products (SOP) and Product of Sums (POS) forms algebraically, and verify our results using Karnaugh Maps.

Step 1: Algebraic Simplification to Sum of Products (SOP)
We simplify the given expression step-by-step using fundamental Boolean algebra laws:

$$\begin{aligned}
F &= C'D + AB' + ACD' + ABCD && \text{(Commutative Law)} \\
F &= C'D + AB' + AC(D' + BD) && \text{(Distributive Law: factoring } AC) \\
F &= C'D + AB' + AC(D' + B) && \text{(Simplification Law: } X' + XY = X' + Y) \\
F &= C'D + AB' + ACD' + ABC && \text{(Distributive Law)} \\
F &= C'D + A(B' + BC) + ACD' && \text{(Factoring } A \text{ from 2nd and 4th terms)} \\
F &= C'D + A(B' + C) + ACD' && \text{(Simplification Law: } X' + XY = X' + Y) \\
F &= C'D + AB' + AC + ACD' && \text{(Distributive Law)} \\
F &= C'D + AB' + AC(1 + D') && \text{(Factoring } AC \text{ from 3rd and 4th terms)} \\
F &= C'D + AB' + AC(1) && \text{(Null Law: } 1 + X = 1) \\
F &= C'D + AB' + AC && \text{(Identity Law)}
\end{aligned}$$

Thus, the simplified **SOP** expression is:

$$\mathbf{F_{SOP} = C'D + AB' + AC}$$

Step 2: Algebraic Simplification to Product of Sums (POS)
We can derive the POS form directly from the simplified SOP expression by repeatedly applying the Distributive Law ($X + YZ = (X + Y)(X + Z)$):

$$\begin{aligned}
F &= C'D + A(B' + C) && \text{(Factoring } A \text{ from } AB' + AC) \\
F &= (C'D + A) \cdot (C'D + B' + C) && \text{(Distributive Law)}
\end{aligned}$$

Now, we simplify the two resulting terms separately.
First term:

$$C'D + A = (A + C')(A + D) \qquad \text{(Distributive Law)}$$

Second term:

$$\begin{aligned} C'D + B' + C &= (C + C'D) + B' && \text{(Commutative Law)} \\ &= (C + C')(C + D) + B' && \text{(Distributive Law)} \\ &= (1)(C + D) + B' && \text{(Complement Law)} \\ &= B' + C + D \end{aligned}$$

Substituting these simplified factors back into the equation for F:

$$\mathbf{F_{POS} = (A + C')(A + D)(B' + C + D)}$$

Step 3: Karnaugh Map Verification
To rigorously verify our algebraic minimizations, we plot the unsimplified function on a 4-variable K-map. We can derive SOP by grouping the 1s and POS by grouping the 0s.

SOP Grouping (1s)

AB \ CD	00	01	11	10
00	0	1	0	0
01	0	1	0	0
11	0	1	1	1
10	1	1	1	1

POS Grouping (0s)

AB \ CD	00	01	11	10
00	0	1	0	0
01	0	1	0	0
11	0	1	1	1
10	1	1	1	1

- **SOP Extraction:** By grouping the 1s, we extract the minterms.
  - **Blue Column (01):** Constant $C = 0, D = 1 \implies C'D$
  - **Red Row (10):** Constant $A = 1, B = 0 \implies AB'$
  - **Green 2x2 Square:** Constant $A = 1, C = 1 \implies AC$

  Summing these groups confirms $\mathbf{F_{SOP} = C'D + AB' + AC}$.
- **POS Extraction:** By grouping the 0s, we extract the maxterms. Recall that for maxterms, logic is inverted ($0 \to$ uncomplemented, $1 \to$ complemented).
  - **Magenta 2x2 Square:** Constant $A = 0, C = 1 \implies (A + C')$
  - **Orange Edges (Top Corners):** Constant $A = 0, D = 0 \implies (A + D)$
  - **Cyan 1x2 Rectangle:** Constant $B = 1, C = 0, D = 0 \implies (B' + C + D)$

  Multiplying these groups confirms $\mathbf{F_{POS} = (A + C')(A + D)(B' + C + D)}$.

Q6. Simplify the following expression to POS and SOP: $BCD' + ABC' + ACD$. **(8 marks)** **[CSE 205 | L-2/T-1 | 2017–18]**

Answer

Simplification of the Boolean Expression
Given the Boolean function:

$$F = BCD' + ABC' + ACD$$

We will simplify this expression into its minimum Sum of Products (SOP) and Product of Sums (POS) forms algebraically, and then verify our results using Karnaugh Maps.

Step 1: Algebraic Simplification to Sum of Products (SOP)
We can simplify the given expression step-by-step using fundamental Boolean algebra laws, specifically relying on the **Consensus Theorem** ($XY + X'Z + YZ = XY + X'Z$).
Observe the terms BCD' and ACD. The variable D is uncomplemented in the latter and complemented in the former. The consensus of these two terms is $(BC)(AC) = ABC$. By the Consensus Theorem, we can add this redundant term without changing

the function's value:

$$\begin{aligned}
F &= BCD' + ABC' + ACD \\
F &= BCD' + ABC' + ACD + ABC && \text{(Adding consensus of } BCD' \text{ and } ACD) \\
F &= BCD' + ACD + ABC' + ABC && \text{(Rearranging terms)} \\
F &= BCD' + ACD + AB(C' + C) && \text{(Factoring } AB) \\
F &= BCD' + ACD + AB(1) && \text{(Complement Law: } C' + C = 1) \\
F &= BCD' + ACD + AB && \text{(Identity Law)}
\end{aligned}$$

Rearranging to a standard order, the simplified **SOP** expression is:

$$\mathbf{F_{SOP} = AB + ACD + BCD'}$$

Step 2: Algebraic Simplification to Product of Sums (POS)
We derive the POS form directly from our minimal SOP expression using the Distributive Law ($X + YZ = (X + Y)(X + Z)$) and the Consensus Theorem.

$$\begin{aligned}
F &= AB + ACD + BCD' \\
F &= AB + C(AD + BD') && \text{(Factoring } C \text{ from the last two terms)} \\
F &= (AB + C)(AB + AD + BD') && \text{(Distributive Law)}
\end{aligned}$$

Now, we simplify both factors:

(a) **First factor:** By the Distributive Law, $(AB + C) = (A + C)(B + C)$.

(b) **Second factor:** Notice that AB is the consensus of AD and BD'. Therefore, $AD + BD' + AB = AD + BD'$. We can use this property to eliminate AB:

$$AB + AD + BD' = AD + BD'$$

Furthermore, we can factor $AD + BD'$ by recognizing it is the expanded form of $(A + D')(B + D)$:

$$(A + D')(B + D) = AB + AD + BD' + DD' = AB + AD + BD' = AD + BD'$$

Substituting these simplified factors back into our expression for F:

$$F = (A + C)(B + C) \cdot (A + D')(B + D)$$

Thus, the simplified **POS** expression is:

$$\mathbf{F_{POS} = (A + C)(B + C)(B + D)(A + D')}$$

Step 3: Karnaugh Map Verification
To rigorously verify our algebraic minimizations, we plot the function on a 4-variable K-map. We derive SOP by grouping the 1s and POS by grouping the 0s.

SOP Grouping (1s)

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	1
11	1	1	1	1
10	0	0	1	0

POS Grouping (0s)

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	1
11	1	1	1	1
10	0	0	1	0

- **SOP Extraction (1s):**
  - **Blue Row:** Constant $A = 1, B = 1 \implies AB$
  - **Red Column Pair:** Constant $A = 1, C = 1, D = 1 \implies ACD$
  - **Green Column Pair:** Constant $B = 1, C = 1, D = 0 \implies BCD'$

  This confirms $\mathbf{F_{SOP} = AB + ACD + BCD'}$.
- **POS Extraction (0s):** (Note: $0 \to$ uncomplemented, $1 \to$ complemented)
  - **Magenta 2x2:** Constant $A = 0, C = 0 \implies (A + C)$
  - **Brown 2x2:** Constant $A = 0, D = 1 \implies (A + D')$
  - **Orange Split 2x2:** Constant $B = 0, C = 0 \implies (B + C)$
  - **Cyan Four Corners:** Constant $B = 0, D = 0 \implies (B + D)$

  This confirms $\mathbf{F_{POS} = (A + C)(B + C)(B + D)(A + D')}$.

Q7. Prove the following Boolean algebra theorem:

$$(x+y)(x'+z)(y+z) = (x+y)(x'+z)$$

(10 marks) [CSE 205 | L-2/T-1 | 2015–16]

Answer

Proof of the Boolean Algebra Theorem (Consensus Theorem in POS Form)
The given theorem is the Product of Sums (POS) form of the **Consensus Theorem**. We can prove this theorem using two rigorous methods: Algebraic Manipulation and Truth Table Verification.

Method 1: Algebraic Proof
We start with the Left-Hand Side (LHS) of the equation and apply standard Boolean algebra postulates and theorems to reduce it to the Right-Hand Side (RHS).

$$\begin{aligned}
\text{LHS} &= (x+y)(x'+z)(y+z) \\
&= (x+y)(x'+z)(y+z+0) && \text{(Identity Law: } A+0=A\text{)} \\
&= (x+y)(x'+z)(y+z+xx') && \text{(Complement Law: } xx'=0\text{)} \\
&= (x+y)(x'+z)(x+y+z)(x'+y+z) && \text{(Distributive Law: } A+BC=(A+B)(A+C)\text{)}
\end{aligned}$$

Now, we rearrange the terms using the Commutative Law to group related sums together:

$$\text{LHS} = \big[(x+y)(x+y+z)\big] \cdot \big[(x'+z)(x'+z+y)\big] \qquad \text{(Commutative Law)}$$

Next, we apply the **Absorption Law**, which states that $A(A+B)=A$.

- For the first group, let $A=(x+y)$ and $B=z$. Thus, $(x+y)(x+y+z)=(x+y)$.
- For the second group, let $A=(x'+z)$ and $B=y$. Thus, $(x'+z)(x'+z+y)=(x'+z)$.

Substituting these absorbed expressions back into our main equation gives:

$$\begin{aligned}
\text{LHS} &= (x+y)(x'+z) \\
&= \text{RHS}
\end{aligned}$$

Thus, the theorem is algebraically proven.

Method 2: Verification via Truth Table
To ensure complete verification of the circuit behavior, we can exhaustively test all possible combinations of the variables x, y, and z.

x	y	z	x'	**Term 1:** $(x+y)$	**Term 2:** $(x'+z)$	**Term 3:** $(y+z)$	**LHS:** T1 · T2 · T3	**RHS:** T1 · T2
0	0	0	1	0	1	0	**0**	**0**
0	0	1	1	0	1	1	**0**	**0**
0	1	0	1	1	1	1	**1**	**1**
0	1	1	1	1	1	1	**1**	**1**
1	0	0	0	1	0	0	**0**	**0**
1	0	1	0	1	1	1	**1**	**1**
1	1	0	0	1	0	1	**0**	**0**
1	1	1	0	1	1	1	**1**	**1**

By comparing the last two columns of the truth table, we observe that the output values for the LHS $(x+y)(x'+z)(y+z)$ and the RHS $(x+y)(x'+z)$ are identical for every possible input combination of x, y, and z.
Conclusion: Both the rigorous algebraic derivation and the exhaustive truth table prove that the expression $(x+y)(x'+z)(y+z)$ accurately simplifies to $(x+y)(x'+z)$.

Q8. How many distinct Boolean functions are there of n Boolean variables? **(5 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Answer

Total Number of Distinct Boolean Functions
For n Boolean variables, the total number of distinct Boolean functions is given by the formula:

$$\mathbf{2^{2^n}}$$

Step-by-Step Derivation:
To understand how this formula is derived mathematically, we can analyze the structure of a standard logic truth table.

(a) **Determine the number of input combinations:** A Boolean function of n variables (e.g., $x_1, x_2, \ldots, x_n$) operates on inputs that can each take on exactly two possible values in binary logic: 0 or 1. Therefore, the total number of unique input combinations (which corresponds to the number of rows in the truth table) is:

$$\text{Number of Rows} = 2 \times 2 \times \cdots \times 2 \ (n \text{ times}) = 2^n$$

(b) **Determine the number of output assignments:** A specific Boolean function is uniquely defined by the output value it maps to each of these 2^n input combinations. For every single row in the truth table, the function's output can be either a 0 or a 1. This gives exactly 2 independent choices per row.

(c) **Calculate the total possible functions:** Since there are 2 choices for the output of the first row, 2 choices for the second row, and so on, spanning across all 2^n rows, the fundamental counting principle dictates that we multiply the number of choices together:

$$\text{Total Functions} = \underbrace{2 \times 2 \times 2 \times \cdots \times 2}_{2^n \text{ times}} = 2^{(2^n)}$$

Illustrative Examples:
To verify this intuitively, we can evaluate the formula for small values of n.

- **Case for $n = 1$ (One variable, e.g., A):**
  - Number of input rows: $2^1 = 2$ rows (Inputs $A = 0$ and $A = 1$).
  - Number of possible functions: $2^{2^1} = 2^2 = \mathbf{4}$ distinct functions.
  - These 4 functions are: $F = 0$ (Constant 0), $F = 1$ (Constant 1), $F = A$ (Buffer/Identity), and $F = A'$ (NOT/Inverter).

- **Case for $n = 2$ (Two variables, e.g., A, B):**
  - Number of input rows: $2^2 = 4$ rows (Inputs $00, 01, 10, 11$).
  - Number of possible functions: $2^{2^2} = 2^4 = \mathbf{16}$ distinct functions.
  - These 16 functions represent all possible 2-input logic mappings, including standard logic gates such as AND, OR, NAND, NOR, XOR, XNOR, Implications, Constants, and single-variable passes/inversions.

- **Case for $n = 3$ (Three variables):**
  - Number of input rows: $2^3 = 8$ rows.
  - Number of possible functions: $2^{2^3} = 2^8 = \mathbf{256}$ distinct Boolean functions.

Summary Table of Combinatorial Growth:
The number of functions exhibits double exponential growth, as shown below:

Number of Variables (n)	**Truth Table Rows (2^n)**	**Distinct Functions (2^{2^n})**
1	2	4
2	4	16
3	8	256
4	16	$65,536$
5	32	$4,294,967,296$

Q9. Simplify the Boolean function using algebraic manipulation:

$$f = A'BC + AB'C + ABC' + ABC$$

(10 marks) **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

Algebraic Simplification of the Boolean Function
The given Boolean function is:

$$f = A'BC + AB'C + ABC' + ABC$$

This function represents the Sum of Products (SOP) form of the logical Majority Function (or the Carry-Out of a Full Adder). We can simplify this expression step-by-step using standard Boolean algebraic laws.

Step-by-Step Derivation:

$$f = A'BC + AB'C + ABC' + ABC$$

According to the **Idempotent Law** ($X + X = X$), we can duplicate the term ABC multiple times without changing the value of the function. We will add two extra copies of ABC to group with the other three terms:

$$f = A'BC + AB'C + ABC' + (ABC + ABC + ABC)$$

Now, we use the **Commutative Law** to rearrange the terms, pairing one ABC with each of the other terms:

$$f = (A'BC + ABC) + (AB'C + ABC) + (ABC' + ABC)$$

Next, we apply the **Distributive Law** ($XY + XZ = X(Y + Z)$) to factor out the common variables in each grouped pair:

$$f = BC(A' + A) + AC(B' + B) + AB(C' + C)$$

According to the **Complement Law**, the OR sum of a variable and its complement is always 1 ($X + X' = 1$):

$$f = BC(1) + AC(1) + AB(1)$$

Finally, applying the **Identity Law** ($X \cdot 1 = X$), we remove the 1s to obtain the simplified minimal Sum of Products expression:

$$f = BC + AC + AB$$

Rearranging the terms alphabetically for standard presentation, we get the final simplified expression:

$$\mathbf{f = AB + BC + AC}$$

Verification using Karnaugh Map (Visualization)
To visually verify our algebraic simplification, we can map the original standard SOP function onto a 3-variable Karnaugh Map. The original function $f = A'BC + AB'C + ABC' + ABC$ corresponds to minterms m_3, m_5, m_6, and m_7.

A \ BC	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Group 1 (Vertical): BC
Group 2 (Horizontal Left): AC
Group 3 (Horizontal Right): AB

From the K-map, we form three overlapping groups of two adjacent 1s:

- **Red Group** covers m_3 and m_7, where B and C remain constant (11). This gives the term BC.
- **Blue Group** covers m_5 and m_7, where A and C remain constant (1 and 1). This gives the term AC.
- **Green Group** covers m_6 and m_7, where A and B remain constant (11). This gives the term AB.

Combining these groupings yields $f = AB + BC + AC$, which perfectly matches our algebraic derivation.

Q10. Simplify the following Boolean function using algebraic manipulation:

$$(A + BC)' + AB'$$

(7 marks) **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

Algebraic Simplification of the Boolean Function
The given Boolean expression is:

$$f = (A + BC)' + AB'$$

We can simplify this function step-by-step using standard Boolean algebraic theorems and postulates.

Step-by-Step Derivation:

$$f = (A + BC)' + AB'$$

Step 1: Apply **De Morgan's Law** $(X + Y)' = X'Y'$ to the first term, treating A as X and BC as Y.

$$f = A'(BC)' + AB'$$

Step 2: Apply **De Morgan's Law** $(XY)' = X' + Y'$ to the term $(BC)'$.

$$f = A'(B' + C') + AB'$$

Step 3: Apply the **Distributive Law** $X(Y+Z) = XY + XZ$ to expand the first part of the expression.

$$f = A'B' + A'C' + AB'$$

Step 4: Rearrange the terms using the **Commutative Law** $(X+Y = Y+X)$ to place terms containing B' adjacent to each other.

$$f = A'B' + AB' + A'C'$$

Step 5: Apply the **Distributive Law** in reverse to factor out the common literal B' from the first two terms.

$$f = B'(A' + A) + A'C'$$

Step 6: Apply the **Complement Law**, which states that the logical OR of a variable and its complement is always true $(X'+X = 1)$.

$$f = B'(1) + A'C'$$

Step 7: Finally, apply the **Identity Law** $(X \cdot 1 = X)$ to eliminate the 1.

$$f = B' + A'C'$$

The final simplified Boolean expression is:

$$\mathbf{f = B' + A'C'}$$

Verification using Truth Table

To ensure the accuracy of our algebraic simplification, we can construct a truth table comparing the original function to our simplified result across all possible input combinations for A, B, and C.

A	B	C	A'	B'	C'	**Term 1:** $(A+BC)'$	**Term 2:** AB'	**Original:** $(A+BC)'+AB'$	**Simplified:** $B'+A'C'$
0	0	0	1	1	1	$(0+0)'=1$	$0\cdot1=0$	$1+0=\mathbf{1}$	$1+(1\cdot1)=\mathbf{1}$
0	0	1	1	1	0	$(0+0)'=1$	$0\cdot1=0$	$1+0=\mathbf{1}$	$1+(1\cdot0)=\mathbf{1}$
0	1	0	1	0	1	$(0+0)'=1$	$0\cdot0=0$	$1+0=\mathbf{1}$	$0+(1\cdot1)=\mathbf{1}$
0	1	1	1	0	0	$(0+1)'=0$	$0\cdot0=0$	$0+0=\mathbf{0}$	$0+(1\cdot0)=\mathbf{0}$
1	0	0	0	1	1	$(1+0)'=0$	$1\cdot1=1$	$0+1=\mathbf{1}$	$1+(0\cdot1)=\mathbf{1}$
1	0	1	0	1	0	$(1+0)'=0$	$1\cdot1=1$	$0+1=\mathbf{1}$	$1+(0\cdot0)=\mathbf{1}$
1	1	0	0	0	1	$(1+0)'=0$	$1\cdot0=0$	$0+0=\mathbf{0}$	$0+(0\cdot1)=\mathbf{0}$
1	1	1	0	0	0	$(1+1)'=0$	$1\cdot0=0$	$0+0=\mathbf{0}$	$0+(0\cdot0)=\mathbf{0}$

Conclusion: By observing the last two columns of the truth table, we see that the output vectors for the original function $(A+BC)'+AB'$ and the simplified expression $B'+A'C'$ match perfectly for all $2^3 = 8$ possible input combinations. This conclusively proves that our algebraic simplification is correct.

Q11. State and prove the consensus theorem using Boolean algebra. **(8 marks)** **[CSE 205 | L-2/T-1 | 2014–15]**

Answer

Statement of the Consensus Theorem

The Consensus Theorem is a powerful rule in Boolean algebra used to minimize logic expressions by eliminating redundant terms. It states that if a Boolean expression contains two terms where one variable appears uncomplemented in one term and complemented in the other, and a third term contains the remaining variables from the first two terms, this third term is redundant and can be omitted. The redundant third term is known as the **consensus term**.

The theorem exists in two dual forms:

- **Sum of Products (SOP) Form:**

$$XY + X'Z + YZ = XY + X'Z$$

- **Product of Sums (POS) Form:**

$$(X+Y)(X'+Z)(Y+Z) = (X+Y)(X'+Z)$$

Proof of the Sum of Products (SOP) Form

We will prove the SOP form using standard Boolean algebraic postulates and theorems, starting with the Left-Hand Side (LHS) and simplifying it to match the Right-Hand Side (RHS).

$$\text{LHS} = XY + X'Z + YZ$$

Step 1: Multiply the consensus term (YZ) by 1 using the **Identity Law** $(A \cdot 1 = A)$.

$$= XY + X'Z + YZ \cdot 1$$

Step 2: Substitute 1 with $(X + X')$ using the **Complement Law** $(A + A' = 1)$.

$$= XY + X'Z + YZ(X + X')$$

Step 3: Expand the expression using the **Distributive Law** $(A(B + C) = AB + AC)$.

$$= XY + X'Z + XYZ + X'YZ$$

Step 4: Rearrange the terms using the **Commutative Law** to group related terms together.

$$= XY + XYZ + X'Z + X'YZ$$

Step 5: Factor out common variables using the **Distributive Law** in reverse.

$$= XY(1 + Z) + X'Z(1 + Y)$$

Step 6: Apply the **Annulment Law** (also known as the Null Law), which states that $1 + A = 1$.

$$= XY(1) + X'Z(1)$$

Step 7: Finally, apply the **Identity Law** $(A \cdot 1 = A)$ to yield the RHS.

$$= XY + X'Z$$
$$= \text{RHS}$$

Thus, the SOP form of the Consensus Theorem is algebraically proven.

Proof of the Product of Sums (POS) Form
Similarly, we can prove the POS form by direct algebraic manipulation.

$$\text{LHS} = (X + Y)(X' + Z)(Y + Z)$$

Step 1: Add 0 to the consensus term $(Y + Z)$ using the **Identity Law** $(A + 0 = A)$.

$$= (X + Y)(X' + Z)(Y + Z + 0)$$

Step 2: Substitute 0 with XX' using the **Complement Law** $(A \cdot A' = 0)$.

$$= (X + Y)(X' + Z)(Y + Z + XX')$$

Step 3: Expand using the **Distributive Law** $(A + BC = (A + B)(A + C))$.

$$= (X + Y)(X' + Z)(X + Y + Z)(X' + Y + Z)$$

Step 4: Rearrange terms using the **Commutative Law** to pair terms efficiently.

$$= \big[(X + Y)(X + Y + Z)\big] \cdot \big[(X' + Z)(X' + Z + Y)\big]$$

Step 5: Apply the **Absorption Law** $A(A+B) = A$. In the first bracketed group, let $A = X+Y$ and $B = Z$. In the second group, let $A = X' + Z$ and $B = Y$.

$$= (X + Y) \cdot (X' + Z)$$
$$= \text{RHS}$$

Thus, the POS form of the Consensus Theorem is also algebraically proven.

Q12. Simplify the following function using Boolean algebra:

$$F(A, B, C) = A'BC + AB'C + ABC$$

(7 marks) **[CSE 205 | L-2/T-1 | 2014–15]**

Answer

Algebraic Simplification of the Boolean Function
The given Boolean function is:

$$F(A, B, C) = A'BC + AB'C + ABC$$

We can simplify this expression step-by-step using standard Boolean algebraic postulates and theorems. There are multiple valid algebraic paths to reach the minimal form; we will use the highly effective Redundancy Law (also known as the Simplification Theorem).

Step-by-Step Derivation:

$$F = A'BC + AB'C + ABC$$

Step 1: Apply the **Distributive Law** ($XY + XZ = X(Y + Z)$) to factor out the common variables AC from the second and third terms.

$$F = A'BC + AC(B' + B)$$

Step 2: Apply the **Complement Law**, which states that the logical OR of a variable and its complement is always 1 ($X + X' = 1$). Here, $B' + B = 1$.

$$F = A'BC + AC(1)$$

Step 3: Apply the **Identity Law** ($X \cdot 1 = X$) to eliminate the 1.

$$F = A'BC + AC$$

Step 4: Apply the **Distributive Law** again to factor out the common variable C from both remaining terms.

$$F = C(A'B + A)$$

Step 5: Apply the **Commutative Law** to rearrange the terms inside the parenthesis for clarity.

$$F = C(A + A'B)$$

Step 6: Apply the **Redundancy Law** (or Rule of Simplification), which states that $X + X'Y = X + Y$. Let $X = A$ and $Y = B$.

$$F = C(A + B)$$

Step 7: Finally, expand the expression using the **Distributive Law** to present the answer in standard Sum of Products (SOP) form.

$$F = AC + BC$$

The final simplified Boolean expression is:

$$\mathbf{F = AC + BC} \quad \text{or} \quad \mathbf{F = C(A + B)}$$

Verification using Karnaugh Map (Visualization)
To rigorously verify our algebraic simplification, we can map the original function onto a 3-variable Karnaugh Map.
The original function $F = A'BC + AB'C + ABC$ corresponds to the minterms m_3 (011), m_5 (101), and m_7 (111).

A \ BC	00	01	11	10
0	0	0	1	0
1	0	1	1	0

Group 1 (Vertical): BC
Group 2 (Horizontal): AC

By analyzing the K-map, we can form two optimal groups of adjacent 1s:

- **Red Group** covers minterms m_3 and m_7. In this vertical group, A changes from 0 to 1 (and is eliminated), while B and C remain constant at 11. This yields the term BC.
- **Blue Group** covers minterms m_5 and m_7. In this horizontal group, B changes from 0 to 1 (and is eliminated), while A and C remain constant at 1 and 1. This yields the term AC.

Combining the terms derived from the logical groupings yields $F = AC + BC$. This visual minimization perfectly corroborates our step-by-step algebraic simplification.

Q13. Simplify the following Boolean expressions to a minimum number of literals using Boolean algebra:

(a) $AB + A(CD + CD') + A'$

(b) $(A + B)'(A' + B')'$

(10 marks) **[CSE 205 | L-2/T-1 | 2013–14]**

Answer

Boolean Algebra Simplification

We are tasked with simplifying two Boolean expressions to their minimum number of literals. We will proceed step-by-step, clearly stating the Boolean theorems and postulates used at each stage.

Part 1: Simplify $\mathbf{AB + A(CD + CD') + A'}$

Let $F_1 = AB + A(CD + CD') + A'$.

$$F_1 = AB + A(CD + CD') + A'$$

Step 1: Factor out the common variable C from the terms inside the parentheses using the **Distributive Law** ($XY + XZ = X(Y + Z)$).

$$= AB + AC(D + D') + A'$$

Step 2: Apply the **Complement Law**, which states that the logical OR of a variable and its complement is always 1 ($X + X' = 1$). Here, $D + D' = 1$.

$$= AB + AC(1) + A'$$

Step 3: Apply the **Identity Law** ($X \cdot 1 = X$) to eliminate the 1.

$$= AB + AC + A'$$

Step 4: Rearrange the terms using the **Commutative Law** ($X + Y = Y + X$) to position A' next to AB.

$$= A' + AB + AC$$

Step 5: Apply the **Distributive Law** in the form $X + YZ = (X + Y)(X + Z)$ to the first two terms: $A' + AB = (A' + A)(A' + B)$.

$$= (A' + A)(A' + B) + AC$$

Step 6: Apply the **Complement Law** again ($A' + A = 1$).

$$= 1 \cdot (A' + B) + AC$$

Step 7: Apply the **Identity Law** ($1 \cdot X = X$).

$$= A' + B + AC$$

Step 8: Rearrange terms again using the **Commutative Law** to group A' and AC.

$$= B + A' + AC$$

Step 9: Apply the **Distributive Law** to $A' + AC$, expanding it to $(A' + A)(A' + C)$.

$$= B + (A' + A)(A' + C)$$

Step 10: Use the **Complement Law** ($A' + A = 1$) and the **Identity Law** to simplify the term.

$$= B + 1 \cdot (A' + C)$$
$$= B + A' + C$$

Rearranging the variables alphabetically, the final simplified expression is:

$$\mathbf{F_1 = A' + B + C}$$

Part 2: Simplify $\mathbf{(A + B)'(A' + B')'}$

Let $F_2 = (A + B)'(A' + B')'$.

$$F_2 = (A + B)'(A' + B')'$$

Step 1: Apply **De Morgan's Law** $[(X+Y)' = X'Y']$ to the first term, $(A+B)'$.

$$= (A'B') \cdot (A'+B')'$$

Step 2: Apply **De Morgan's Law** again to the second term, $(A'+B')'$.

$$= (A'B') \cdot ((A')'(B')')$$

Step 3: Apply the **Involution Law** (or Double Negation Law), which states $(X')' = X$, to simplify $(A')'$ to A and $(B')'$ to B.

$$= (A'B') \cdot (AB)$$

Step 4: Remove the parentheses using the **Associative Law** and rearrange the variables using the **Commutative Law** to group identical variables together.

$$= (A \cdot A') \cdot (B \cdot B')$$

Step 5: Apply the **Complement Law** (AND form), which states that the logical AND of a variable and its complement is always 0 $(X \cdot X' = 0)$.

$$= 0 \cdot 0$$

Step 6: Apply the **Idempotent Law** (or Null Law) to conclude the simplification.

$$= 0$$

The final simplified expression is:

$$\mathbf{F_2 = 0}$$

Q14. The Boolean expression for an X-OR gate is $AB' + A'B$. With this as a starting point, use DeMorgan's theorems and other rules or laws to find a simplified expression for an X-NOR gate. **(8 marks)** **[CSE 205 | L-2/T-1 | 2011–12]**

Answer

Derivation of the X-NOR Expression

The Exclusive-NOR (X-NOR) gate operates as the logical complement (inversion) of the Exclusive-OR (X-OR) gate. Therefore, we can derive the Boolean expression for the X-NOR gate by taking the complement of the given X-OR expression and simplifying it using Boolean algebra.

Given the X-OR expression:

$$\text{X-OR} = AB' + A'B$$

The X-NOR expression is:

$$\text{X-NOR} = (AB' + A'B)'$$

Step-by-Step Algebraic Simplification:

$$F_{\text{XNOR}} = (AB' + A'B)'$$

Step 1: Apply **De Morgan's Law** $[(X+Y)' = X' \cdot Y']$ to the entire expression, treating the term AB' as X and the term $A'B$ as Y.

$$= (AB')' \cdot (A'B)'$$

Step 2: Apply **De Morgan's Law** $[(XY)' = X' + Y']$ individually to both grouped product terms.

$$= (A' + (B')') \cdot ((A')' + B')$$

Step 3: Apply the **Involution Law** (Double Negation Law), which states that $(X')' = X$, to eliminate the double primes on variables A and B.

$$= (A' + B) \cdot (A + B')$$

Step 4: Apply the **Distributive Law** to expand the product of sums (multiplying out the terms: First, Outer, Inner, Last).

$$= A'A + A'B' + BA + BB'$$

Step 5: Apply the **Commutative Law** to rearrange the term BA to AB for standard presentation.

$$= A'A + A'B' + AB + BB'$$

Step 6: Apply the **Complement Law** (AND form), which states that a variable ANDed with its logical complement is always 0 ($X \cdot X' = 0$). Therefore, $A'A = 0$ and $BB' = 0$.

$$= 0 + A'B' + AB + 0$$

Step 7: Apply the **Identity Law** ($X + 0 = X$) to remove the zeros from the sum.

$$= A'B' + AB$$

Step 8: Apply the **Commutative Law** one final time to arrange the terms into the universally recognized standard format.

$$= AB + A'B'$$

The simplified Boolean expression for an X-NOR gate is:

$$\mathbf{F_{XNOR} = AB + A'B'}$$

Verification via Truth Table
To rigorously verify that this derived expression correctly models the X-NOR behavior (often called the Equivalence gate, outputting 1 when inputs match), we can evaluate its truth table:

A	B	A'	B'	**Term 1:** AB	**Term 2:** $A'B'$	**Output:** $AB + A'B'$
0	0	1	1	0	1	**1**
0	1	1	0	0	0	**0**
1	0	0	1	0	0	**0**
1	1	0	0	1	0	**1**

Conclusion: The truth table explicitly confirms that the output evaluates to 1 only when both inputs are identical ($A = B = 0$ or $A = B = 1$). This fully validates that our algebraically derived expression $\mathbf{AB + A'B'}$ acts correctly as an X-NOR function.

Q15. Give an example of the non-associativity of the 3-input NOR operator. **(8 marks)** **[CSE 205 | L-2/T-1 | 2011–12]**

Answer

Non-Associativity of the 3-Input NOR Operator
For a binary operator to be associative, the order in which grouping is applied must not affect the final result. That is, an operator $\downarrow$ (representing NOR) is associative if and only if for all possible Boolean values A, B, and C:

$$(A \downarrow B) \downarrow C = A \downarrow (B \downarrow C)$$

We will demonstrate that the NOR operator is **non-associative** by using two methods: algebraic expansion and a definitive truth-table counterexample.

1. Algebraic Demonstration
The NOR operation between two variables is defined as the complement of their logical sum:

$$X \downarrow Y = (X + Y)'$$

Let us expand both sides of the associative equation using De Morgan's laws:

- **Left-Hand Side (LHS):**

$$\begin{aligned} \text{LHS} &= (A \downarrow B) \downarrow C \\ &= ((A + B)') \downarrow C \\ &= \big((A + B)' + C\big)' \end{aligned}$$

  Applying De Morgan's Law $(X + Y)' = X'Y'$ and the Involution Law $(X'') = X$:

$$\begin{aligned} &= ((A + B)')' \cdot C' \\ &= (A + B)C' \\ &= AC' + BC' \end{aligned}$$

- **Right-Hand Side (RHS):**

$$\begin{aligned} \text{RHS} &= A \downarrow (B \downarrow C) \\ &= A \downarrow ((B + C)') \\ &= \big(A + (B + C)'\big)' \end{aligned}$$

Applying De Morgan's Law and the Involution Law:

$$\begin{aligned} &= A' \cdot ((B + C)')' \\ &= A'(B + C) \\ &= A'B + A'C \end{aligned}$$

Since the expanded expressions $\mathbf{AC' + BC'}$ (LHS) and $\mathbf{A'B + A'C}$ (RHS) are distinct and not logically equivalent, the operator is non-associative.

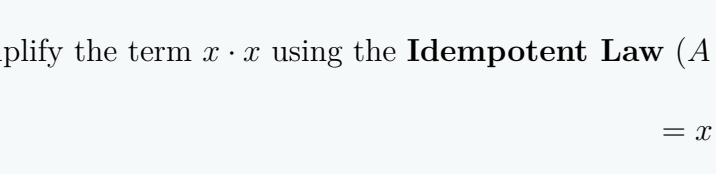
2. Concrete Truth Table Counterexample
To explicitly prove non-associativity, we evaluate all $2^3 = 8$ combinations of inputs. A single mismatch between the LHS and RHS columns is sufficient to prove non-associativity.

A	B	C	$A \downarrow B$	**LHS** : $(A \downarrow B) \downarrow C$	$B \downarrow C$	**RHS** : $A \downarrow (B \downarrow C)$	**Status**
0	0	0	1	$(1+0)' = \mathbf{0}$	1	$(0+1)' = \mathbf{0}$	Match
0	0	1	1	$(1+1)' = \mathbf{0}$	0	$(0+0)' = \mathbf{1}$	**Mismatch**
0	1	0	0	$(0+0)' = \mathbf{1}$	0	$(0+0)' = \mathbf{1}$	Match
0	1	1	0	$(0+1)' = \mathbf{0}$	0	$(0+0)' = \mathbf{1}$	**Mismatch**
1	0	0	0	$(0+0)' = \mathbf{1}$	1	$(1+1)' = \mathbf{0}$	**Mismatch**
1	0	1	0	$(0+1)' = \mathbf{0}$	0	$(1+0)' = \mathbf{0}$	Match
1	1	0	0	$(0+0)' = \mathbf{1}$	0	$(1+0)' = \mathbf{0}$	**Mismatch**
1	1	1	0	$(0+1)' = \mathbf{0}$	0	$(1+0)' = \mathbf{0}$	Match

As highlighted in the table, rows such as $(A = 0, B = 0, C = 1)$ clearly show a conflict where LHS $= 0$ while RHS $= 1$.

3. Structural Visualization via Logic Circuits
The structural asymmetry causing this functional difference can be easily understood by visualizing how the inputs cascade through the hardware connections in each grouping configuration.

Circuit 1: $((A \downarrow B) \downarrow C)$



Circuit 2: $(A \downarrow (B \downarrow C))$

Conclusion: Because the grouping changes the propagation structure and produces asymmetric Boolean sub-expressions, **the 3-input NOR operator is inherently non-associative**.

Q16. Prove that $x(x + y) = x$. **(5 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Answer

Proof of the Absorption Law
The given Boolean equation, $x(x + y) = x$, is formally known as the **Absorption Law**. It demonstrates how a term can "absorb" another term under specific logical conditions. We can prove this theorem rigorously using two standard methods: algebraic manipulation and perfect induction (truth table verification).

Method 1: Algebraic Proof
We begin with the Left-Hand Side (LHS) of the equation and apply fundamental Boolean postulates and theorems to reduce it to the Right-Hand Side (RHS).

$$\text{LHS} = x(x + y)$$

Step 1: Expand the expression using the **Distributive Law** $(A(B + C) = AB + AC)$.

$$= x \cdot x + x \cdot y$$

Step 2: Simplify the term $x \cdot x$ using the **Idempotent Law** $(A \cdot A = A)$.

$$= x + xy$$

Step 3: Express the single variable x as $x \cdot 1$ using the **Identity Law** $(A \cdot 1 = A)$. This makes factoring easier.

$$= x \cdot 1 + x \cdot y$$

Step 4: Factor out the common variable x by applying the **Distributive Law** in reverse.

$$= x(1 + y)$$

Step 5: Apply the **Annulment Law** (also known as the Dominance Law), which states that the logical OR of 1 and any Boolean variable is always 1 ($1 + A = 1$).

$$= x(1)$$

Step 6: Finally, apply the **Identity Law** ($A \cdot 1 = A$) to eliminate the 1.

$$= x$$
$$= \text{RHS}$$

Thus, the algebraic equivalence is successfully proven.

Method 2: Proof by Perfect Induction (Truth Table)
Another definitive way to prove a Boolean theorem is by evaluating the expression for all possible input combinations of its variables. Since there are two variables (x and y), there are $2^2 = 4$ possible combinations.

Input x	**Input** y	**Intermediate Term:** $x + y$	**LHS Expression:** $x(x + y)$
0	0	$0 + 0 = 0$	$0 \cdot 0 = \mathbf{0}$
0	1	$0 + 1 = 1$	$0 \cdot 1 = \mathbf{0}$
1	0	$1 + 0 = 1$	$1 \cdot 1 = \mathbf{1}$
1	1	$1 + 1 = 1$	$1 \cdot 1 = \mathbf{1}$

Conclusion: By comparing the first column (**Input** x) with the last column (**LHS Expression:** $x(x + y)$), we observe that the logical values are identical for every possible combination of inputs. Because the output of $x(x + y)$ perfectly mirrors the input x, the identity $x(x + y) = x$ is unconditionally true.

De Morgan's Theorem

Q1. Demonstrate the validity of the following identities by means of truth tables:

(a) De Morgan's theorem for three variables: $(x + y + z)' = x'y'z'$

(b) The distributive law: $x(y + z) = xy + xz$

(10 marks) [CSE 205 | L-2/T-1 | 2019–20]

Answer

Proof of Boolean Identities Using Truth Tables
To demonstrate the validity of a Boolean identity using a truth table, we evaluate both the Left-Hand Side (LHS) and the Right-Hand Side (RHS) expressions for all possible combinations of the input variables. Since both identities contain three binary variables (x, y, and z), there are $2^3 = 8$ distinct input combinations. If the column corresponding to the LHS is identical to the column corresponding to the RHS for every row, the identity is verified.

Part (a): De Morgan's Theorem for Three Variables: $(x + y + z)' = x'y'z'$
De Morgan's theorem states that the complement of a logical sum (OR) is equal to the logical product (AND) of the complements. The table below presents the step-by-step evaluation of both sides of the identity.

x	y	z	$x + y + z$	**LHS:** $(x + y + z)'$	x'	y'	z'	**RHS:** $x'y'z'$
0	0	0	0	**1**	1	1	1	**1**
0	0	1	1	**0**	1	1	0	**0**
0	1	0	1	**0**	1	0	1	**0**
0	1	1	1	**0**	1	0	0	**0**
1	0	0	1	**0**	0	1	1	**0**
1	0	1	1	**0**	0	1	0	**0**
1	1	0	1	**0**	0	0	1	**0**
1	1	1	1	**0**	0	0	0	**0**

Conclusion for Part (a): Comparing the fifth column, **LHS:** $(x + y + z)'$, and the ninth column, **RHS:** $x'y'z'$, we observe that they contain identical values for all eight rows. **Thus, the identity** $(x + y + z)' = x'y'z'$ **is proven to be valid.**

Part (b): The Distributive Law: $x(y+z) = xy + xz$
The distributive law states that the logical AND operation distributes over the logical OR operation. The table below displays the logical operations to verify this distribution.

x	y	z	$y+z$	**LHS:** $x(y+z)$	xy	xz	**RHS:** $xy+xz$
0	0	0	0	**0**	0	0	**0**
0	0	1	1	**0**	0	0	**0**
0	1	0	1	**0**	0	0	**0**
0	1	1	1	**0**	0	0	**0**
1	0	0	0	**0**	0	0	**0**
1	0	1	1	**1**	0	1	**1**
1	1	0	1	**1**	1	0	**1**
1	1	1	1	**1**	1	1	**1**

Conclusion for Part (b): Comparing the fifth column, **LHS:** $x(y+z)$, and the eighth column, **RHS:** $xy+xz$, we observe that they possess exactly the same truth values for every row configuration. **Thus, the identity $x(y+z) = xy + xz$ is proven to be valid.**

Q2. Given De Morgan's theorem for two variables: $(A+B)' = A'B'$, prove De Morgan's theorem for three variables by successive substitution. **(10 marks)** **[CSE 205 | L-2/T-1 | 2013–14]**

Answer

Proof by Successive Substitution
De Morgan's theorem for three variables (in its NOR form) states that the complement of the logical sum of three variables is equal to the logical product of their individual complements:

$$(A+B+C)' = A'B'C'$$

We are given the foundation of De Morgan's theorem for two variables:

$$(X+Y)' = X'Y'$$

We can prove the three-variable theorem mathematically by temporarily grouping variables and applying the two-variable theorem in succession.

Step-by-Step Derivation:
We start by defining the Left-Hand Side (LHS) of the three-variable theorem:

$$\text{LHS} = (A+B+C)'$$

Step 1: Apply the **Associative Law** of Boolean algebra for the OR operation $(A+B+C = (A+B)+C)$ to group the first two variables together. Let us define a temporary substitute variable $X = A+B$. Substituting X into the expression yields:

$$= (X+C)'$$

Step 2: The expression is now in the form of the complement of two variables (X and C). Apply the given two-variable **De Morgan's theorem** $(X+Y)' = X'Y'$.

$$= X' \cdot C'$$

Step 3: Substitute the original sub-expression $(A+B)$ back in place of the temporary variable X.

$$= (A+B)' \cdot C'$$

Step 4: Apply the two-variable **De Morgan's theorem** a second time, specifically to the first term $(A+B)'$.

$$= (A' \cdot B') \cdot C'$$

Step 5: Finally, apply the **Associative Law** for the logical AND operation to drop the unnecessary parentheses, yielding the Right-Hand Side (RHS).

$$= A' \cdot B' \cdot C'$$
$$= A'B'C'$$

Conclusion: By grouping variables and applying the two-variable De Morgan's theorem successively, we have rigorously proven that $(\mathbf{A+B+C})' = \mathbf{A'B'C'}$.

Extension: The Dual Form (NAND Form)
For absolute completeness in an exam setting, the dual of De Morgan's theorem for three variables states that $(ABC)' = A'+B'+C'$. This can be proven using the exact same successive substitution logic, provided the two-variable dual $(XY)' = X'+Y'$ is established.

$$\begin{aligned}
(ABC)' &= ((AB)C)' && \text{(Associative Law)} \\
&= (AB)' + C' && \text{(Two-variable De Morgan's on } (AB) \text{ and } C) \\
&= (A' + B') + C' && \text{(Two-variable De Morgan's on } AB) \\
&= A' + B' + C' && \text{(Associative Law)}
\end{aligned}$$

Both forms rely purely on the principle of extending a two-variable theorem to n-variables through strategic substitution.

Q3. State and prove De Morgan's theorem using a truth table. **(10 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Answer

Statement of De Morgan's Theorems
De Morgan's Theorems are two fundamental principles in Boolean algebra that relate the logical AND and OR operations through logical complementation (inversion). They are stated as follows:

- **Theorem 1 (NAND to Negative-OR):** The complement of a logical product (AND) of variables is equal to the logical sum (OR) of their individual complements.
$$(\mathbf{A} \cdot \mathbf{B})' = \mathbf{A}' + \mathbf{B}'$$
- **Theorem 2 (NOR to Negative-AND):** The complement of a logical sum (OR) of variables is equal to the logical product (AND) of their individual complements.
$$(\mathbf{A} + \mathbf{B})' = \mathbf{A}' \cdot \mathbf{B}'$$

These theorems can be extended to any number of variables (e.g., $(A + B + C + \dots)' = A'B'C' \dots$).

Proof by Perfect Induction (Truth Table)
To prove a Boolean theorem using a truth table, we must evaluate both the Left-Hand Side (LHS) and the Right-Hand Side (RHS) of the equation for all possible combinations of the input variables. If the column representing the LHS matches the column representing the RHS for every row, the theorem is proven valid.

Proof of Theorem 1: $(A \cdot B)' = A' + B'$

A	B	A'	B'	$A \cdot B$	**LHS:** $(A \cdot B)'$	**RHS:** $A' + B'$
0	0	1	1	0	**1**	**1**
0	1	1	0	0	**1**	**1**
1	0	0	1	0	**1**	**1**
1	1	0	0	1	**0**	**0**

Observation: The values in the column for $(A \cdot B)'$ are identical to the values in the column for $A' + B'$ for all input combinations. Thus, Theorem 1 is proven.

Proof of Theorem 2: $(A + B)' = A' \cdot B'$

A	B	A'	B'	$A + B$	**LHS:** $(A + B)'$	**RHS:** $A' \cdot B'$
0	0	1	1	0	**1**	**1**
0	1	1	0	1	**0**	**0**
1	0	0	1	1	**0**	**0**
1	1	0	0	1	**0**	**0**

Observation: The values in the column for $(A + B)'$ are identical to the values in the column for $A' \cdot B'$ for all input combinations. Thus, Theorem 2 is proven.

Conclusion
Through the exhaustive evaluation of all binary states (perfect induction), both of De Morgan's theorems are structurally verified and proven universally valid.

Logic Gates, Universal Gates & Definitions

Q1. Why are NAND and NOR gates called universal gates? Explain how hardware cost is related to Boolean simplification. If a function is already minimal, how can you verify it? **(10 marks)** **[CSE 205 | L-2/T-1 | 2024–25]**

Answer

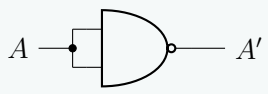
1. Universality of NAND and NOR Gates

NAND and NOR gates are designated as **universal gates** because either one can be used exclusively to implement *any* Boolean function, without requiring any other type of logic gate.

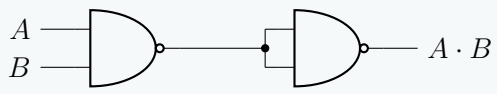
Any Boolean expression can be written in standard forms such as Sum-of-Products (SOP) or Product-of-Sums (POS), which fundamentally only require NOT, AND, and OR operations. Because the NOT, AND, and OR gates can be constructed entirely out of NAND gates (or entirely out of NOR gates), it follows that any digital circuit can be implemented using only NAND or only NOR gates.

To demonstrate this universality, the realization of the basic gates using only **NAND gates** is shown below:

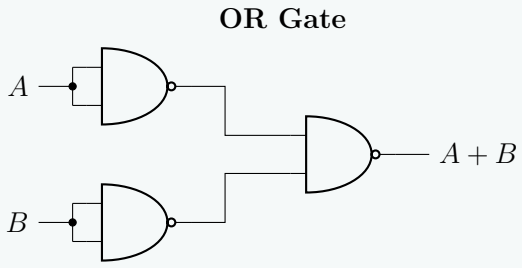
NOT Gate (Inverter)



AND Gate



OR Gate



2. Relationship Between Hardware Cost and Boolean Simplification

In digital logic design, the mathematical structure of a Boolean expression directly correlates to the physical structure of its corresponding hardware circuit. Simplifying a Boolean expression reduces the hardware cost in several critical ways:

- **Number of Terms:** In an SOP expression, each product term represents an AND gate. By minimizing the number of terms using Boolean algebra or K-maps, we directly reduce the *total number of logic gates* required on the silicon chip or Printed Circuit Board (PCB).
- **Number of Literals:** A literal is a primed or unprimed variable in a term (e.g., $A'B$ has two literals). The number of literals in a term dictates the *number of inputs* required for the corresponding gate. Minimizing literals reduces gate complexity, allowing the use of cheaper, standard 2-input or 3-input gates rather than custom multi-input gates.
- **Physical Footprint and Power:** Fewer gates and fewer inputs translate to fewer transistors. This reduces the physical area of the Integrated Circuit (IC), lowers static and dynamic power consumption, and decreases heat dissipation.
- **Propagation Delay:** Simplification often reduces the number of logical levels a signal must traverse, which decreases the overall propagation delay and improves circuit operational speed.

3. Verifying the Minimality of a Boolean Function

If a Boolean function is believed to be minimal, its minimality can be systematically verified using one of the following methods:

(a) **Karnaugh Map (K-map) Inspection (For up to 4-5 variables):**

- **Check for Prime Implicants:** Ensure that every grouping of 1s (for SOP) or 0s (for POS) is as large as possible (sizes of 1, 2, 4, 8, etc.). If any group can be expanded by combining it with adjacent groups, the function is not yet minimal.
- **Check for Essential Prime Implicants:** Ensure all selected groups are necessary. If all the 1s in a specific group are already covered by other necessary groups, that group is redundant. A strictly minimal expression contains only essential prime implicants and the minimum number of non-essential prime implicants required to cover any remaining 1s.

(b) **Quine-McCluskey (Tabulation) Method (For any number of variables):**

- This is a rigorous algorithmic approach. You construct a prime implicant chart. If the given expression exactly matches the optimal selection of prime implicants derived from solving the chart (ensuring all minterms are covered with the lowest literal/term count), the expression is proven minimal. Since it is algorithmic, it provides an absolute mathematical proof of minimality, suitable for computer-aided verification.

Q2. How does a three-state gate work? What can be the advantage of using a three-state gate? **(6 marks)** **[CSE 205 | L-2/T-1 | 2024–25]**

Answer

Operation of a Three-State Gate

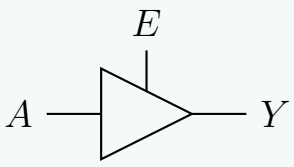
A standard digital logic gate outputs one of two binary states: logic Low (0) or logic High (1). A **three-state gate** (or tri-state

gate), however, exhibits a third stable state called the **High-Impedance state**, denoted by **Z**.
A three-state gate features two primary inputs: a standard **Data Input** (A) and a **Control/Enable Input** (E). The operation depends entirely on the condition of the enable signal:

(a) **Enabled State ($E = 1$ for active-high):** The gate is fully functional and acts as a standard logic buffer. The output follows the input directly ($Y = A$). The gate actively drives the output line to either a low or high voltage.

(b) **Disabled/High-Impedance State ($E = 0$ for active-high):** The output is placed into the high-impedance state (Z). In this state, the internal output transistors are turned off simultaneously, effectively creating an **electrical open circuit**. The gate is disconnected from its output pin, meaning it neither sinks nor sources current, allowing external circuits to dictate the voltage level of the shared line.

Logic Symbol and Truth Table
Below are the schematic representation and the corresponding functional truth table for an active-high three-state buffer:

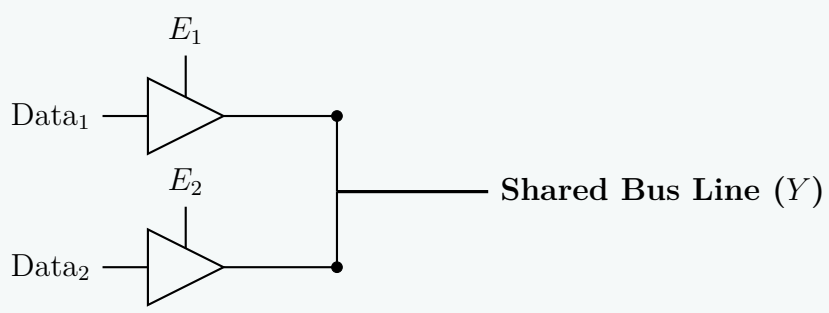


Enable (E)	Input (A)	Output (Y)	Operating State
0	X (Don't Care)	**Z**	High-Impedance (Disconnected)
1	0	**0**	Logic Low (Active Drive)
1	1	**1**	Logic High (Active Drive)

Advantages of Using Three-State Gates
Three-state logic elements provide several critical structural and systemic benefits in modern digital architecture:

- **Implementation of Shared Bus Architectures:** The foremost advantage of tri-state gates is their ability to connect multiple gate outputs to a single, common wire or communication bus. Under normal circumstances, connecting two standard gate outputs directly together is forbidden because if one outputs 1 (high voltage) and the other outputs 0 (low voltage), a damaging short-circuit occurs (known as bus contention). By ensuring that the control logic enables **only one tri-state gate at a time** while keeping all others disabled (Z), multiple components can safely share the same channel.
- **Bidirectional Data Transmission:** Tri-state gates allow a single physical pin or line to function both as an input and an output. This bidirectional capability is foundational for microprocessor data buses and memory interfaces (RAM/ROM modules), where data must be written to and read from the same location over the same wiring infrastructure.
- **Reduction in Routing Overhead and Area Optimization:** Without tri-state logic, multiplexing data from N different registers would require large, multi-stage combinational multiplexing logic networks consisting of dozens of AND-OR gates. Tri-state gates allow multiplexing to occur right at the source directly onto the wire, eliminating excessive gating logic and drastically lowering routing complexity on modern silicon chips.

Visualizing a Tri-State Shared Bus System
The circuit diagram below demonstrates how two independent modules can safely drive a single shared bus line using mutual exclusion on their enable signals ($E_1 \cdot E_2 = 0$):



Q3. What do you mean by Universal gate? Show that both NAND gate and NOR gate are universal gates. **(14 marks)** **[CSE 205 | L-2/T-1 | 2022–23, 2013–14]**

↪ *Similar: 2019–20*

Answer

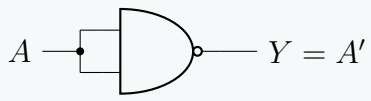
1. Definition of a Universal Gate
A **Universal Gate** is a logic gate that can be used to implement any Boolean function without requiring the use of any other type of logic gate. In digital logic design, any Boolean expression can be realized using a combination of the three fundamental logic gates: **NOT**, **AND**, and **OR**.
Therefore, if we can demonstrate that a specific gate can successfully implement the NOT, AND, and OR operations purely by connecting multiple instances of itself, we prove that it is a universal gate. The two standard universal gates are the **NAND** gate and the **NOR** gate.

2. Proof: NAND as a Universal Gate

The NAND gate produces the complement of the logical product of its inputs: $Y = (A \cdot B)'$. We can construct the basic gates as follows:

a) Implementing a NOT Gate using a NAND Gate To invert a signal, we apply the same input signal A to all inputs of the NAND gate. According to the *Idempotent Law* $(A \cdot A = A)$, the operation simplifies to the logical complement.

$$Y = (A \cdot A)'$$
$$Y = A'$$

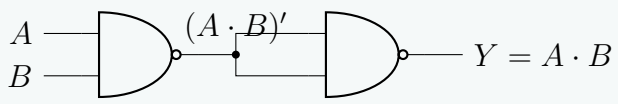


b) Implementing an AND Gate using NAND Gates An AND operation is simply a NAND operation followed by a NOT operation. Since we already know how to create a NOT gate using a NAND gate, we cascade two NAND gates.

$$Y = ((A \cdot B)')'$$

Using the *Involution Law* $(X'') = X$:

$$Y = A \cdot B$$

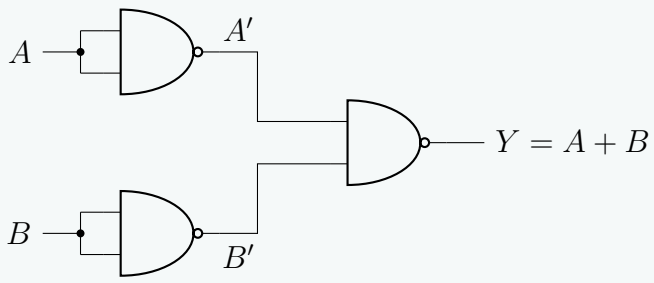


c) Implementing an OR Gate using NAND Gates To perform an OR operation, we first invert both inputs using NAND-based NOT gates, and then feed those inverted signals into a third NAND gate. We apply *De Morgan's Theorem* to prove the equivalence.

$$Y = (A' \cdot B')'$$

Applying De Morgan's Law $(X \cdot Y)' = X' + Y'$:

$$Y = (A')' + (B')'$$
$$Y = A + B$$



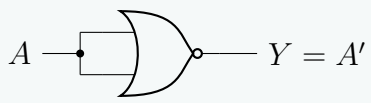
---

3. Proof: NOR as a Universal Gate

The NOR gate produces the complement of the logical sum of its inputs: $Y = (A + B)'$. The proofs follow a symmetric logic to the NAND gate implementations.

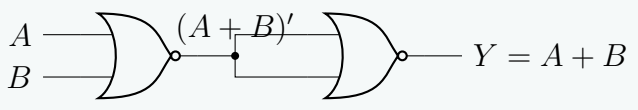
a) Implementing a NOT Gate using a NOR Gate Applying the same input to all terminals of a NOR gate acts as an inverter. According to the *Idempotent Law* $(A + A = A)$.

$$Y = (A + A)'$$
$$Y = A'$$



b) Implementing an OR Gate using NOR Gates An OR operation is a NOR operation followed by an inversion (NOT gate created from a NOR gate).

$$Y = ((A + B)')'$$
$$Y = A + B$$

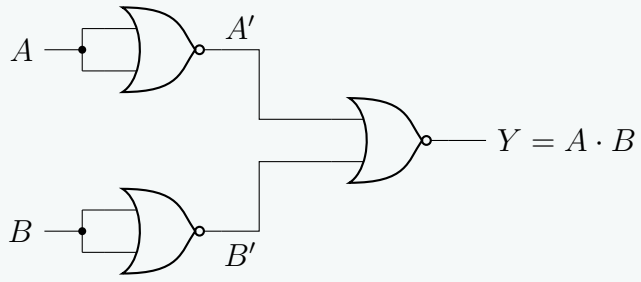


c) Implementing an AND Gate using NOR Gates To perform an AND operation, we invert both inputs using NOR gates, and then supply them to a third NOR gate. We prove this mathematically using *De Morgan's Theorem*.

$$Y = (A' + B')'$$

Applying De Morgan's Law $(X + Y)' = X' \cdot Y'$:

$$Y = (A')' \cdot (B')'$$
$$Y = A \cdot B$$



Conclusion: Because we have successfully mapped the NOT, AND, and OR functional operators entirely to NAND gates and NOR gates, both are formally validated as universal logic gates.

Q4. Show the circuit diagram of the expression $AB + CDE$ using only NAND gates. **(10 marks)** **[CSE 205 | L-2/T-1 | 2021–22]**

Answer

1. Mathematical Derivation

To implement a given Sum-of-Products (SOP) expression using only NAND gates, we must transform the expression into a form that exclusively uses the NAND operation $((X \cdot Y)')$. We achieve this by applying double negation and De Morgan's laws.
Let the output function be Y:

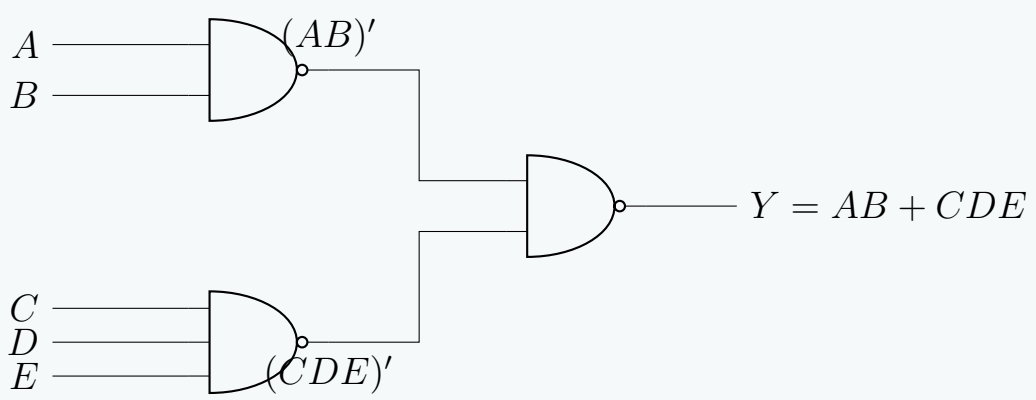
$$Y = AB + CDE \qquad \text{(Original SOP expression)}$$
$$Y = ((AB + CDE)')' \qquad \text{(Applying the Involution Law: } X = (X')')$$
$$Y = ((AB)' \cdot (CDE)')' \qquad \text{(Applying De Morgan's Law: } (X + Y)' = X' \cdot Y')$$

Circuit Analysis: The resulting equation $Y = ((AB)' \cdot (CDE)')'$ clearly defines a two-level NAND-NAND implementation:

- **Level 1:** Two NAND gates generate the terms $(AB)'$ and $(CDE)'$. We will use a 2-input NAND gate for AB and a 3-input NAND gate for CDE.
- **Level 2:** A final 2-input NAND gate takes the outputs from Level 1 and produces the target expression.

2. Logic Circuit Diagram

The circuit diagram below demonstrates the realization of the Boolean expression using standard 2-input and 3-input NAND gates.



Note: If the design constraints strictly required only 2-input NAND gates, the 3-input NAND gate $(CDE)'$ would be expanded. We would first compute $(CD)'$, invert it using a NAND gate acting as a NOT gate to get CD, and then feed CD and E into another 2-input NAND gate to produce $(CDE)'$. However, 3-input NAND gates are standard components (e.g., the 7410 IC), so utilizing one directly provides the most optimal and generalized solution.

Q5. A majority circuit is a combinational circuit whose output is equal to 1 if the input variables have more 1's than 0's, and 0 otherwise. Design a 3-input majority circuit by finding the circuit's truth table, simplified Boolean equation, and a logic diagram. Use only NAND gate(s) for your design. **(18 marks)** **[CSE 205 | L-2/T-1 | 2019–20]**

Answer

Design of a 3-Input Majority Circuit

A majority circuit output is equal to 1 if the majority of its inputs are 1. For a 3-input combinational circuit with inputs A, B, and C, a majority occurs when two or more inputs are high (≥ 2).

—

1. Truth Table

Based on the design criteria, the output F is high for the minterms where the binary count of 1s is greater than or equal to 2. This corresponds to the minterms m_3, m_5, m_6, and m_7.

Inputs			Output	
A	B	C	F	**Minterm**
0	0	0	0	m_0
0	0	1	0	m_1
0	1	0	0	m_2
0	1	1	1	$m_3 = \overline{A}BC$
1	0	0	0	m_4
1	0	1	1	$m_5 = A\overline{B}C$
1	1	0	1	$m_6 = AB\overline{C}$
1	1	1	1	$m_7 = ABC$

—

2. Boolean Simplification (Karnaugh Map)

To find the simplified Sum-of-Products (SOP) expression, we map the 1s into a 3-variable Karnaugh Map:

A \ BC	00	01	11	10
0	0	0	1	0
1	0	1	1	1

From the loops grouped above, we obtain three essential prime implicants:

- Vertical loop grouping m_3 and m_7: BC
- Horizontal loop grouping m_5 and m_7: AC
- Bottom-right loop grouping m_6 and m_7: AB

Thus, the minimal SOP expression is:

$$F = AB + BC + AC$$

—

3. Conversion to NAND-Only Logic

To implement the circuit using exclusively NAND gates, we apply the Double Negation Law followed by De Morgan's Theorem to transform the SOP expression into a Product-of-Inversions format:

$$\begin{aligned} F &= AB + BC + AC \\ &= \overline{\overline{AB + BC + AC}} && \text{(Double Negation Law)} \\ &= \overline{\overline{AB} \cdot \overline{BC} \cdot \overline{AC}} && \text{(De Morgan's Law)} \end{aligned}$$

This final expression indicates that the circuit can be implemented using:

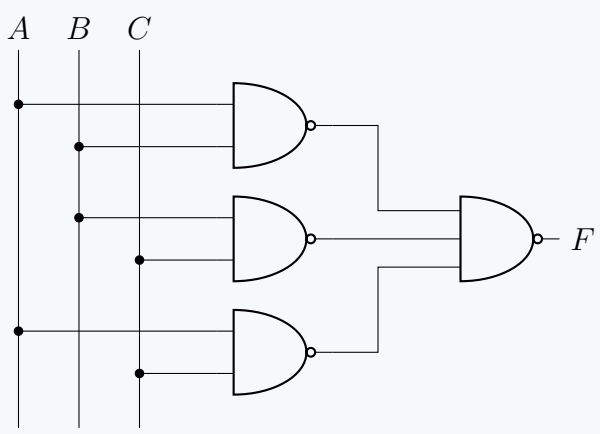
(a) Three 2-input NAND gates to generate $\overline{AB}$, $\overline{BC}$, and $\overline{AC}$.

(b) One 3-input NAND gate to combine the intermediate outputs.

—

4. Logic Diagram

The professional schematic below illustrates the NAND-only gate realization of the 3-input majority circuit.



Q6. Show that NAND gate is a universal logic gate. Also show that NAND operation does not follow the associative law. **(5+5 marks)**
[CSE 205 | L-2/T-1 | 2015–16]

Answer

Part 1: Universality of the NAND Gate

A logic gate is considered **universal** if it can be used to implement the three fundamental Boolean operations: **NOT**, **AND**, and **OR**. By demonstrating that a NAND gate can synthesize these three basic gates, we prove its universality.

1. Implementing a NOT Gate

A NOT gate (inverter) takes a single input A and produces its complement $\overline{A}$. To achieve this using a 2-input NAND gate, we tie both inputs together.

Boolean Derivation:

$$
\begin{aligned}
F &= \text{NAND}(A, A) \\
F &= \overline{A \cdot A} && \text{(Definition of NAND)} \\
F &= \overline{A} && \text{(Idempotent Law: } A \cdot A = A\text{)}
\end{aligned}
$$

Logic Diagram:

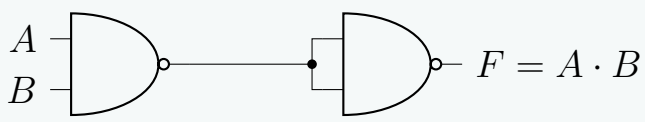
2. Implementing an AND Gate

An AND gate performs the Boolean product $A \cdot B$. Since a NAND gate produces $\overline{A \cdot B}$, we simply need to invert its output using a second NAND gate configured as a NOT gate.

Boolean Derivation:

$$
\begin{aligned}
F &= \text{NOT}(\text{NAND}(A, B)) \\
F &= \overline{(\overline{A \cdot B})} && \text{(Definition of NAND)} \\
F &= A \cdot B && \text{(Double Negation Law)}
\end{aligned}
$$

Logic Diagram:



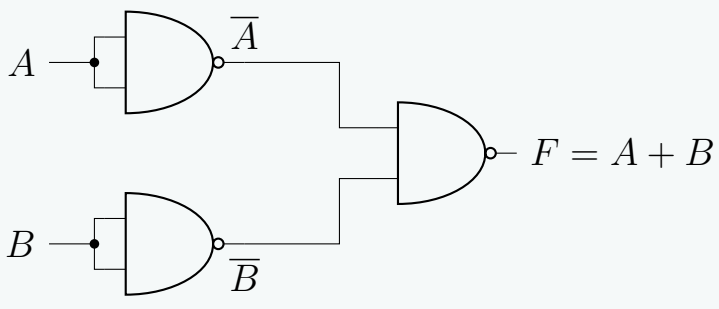
3. Implementing an OR Gate

An OR gate performs the Boolean sum $A + B$. We can implement this by first inverting both inputs using NAND gates, and then passing those inverted signals into a third NAND gate.

Boolean Derivation:

$$
\begin{aligned}
F &= \text{NAND}(\text{NOT}(A), \text{NOT}(B)) \\
F &= \text{NAND}(\overline{A}, \overline{B}) && \text{(From NOT implementation)} \\
F &= \overline{\overline{A} \cdot \overline{B}} && \text{(Definition of NAND)} \\
F &= \overline{\overline{A}} + \overline{\overline{B}} && \text{(De Morgan's Law)} \\
F &= A + B && \text{(Double Negation Law)}
\end{aligned}
$$

Logic Diagram:



Because the NAND gate can implement NOT, AND, and OR functions, it is formally proven to be a **universal gate**.

Part 2: Non-Associativity of the NAND Operation

The associative law states that for a given binary operator $\uparrow$, the grouping of operands does not affect the result:

$$(A \uparrow B) \uparrow C = A \uparrow (B \uparrow C)$$

Let the operator $\uparrow$ denote the NAND operation, where $A \uparrow B = \overline{A \cdot B}$. We will evaluate the Left-Hand Side (LHS) and Right-Hand Side (RHS) of the associative equation independently.

Evaluating the Left-Hand Side (LHS):

$$\begin{aligned}
\text{LHS} &= (A \uparrow B) \uparrow C \\
&= (\overline{A \cdot B}) \uparrow C && \text{(Definition of NAND)} \\
&= \overline{(\overline{A \cdot B}) \cdot C} && \text{(Definition of NAND)} \\
&= \overline{(\overline{A \cdot B})} + \overline{C} && \text{(De Morgan's Law)} \\
&= (A \cdot B) + \overline{C} && \text{(Double Negation Law)}
\end{aligned}$$

Evaluating the Right-Hand Side (RHS):

$$\begin{aligned}
\text{RHS} &= A \uparrow (B \uparrow C) \\
&= A \uparrow (\overline{B \cdot C}) && \text{(Definition of NAND)} \\
&= \overline{A \cdot (\overline{B \cdot C})} && \text{(Definition of NAND)} \\
&= \overline{A} + \overline{(\overline{B \cdot C})} && \text{(De Morgan's Law)} \\
&= \overline{A} + (B \cdot C) && \text{(Double Negation Law)}
\end{aligned}$$

Comparing the simplified expressions, we clearly see that:

$$(A \cdot B) + \overline{C} \neq \overline{A} + (B \cdot C)$$

Counterexample via Truth Values: To definitively prove the inequality, we can assign an arbitrary set of values, such as $A = 0$, $B = 0$, and $C = 1$.

- **LHS:** $((0 \cdot 0) + \overline{1}) = (0 + 0) = \mathbf{0}$
- **RHS:** $(\overline{0} + (0 \cdot 1)) = (1 + 0) = \mathbf{1}$

Since $0 \neq 1$, the LHS and RHS do not yield the same result for the same input combination. Therefore, **the NAND operation does not follow the associative law.**

Q7. Define: Odd Function, Even Function, Don't-care conditions. **(5 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Answer

Definitions of Logic Function Properties

1. Odd Function

In digital logic, an **odd function** is a Boolean function that evaluates to logic 1 if and only if an **odd number** of its input variables are equal to logic 1.

- **Operation:** The multiple-input exclusive-OR (XOR) operation inherently implements an odd function.
- **Example (3-Variable):** $F_{\text{odd}}(A, B, C) = A \oplus B \oplus C = \sum m(1, 2, 4, 7)$

2. Even Function

An **even function** is a Boolean function that evaluates to logic 1 if and only if an **even number** of its input variables are equal to logic 1. In binary logic, zero is considered an even number, so the function also outputs 1 when all inputs are 0.

- **Operation:** For a 3-variable system, the complement of the XOR function (which is the exclusive-NOR, or XNOR, evaluated sequentially) yields an even function.
- **Example (3-Variable):** $F_{\text{even}}(A, B, C) = (A \oplus B \oplus C)' = \sum m(0, 3, 5, 6)$

Truth Table Comparison (3 Variables):

Inputs			Count	Odd Function	Even Function	
A	B	C	of 1s	$F = A \oplus B \oplus C$	$F = (A \oplus B \oplus C)'$	Minterm
0	0	0	0	**0**	**1**	m_0
0	0	1	1	**1**	**0**	m_1
0	1	0	1	**1**	**0**	m_2
0	1	1	2	**0**	**1**	m_3
1	0	0	1	**1**	**0**	m_4
1	0	1	2	**0**	**1**	m_5
1	1	0	2	**0**	**1**	m_6
1	1	1	3	**1**	**0**	m_7

3. Don't-Care Conditions

In certain digital systems, specific input combinations may either never physically occur (e.g., states 1010 to 1111 in a BCD, Binary-Coded Decimal, system) or the system's output for those specific input combinations does not affect the overall circuit behavior. The unassigned output states for these combinations are defined as **don't-care conditions**.

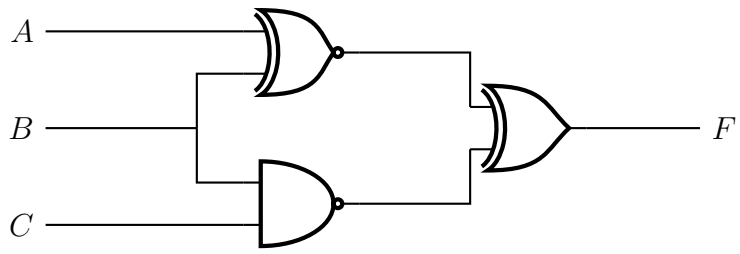
- **Notation:** They are typically denoted by **X**, **d**, or Φ in truth tables and Boolean expressions. For example: $F(A, B, C) = \sum m(0, 1) + \sum d(2, 3)$.
- **Application in Minimization:** During logic simplification (e.g., using Karnaugh Maps or the Quine-McCluskey algorithm), a designer can arbitrarily treat a don't-care condition as either a 1 or a 0. The strategic choice is to assign them the value that leads to the largest possible groupings (implicants), thereby yielding the most minimized and cost-effective logic circuit.

Example of K-Map Optimization using Don't-Cares: Consider $F(A, B, C) = \sum m(0, 1) + \sum d(2, 3)$. Without don't-cares, m_0 and m_1 group to form $\overline{A}B$. By treating the don't-cares (d_2, d_3) as 1s, we can form a quad, simplifying the expression significantly to $F = \overline{A}$.

		BC = 00	01	11	10
A	0	1	1	-	-
	1	0	0	0	0

Circuit Analysis

Q1. Analyze the following circuit and determine the output F as sum of minterms and product of maxterms.



(9 marks) [CSE 205 | L-2/T-1 | 2011–12]

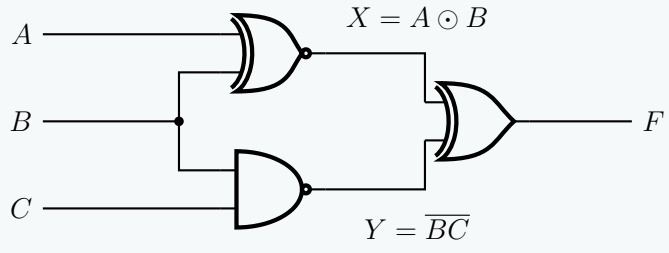
Answer

Circuit Analysis: Sum of Minterms and Product of Maxterms

To determine the output F in its canonical forms, we will first extract the Boolean expression from the logic circuit, simplify it algebraically, and verify the results using a Truth Table.

1. Boolean Expression Extraction

Let us define the intermediate signals at the outputs of the first-stage gates as X and Y.



From the circuit diagram, we can write the equations for the intermediate nodes and the final output:

- Top Gate (XNOR): $X = A \odot B$
- Bottom Gate (NAND): $Y = \overline{BC}$
- Output Gate (XOR): $F = X \oplus Y$

2. Algebraic Derivation

Substitute X and Y into the XOR expression. By definition, $X \oplus Y = X\overline{Y} + \overline{X}Y$.
First, find the complements $\overline{X}$ and $\overline{Y}$:

$$\overline{X} = \overline{A \odot B} = A \oplus B = A\overline{B} + \overline{A}B$$

$$\overline{Y} = \overline{\overline{BC}} = BC \qquad \text{(Double Negation Law)}$$

Now, substitute X, $\overline{X}$, Y, and $\overline{Y}$ into the expression for F:

$$F = (A \odot B)(BC) + (A \oplus B)(\overline{BC})$$
$$F = (AB + \overline{A}\overline{B})(BC) + (A\overline{B} + \overline{A}B)(\overline{B} + \overline{C}) \qquad \text{(De Morgan's Law on } \overline{BC})$$

Expand both terms using the Distributive Law:

$$\begin{aligned} \text{Term 1: } & (AB)(BC) + (\overline{A}\overline{B})(BC) \\ &= ABC + \overline{A}(\overline{B}B)C && \text{(Idempotent Law: } BB = B) \\ &= ABC + 0 && \text{(Complement Law: } \overline{B}B = 0) \\ &= ABC \end{aligned}$$

$$\begin{aligned} \text{Term 2: } & A\overline{B}\overline{B} + A\overline{B}\overline{C} + \overline{A}B\overline{B} + \overline{A}B\overline{C} \\ &= A\overline{B} + A\overline{B}\overline{C} + 0 + \overline{A}B\overline{C} && \text{(Idempotent } \overline{B}\overline{B} = \overline{B}\text{, Complement } B\overline{B} = 0) \\ &= A\overline{B}(1 + \overline{C}) + \overline{A}B\overline{C} && \text{(Factoring } A\overline{B}) \\ &= A\overline{B}(1) + \overline{A}B\overline{C} && \text{(Identity Law: } 1 + X = 1) \\ &= A\overline{B} + \overline{A}B\overline{C} \end{aligned}$$

Combine the simplified terms:

$$F = ABC + A\overline{B} + \overline{A}B\overline{C}$$

To express F as a sum of minterms, we expand the incomplete term $(A\overline{B})$ by multiplying it by $(C + \overline{C})$:

$$F = ABC + A\overline{B}(C + \overline{C}) + \overline{A}B\overline{C}$$
$$F = ABC + A\overline{B}C + A\overline{B}\overline{C} + \overline{A}B\overline{C}$$

Translating the boolean products to their binary equivalents (where uncomplemented = 1, complemented = 0):

- $ABC \rightarrow 111 = m_7$
- $A\overline{B}C \rightarrow 101 = m_5$
- $A\overline{B}\overline{C} \rightarrow 100 = m_4$
- $\overline{A}B\overline{C} \rightarrow 010 = m_2$

3. Truth Table Verification

We construct a truth table to double-check the logic levels for all 8 combinations of A, B, C.

Inputs			**XNOR**	**NAND**	**Output**	**Canonical Forms**	
A	B	C	$X = A \odot B$	$Y = \overline{BC}$	$F = X \oplus Y$	**Minterm**	**Maxterm**
0	0	0	1	1	**0**		M_0
0	0	1	1	1	**0**		M_1
0	1	0	0	1	**1**	m_2	
0	1	1	0	0	**0**		M_3
1	0	0	0	1	**1**	m_4	
1	0	1	0	1	**1**	m_5	
1	1	0	1	1	**0**		M_6
1	1	1	1	0	**1**	m_7	

4. Final Answer

From both the algebraic derivation and the truth table, the function equals 1 for minterms 2, 4, 5, and 7, and equals 0 for maxterms 0, 1, 3, and 6.

Sum of Minterms (Σm):

$$F(A, B, C) = \sum m(2, 4, 5, 7)$$

Product of Maxterms (ΠM):

$$F(A, B, C) = \prod M(0, 1, 3, 6)$$

Two-Level & Multi-Level Logic Implementation

Q1. Simplify the following function and draw the circuit using only NAND gates:

$$F(w, x, y, z) = \Sigma\,(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$

(16 marks) [CSE 205 | L-2/T-1 | 2023–24]

Q2. Implement $F(x, y, z) = x'y'z' + xyz'$ using two-level forms of logic by using NAND-AND gates. Assume that the complement of x, y and z are readily available. **(14 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Q3. Express $f(A, B, C, D) = \Sigma\,(0, 4, 8, 9, 10, 11, 12, 14)$ with the two-level forms of logic "NAND-AND" and "OR-NAND". **(6 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

Two-Level Logic Design: NAND-AND and OR-NAND Forms

To implement a logic function in specific two-level forms, we must first derive the simplified Boolean expressions for the function f and its complement f'. We achieve this using Karnaugh Maps.
The given Boolean function is:

$$f(A, B, C, D) = \Sigma\,(0, 4, 8, 9, 10, 11, 12, 14)$$

The unlisted minterms form the complement function f':

$$f'(A, B, C, D) = \Sigma\,(1, 2, 3, 5, 6, 7, 13, 15)$$

—

1. Karnaugh Map Simplification

Minimizing f (Sum of Products for f): We plot the minterms of f on a 4-variable K-Map to find the minimal Sum of Products (SOP).

AB \ CD	00	01	11	10
00	1	0	0	0
01	1	0	0	0
11	1	0	0	1
10	1	1	1	1

From the groupings above, we extract three essential prime implicants:

- Left column group $(0, 4, 12, 8)$: $C'D'$
- Bottom row group $(8, 9, 11, 10)$: AB'
- Wrapped edge quad $(12, 14, 8, 10)$: AD'

Thus, the simplified SOP expression for f is:

$$f = C'D' + AB' + AD' \tag{1}$$

Minimizing f' (Sum of Products for f'): We plot the minterms of f' to find its minimal SOP.

AB \ CD	00	01	11	10
00	0	1	1	1
01	0	1	1	1
11	0	1	1	0
10	0	0	0	0

From the K-Map for f', we obtain:

- Group $(1, 3, 5, 7)$: $A'D$
- Group $(2, 3, 6, 7)$: $A'C$

- Group $(5, 7, 13, 15)$: BD

Thus, the simplified SOP expression for f' is:

$$f' = A'D + A'C + BD \tag{2}$$

—

2. NAND-AND Logic Form

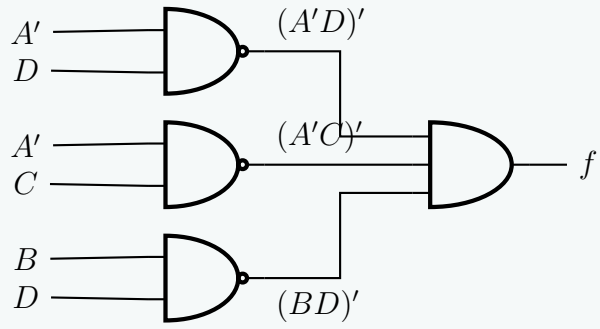
The **NAND-AND** form requires the logic equation to be a product of inverted terms (First Level: NAND, Second Level: AND). We derive this naturally from the SOP of the complemented function f'.
By taking the complement of both sides of Equation (2):

$$f = (f')'$$
$$f = (A'D + A'C + BD)'$$

Applying **De Morgan's Law** to distribute the inversion over the OR terms:

$$f = (A'D)' \cdot (A'C)' \cdot (BD)'$$

This equation maps directly to a two-level NAND-AND circuit:



—

3. OR-NAND Logic Form

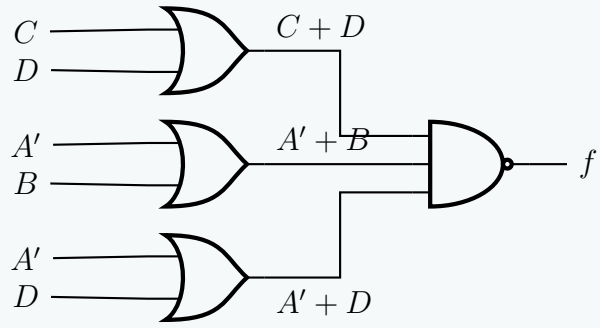
The **OR-NAND** form requires the logic equation to be an inverted product of sums (First Level: OR, Second Level: NAND). This is obtained by expressing f' as a Product of Sums (POS), and then taking its complement.
The POS of f' is the logical complement of the SOP of f (Equation 1). Applying **De Morgan's Law** to Equation (1):

$$f' = (C'D' + AB' + AD')'$$
$$f' = (C + D) \cdot (A' + B) \cdot (A' + D)$$

To find f, we complement f' again:

$$f = (f')'$$
$$f = \big((C + D) \cdot (A' + B) \cdot (A' + D)\big)'$$

This equation translates perfectly into a two-level OR-NAND circuit:



Q4. Implement the function $F(w, x, y, z) = w'x' + w'x'z + w'yz'$ using two-level forms of logic by

(a) NOR gates only

(b) AND-NOR

(c) NAND gates only

(d) OR-NAND

(20 marks) **[CSE 205 | L-2/T-1 | 2014–15]**

Answer

Simplification and Two-Level Logic Implementations

First, we simplify the given Boolean function $F(w, x, y, z)$ to its minimal Sum of Products (SOP) form. This will serve as the foundation for our derivations.

$$
\begin{aligned}
F &= w'x' + w'x'z + w'yz' \\
F &= w'x'(1 + z) + w'yz' && \text{(Distributive Law)} \\
F &= w'x'(1) + w'yz' && \text{(Null Element Law: } 1 + X = 1\text{)} \\
F &= w'x' + w'yz' && \text{(Identity Law: } X \cdot 1 = X\text{)}
\end{aligned}
$$

---

1. NOR Gates Only (NOR-NOR Form)

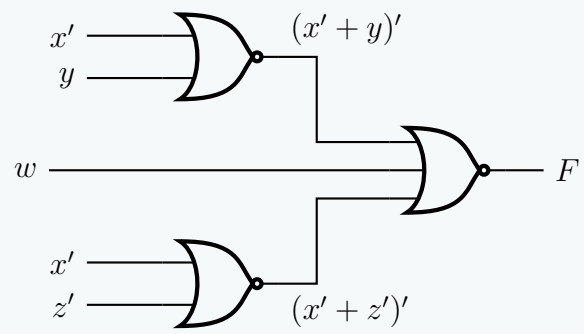
A two-level NOR-NOR circuit requires the function to be in the form of a complemented sum of negated sums. This is derived from the Product of Sums (POS) form of F.

Step 1: Find the POS of F

$$
\begin{aligned}
F &= w'x' + w'yz' \\
F &= w'(x' + yz') && \text{(Distributive Law)} \\
F &= w'(x' + y)(x' + z') && \text{(Distributive Law: } A + BC = (A + B)(A + C)\text{)}
\end{aligned}
$$

Step 2: Convert to NOR-NOR logic

$$
\begin{aligned}
F &= \overline{\overline{w'(x' + y)(x' + z')}} && \text{(Double Negation Law)} \\
F &= \overline{w + (x' + y)' + (x' + z')'} && \text{(De Morgan's Law)}
\end{aligned}
$$



---

2. AND-NOR Form

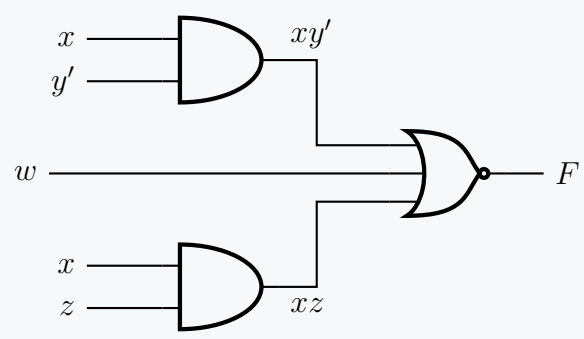
A two-level AND-NOR circuit is structurally equivalent to the complement of the SOP of F'. Therefore, we first find the SOP of the complemented function F'.

Step 1: Find the SOP of F'

$$
\begin{aligned}
F' &= (w'(x' + y)(x' + z'))' && \text{(Complementing the POS of } F\text{)} \\
F' &= w + (x' + y)' + (x' + z')' && \text{(De Morgan's Law)} \\
F' &= w + xy' + xz && \text{(De Morgan's Law and Double Negation)}
\end{aligned}
$$

Step 2: Express F as an AND-NOR equation

$$
\begin{aligned}
F &= (F')' \\
F &= \overline{w + xy' + xz} && \text{(Substitution)}
\end{aligned}
$$



---

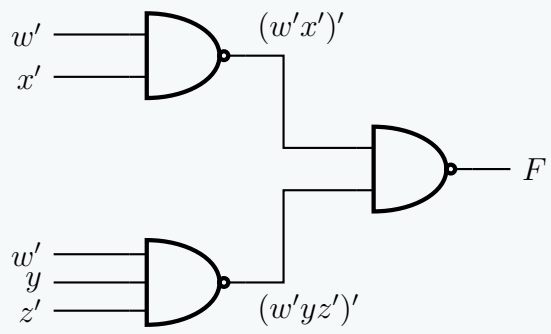
3. NAND Gates Only (NAND-NAND Form)

A two-level NAND-NAND circuit translates directly from the SOP form of F.
Step 1: Start with SOP of *F*

$$F = w'x' + w'yz'$$

Step 2: Convert to NAND-NAND logic

$$F = \overline{\overline{w'x' + w'yz'}} \qquad \text{(Double Negation Law)}$$
$$F = \overline{(w'x')' \cdot (w'yz')'} \qquad \text{(De Morgan's Law)}$$



---

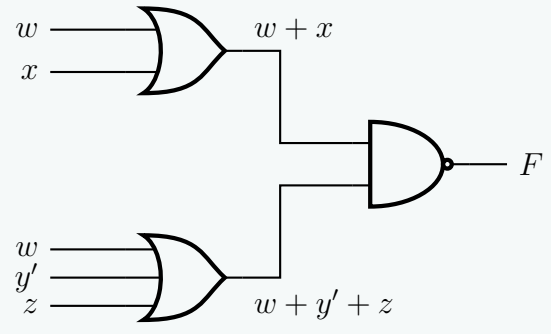
4. OR-NAND Form

A two-level OR-NAND circuit corresponds to the inversion of the POS form of F'. We must derive the POS of the complement function.
Step 1: Find the POS of *F′* Using the SOP of F' determined previously ($F' = w + xy' + xz$):

$$F' = w + x(y' + z) \qquad \text{(Distributive Law)}$$
$$F' = (w + x)(w + y' + z) \qquad \text{(Distributive Law: } A + BC = (A + B)(A + C)\text{)}$$

Step 2: Express *F* as an OR-NAND equation

$$F = (F')'$$
$$F = \overline{(w + x)(w + y' + z)} \qquad \text{(Substitution)}$$



Q5. Show the truth table for the following function:

$$F(x, y, z) = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$$

(8 marks) [CSE 205 | L-2/T-1 | 2014–15]

Answer

Truth Table Derivation

The given Boolean function is presented in its canonical **Product of Sums (POS)** form:

$$F(x, y, z) = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$$

In a POS expression, each sum term is called a **maxterm** (M_i). A maxterm evaluates to logic 0 for exactly one combination of the input variables. To determine the binary value of each maxterm, we map uncomplemented variables to 0 and complemented variables to 1.
Let us evaluate each term to identify its corresponding row in the truth table where the function outputs a 0:

$$(x + y + z) \implies 000_2 \implies M_0$$
$$(x + y + z') \implies 001_2 \implies M_1$$
$$(x + y' + z) \implies 010_2 \implies M_2$$
$$(x' + y + z) \implies 100_2 \implies M_4$$

Thus, the function can be compactly written using the Pi-notation for maxterms:

$$F(x, y, z) = \Pi\,(0, 1, 2, 4)$$

This analysis establishes that the function F will output **0** for the decimal input combinations 0, 1, 2, and 4. For all remaining input combinations (3, 5, 6, and 7), the function will output **1**.

Truth Table

Constructing the table based on our derived maxterms:

Decimal (i)	x	y	z	$F(x,y,z)$
0	0	0	0	**0**
1	0	0	1	**0**
2	0	1	0	**0**
3	0	1	1	1
4	1	0	0	**0**
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

Q6. Given function $F = A'B + BC + AC$:

(a) Implement F using AND, OR and NOT gates.

(b) Minimize F in SOP form using K-Map.

(c) Implement the minimized F using only NAND gates.

(15 marks) **[CSE 205 | L-2/T-1 | 2013–14]**

Answer

Logic Circuit Design and Minimization

Given the Boolean function:

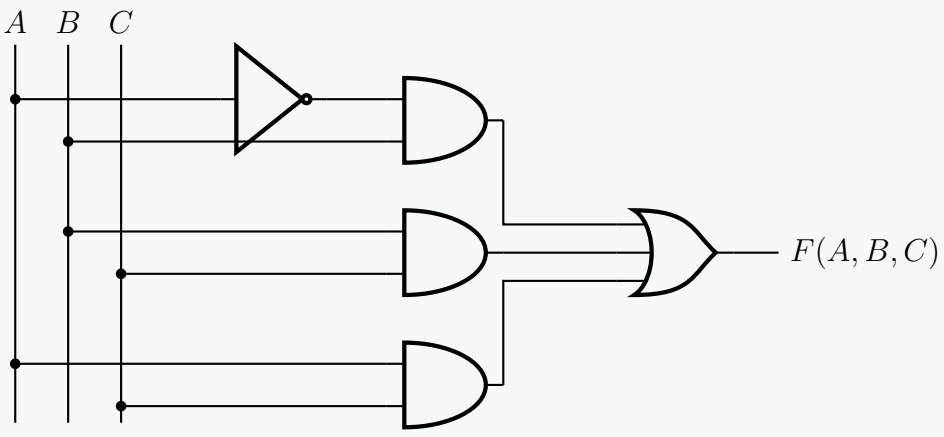
$$F(A, B, C) = A'B + BC + AC$$

---

1. Implementation using AND, OR, and NOT gates

To implement the unminimized function F, we analyze its structure:

- One **NOT** gate is required to generate the complement of A (A').
- Three 2-input **AND** gates are required to generate the product terms $A'B$, BC, and AC.
- One 3-input **OR** gate is required to sum the product terms together.



---

2. K-Map Minimization in SOP Form

To plot the function on a Karnaugh Map, we first expand the terms into canonical minterms (Sum of Minterms) by introducing missing variables:

$$A'B = A'B(C + C') = A'BC + A'BC' \implies m_3 + m_2$$
$$BC = (A + A')BC = ABC + A'BC \implies m_7 + m_3$$
$$AC = A(B + B')C = ABC + AB'C \implies m_7 + m_5$$

Thus, the canonical function is $F(A, B, C) = \Sigma m(2, 3, 5, 7)$. Let us map these 1s onto a 3-variable K-Map.

A \ BC	00	01	11	10
0	0	0	1	1
1	0	1	1	0

Group Extraction:

- **Group 1** (Cells 2, 3): This horizontal pair yields the term $\mathbf{A'B}$.
- **Group 2** (Cells 5, 7): This horizontal pair yields the term $\mathbf{AC}$.

Notice that cells 3 and 7 form a vertical pair (BC), but this is a *redundant group* since all its 1s are already covered by essential prime implicants. This visual redundancy perfectly illustrates the **Consensus Theorem**.
The minimized SOP function is:

$$F_{\text{min}} = A'B + AC$$

---

3. Implementation of Minimized F using only NAND gates

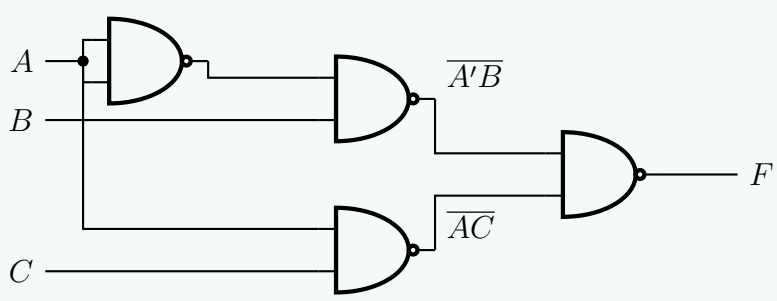
To construct the circuit using strictly NAND gates, we must convert the minimized SOP expression into a NAND-NAND form. We achieve this by applying the **Double Negation Law** and **De Morgan's Laws**:

$$\begin{aligned} F_{\text{min}} &= A'B + AC \\ F_{\text{min}} &= \overline{\overline{A'B + AC}} && \text{(Double Negation Law)} \\ F_{\text{min}} &= \overline{\overline{(A'B)} \cdot \overline{(AC)}} && \text{(De Morgan's Law: } \overline{X+Y} = \overline{X} \cdot \overline{Y}\text{)} \end{aligned}$$

This equation dictates a two-level NAND structure:

(a) The first level generates the terms $\overline{A'B}$ and $\overline{AC}$. Note that A' must also be created by tying the inputs of a NAND gate together (since $\overline{A \cdot A} = A'$).

(b) The second level NAND gate takes these inverted products and produces the final output F.



Q7. Given the Boolean function $F = xy + x'y' + y'z$, implement it with (i) AND, OR and NOT gates, and (ii) only NAND gates. **(5+5 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

Logic Circuit Implementations

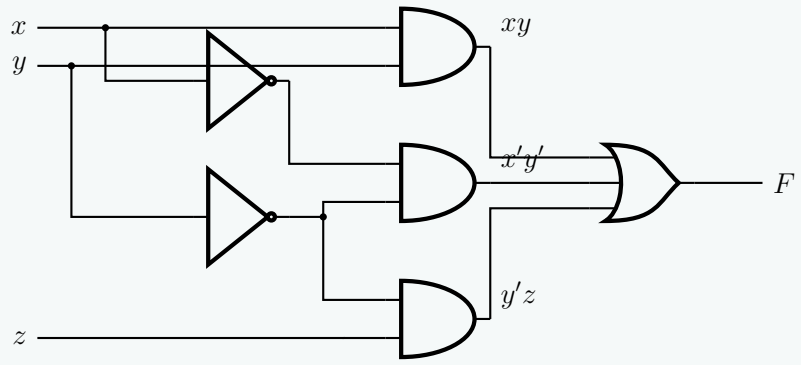
We are given the Boolean function in Sum of Products (SOP) form:

$$F(x, y, z) = xy + x'y' + y'z$$

(i) Implementation using AND, OR, and NOT gates

To implement F directly using standard logic gates, we require:

- **NOT Gates:** To generate the complements x' and y'.
- **AND Gates:** Three 2-input AND gates to compute the product terms (xy), $(x'y')$, and $(y'z)$.
- **OR Gate:** One 3-input OR gate to sum the product terms together.



(ii) Implementation using ONLY NAND gates

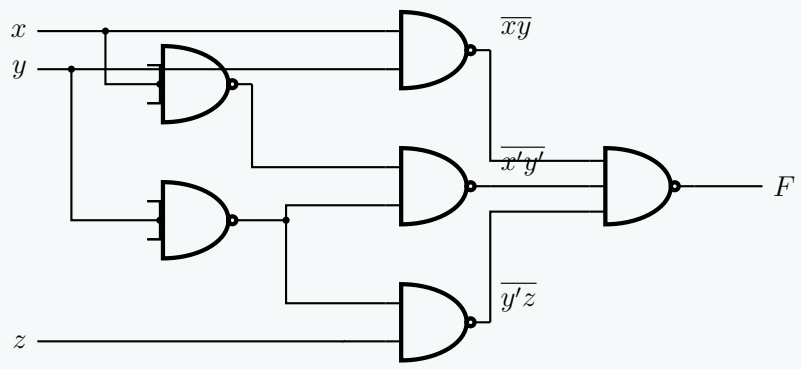
To implement the circuit exclusively with NAND gates, we must convert the SOP expression into a **NAND-NAND** structure. We achieve this by applying the *Double Negation Law* and *De Morgan's Laws*:

$$F = xy + x'y' + y'z$$

$$F = \overline{\overline{xy + x'y' + y'z}} \qquad \text{(Double Negation Law)}$$

$$F = \overline{\overline{(xy)} \cdot \overline{(x'y')} \cdot \overline{(y'z)}} \qquad \text{(De Morgan's Law: } \overline{A + B + C} = \overline{A} \cdot \overline{B} \cdot \overline{C}\text{)}$$

This equation dictates a two-level NAND structure. Additionally, the complements x' and y' must be generated using NAND gates with their inputs tied together (since $\overline{A \cdot A} = A'$).



Q8. Using the two-level forms of logic (i) NOR-OR, and (ii) OR-NAND, implement the following Boolean function F:

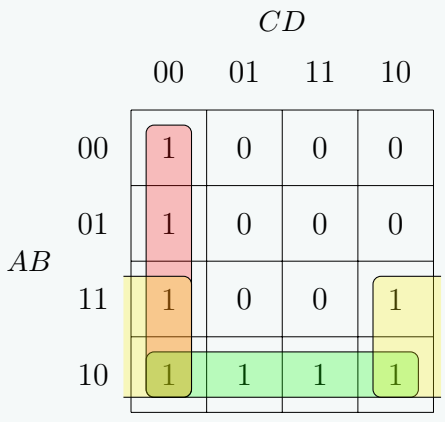
$$F(A, B, C, D) = \Sigma\,(0, 4, 8, 9, 10, 11, 12, 14)$$

Assume that both the normal and complement inputs are available. **(6+6 marks)** **[CSE 205 | L-2/T-1 | 2011–12]**

Answer

Step 1: K-Map Minimization in SOP Form

Before implementing the function in the required two-level forms, we must first minimize the Boolean function $F(A, B, C, D) = \Sigma\,(0, 4, 8, 9, 10, 11, 12, 14)$ into its minimal Sum of Products (SOP) form. We map the given minterms onto a 4-variable Karnaugh Map:



Group Extraction:

- **Group 1 (Vertical Col 00):** Minterms (0, 4, 12, 8). This eliminates A and B, yielding the prime implicant **C′D′**.
- **Group 2 (Horizontal Row 10):** Minterms (8, 9, 11, 10). This eliminates C and D, yielding the prime implicant **AB′**.
- **Group 3 (Edges):** Minterms (12, 14, 8, 10). This forms a 2×2 block wrapping around the horizontal edges, eliminating B and C, yielding the prime implicant **AD′**.

All three groups are essential prime implicants. The minimized SOP expression is:

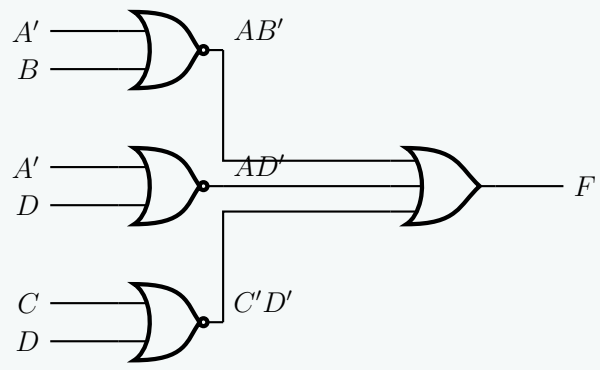
$$F = AB' + AD' + C'D' \tag{1}$$

Step 2: Implementation (i) NOR-OR Logic

A **NOR-OR** implementation produces a Sum of Products (SOP) expression, but requires specific algebraic manipulation of the inputs. We analyze a single NOR gate: it computes the complement of an OR sum, which maps to a product term via De Morgan's Law ($\overline{X+Y} = X'Y'$).
To generate our required product terms using NOR gates, we complement the literals of each product term and feed them into the NOR gates:

$$AB' = \overline{A' + B} \qquad \text{(NOR with inputs } A', B\text{)}$$
$$AD' = \overline{A' + D} \qquad \text{(NOR with inputs } A', D\text{)}$$
$$C'D' = \overline{C + D} \qquad \text{(NOR with inputs } C, D\text{)}$$

We then sum these terms together using a second-level OR gate. Since both normal and complement inputs are available, we construct the circuit directly:



Step 3: Implementation (ii) OR-NAND Logic

An **OR-NAND** structure is logically equivalent to the SOP form. To convert the SOP expression into an OR-NAND format, we apply the **Double Negation Law** followed by **De Morgan's Law** on the entire function:

$$F = AB' + AD' + C'D'$$
$$F = \overline{\overline{AB' + AD' + C'D'}} \qquad \text{(Double Negation Law)}$$
$$F = \overline{\overline{(AB')} \cdot \overline{(AD')} \cdot \overline{(C'D')}} \qquad \text{(De Morgan's Law)}$$

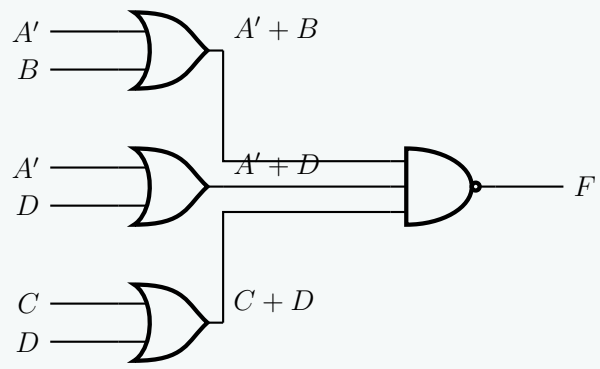
Next, we expand the inverted product terms internally using De Morgan's Law ($\overline{XY} = X' + Y'$):

$$\overline{AB'} = A' + B \qquad \text{(OR with inputs } A', B\text{)}$$
$$\overline{AD'} = A' + D \qquad \text{(OR with inputs } A', D\text{)}$$
$$\overline{C'D'} = C + D \qquad \text{(OR with inputs } C, D\text{)}$$

Substituting these back yields the final OR-NAND equation:

$$F = \overline{(A' + B) \cdot (A' + D) \cdot (C + D)}$$

This dictates a first level of OR gates generating sums, feeding into a final NAND gate:



NOR Gate Implementation

Q1. Express $f(A, B, C) = \Pi\,(0, 2, 3, 5, 7)$ with NOR gates. **(5 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Answer

Step 1: K-Map Minimization in POS Form

We are given the Boolean function in the Product of Maxterms (POS) form:

$$f(A, B, C) = \Pi\,(0, 2, 3, 5, 7)$$

To express this function using **NOR gates**, it is most efficient to first minimize the function into its minimal Product of Sums (POS) form. We achieve this by mapping the Maxterms (0s) onto a 3-variable Karnaugh Map and grouping them.

A \ BC	00	01	11	10
0	0	1	0	0
1	1	0	0	1

Maxterm Group Extraction (Grouping 0s):

- **Group 1 (Cells 0, 2):** Located in row $A = 0$, spanning columns $BC = 00$ and 10. The variable B changes, while $A = 0$ and $C = 0$ remain constant. This yields the sum term $(\mathbf{A + C})$.
- **Group 2 (Cells 3, 7):** Located in column $BC = 11$, spanning both rows. The variable A changes, while $B = 1$ and $C = 1$ remain constant. This yields the sum term $(\mathbf{B' + C'})$.
- **Group 3 (Cells 5, 7):** Located in row $A = 1$, spanning columns $BC = 01$ and 11. The variable B changes, while $A = 1$ and $C = 1$ remain constant. This yields the sum term $(\mathbf{A' + C'})$.

The minimized function in POS form is:

$$f(A, B, C) = (A + C)(B' + C')(A' + C') \tag{1}$$

Step 2: Conversion to NOR Logic

A **NOR-NOR** architecture naturally implements a Product of Sums (POS) expression. To mathematically map the POS equation to NOR gates, we apply the **Double Negation Law** and **De Morgan's Laws**:

$$\begin{aligned} f &= (A + C)(B' + C')(A' + C') \\ f &= \overline{\overline{(A + C) \cdot (B' + C') \cdot (A' + C')}} && \text{(Double Negation Law: } \overline{\overline{X}} = X) \\ f &= \overline{\overline{(A + C)} + \overline{(B' + C')} + \overline{(A' + C')}} && \text{(De Morgan's Law: } \overline{X \cdot Y \cdot Z} = \overline{X} + \overline{Y} + \overline{Z}) \end{aligned}$$

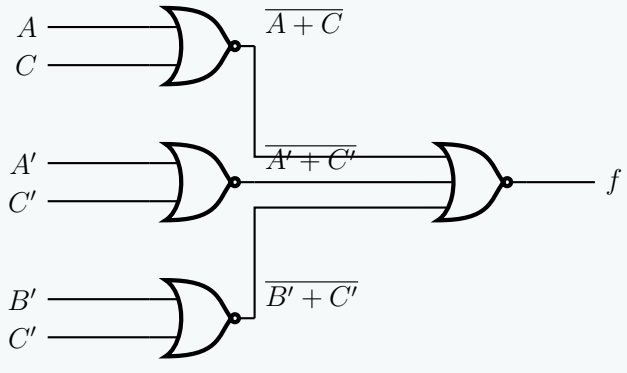
This final equation dictates a two-level NOR structure:

(a) The first level consists of three 2-input NOR gates generating the terms $\overline{A + C}$, $\overline{B' + C'}$, and $\overline{A' + C'}$.

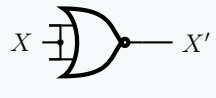
(b) The second level consists of a single 3-input NOR gate that sums and inverts these terms to produce the final output f.

Step 3: Circuit Diagram

Below is the complete implementation of f using exclusively NOR gates.
Note: If the complemented literals (A', B', C') are not directly available from the inputs, they can be generated using a 2-input NOR gate with its inputs tied together (e.g., $A' = \overline{A + A}$).



Complement Generation
(Using NOR as NOT)



Q2. Express $f(A, B, C, D) = \text{SOP}(0, 4, 8, 9, 10, 11, 12, 14)$ with the two-level forms of logic "NOR-OR" and "AND-NOR". **(8 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

Step 1: K-Map Minimization for f and f'

To implement the target Boolean function using the required two-level forms, we need the minimal Sum of Products (SOP) for both the function f and its complement f'.
The given function is $f(A, B, C, D) = \Sigma\,(0, 4, 8, 9, 10, 11, 12, 14)$. Its complement is $f'(A, B, C, D) = \Sigma\,(1, 2, 3, 5, 6, 7, 13, 15)$.
We map both sets onto 4-variable Karnaugh Maps.

1.1 Minimized SOP for f

We group the 1s to find the SOP expression for f:

AB \ CD	00	01	11	10
00	1	0	0	0
01	1	0	0	0
11	1	0	0	1
10	1	1	1	1

Group Extraction for f:

- **Group 1 (Col 00):** Minterms (0, 4, 12, 8) $\implies$ **C′D′**
- **Group 2 (Row 10):** Minterms (8, 9, 11, 10) $\implies$ **AB′**
- **Group 3 (Edges):** Minterms (12, 14, 8, 10) $\implies$ **AD′**

$$f = C'D' + AB' + AD' \tag{1}$$

1.2 Minimized SOP for f'

We group the 0s of f (which are the 1s of f') to find the SOP expression for f':

AB \ CD	00	01	11	10
00	0	1	1	1
01	0	1	1	1
11	0	1	1	0
10	0	0	0	0

Group Extraction for f':

- **Group 1:** Minterms (1, 3, 5, 7) $\implies$ **A′D**
- **Group 2:** Minterms (2, 3, 6, 7) $\implies$ **A′C**
- **Group 3:** Minterms (5, 7, 13, 15) $\implies$ **BD**

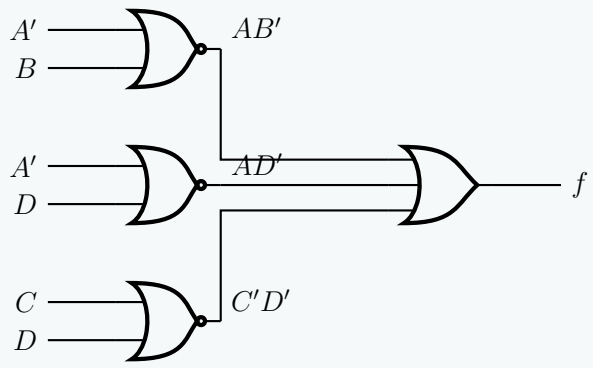
$$f' = A'D + A'C + BD \tag{2}$$

Step 2: (i) NOR-OR Logic Implementation

A **NOR-OR** implementation generates a Sum of Products (SOP). We start with the SOP equation for f and apply the **Double Negation Law** and **De Morgan's Law** to convert the product terms into NOR operations.

$$
\begin{aligned}
f &= AB' + AD' + C'D' \\
f &= \overline{\overline{AB'}} + \overline{\overline{AD'}} + \overline{\overline{C'D'}} && \text{(Double Negation Law)} \\
f &= \overline{(A' + B)} + \overline{(A' + D)} + \overline{(C + D)} && \text{(De Morgan's Law: } \overline{XY} = \overline{X} + \overline{Y}\text{)}
\end{aligned}
$$

This gives us a first level of NOR gates generating the required product terms, which are then summed using a second-level OR gate.

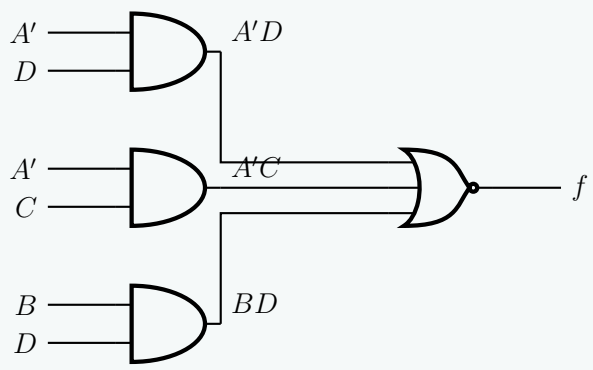


Step 3: (ii) AND-NOR Logic Implementation

An **AND-NOR** structure essentially performs an AND-OR-INVERT (AOI) operation. It computes the complement of the SOP expression fed into it. Therefore, to output f, we must implement f' using a first level of AND gates and a second level NOR gate. Taking the complement of both sides of our f' equation:

$$
\begin{aligned}
f' &= A'D + A'C + BD \\
\overline{f'} &= \overline{A'D + A'C + BD} \\
f &= \overline{(A' \cdot D) + (A' \cdot C) + (B \cdot D)}
\end{aligned}
$$

This equation maps exactly to three AND gates (computing the products) feeding into a single NOR gate (computing the sum and inverting the result).



Q3. Using NOR gates, evaluate the following expression:

$$p \oplus (q \vee r) \oplus ((p \wedge q) \rightarrow r)$$

(5 marks) **[CSE 205 | L-2/T-1 | 2016–17]**

Answer

Step 1: Formal Algebraic Simplification

We begin by evaluating and simplifying the given propositional expression E:

$$E = p \oplus (q \vee r) \oplus ((p \wedge q) \rightarrow r)$$

First, we apply the **Material Implication** identity to rewrite the conditional term in terms of basic Boolean operators:

$$
\begin{aligned}
(p \wedge q) \rightarrow r &= \neg(p \wedge q) \vee r && \text{(Material Implication: } X \rightarrow Y = \bar{X} \vee Y\text{)} \\
&= \bar{p} \vee \bar{q} \vee r && \text{(De Morgan's Law)}
\end{aligned}
$$

Substituting this back into our primary expression yields:

$$E = p \oplus (q \vee r) \oplus (\bar{p} \vee \bar{q} \vee r) \tag{1}$$

Step 2: Truth Table Evaluation

To systematically evaluate the expression for all possible combinations of the input variables p, q, and r, we construct a truth table. This enables us to determine the canonical minterms of the system.

p	q	r	$q \vee r$	$p \oplus (q \vee r)$	$\bar{p} \vee \bar{q} \vee r$	E	Minterm
0	0	0	0	0	1	**1**	m_0
0	0	1	1	1	1	0	—
0	1	0	1	1	1	0	—
0	1	1	1	1	1	0	—
1	0	0	0	1	1	0	—
1	0	1	1	0	1	**1**	m_5
1	1	0	1	0	0	0	—
1	1	1	1	0	1	**1**	m_7

From the truth table, the function evaluates to 1 at minterms $0, 5$, and 7. Thus, the canonical Sum-of-Products (SOP) expression is:

$$E(p, q, r) = \Sigma\,(0, 5, 7) = \bar{p}\bar{q}\bar{r} + p\bar{q}r + pqr \tag{2}$$

Step 3: Karnaugh Map Minimization

We map the extracted minterms onto a 3-variable Karnaugh Map to determine the minimal SOP expression.

p \ qr	00	01	11	10
0	1	0	0	0
1	0	1	1	0

Prime Implicant Extraction:

- **Group 1 (Cell 0):** Isolated minterm yields $\bar{p}\bar{q}\bar{r}$.
- **Group 2 (Cells 5, 7):** Combines to eliminate the variable q, yielding the product term pr.

The minimized SOP expression is:

$$E = \bar{p}\bar{q}\bar{r} + pr \tag{3}$$

Step 4: Mathematical Conversion to NOR-Only Logic

To realize the function using exclusively NOR gates, we must recall that a standard NOR gate computes the negated sum: $\text{NOR}(X, Y) = \overline{X + Y}$. We use Boolean algebra laws to massage our minimized equation into nested negated sums.
First, observe that our product terms can be rewritten via **De Morgan's Law** as purely inverted sums:

$$\bar{p}\bar{q}\bar{r} = \overline{p + q + r}$$
$$pr = \overline{\bar{p} + \bar{r}}$$

Substituting these back into E:

$$E = \overline{(p + q + r)} + \overline{(\bar{p} + \bar{r})}$$

Currently, the outer operation is an OR. To convert this final step into NOR gates, we apply the **Double Negation Law** ($X = \overline{\overline{X}}$) to the entire expression:

$$E = \overline{\overline{(p + q + r)} + \overline{(\bar{p} + \bar{r})}} \tag{4}$$

This maps perfectly to a multi-stage NOR architecture:

(a) $H_1 = \text{NOR}(p, q, r) = \overline{p + q + r}$

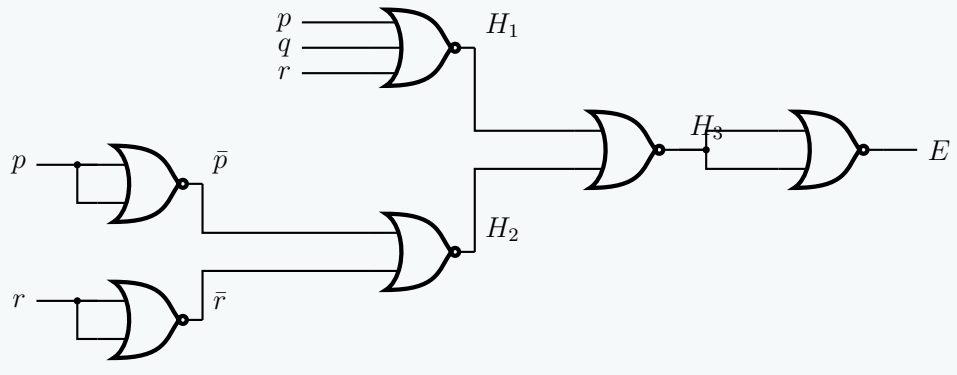
(b) $H_2 = \text{NOR}(\bar{p}, \bar{r}) = \overline{\bar{p} + \bar{r}}$

(c) $H_3 = \text{NOR}(H_1, H_2) = \overline{H_1 + H_2}$

(d) $E = \text{NOT}(H_3) = \text{NOR}(H_3, H_3)$ *(Applying a final NOR gate as an inverter)*

Step 5: Logic Gate Schematic

Below is the complete gate-level realization using only standard NOR gates. The required complements $\bar{p}$ and $\bar{r}$ are locally generated by using 2-input NOR gates with tied inputs.



Q4. Implement the following Boolean function using only NOR gates:

$$f = xy + yz$$

Your implementation should use as few NOR gates as possible. **(6 marks)** **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

Step 1: Boolean Function Simplification

We are given the Boolean function:

$$f = xy + yz$$

To minimize the number of required logic gates, we first simplify the expression by factoring out the common variable y using the **Distributive Law**:

$$f = y(x + z) \tag{1}$$

Step 2: Conversion to NOR Logic

A standard NOR gate performs the negated sum of its inputs: $\text{NOR}(A, B) = \overline{A + B}$. To implement our factored function using exclusively NOR gates, we must manipulate the equation using the **Double Negation Law** and **De Morgan's Laws** until all operations are in the form of negated sums.
Starting with the simplified function:

$$\begin{aligned} f &= y(x + z) \\ &= \overline{\overline{y(x + z)}} && \text{(Double Negation Law: } A = \overline{\overline{A}}) \\ &= \overline{\overline{y} + \overline{(x + z)}} && \text{(De Morgan's Law: } \overline{A \cdot B} = \overline{A} + \overline{B}) \end{aligned}$$

Let us analyze the components of this final equation to map them to NOR gates:
(a) $\overline{x + z}$: This is the direct output of a NOR gate with inputs x and z. Let $N_1 = \overline{x + z}$.

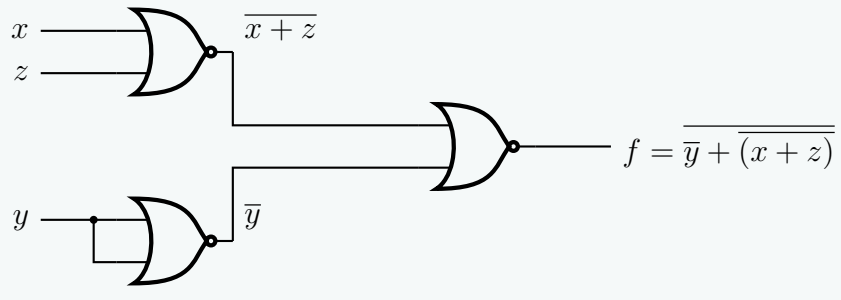
(b) $\overline{y}$: We can generate the complement of y by tying the two inputs of a NOR gate together. $\text{NOR}(y, y) = \overline{y + y} = \overline{y}$. Let $N_2 = \overline{y}$.

(c) $\overline{N_2 + N_1}$: This is the final NOR gate that takes the outputs of N_1 and N_2 as its inputs.

Thus, the function can be perfectly realized using exactly **three NOR gates**.

Step 3: Logic Gate Implementation

Below is the corresponding schematic drawing using the minimum number of NOR gates (3 gates).



Q5. Implement the following Boolean function together with the don't-care conditions using only two-input NOR gates (complements of literals are available as input):

$$F(A, B, C, D) = \Sigma\,(0, 1, 9, 11), \quad d(A, B, C, D) = \Sigma\,(2, 8, 10, 14, 15)$$

(15 marks) **[CSE 205 | L-2/T-1 | 2013–14]**

Answer

Step 1: Karnaugh Map Minimization

We are given the minterms and don't-care conditions for a 4-variable function:

$$F(A,B,C,D) = \Sigma\,(0,1,9,11)$$
$$d(A,B,C,D) = \Sigma\,(2,8,10,14,15)$$

We plot these on a 4-variable Karnaugh map to find the minimal Sum-of-Products (SOP) expression.

AB \ CD	00	01	11	10
00	1	1	0	-
01	0	0	0	0
11	0	0	-	-
10	-	1	1	-

Prime Implicant Extraction:

- **Group 1** (Cells 0, 1, 8, 9): Covers the top and bottom rows of the first two columns. The variables that remain constant are $B = 0$ and $C = 0$. This yields the term $\bar{B}\bar{C}$.
- **Group 2** (Cells 8, 9, 10, 11): Covers the entire $AB = 10$ row. The variables that remain constant are $A = 1$ and $B = 0$. This yields the term $A\bar{B}$.

The minimized SOP expression is:

$$F = \bar{B}\bar{C} + A\bar{B} \tag{1}$$

Step 2: Boolean Factoring & NOR Conversion

Our goal is to implement this function using **only two-input NOR gates**. A NOR gate performs the negated sum of its inputs: $\text{NOR}(X,Y) = \overline{X+Y}$. To minimize the gate count, we first factor the SOP expression and then convert it into a nested negated-sum format.

1. Factoring out common literals:

$$F = \bar{B}\bar{C} + A\bar{B}$$
$$F = \bar{B}(A + \bar{C}) \qquad \text{(Distributive Law)}$$

2. Applying Involution and De Morgan's Law:

$$F = \overline{\overline{\bar{B}(A+\bar{C})}} \qquad \text{(Double Negation Law: } X = \overline{\overline{X}}\text{)}$$
$$F = \overline{\overline{\bar{B}} + \overline{(A+\bar{C})}} \qquad \text{(De Morgan's Law: } \overline{X\cdot Y} = \bar{X} + \bar{Y}\text{)}$$
$$F = \overline{B + \overline{(A+\bar{C})}} \qquad \text{(De Morgan's Law: } \overline{X\cdot Y} = \bar{X} + \bar{Y}\text{)}$$
$$F = \overline{B + \overline{A+\bar{C}}}$$

Let us analyze the required operations mapping perfectly to two-input NOR gates:

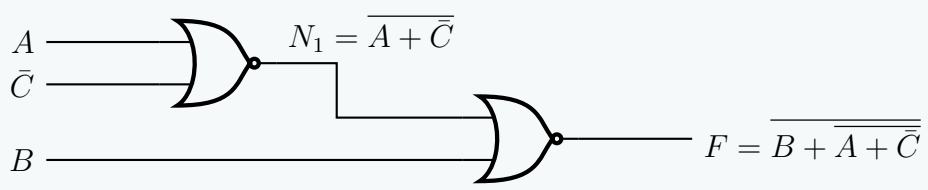
(a) $N_1 = \overline{A+\bar{C}} \implies$ The output of a 2-input NOR gate with inputs A and $\bar{C}$.

(b) $F = \overline{B+N_1} \implies$ The output of a second 2-input NOR gate with inputs B and N_1.

Since we are given that the **complements of literals are available as input**, we do not need to use an additional NOR gate to generate $\bar{C}$. The complete function requires exactly **two** NOR gates.

Step 3: Logic Gate Implementation

Below is the highly optimized schematic using exclusively two-input NOR gates.



XOR Properties

Q1. Show that the dual of the exclusive-OR is equal to its complement. **(7 marks)** [CSE 205 | L-2/T-1 | 2022–23, 2019–20]

Answer

Step 1: Definition of the Exclusive-OR (XOR) Operation

Let $f(A, B)$ represent the Boolean function for the Exclusive-OR (XOR) operation. By definition, the sum-of-products (SOP) expression for XOR is:

$$f(A, B) = A \oplus B = A\bar{B} + \bar{A}B \tag{1}$$

Step 2: Deriving the Dual of the XOR Function

The **Principle of Duality** states that the dual of a Boolean expression is obtained by:

- Interchanging OR ($+$) and AND ($\cdot$) operators.
- Interchanging the constants 0 and 1 (though none are present in this specific equation).
- **Leaving all variables and their complements unchanged.**

Applying the duality principle to $f(A, B) = A\bar{B} + \bar{A}B$:

$$f^D(A, B) = (A + \bar{B}) \cdot (\bar{A} + B) \tag{2}$$

Now, we expand and simplify the dual expression using Boolean algebra laws:

$$\begin{aligned} f^D(A, B) &= (A + \bar{B}) \cdot (\bar{A} + B) \\ &= A\bar{A} + AB + \bar{B}\bar{A} + \bar{B}B && \text{(Distributive Law)} \\ &= 0 + AB + \bar{A}\bar{B} + 0 && \text{(Complementarity Law: } X\bar{X} = 0\text{)} \\ &= AB + \bar{A}\bar{B} && \text{(Identity Law: } X + 0 = X\text{)} \end{aligned}$$

Thus, the minimized dual of the XOR function is:

$$f^D(A, B) = AB + \bar{A}\bar{B} \tag{3}$$

Step 3: Deriving the Complement of the XOR Function

Now, we find the algebraic complement (NOT) of the original XOR function, denoted as $\overline{f(A, B)}$ or $\overline{A \oplus B}$. We apply **De Morgan's Laws** and Boolean simplification:

$$\begin{aligned} \overline{f(A, B)} &= \overline{A\bar{B} + \bar{A}B} \\ &= (\overline{A\bar{B}}) \cdot (\overline{\bar{A}B}) && \text{(De Morgan's Law: } \overline{X + Y} = \bar{X} \cdot \bar{Y}\text{)} \\ &= (\bar{A} + \overline{\overline{B}}) \cdot (\overline{\overline{A}} + \bar{B}) && \text{(De Morgan's Law: } \overline{X \cdot Y} = \bar{X} + \bar{Y}\text{)} \\ &= (\bar{A} + B) \cdot (A + \bar{B}) && \text{(Double Negation Law: } \overline{\overline{X}} = X\text{)} \end{aligned}$$

Notice that this expression is structurally identical to the unexpanded dual we found in Step 2. Expanding it yields:

$$\begin{aligned} \overline{f(A, B)} &= \bar{A}A + \bar{A}\bar{B} + BA + B\bar{B} && \text{(Distributive Law)} \\ &= 0 + \bar{A}\bar{B} + AB + 0 && \text{(Complementarity Law)} \\ &= AB + \bar{A}\bar{B} && \text{(Commutative Law \& Identity Law)} \end{aligned}$$

Thus, the complement of the XOR function is the Exclusive-NOR (XNOR) function:

$$\overline{f(A, B)} = AB + \bar{A}\bar{B} \tag{4}$$

Step 4: Conclusion

Comparing the results from Step 2 and Step 3:

$$\begin{aligned} f^D(A, B) &= AB + \bar{A}\bar{B} \\ \overline{f(A, B)} &= AB + \bar{A}\bar{B} \end{aligned}$$

Since $f^D(A, B) = \overline{f(A, B)}$, we have successfully demonstrated that the **dual of the Exclusive-OR operation is mathematically identical to its complement**.

Q2. Show that XOR operation is not distributive over AND operation. **(7 marks)** **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

Step 1: Formulating the Distributive Property

To determine if an operation $*$ is distributive over another operation $\circ$, the following property must hold true for all possible values of the variables:

$$A * (B \circ C) = (A * B) \circ (A * C)$$

Therefore, for the Exclusive-OR ($\oplus$) operation to be distributive over the AND ($\cdot$) operation, the following Boolean equation must be an identity:

$$A \oplus (B \cdot C) \stackrel{?}{=} (A \oplus B) \cdot (A \oplus C) \tag{1}$$

We will test this hypothesis using both algebraic expansion and a truth table.

Step 2: Algebraic Expansion and Comparison

We will expand both the Left-Hand Side (LHS) and the Right-Hand Side (RHS) of the proposed identity into Sum-of-Products (SOP) form to see if they are mathematically equivalent.

Expanding the LHS:

$$\begin{aligned}
\text{LHS} &= A \oplus (BC) \\
&= A\overline{(BC)} + \bar{A}(BC) && \text{(Definition of XOR: } X \oplus Y = X\bar{Y} + \bar{X}Y\text{)} \\
&= A(\bar{B} + \bar{C}) + \bar{A}BC && \text{(De Morgan's Law: } \overline{XY} = \bar{X} + \bar{Y}\text{)} \\
\text{LHS} &= A\bar{B} + A\bar{C} + \bar{A}BC && \text{(Distributive Law)}
\end{aligned}$$

Expanding the RHS:

$$\begin{aligned}
\text{RHS} &= (A \oplus B) \cdot (A \oplus C) \\
&= (A\bar{B} + \bar{A}B) \cdot (A\bar{C} + \bar{A}C) && \text{(Definition of XOR)} \\
&= (A\bar{B})(A\bar{C}) + (A\bar{B})(\bar{A}C) + (\bar{A}B)(A\bar{C}) + (\bar{A}B)(\bar{A}C) && \text{(Distributive Law / FOIL)} \\
&= (AA)\bar{B}\bar{C} + (A\bar{A})\bar{B}C + (\bar{A}A)B\bar{C} + (\bar{A}\bar{A})BC && \text{(Commutative \& Associative Laws)} \\
&= A\bar{B}\bar{C} + (0)\bar{B}C + (0)B\bar{C} + \bar{A}BC && \text{(Idempotent } AA = A\text{, Complementarity } A\bar{A} = 0\text{)} \\
\text{RHS} &= A\bar{B}\bar{C} + \bar{A}BC && \text{(Identity Law: } X + 0 = X\text{)}
\end{aligned}$$

Comparison: Comparing the minimized forms, we clearly see that:

$$A\bar{B} + A\bar{C} + \bar{A}BC \neq A\bar{B}\bar{C} + \bar{A}BC$$

Because the expressions are not identical, the XOR operation is **not** distributive over the AND operation.

Step 3: Proof by Truth Table & Counterexample

To provide absolute proof, we evaluate both sides of the equation for all $2^3 = 8$ combinations of the variables A, B, and C.

A	B	C	$B \cdot C$	$\mathbf{A \oplus (B \cdot C)}$ **(LHS)**	$A \oplus B$	$A \oplus C$	$\mathbf{(A \oplus B) \cdot (A \oplus C)}$ **(RHS)**	Match?
0	0	0	0	0	0	0	0	Yes
0	0	1	0	0	0	1	0	Yes
0	1	0	0	0	1	0	0	Yes
0	1	1	1	1	1	1	1	Yes
1	0	0	0	1	1	1	1	Yes
1	**0**	**1**	**0**	**1**	**1**	**0**	**0**	**No**
1	**1**	**0**	**0**	**1**	**0**	**1**	**0**	**No**
1	1	1	1	0	0	0	0	Yes

Counterexample Analysis: As highlighted in the truth table, consider the case where $A = 1, B = 0$, and $C = 1$:

- **LHS:** $1 \oplus (0 \cdot 1) = 1 \oplus 0 = \mathbf{1}$
- **RHS:** $(1 \oplus 0) \cdot (1 \oplus 1) = 1 \cdot 0 = \mathbf{0}$

Since $1 \neq 0$, the equality fails. **Thus, the XOR operation does not distribute over the AND operation.**

Q3. Why is the XOR operation called an odd function? Convert the following Boolean expression to a canonical maxterm form:

$$a \oplus b \oplus c \oplus d$$

(10 marks) **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

Part 1: Why the XOR Operation is an Odd Function

The Exclusive-OR (XOR) operation, denoted by $\oplus$, is called an **odd function** (or an odd parity function) because its output evaluates to 1 (TRUE) *if and only if an odd number of its input variables are equal to* 1. Conversely, if the number of inputs containing 1 is even (including zero), the output evaluates to 0 (FALSE).
Mathematically, for a multiple-variable XOR function:

$$f(x_1, x_2, \ldots, x_n) = x_1 \oplus x_2 \oplus \cdots \oplus x_n = \begin{cases} 1, & \text{if } \sum_{i=1}^{n} x_i \text{ is odd} \\ 0, & \text{if } \sum_{i=1}^{n} x_i \text{ is even} \end{cases} \tag{1}$$

This property makes XOR gates the fundamental building blocks of parity generators and checkers in digital circuits.

Part 2: Conversion to Canonical Maxterm Form

We are tasked with converting the 4-variable Boolean expression into its canonical maxterm form (Product of Maxterms, ΠM):

$$F(a, b, c, d) = a \oplus b \oplus c \oplus d$$

Since XOR is an odd function, $F(a, b, c, d) = 1$ when an odd number of variables are 1 (i.e., 1 or 3 inputs are 1). The maxterms correspond to the input combinations where the function evaluates to 0. Therefore, $F = 0$ when an **even number** of variables are 1 (i.e., 0, 2, or 4 inputs are 1).

Truth Table Analysis

Let us evaluate all 16 possible combinations for the 4 variables to explicitly extract the maxterms (M_i).

Dec	a	b	c	d	Count of 1s	$F = a \oplus b \oplus c \oplus d$	Maxterm (M_i)
0	0	0	0	0	0 (Even)	0	$M_0 = a + b + c + d$
1	0	0	0	1	1 (Odd)	1	-
2	0	0	1	0	1 (Odd)	1	-
3	0	0	1	1	2 (Even)	0	$M_3 = a + b + \bar{c} + \bar{d}$
4	0	1	0	0	1 (Odd)	1	-
5	0	1	0	1	2 (Even)	0	$M_5 = a + \bar{b} + c + \bar{d}$
6	0	1	1	0	2 (Even)	0	$M_6 = a + \bar{b} + \bar{c} + d$
7	0	1	1	1	3 (Odd)	1	-
8	1	0	0	0	1 (Odd)	1	-
9	1	0	0	1	2 (Even)	0	$M_9 = \bar{a} + b + c + \bar{d}$
10	1	0	1	0	2 (Even)	0	$M_{10} = \bar{a} + b + \bar{c} + d$
11	1	0	1	1	3 (Odd)	1	-
12	1	1	0	0	2 (Even)	0	$M_{12} = \bar{a} + \bar{b} + c + d$
13	1	1	0	1	3 (Odd)	1	-
14	1	1	1	0	3 (Odd)	1	-
15	1	1	1	1	4 (Even)	0	$M_{15} = \bar{a} + \bar{b} + \bar{c} + \bar{d}$

Final Canonical Maxterm Expression

By gathering the indices where $F = 0$, we form the canonical Product of Maxterms (ΠM):

$$F(a, b, c, d) = \prod M(0, 3, 5, 6, 9, 10, 12, 15) \tag{2}$$

Expanded into its pure algebraic form (Product of Sums), this is:

$$\begin{aligned} F(a, b, c, d) = {} & (a + b + c + d) \cdot (a + b + \bar{c} + \bar{d}) \cdot (a + \bar{b} + c + \bar{d}) \cdot (a + \bar{b} + \bar{c} + d) \cdot \\ & (\bar{a} + b + c + \bar{d}) \cdot (\bar{a} + b + \bar{c} + d) \cdot (\bar{a} + \bar{b} + c + d) \cdot (\bar{a} + \bar{b} + \bar{c} + \bar{d}) \end{aligned}$$

Parity Checker and Generator

Q1. In a network, for a 3-bit data transmission, an error will be generated if there is an odd parity. Design this logical circuit. **(5 marks)**
[CSE 205 | L-2/T-1 | 2020–21]

Answer

Step 1: Problem Analysis and Truth Table

We need to design a logic circuit for a 3-bit data transmission system that generates an error signal when the data contains an **odd parity** (i.e., an odd number of 1s in the 3-bit sequence).
Let the 3-bit input data be A, B, and C, and let the error output be E. The output E will be 1 when the number of 1s in the input

is exactly 1 or 3.

A	B	C	**Number of 1s**	**Error Output (E)**
0	0	0	0 (Even)	0
0	**0**	**1**	**1 (Odd)**	**1**
0	**1**	**0**	**1 (Odd)**	**1**
0	1	1	2 (Even)	0
1	**0**	**0**	**1 (Odd)**	**1**
1	0	1	2 (Even)	0
1	1	0	2 (Even)	0
1	**1**	**1**	**3 (Odd)**	**1**

From the truth table, the Sum-of-Minterms expression for the error function is:

$$E(A, B, C) = \sum m(1, 2, 4, 7) \tag{1}$$

Step 2: Karnaugh Map Minimization

We map the canonical expression onto a 3-variable Karnaugh Map to explore potential simplifications.

		BC = 00	01	11	10
A	0	0	1	0	1
	1	1	0	1	0

The K-map reveals a classic **checkerboard pattern**. None of the 1s are logically adjacent, which implies that no prime implicants can be grouped to eliminate variables. This specific diagonal pattern is the definitive signature of a parity function, indicating an Exclusive-OR (XOR) or Exclusive-NOR (XNOR) relationship.

Step 3: Algebraic Derivation

To implement this with standard logic gates, we will factor the unsimplified Sum-of-Products (SOP) expression algebraically:

$$E = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC \tag{2}$$

Factor out $\bar{A}$ from the first two terms and A from the last two terms:

$$E = \bar{A}(\bar{B}C + B\bar{C}) + A(\bar{B}\bar{C} + BC) \qquad \text{(Distributive Law)}$$

Recognizing the fundamental identities for XOR ($B \oplus C = \bar{B}C + B\bar{C}$) and XNOR ($\overline{B \oplus C} = \bar{B}\bar{C} + BC$), we substitute them into the equation:

$$E = \bar{A}(B \oplus C) + A(\overline{B \oplus C}) \qquad \text{(Definition of XOR and XNOR)}$$

Let $X = B \oplus C$. The equation simplifies structurally to:

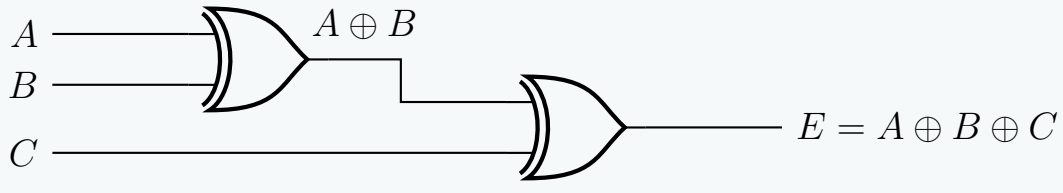
$$E = \bar{A}X + A\bar{X}$$
$$E = A \oplus X \qquad \text{(Definition of XOR)}$$

Substituting X back into the expression gives the final minimized logic equation:

$$E = A \oplus B \oplus C \tag{3}$$

Step 4: Logic Circuit Implementation

The odd parity error-detection circuit can be cleanly implemented using a cascade of two 2-input XOR gates.



Q2. What are the differences between an odd parity checker and an odd parity generator? **(8 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Answer

1. Conceptual Differences

Both parity generators and parity checkers are fundamental combinational logic circuits used for error detection in digital communication. However, they serve complementary roles at opposite ends of a transmission channel.

- An **Odd Parity Generator** is located at the *transmitter*. It evaluates the original data bits and generates an additional bit (the parity bit) to ensure that the total number of 1s in the transmitted message (data + parity) is always **odd**.
- An **Odd Parity Checker** is located at the *receiver*. It evaluates the incoming data alongside the received parity bit to verify if the total number of 1s remains odd. If it is even, it flags an error.

2. Comparative Analysis

Feature	Odd Parity Generator	Odd Parity Checker
Location	Transmitter side.	Receiver side.
Primary Purpose	To *create* the parity bit before transmission.	To *validate* the received bits and detect single-bit errors.
Input Data	n bits (Original message).	$n+1$ bits (Message + Parity bit).
Output Data	1 bit (The calculated Parity bit, P).	1 bit (The Error flag, E).
Output Condition	Outputs 1 if the message has an *even* number of 1s. Outputs 0 if it is *odd*.	Outputs 1 (Error) if the received frame has an *even* number of 1s.
Logic Equivalent	Exclusive-NOR (XNOR) of the n data bits.	Exclusive-NOR (XNOR) of all $n+1$ received bits.

3. Mathematical Formulation (3-bit Example)

Consider a 3-bit data message A, B, C.

The Odd Parity Generator

The generator produces a parity bit P such that the group $\{A, B, C, P\}$ contains an odd number of 1s.

$$P = \begin{cases} 1 & \text{if } A+B+C \text{ contains an even number of 1s} \\ 0 & \text{if } A+B+C \text{ contains an odd number of 1s} \end{cases}$$

$P = \overline{A \oplus B \oplus C}$ (XNOR Function)

The Odd Parity Checker

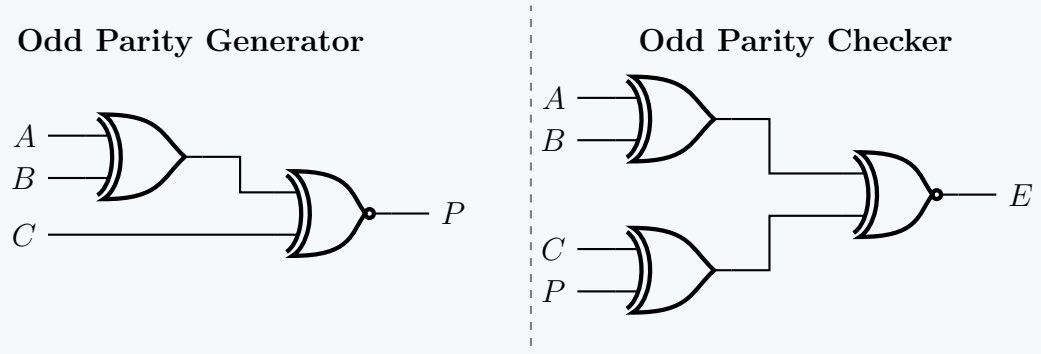
The checker receives the bits A, B, C and the parity bit P. It produces an error signal E that is 1 if an error occurred (i.e., the total number of 1s is now even).

$$E = \begin{cases} 1 \text{ (Error)} & \text{if } A+B+C+P \text{ contains an even number of 1s} \\ 0 \text{ (Valid)} & \text{if } A+B+C+P \text{ contains an odd number of 1s} \end{cases}$$

$E = \overline{A \oplus B \oplus C \oplus P}$ (XNOR Function)

4. Logic Circuit Implementations

Below are the logic circuit realizations for both the generator (transmitter side) and the checker (receiver side) using standard 2-input logic gates.



Q3. For a 3-bit Excess-3 code, design an odd parity checker and odd parity generator. **(8 marks)** **[CSE 205 | L-2/T-1 | 2017–18]**

Answer

Conceptual Note on Data Encoding and Parity

A critical principle in Digital Logic Design is that **parity generation and checking are independent of the semantic meaning of the data**. Whether the 3-bit data represents a standard binary number, a Gray code, or an **Excess-3 code**, the parity circuitry evaluates only the physical logic levels (0s and 1s) of the bits. Therefore, we will define our 3-bit Excess-3 code word as X, Y, and Z, and design the standard 3-bit odd parity logic.

Part 1: Odd Parity Generator Design (Transmitter Side)

The odd parity generator evaluates the 3-bit data (X, Y, Z) and produces a parity bit P such that the total number of 1s in the transmitted 4-bit message (X, Y, Z, P) is **odd**.

1.1 Truth Table for Odd Parity Generator

If the number of 1s in the input data is even (0 or 2), P must be 1 to make the total count odd. If the number of 1s in the input is odd (1 or 3), P must be 0.

X	Y	Z	**Number of 1s in Data**	**Generated Parity Bit (P)**
0	0	0	0 (Even)	**1**
0	0	1	1 (Odd)	0
0	1	0	1 (Odd)	0
0	1	1	2 (Even)	**1**
1	0	0	1 (Odd)	0
1	0	1	2 (Even)	**1**
1	1	0	2 (Even)	**1**
1	1	1	3 (Odd)	0

1.2 Boolean Algebra Derivation

From the truth table, the Sum-of-Minterms expression is:

$$\begin{aligned} P(X, Y, Z) &= \sum m(0, 3, 5, 6) \\ &= \bar{X}\bar{Y}\bar{Z} + \bar{X}YZ + X\bar{Y}Z + XY\bar{Z} \end{aligned}$$

Factoring out $\bar{X}$ and X:

$$\begin{aligned} P &= \bar{X}(\bar{Y}\bar{Z} + YZ) + X(\bar{Y}Z + Y\bar{Z}) && \text{(Distributive Law)} \\ P &= \bar{X}(\overline{Y \oplus Z}) + X(Y \oplus Z) && \text{(Definition of XOR and XNOR)} \end{aligned}$$

Let $W = Y \oplus Z$. Substituting W into the equation yields the standard definition of the XNOR operation:

$$\begin{aligned} P &= \bar{X}\bar{W} + XW \\ P &= \overline{X \oplus W} \\ P &= \overline{X \oplus Y \oplus Z} && \text{(Odd Parity Generator Equation)} \end{aligned}$$

Part 2: Odd Parity Checker Design (Receiver Side)

The checker receives the 4-bit sequence (X, Y, Z, P). If no error occurred, the total number of 1s must be odd. If an error alters a bit, the total number of 1s becomes **even**, and the circuit must flag an error ($E = 1$).

2.1 Boolean Algebraic Logic

An even parity condition across 4 variables (X, Y, Z, P) triggers the error. The Boolean function that outputs 1 for an even number of 1s across n variables is the generalized XNOR function.

$$E = \overline{X \oplus Y \oplus Z \oplus P} \tag{1}$$

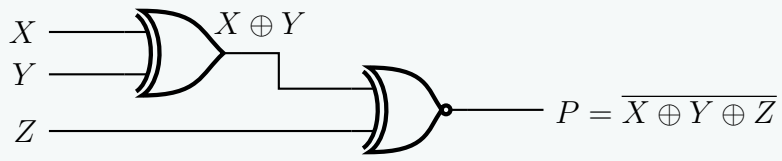
To optimize the logic circuit implementation, we can group the terms:

$$E = \overline{(X \oplus Y) \oplus (Z \oplus P)} \tag{2}$$

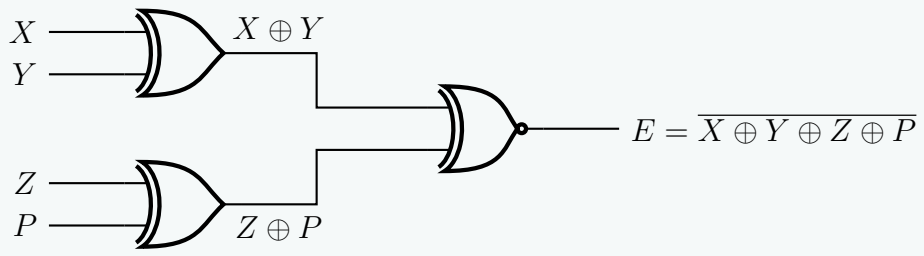
Part 3: Logic Circuit Implementations

Below are the highly optimized schematic diagrams for both the generator and the checker using minimum logic gates.

Odd Parity Generator (Transmitter)



Odd Parity Checker (Receiver)



Q4. Design a negative logic 4-bit parity checker and generator. **(6 marks)** **[CSE 205 | L-2/T-1 | 2016–17]**

Answer

System Assumptions & Negative Logic Principles

We will design an **Even Parity** generator and checker for a **4-bit data word** (A, B, C, D).
In **negative logic**, the logical states (True/False or 1/0) are inversely mapped to physical voltage levels compared to standard positive logic.

- **Logical** 1 is represented by a **Low Voltage** (0).
- **Logical** 0 is represented by a **High Voltage** (1).

Mathematically, if X_L represents the logical Boolean variable and X_V represents the physical voltage state (the hardware gate behavior), their relationship is defined by:

$$X_L = \overline{X_V} \tag{1}$$

Part 1: Negative Logic 4-Bit Parity Generator

The generator calculates a parity bit P such that the total number of logical 1s in the 5-bit word (A, B, C, D, P) is even. The logical Boolean equation for an even parity generator is the continuous XOR of the data bits:

$$P_L = A_L \oplus B_L \oplus C_L \oplus D_L \tag{2}$$

To determine the required physical logic gates (which are defined by positive voltage truth tables), we substitute the negative logic definition $X_L = \overline{X_V}$ into the equation:

$$\overline{P_V} = \overline{A_V} \oplus \overline{B_V} \oplus \overline{C_V} \oplus \overline{D_V}$$

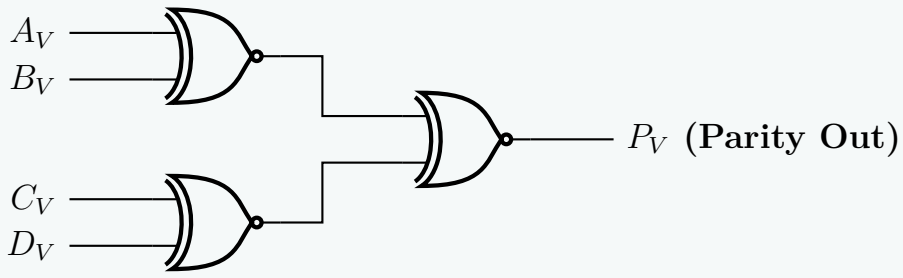
We apply the Boolean XOR property $\bar{X} \oplus \bar{Y} = X \oplus Y$:

$$\begin{aligned} \overline{P_V} &= (\overline{A_V} \oplus \overline{B_V}) \oplus (\overline{C_V} \oplus \overline{D_V}) \\ \overline{P_V} &= (A_V \oplus B_V) \oplus (C_V \oplus D_V) \\ \overline{P_V} &= A_V \oplus B_V \oplus C_V \oplus D_V \end{aligned}$$

To isolate the physical output P_V, we invert both sides:

$$\begin{aligned} P_V &= \overline{A_V \oplus B_V \oplus C_V \oplus D_V} \\ P_V &= (A_V \odot B_V) \odot (C_V \odot D_V) \qquad \text{(Hardware Equation)} \end{aligned} \tag{3}$$

Conclusion for Generator: Due to the dual nature of negative logic, an even parity generator for an **even** number of input bits requires physical **XNOR gates**.



Part 2: Negative Logic 4-Bit Parity Checker

The parity checker receives the 5 bits (A, B, C, D, P). If the total number of logical 1s is odd, it outputs an error flag $E_L = 1$. The logical equation is:

$$E_L = A_L \oplus B_L \oplus C_L \oplus D_L \oplus P_L \tag{4}$$

Substituting the negative logic mappings ($X_L = \overline{X_V}$):

$$\overline{E_V} = \overline{A_V} \oplus \overline{B_V} \oplus \overline{C_V} \oplus \overline{D_V} \oplus \overline{P_V}$$

We group the first four terms and apply $\bar{X} \oplus \bar{Y} = X \oplus Y$ as derived previously:

$$\overline{E_V} = (A_V \oplus B_V \oplus C_V \oplus D_V) \oplus \overline{P_V}$$

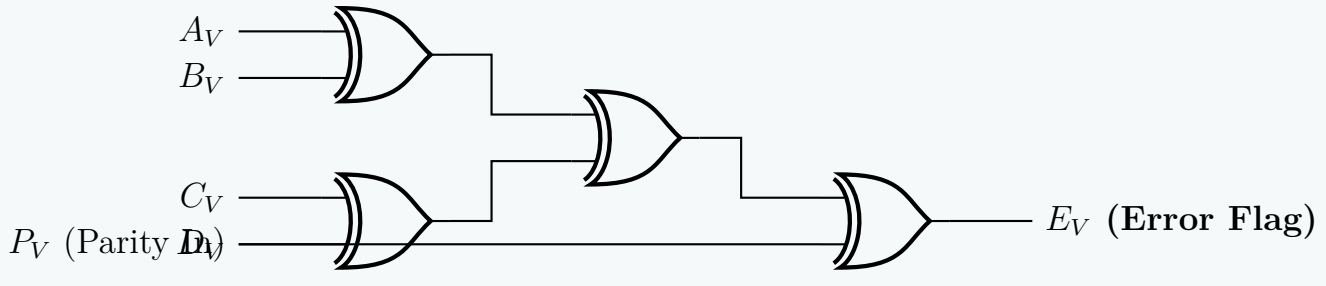
Next, we apply the property $X \oplus \bar{Y} = \overline{X \oplus Y}$:

$$\overline{E_V} = \overline{(A_V \oplus B_V \oplus C_V \oplus D_V) \oplus P_V}$$
$$\overline{E_V} = \overline{A_V \oplus B_V \oplus C_V \oplus D_V \oplus P_V}$$

Inverting both sides to isolate the physical voltage output E_V:

$$E_V = A_V \oplus B_V \oplus C_V \oplus D_V \oplus P_V \qquad \text{(Hardware Equation)} \tag{5}$$

Conclusion for Checker: Interestingly, because the checker evaluates an **odd** number of total variables (5), the negative logic inversions cancel out across the XOR boundaries. The physical implementation uses standard **XOR gates**.



BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Canonical & Standard Forms

Minterms, Maxterms, SOP, POS, Conversion

S2 Canonical Forms & Boolean Functions

Canonical vs Standard Forms

Q1. Compare the different types of canonical and standard forms used in Boolean Algebra with examples. **(8 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

Overview of Boolean Algebraic Forms

In Boolean Algebra, logic functions can be expressed in various algebraic formats. These formats are broadly classified into two main categories: **Canonical Forms** and **Standard Forms**. Understanding the distinction between them is crucial for both theoretical analysis and practical logic gate minimization.

1. Canonical Forms

In a canonical form, every individual term in the expression **must contain all the variables** present in the function, either in their normal (uncomplemented) or complemented state. Canonical forms are unique for a given Boolean function, meaning a specific logic function has exactly one canonical Sum of Products and one canonical Product of Sums.

A. Sum of Minterms (Canonical SOP)

A minterm is a product (AND) term that contains all variables of the function exactly once. The Canonical SOP form is the logical OR (Sum) of these minterms. It represents all the input combinations for which the function evaluates to 1.

- **Example:** For a 3-variable function $F(A, B, C)$, a canonical SOP expression is:

$$F(A, B, C) = \bar{A}B\bar{C} + A\bar{B}C + ABC$$
$$F(A, B, C) = m_2 + m_5 + m_7$$
$$F(A, B, C) = \sum m(2, 5, 7)$$

B. Product of Maxterms (Canonical POS)

A maxterm is a sum (OR) term that contains all variables of the function exactly once. The Canonical POS form is the logical AND (Product) of these maxterms. It represents all the input combinations for which the function evaluates to 0.

- **Example:** For the same 3-variable function, the canonical POS expression consists of the missing terms from the sum of minterms:

$$F(A, B, C) = (A + B + C) \cdot (A + B + \bar{C}) \cdot (A + \bar{B} + \bar{C}) \cdot (\bar{A} + B + C) \cdot (\bar{A} + \bar{B} + C)$$
$$F(A, B, C) = M_0 \cdot M_1 \cdot M_3 \cdot M_4 \cdot M_6$$
$$F(A, B, C) = \prod M(0, 1, 3, 4, 6)$$

---

2. Standard Forms

In a standard form, the terms in the expression **do not necessarily contain all the variables**. Standard forms are generally the simplified or minimized versions of canonical forms (e.g., using Karnaugh Maps or Boolean algebraic theorems). They are heavily preferred for actual hardware implementation because they require fewer logic gates and inputs.

A. Sum of Products (Standard SOP)

An expression consisting of two or more AND terms that are ORed together. Unlike minterms, these product terms can have any number of variables.

- **Example:** A minimized version of a 3-variable function might look like:

$$F(A, B, C) = AB + \bar{A}C$$

  Note that the first term lacks C, and the second term lacks B.

B. Product of Sums (Standard POS)

An expression consisting of two or more OR terms that are ANDed together. These sum terms can contain any number of literals.

- **Example:** A minimized POS expression for a function:

$$F(A, B, C) = (A + C) \cdot (\bar{A} + B)$$

---

3. Summary Comparison Table

Feature	Canonical Form	Standard Form
Variable Presence	Every term must contain **all** variables of the function.	Terms can contain **one, some, or all** variables of the function.
Uniqueness	**Unique** for a specific Boolean function.	**Not necessarily unique** (multiple equivalent simplified forms can exist).
Hardware Implementation	Requires maximum hardware (more gates and more inputs per gate). Highly inefficient.	Requires minimized hardware. Highly efficient and practical for IC design.
Derivation Method	Derived directly from the Truth Table.	Derived using minimization techniques like K-Maps or Quine-McCluskey.
Notation	Expressed easily using $\sum m$ (minterms) or $\prod M$ (maxterms).	Expressed purely as algebraic equations.

Q2. (a) What are the differences between canonical form and standard form?

(b) Which form is preferable when implementing a Boolean function with gates? Why?

(c) Which form is obtained when reading a function from a truth table?

(8+4+3 marks) [CSE 205 | L-2/T-1 | 2012–13]

Answer

(a) Differences between Canonical Form and Standard Form

In Boolean Algebra, logic expressions can be written in different algebraic formats. The fundamental differences between **Canonical** and **Standard** forms revolve around variable presence, uniqueness, and structural complexity.

Criteria	Canonical Form	Standard Form
Variable Inclusion	Every individual term must contain **all** variables of the Boolean function (either in normal or complemented form).	Terms do **not** need to contain all variables. They can consist of one, some, or all variables.
Terminology	Composed of *Minterms* (Canonical Sum of Products) or *Maxterms* (Canonical Product of Sums).	Composed of standard product terms (SOP) or standard sum terms (POS).
Uniqueness	**Strictly Unique** for a given Boolean function. There is exactly one canonical SOP and one canonical POS.	**Not unique**. A function can have multiple equivalent standard forms depending on how it is simplified.
Complexity	Generally longer, redundant, and more complex.	Generally simpler, often representing the minimized version of the function.
Example (for 3 variables A, B, C)	$F = \bar{A}B\bar{C} + A\bar{B}C + ABC$	$F = B\bar{C} + AC$

(b) Preferable Form for Logic Gate Implementation

When implementing a Boolean function with physical hardware (logic gates), the **Standard Form** (specifically the *minimized* standard form) is vastly preferable.

Reasons for preferring the Standard Form:

- **Reduced Gate Count:** Standard forms eliminate redundant terms and literals using Boolean simplification techniques (like K-Maps or Quine-McCluskey). Fewer terms directly translate to fewer logic gates in the physical circuit.
- **Lower Fan-in:** Because standard terms contain fewer literals, the corresponding logic gates require fewer input pins. High fan-in gates are slower, consume more area, and are more difficult to manufacture reliably.

- **Cost and Area Efficiency:** Minimizing the gate count and internal wire routing drastically reduces the silicon real estate required on an Integrated Circuit (IC), which lowers the overall manufacturing cost.
- **Improved Performance:** A simpler circuit with fewer components generally exhibits lower power consumption and shorter propagation delays.

(c) Form Obtained Directly from a Truth Table

When extracting a Boolean function directly from a truth table, the **Canonical Form** is inherently obtained.
Explanation: A truth table exhaustively lists every possible combination of the input variables. Each individual row dictates a specific, unique state for *all* variables.

- **Sum of Minterms (Canonical SOP):** If we write the function by taking the logical OR of all the rows where the output is 1, we generate an AND term containing all variables for each row.
- **Product of Maxterms (Canonical POS):** If we write the function by taking the logical AND of all the rows where the output is 0, we generate an OR term containing all variables for each row.

Because every row explicitly defines the state of every single variable in the system, the resulting algebraic expression is strictly canonical prior to any simplification.

Minterms, Maxterms

Q1. Express the following function in terms of minterms:

$$F = (x' + y)(x + z)(y + z)$$

(10 marks) [CSE 205 | L-2/T-1 | 2023–24]

Answer

Step 1: Simplify the Boolean Function

We are given the Boolean function in a mixed form:

$$F(x, y, z) = (x' + y)(x + z)(y + z)$$

To express the function in terms of minterms (Canonical Sum of Products), it is often easier to first expand and simplify the expression into a Standard Sum of Products (SOP) form.
We can apply the Distributive Law to the last two terms, observing that $(A + B)(A + C) = A + BC$, where $A = z$, $B = x$, and $C = y$:

$$F = (x' + y)(z + xy) \qquad \text{(Distributive Law)}$$

Next, we distribute $(x' + y)$ across $(z + xy)$:

$$F = x'z + x'xy + yz + y(xy) \qquad \text{(Distributive Law)}$$

Now, we apply fundamental Boolean algebraic theorems to simplify the terms:

$$\begin{aligned} F &= x'z + (x'x)y + yz + x(yy) && \text{(Associative \& Commutative Laws)} \\ F &= x'z + (0)y + yz + xy && \text{(Complementarity Law: } x'x = 0\text{; Idempotent Law: } yy = y\text{)} \\ F &= x'z + 0 + yz + xy && \text{(Null Law: } 0 \cdot y = 0\text{)} \\ F &= xy + x'z + yz && \text{(Identity Law)} \end{aligned}$$

Notice the presence of the term yz. According to the **Consensus Theorem** ($AB + A'C + BC = AB + A'C$), the term yz is redundant and can be eliminated:

$$F = xy + x'z \qquad \text{(Consensus Theorem)}$$

Step 2: Expand to Canonical Sum of Minterms

The function $F = xy + x'z$ is currently in standard SOP form. To convert it into canonical form (where every term contains all three variables x, y, z), we multiply each term by $(V + V') = 1$, where V is the missing variable.

- The first term xy is missing z. We multiply by $(z + z')$.
- The second term $x'z$ is missing y. We multiply by $(y + y')$.

$$F = xy(z + z') + x'z(y + y') \qquad \text{(Identity Law \& Complementarity Law)}$$
$$F = xyz + xyz' + x'yz + x'y'z \qquad \text{(Distributive Law)}$$

Rearranging the terms in logical order of their binary equivalents (x as MSB, z as LSB):

$$F = x'y'z + x'yz + xyz' + xyz \qquad \text{(Commutative Law)}$$

Step 3: Convert to Minterm Notation

Each individual product term containing all variables is a minterm m_i, where i is the decimal equivalent of the binary string formed by the variables (Uncomplemented = 1, Complemented = 0).

Product Term	Binary Value (x, y, z)	Decimal Index	Minterm
$x'y'z$	001	1	m_1
$x'yz$	011	3	m_3
xyz'	110	6	m_6
xyz	111	7	m_7

By summing these minterms, we arrive at the final canonical expression:

$$\mathbf{F(x, y, z) = \sum m(1, 3, 6, 7)} \tag{1}$$

Q2. Convert the Boolean function $F = xy + x'z$ as a product of maxterms using the distributive law. **(10 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Answer

Step 1: Convert to Standard Product of Sums (POS) Form

We begin with the Boolean function in Sum of Products (SOP) form:

$$F(x, y, z) = xy + x'z$$

To convert this into a product form, we apply the **Distributive Law** ($A + BC = (A + B)(A + C)$). Let us treat xy as a single term A, and distribute it over $x'z$:

$$F = (xy + x')(xy + z) \qquad \text{(Distributive Law)}$$

Next, we apply the Distributive Law again to both of the resulting terms:

$$F = (x' + x)(x' + y)(z + x)(z + y) \qquad \text{(Distributive Law)}$$

Now, we simplify using fundamental Boolean theorems:

$$F = (1)(x' + y)(x + z)(y + z) \qquad \text{(Complementarity Law: } x' + x = 1\text{)}$$
$$F = (x' + y)(x + z)(y + z) \qquad \text{(Identity Law: } 1 \cdot A = A\text{)}$$

The function is now in a standard Product of Sums (POS) form.

Step 2: Expand to Canonical Product of Maxterms

A canonical maxterm must contain all three variables (x, y, z). We expand the standard POS terms by adding 0 in the form of $VV' = 0$ for any missing variable V, and then distributing.

- **First term** $(x' + y)$**:** Missing z. We add zz'.

$$\begin{aligned} x' + y &= x' + y + 0 && \text{(Identity Law)} \\ &= x' + y + zz' && \text{(Complementarity Law: } zz' = 0\text{)} \\ &= (x' + y + z)(x' + y + z') && \text{(Distributive Law)} \end{aligned}$$

- **Second term** $(x + z)$**:** Missing y. We add yy'.

$$\begin{aligned} x + z &= x + z + yy' \\ &= (x + y + z)(x + y' + z) && \text{(Distributive Law)} \end{aligned}$$

- **Third term** $(y + z)$**:** Missing x. We add xx'.

$$\begin{aligned} y + z &= y + z + xx' \\ &= (x + y + z)(x' + y + z) && \text{(Distributive Law)} \end{aligned}$$

Substituting these expanded terms back into the function F:

$$F = [(x' + y + z)(x' + y + z')] \cdot [(x + y + z)(x + y' + z)] \cdot [(x + y + z)(x' + y + z)]$$

By the **Idempotent Law** $(A \cdot A = A)$, we can eliminate duplicate terms. The term $(x + y + z)$ appears twice, and $(x' + y + z)$ appears twice:

$$F = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z')$$

Step 3: Convert to Maxterm Notation

Each sum term containing all variables is a maxterm M_i. For maxterms, an uncomplemented variable represents 0, and a complemented variable represents 1. We find the binary and decimal equivalents:

Sum Term	Binary Value (x, y, z)	Decimal Index	Maxterm
$(x + y + z)$	000	0	M_0
$(x + y' + z)$	010	2	M_2
$(x' + y + z)$	100	4	M_4
$(x' + y + z')$	101	5	M_5

Taking the logical product of these maxterms gives the final canonical expression:

$$\mathbf{F(x, y, z) = \prod M(0, 2, 4, 5)} \tag{1}$$

Q3. Using Boolean Algebra, express the following Boolean function as a sum of minterms:

$$F(A, B, C) = A + B'C$$

(7 marks) **[CSE 205 | L-2/T-1 | 2021–22]**

Answer

Step 1: Identify Missing Variables in Each Term

We are given the Boolean function in Standard Sum of Products (SOP) form:

$$F(A, B, C) = A + B'C$$

To express this function as a Sum of Minterms (Canonical SOP form), every term must contain all three variables (A, B, and C).

- The first term, A, is missing variables B and C.
- The second term, $B'C$, is missing variable A.

Step 2: Algebraic Expansion into Canonical Form

We will expand each term by multiplying it by 1 in the form of $(V + V') = 1$, where V is the missing variable. This is justified by the **Identity Law** $(X \cdot 1 = X)$ and the **Complementarity Law** $(X + X' = 1)$.

Expanding the first term (A):

First, we introduce the missing variable B:

$$\begin{aligned} A &= A \cdot (B + B') && \text{(Identity \& Complementarity Laws)} \\ &= AB + AB' && \text{(Distributive Law)} \end{aligned}$$

Next, we introduce the missing variable C to both resulting terms:

$$\begin{aligned} A &= AB(C + C') + AB'(C + C') && \text{(Identity \& Complementarity Laws)} \\ &= ABC + ABC' + AB'C + AB'C' && \text{(Distributive Law)} \end{aligned}$$

Expanding the second term ($B'C$):

We introduce the missing variable A:

$$\begin{aligned} B'C &= (A + A')B'C && \text{(Identity \& Complementarity Laws)} \\ &= AB'C + A'B'C && \text{(Distributive Law)} \end{aligned}$$

Step 3: Combine and Simplify

Now, substitute the expanded terms back into the original function F:

$$F = (ABC + ABC' + AB'C + AB'C') + (AB'C + A'B'C)$$

Rearrange the terms (Commutative Law) and apply the **Idempotent Law** $(X + X = X)$ to eliminate the duplicate term $AB'C$:

$$F = A'B'C + AB'C' + AB'C + ABC' + ABC$$

Step 4: Convert to Minterm Notation

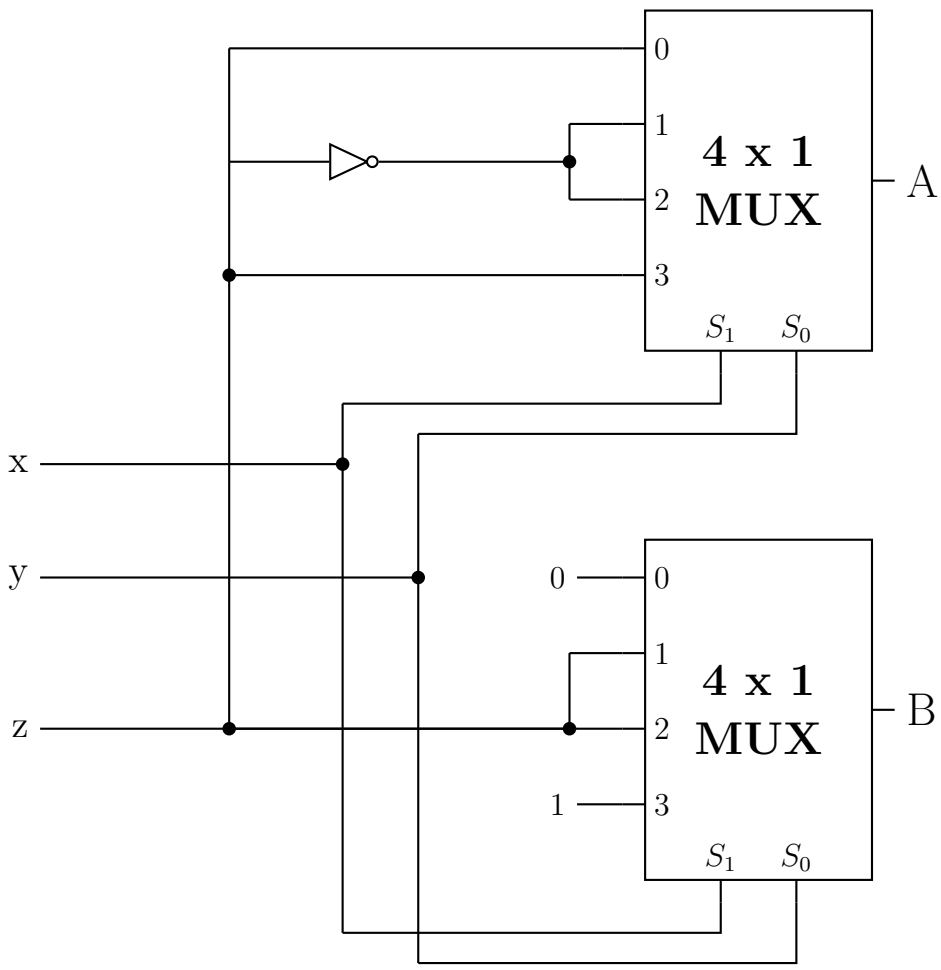
Each product term now contains all three variables, making them valid minterms (m_i). We determine the index i by treating uncomplemented variables as 1 and complemented variables as 0:

Product Term	**Binary Equivalent** (A, B, C)	**Decimal Index**	**Minterm Notation**
$A'B'C$	001	1	m_1
$AB'C'$	100	4	m_4
$AB'C$	101	5	m_5
ABC'	110	6	m_6
ABC	111	7	m_7

By summing these designated minterms, we arrive at the final canonical expression:

$$\mathbf{F(A, B, C) = \sum m(1, 4, 5, 6, 7)} \tag{1}$$

Q4. Determine the output functions A as the sum of minterms and the output function B as the product of maxterms:



(16 marks) [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Circuit Analysis and Multiplexer Foundation

A 4×1 Multiplexer (MUX) routes one of its four data inputs (I_0, I_1, I_2, I_3) to the output based on the value of its two selection lines (S_1, S_0). The canonical logic equation for a 4×1 MUX is:

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3 \tag{1}$$

By tracing the provided logic circuit, we can determine the global inputs applied to the select lines of both MUX A and MUX B:

- The Most Significant Select Bit (S_1) is connected directly to variable x.

- The Least Significant Select Bit (S_0) is connected directly to variable y.

Therefore, for both multiplexers, the select lines are $(S_1, S_0) = (x, y)$.

---

2. Derivation of Output Function A (Sum of Minterms)

By observing the connections to the data pins of **MUX A**, we identify the following inputs from the z bus:

- I_0 is connected directly to $z \implies I_0 = z$
- I_1 is connected through a NOT gate from $z \implies I_1 = z'$
- I_2 is connected through the same NOT gate from $z \implies I_2 = z'$
- I_3 is connected directly to $z \implies I_3 = z$

Substituting these into the MUX equation gives the Boolean function for A:

$$A(x, y, z) = x'y'(z) + x'y(z') + xy'(z') + xy(z)$$

Because each term contains all three variables (x, y, z), the expression is already in canonical Sum of Products (SOP) form. We can map each product term to its corresponding minterm index (where uncomplemented = 1, complemented = 0):

$$\begin{aligned} x'y'z &\implies 001 \implies m_1 \\ x'yz' &\implies 010 \implies m_2 \\ xy'z' &\implies 100 \implies m_4 \\ xyz &\implies 111 \implies m_7 \end{aligned}$$

Thus, the function A as a sum of minterms is:

$$\mathbf{A(x, y, z) = \sum m(1, 2, 4, 7)} \tag{2}$$

---

3. Derivation of Output Function B (Product of Maxterms)

By observing the connections to the data pins of **MUX B**, we identify:

- I_0 is connected to Logic 0 $\implies I_0 = 0$
- I_1 is connected directly to the z bus $\implies I_1 = z$
- I_2 is connected directly to the z bus $\implies I_2 = z$
- I_3 is connected to Logic 1 $\implies I_3 = 1$

Substituting these into the MUX equation gives the Boolean function for B:

$$\begin{aligned} B(x, y, z) &= x'y'(0) + x'y(z) + xy'(z) + xy(1) \\ B(x, y, z) &= x'yz + xy'z + xy \end{aligned}$$

To find the canonical maxterms, it is easiest to first expand the standard SOP expression into canonical minterms. We expand the term xy by introducing the missing variable z:

$$\begin{aligned} B(x, y, z) &= x'yz + xy'z + xy(z + z') && \text{(Identity \& Complementarity Laws)} \\ B(x, y, z) &= x'yz + xy'z + xyz + xyz' && \text{(Distributive Law)} \end{aligned}$$

Now we identify the minterm indices for B:

$$\begin{aligned} x'yz &\implies 011 \implies m_3 \\ xy'z &\implies 101 \implies m_5 \\ xyz' &\implies 110 \implies m_6 \\ xyz &\implies 111 \implies m_7 \end{aligned}$$

So, the minterms for B are $\sum m(3, 5, 6, 7)$.
In Boolean algebra, any term that is *not* a minterm is a maxterm (where the function evaluates to 0). For a 3-variable system, the universal set of indices is 0 through 7. The missing indices are $0, 1, 2$, and 4.
Thus, the function B as a product of maxterms is:

$$\mathbf{B(x, y, z) = \prod M(0, 1, 2, 4)} \tag{3}$$

---

4. Comprehensive Truth Table Verification

The derivations can be validated by mapping the entire system to a truth table.

Select Inputs			MUX A		MUX B	
x (S_1)	y (S_0)	z	Active Input	Output A	Active Input	Output B
0	0	0	$I_0 = z$	**0**	$I_0 = 0$	**0**
0	0	1	$I_0 = z$	**1**	$I_0 = 0$	**0**
0	1	0	$I_1 = z'$	**1**	$I_1 = z$	**0**
0	1	1	$I_1 = z'$	**0**	$I_1 = z$	**1**
1	0	0	$I_2 = z'$	**1**	$I_2 = z$	**0**
1	0	1	$I_2 = z'$	**0**	$I_2 = z$	**1**
1	1	0	$I_3 = z$	**0**	$I_3 = 1$	**1**
1	1	1	$I_3 = z$	**1**	$I_3 = 1$	**1**

Final Verification:

- A outputs 1 at decimal rows $1, 2, 4, 7 \implies \mathbf{A = \sum m(1, 2, 4, 7)}$
- B outputs 0 at decimal rows $0, 1, 2, 4 \implies \mathbf{B = \prod M(0, 1, 2, 4)}$

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Minimization Techniques

K-Maps (2/3/4/5-variable), Quine-McCluskey, Don't-Care Conditions

S3 Minimization Techniques

Prime Implicants

Q1. Differentiate between prime implicant and essential prime implicant of a K-Map. **(7 marks)** [CSE 205 | L-2/T-1 | 2023–24, 2011–12]

Answer

Core Definitions

Before differentiating the two, we must define an **Implicant**: In Boolean algebra, an implicant is any valid single group of 1s (minterms) in a Karnaugh Map that can be expressed as a product term.

1. Prime Implicant (PI)

A **Prime Implicant** is an implicant that is as large as possible. It represents a group of 2^n minterms that cannot be completely entirely encompassed by any larger valid group.

- **Rule of Thumb:** If you can double the size of the group, the current group is *not* a Prime Implicant.
- **Hardware Meaning:** It represents a logic gate with the absolute minimum number of inputs necessary for that specific grouping.

2. Essential Prime Implicant (EPI)

An **Essential Prime Implicant** is a specific type of Prime Implicant that covers **at least one minterm that is not covered by any other Prime Implicant**.

- **Rule of Thumb:** If removing the group leaves a minterm stranded (uncovered), that group is essential.
- **Hardware Meaning:** Because it is the *only* way to cover certain logic conditions, an EPI **must** be included in the final minimized hardware circuit.

Key Differences

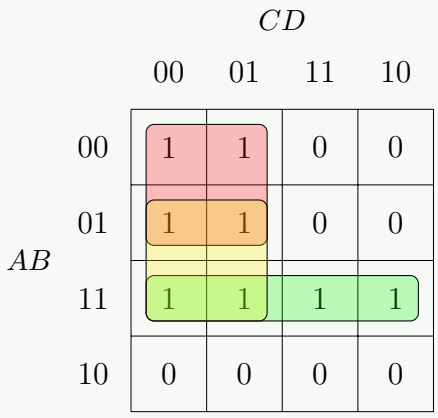
Feature	Prime Implicant (PI)	Essential Prime Implicant (EPI)
Definition	The largest possible grouping of adjacent minterms.	A PI that contains at least one unique, isolated minterm.
Necessity in Final Expression	**Optional.** It may or may not be included in the final minimized Boolean expression.	**Mandatory.** It *must* be included in the final minimized Boolean expression.
Coverage	All its minterms might be overlapped by other PIs.	Has at least one minterm that cannot be grouped by any other PI.
Subsets	All EPIs are PIs, but not all PIs are EPIs.	It is a strict subset of Prime Implicants.

Illustrative Example

Consider the 4-variable Boolean function:

$$F(A, B, C, D) = \sum m(0, 1, 4, 5, 12, 13, 14, 15)$$

Let us map this onto a Karnaugh Map to identify the PIs and EPIs.



Analysis of the K-Map:

(a) **Group 1 (Red/Green Border typically):** Minterms $(0, 1, 4, 5)$.

- It is a maximally sized 2×2 square. It is a **Prime Implicant**.
- Notice minterms 0 and 1. They cannot be covered by any other valid grouping.
- Therefore, this is an **Essential Prime Implicant (EPI)**. (Equation: $A'C'$)

(b) **Group 2 (Red/Green Border typically):** Minterms $(12, 13, 14, 15)$.

- It is a maximally sized 1×4 row. It is a **Prime Implicant**.
- Notice minterms 14 and 15. They cannot be covered by any other valid grouping.
- Therefore, this is an **Essential Prime Implicant (EPI)**. (Equation: AB)

(c) **Group 3 (Overlapping Middle):** Minterms $(4, 5, 12, 13)$.

- It is a maximally sized 2×2 square. It is a **Prime Implicant**.
- However, look closely at its minterms: 4 and 5 are already covered by Group 1. 12 and 13 are already covered by Group 2.
- It does *not* contain any unique minterm.
- Therefore, this is a **Prime Implicant, but NOT an Essential Prime Implicant**. It represents redundant hardware and is discarded in the final answer. (Equation: BC')

Final Minimized Function (Using only EPIs):

$$F_{minimized} = A'C' + AB \tag{1}$$

Q2. Find all the prime implicants for the following Boolean function and determine which are essential:

$$F(A, B, C, D) = \Sigma\,(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

(11 marks) **[CSE 205 | L-2/T-1 | 2014–15]**

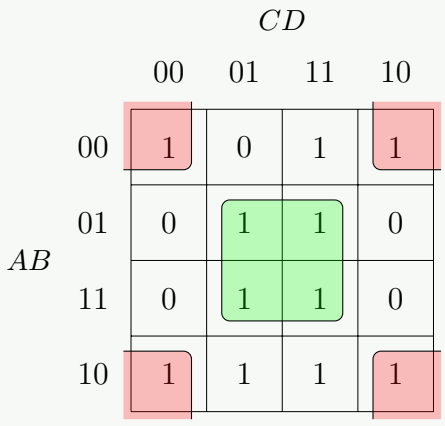
Answer

Step 1: Plotting the Karnaugh Map

We are given the 4-variable Boolean function:

$$F(A, B, C, D) = \Sigma\,(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

We plot the given minterms (1s) onto a 4-variable Karnaugh Map. Empty cells are assumed to be 0. To avoid visual clutter from overlapping groups, only the **Essential Prime Implicants** are graphically highlighted on the map below.



Step 2: Identifying All Prime Implicants (PIs)

A **Prime Implicant** is a maximally sized grouping of 1s (size 2^n) that cannot be entirely encompassed by any larger grouping. By inspecting all possible adjacent rectangular groups on the K-map, we identify **six** distinct Prime Implicants:

- PI_1: Grouping the four corners $(0, 2, 8, 10) \implies \mathbf{B'D'}$
- PI_2: Grouping the center-right square $(5, 7, 13, 15) \implies \mathbf{BD}$
- PI_3: Grouping the entire bottom row $(8, 9, 10, 11) \implies \mathbf{AB'}$
- PI_4: Grouping the entire third column $(3, 7, 11, 15) \implies \mathbf{CD}$
- PI_5: Grouping the bottom-right 2×2 square $(9, 11, 13, 15) \implies \mathbf{AD}$
- PI_6: Grouping the top and bottom edge cells in columns 11 and 10 $(2, 3, 10, 11) \implies \mathbf{B'C}$

Step 3: Prime Implicant Coverage Chart

To systematically determine which of these PIs are **essential**, we construct a Prime Implicant Chart. An Essential Prime Implicant (EPI) must cover at least one minterm that is not covered by any other PI.

PI	Term	0	2	3	5	7	8	9	10	11	13	15
PI_1 (EPI)	**B'D'**	**X**	X				X		X			
PI_2 (EPI)	**BD**				**X**	X					X	X
PI_3	**AB'**						X	X	X	X		
PI_4	**CD**			X		X				X		X
PI_5	**AD**							X		X	X	X
PI_6	**B'C**		X	X					X	X		

Step 4: Conclusion

By examining the columns of the Prime Implicant chart, we look for columns that contain only a single 'X'. These identify minterms that are uniquely covered by only one logic block.

- **Minterm 0** is uniquely covered by PI_1 $(B'D')$.
- **Minterm 5** is uniquely covered by PI_2 (BD).

All other minterms are covered by overlapping groups (two or more PIs). Therefore, PI_1 and PI_2 are strictly essential to the final minimized circuit.

Final Results:

All Prime Implicants: B'D', BD, AB', CD, AD, B'C
Essential Prime Implicants: B'D', BD

Karnaugh Map

Q1. Using K-Map, minimize $f(v, w, x, y, z) = \Sigma\,(2, 7, 8, 15, 22, 23, 25, 29) + d(6, 9, 13, 18, 24, 31)$. Find the Prime Implicants and Essential Prime Implicants, and the Minimal Sums. Write down the responsible minterm for each Essential Prime Implicant. **(15 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Answer

1. 5-Variable Karnaugh Map Setup

For a 5-variable function $f(v, w, x, y, z)$, we construct two 4-variable K-maps. The first map corresponds to $v = 0$ (minterms 0 to 15) and the second map corresponds to $v = 1$ (minterms 16 to 31).
Given:

$$\text{Minterms (1s)} : \Sigma\,(2, 7, 8, 15, 22, 23, 25, 29)$$
$$\text{Don't Cares (Xs)} : d(6, 9, 13, 18, 24, 31)$$

We distribute these into the respective maps. For the $v = 1$ map, the physical cell locations correspond to the given minterm/don't care index minus 16.
To maintain visual clarity, the K-maps below explicitly show the groupings for **one** of the valid Minimal Sums (MS_1). All Prime Implicants will be listed systematically in the next section.

Map for $v = 0$

wx \ yz	00	01	11	10
00	0	0	0	1
01	0	0	1	-
11	0	-	1	0
10	1	-	0	0

Map for $v = 1$

wx \ yz	00	01	11	10
00	0	0	0	-
01	0	0	1	1
11	0	1	-	0
10	-	1	0	0

---

2. Identification of Prime Implicants (PIs)

By systematically grouping the maximum number of adjacent 1s and Xs (in sizes of 2^n), we identify the following **six** Prime Implicants. Notice that these groups span symmetrically across both the $v = 0$ and $v = 1$ maps, thereby eliminating the variable v.

- PI_1**:** Grouping $(2, 6, 18, 22) \implies$ **w′yz′**
- PI_2**:** Grouping $(8, 9, 24, 25) \implies$ **wx′y′**
- PI_3**:** Grouping $(7, 15, 23, 31) \implies$ **xyz**
- PI_4**:** Grouping $(9, 13, 25, 29) \implies$ **wy′z**
- PI_5**:** Grouping $(13, 15, 29, 31) \implies$ **wxz**
- PI_6**:** Grouping $(6, 7, 22, 23) \implies$ **w′xy**

---

3. Essential Prime Implicants (EPIs) and Coverage

To determine the Essential Prime Implicants, we construct a PI Coverage Chart. We only list the true minterms (excluding don't cares) because don't cares do not strictly need to be covered.

PI	**Term**	**2**	**7**	**8**	**15**	**22**	**23**	**25**	**29**
PI_1	**w′yz′**	**X**				X			
PI_2	**wx′y′**			**X**				X	
PI_3	**xyz**		X		X		X		
PI_4	**wy′z**							X	X
PI_5	**wxz**				X				X
PI_6	**w′xy**		X			X	X		

By scanning the columns for single 'X' marks, we find the EPIs and their responsible minterms:

- **Minterm 2** is covered *only* by PI_1. Thus, **w′yz′** is an EPI.
- **Minterm 8** is covered *only* by PI_2. Thus, **wx′y′** is an EPI.

Summary of EPIs & Responsible Minterms:

EPI: **w′yz′** $\longrightarrow$ Responsible minterm: $\mathbf{m_2}$
EPI: **wx′y′** $\longrightarrow$ Responsible minterm: $\mathbf{m_8}$

---

4. Minimal Sums

Selecting the two EPIs (PI_1 and PI_2) automatically covers minterms $2, 8, 22$, and 25. The minterms remaining to be covered are: $\{7, 15, 23, 29\}$.
We must cover these remaining minterms using the fewest possible remaining PIs. Since all PIs are 3-literal terms, cost is identical across choices. Let us evaluate combinations of 2 additional PIs that satisfy the remaining cover:

(a) Selecting PI_3 covers $\{7, 15, 23\}$. We still need 29, which is covered by PI_4. $\implies \{PI_3, PI_4\}$

(b) Selecting PI_5 covers $\{15, 29\}$. We still need $\{7, 23\}$, which is covered by PI_6. $\implies \{PI_5, PI_6\}$

(c) Selecting PI_3 covers $\{7, 15, 23\}$. Selecting PI_5 covers $\{15, 29\}$. Together they cover all remaining. $\implies \{PI_3, PI_5\}$

Therefore, there are exactly **three** valid Minimal Sums for this function:

$$\textbf{Minimal Sum 1:} \quad f = \mathbf{w'yz' + wx'y' + xyz + wy'z}$$
$$\textbf{Minimal Sum 2:} \quad f = \mathbf{w'yz' + wx'y' + w'xy + wxz}$$
$$\textbf{Minimal Sum 3:} \quad f = \mathbf{w'yz' + wx'y' + xyz + wxz}$$

Q2. Convert the following Boolean function from sum-of-products form to a simplified product-of-sums form, then draw a logic circuit for the POS form:

$$F(w, x, y, z) = \Sigma\,(0, 1, 2, 5, 7, 11, 13)$$

(20 marks) [CSE 205 | L-2/T-1 | 2019–20]

Answer

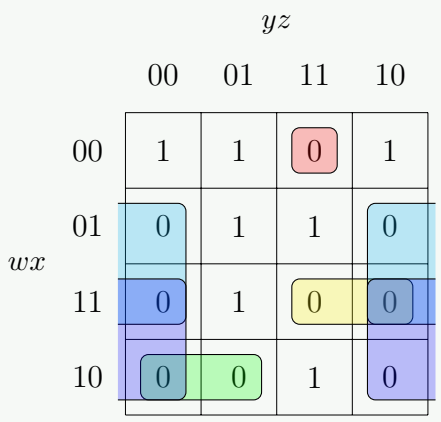
Step 1: Identify the Maxterms
The Boolean function is given in canonical Sum-of-Products (SOP) form with minterms:

$$F(w, x, y, z) = \Sigma\,(0, 1, 2, 5, 7, 11, 13)$$

For a 4-variable function, the total possible minterms range from 0 to 15. The minterms that are *not* present in F correspond to the maxterms (zeros) of the function.

$$F(w, x, y, z) = \Pi\,(3, 4, 6, 8, 9, 10, 12, 14, 15)$$

Step 2: Karnaugh Map for Product-of-Sums (POS) Minimization
To find the simplified POS expression, we map the zeros (maxterms) of F on a Karnaugh map. Grouping the zeros directly yields the minimal sum terms.



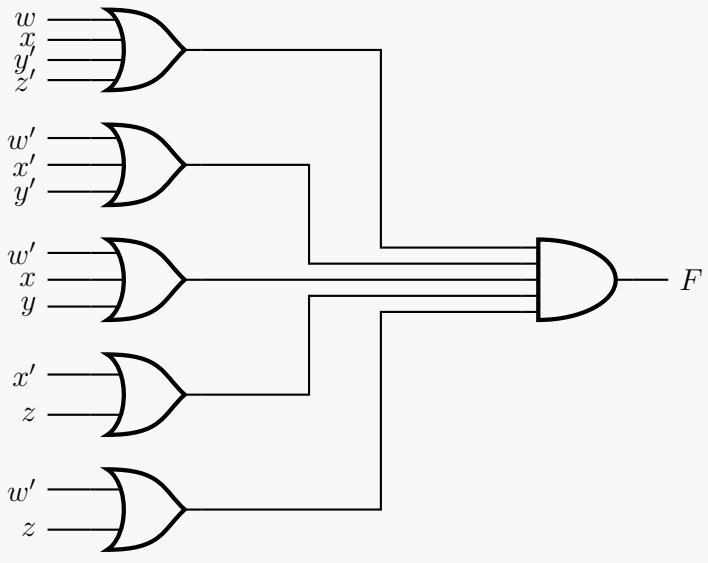
Step 3: Deriving the Sum Terms
From the groups of zeros, we extract the corresponding sum terms. Remember that for zeros (POS), an input value of 0 is represented by the uncomplemented variable, and an input value of 1 is represented by the complemented variable.

- **Group 1 (Single 0 at m_3):** Fixed $w = 0, x = 0, y = 1, z = 1 \implies (\mathbf{w + x + y' + z'})$
- **Group 2 (Pair at 14, 15):** Fixed $w = 1, x = 1, y = 1 \implies (\mathbf{w' + x' + y'})$
- **Group 3 (Pair at 8, 9):** Fixed $w = 1, x = 0, y = 0 \implies (\mathbf{w' + x + y})$
- **Group 4 (Quad at 4, 6, 12, 14):** Fixed $x = 1, z = 0 \implies (\mathbf{x' + z})$
- **Group 5 (Quad at 8, 10, 12, 14):** Fixed $w = 1, z = 0 \implies (\mathbf{w' + z})$

Algebraic Verification (De Morgan's Law): If we grouped the same cells as 1s for the complement function F', we would get $F' = w'x'yz + wxy + wx'y' + xz' + wz'$. Taking the complement of both sides and applying De Morgan's Law gives:

$$\begin{aligned}
F &= (F')' \\
&= (w'x'yz + wxy + wx'y' + xz' + wz')' \\
&= (w'x'yz)' \cdot (wxy)' \cdot (wx'y')' \cdot (xz')' \cdot (wz')' && \text{(De Morgan's Law: } (A + B)' = A'B') \\
&= \mathbf{(w + x + y' + z')(w' + x' + y')(w' + x + y)(x' + z)(w' + z)} && \text{(De Morgan's Law: } (AB)' = A' + B')
\end{aligned}$$

Step 4: Logic Circuit Diagram for POS Form
The POS function is implemented using a 2-level OR-AND logic structure.



Q3. Using K-Map, minimize $f(w, x, y, z) = \Sigma\,(1, 2, 3, 7, 10, 12, 14) + d(0, 6, 8, 13)$. Find the Prime Implicants and Essential Prime Implicants. Write down the responsible minterm for each Essential Prime Implicant. **(12 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Answer

1. Karnaugh Map Construction
The Boolean function is given as $f(w, x, y, z) = \Sigma\,(1, 2, 3, 7, 10, 12, 14) + d(0, 6, 8, 13)$. We map the minterms as '1', the don't cares as 'd', and the remaining cells as '0'.

wx \ yz	00	01	11	10
00	-	1	1	1
01	0	0	1	-
11	1	-	0	1
10	-	0	0	1

2. Identifying Prime Implicants (PIs)
A Prime Implicant is a maximal grouping of 1s and don't cares (i.e., a group that cannot be entirely consumed by a larger valid group). By analyzing the K-Map, we extract all possible maximal groupings:

Prime Implicant	Cells Covered	Minterms Covered (excluding 'd')
$w'x'$	$0, 1, 3, 2$	m_1, m_2, m_3
$w'y$	$3, 2, 7, 6$	m_2, m_3, m_7
$x'z'$	$0, 2, 8, 10$	m_2, m_{10}
yz'	$2, 6, 14, 10$	m_2, m_{10}, m_{14}
wz'	$12, 14, 8, 10$	m_{10}, m_{12}, m_{14}
wxy'	$12, 13$	m_{12}

3. Essential Prime Implicants (EPIs)
An Essential Prime Implicant is a PI that covers at least one minterm (1) that is *not* covered by any other Prime Implicant. This exclusively covered minterm is known as the *responsible minterm* or *distinguished minterm*.
By cross-referencing our PIs:

- Minterm m_1 is covered **only** by the PI $w'x'$.
- Minterm m_7 is covered **only** by the PI $w'y$.
- All other minterms ($m_2, m_{10}, m_{12}, m_{14}$) are covered by at least two different PIs.

Essential Prime Implicant	Responsible Minterm
$\mathbf{w'x'}$	m_1
$\mathbf{w'y}$	m_7

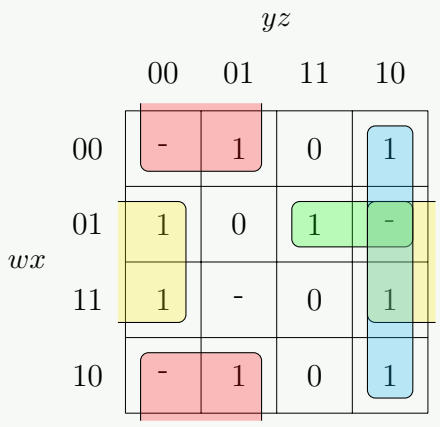
4. Minimal Sum-of-Products (SOP) Expression
To find the final minimized expression, we must first include all Essential Prime Implicants, which cover minterms m_1, m_2, m_3, and m_7.
The remaining uncovered minterms are m_{10}, m_{12}, and m_{14}. Looking at our list of available PIs, the single implicant wz' perfectly covers all three of these remaining minterms simultaneously, making it the most optimal choice to complete the function.

$$f(w,x,y,z) = \text{EPIs} + \text{Remaining Minimal Cover}$$
$$f(w,x,y,z) = \mathbf{w'x' + w'y + wz'}$$

Q4. Using K-Map, minimize $f(w,x,y,z) = \text{SOP}(1,2,4,7,9,10,12,14) + d(0,6,8,13)$. Find the Prime Implicants and Essential Prime Implicants. Write down the responsible minterm for each Essential Prime Implicant. **(12 marks)** **[CSE 205 | L-2/T-1 | 2017–18]**

Answer

1. Karnaugh Map Construction
The Boolean function is given as $f(w,x,y,z) = \text{SOP}(1,2,4,7,9,10,12,14) + d(0,6,8,13)$. We plot the minterms as '1', the don't cares as 'd', and leave the remaining cells as '0'.



2. Identifying Prime Implicants (PIs)
A Prime Implicant is a maximal grouping of 1s and don't cares (i.e., a group that cannot be entirely consumed by a larger valid group). By thoroughly analyzing the K-Map, we extract all possible maximal groupings:

Prime Implicant	Cells Covered	Minterms Covered (excluding 'd')
$x'y'$	$0,1,8,9$	m_1, m_9
wy'	$8,9,12,13$	m_9, m_{12}
$y'z'$	$0,4,8,12$	m_4, m_{12}
xz'	$4,6,12,14$	m_4, m_{12}, m_{14}
yz'	$2,6,10,14$	m_2, m_{10}, m_{14}
$x'z'$	$0,2,8,10$	m_2, m_{10}
$w'xy$	$6,7$	m_7

3. Essential Prime Implicants (EPIs)
An Essential Prime Implicant is a PI that covers at least one minterm (1) that is *not* covered by any other Prime Implicant. This exclusively covered minterm is known as the *responsible minterm*.
By cross-referencing our PIs with the minterms they cover:

- Minterm m_1 is covered **only** by the PI $x'y'$.
- Minterm m_7 is covered **only** by the PI $w'xy$.
- All other minterms $(m_2, m_4, m_9, m_{10}, m_{12}, m_{14})$ are covered by at least two different PIs.

Essential Prime Implicant	Responsible Minterm
$\mathbf{x'y'}$	m_1
$\mathbf{w'xy}$	m_7

4. Minimal Sum-of-Products (SOP) Expression
To find the final minimized expression, we must first include all Essential Prime Implicants, which cover minterms m_1, m_7, and m_9 (since m_9 is absorbed by $x'y'$).
The remaining uncovered minterms are m_2, m_4, m_{10}, m_{12}, and m_{14}. Looking at our list of available PIs, we want to select the minimum number of implicants to cover these five minterms:

- Selecting yz' covers m_2, m_{10}, and m_{14}.

- Selecting xz' covers m_4, m_{12}, and m_{14}.

These two PIs together perfectly cover all of the remaining minterms. Adding them to our EPIs yields the optimal and complete sum-of-products equation.

$$f(w,x,y,z) = \text{EPIs} + \text{Remaining Minimal Cover}$$
$$f(w,x,y,z) = \mathbf{x'y' + w'xy + xz' + yz'}$$

Q5. Using K-map, solve $f(x_1,x_2,x_3,x_4,x_5) = \Sigma\,(0,2,4,5,6,7,8,10,14,17,18,21,29,31) + d\Sigma\,(11,20,22)$. Find the essential prime implicants. (Show the reduction steps.) **(15 marks)** **[CSE 205 | L-2/T-1 | 2016–17]**

Answer

1. Problem Statement & Setup

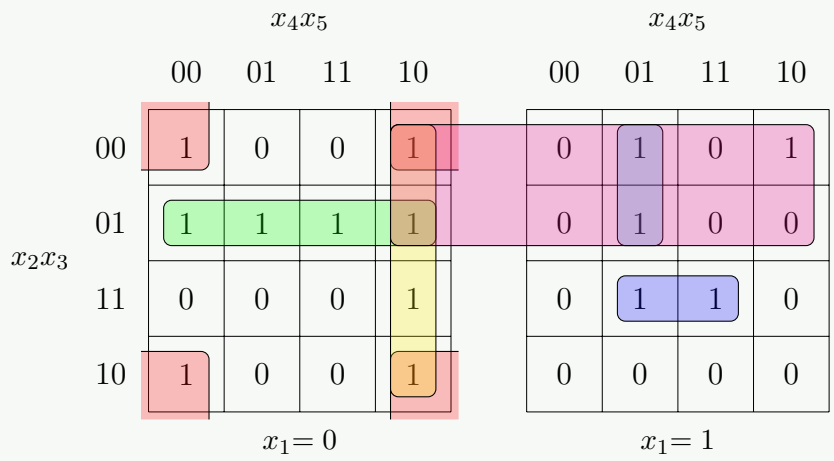
We are given a 5-variable Boolean function defined by its minterms (Sum of Products) and don't care conditions:

$$f(x_1,x_2,x_3,x_4,x_5) = \Sigma\,(0,2,4,5,6,7,8,10,14,17,18,21,29,31) + d\Sigma\,(11,20,22)$$

To minimize this function, we construct a 5-variable Karnaugh Map. A 5-variable K-map is visually represented as two adjacent 4-variable K-maps. The left map corresponds to $x_1 = 0$ and the right map corresponds to $x_1 = 1$. The cells represent the remaining variables x_2, x_3, x_4, x_5.

2. Karnaugh Map Construction

We populate the map with 1 for the minterms, d for the don't care conditions, and 0 for the remaining cells. Groupings are made to cover all 1s using the largest possible combinations (sizes of 2^n), utilizing don't cares (d) only when they help to enlarge a group.



3. Algebraic Foundation of Reduction Steps

Visual grouping on a K-map is mathematically grounded in the **Adjacency Theorem** ($XY + XY' = X$). When we group cells, we systematically eliminate variables that exist in both their true and complemented forms. Let us explicitly demonstrate the Boolean theorems applied during these reductions:

Example 1: A Pair (Group 5 - Minterms 17 and 21)

$$\begin{aligned} m_{17} + m_{21} &= x_1x_2'x_3'x_4'x_5 + x_1x_2'x_3x_4'x_5 \\ &= x_1x_2'x_4'x_5(x_3' + x_3) && \text{(Distributive Law)} \\ &= x_1x_2'x_4'x_5(1) && \text{(Complementarity Law: } A + A' = 1) \\ &= x_1x_2'x_4'x_5 && \text{(Identity Law: } A \cdot 1 = A) \end{aligned}$$

Example 2: A Quad (Group 2 - Minterms 4, 5, 6, 7)

$$\begin{aligned} m_4 + m_5 + m_6 + m_7 &= x_1'x_2'x_3x_4'x_5' + x_1'x_2'x_3x_4'x_5 + x_1'x_2'x_3x_4x_5' + x_1'x_2'x_3x_4x_5 \\ &= x_1'x_2'x_3x_4'(x_5' + x_5) + x_1'x_2'x_3x_4(x_5' + x_5) && \text{(Distributive Law)} \\ &= x_1'x_2'x_3x_4'(1) + x_1'x_2'x_3x_4(1) && \text{(Complementarity Law)} \\ &= x_1'x_2'x_3(x_4' + x_4) && \text{(Distributive \& Identity Laws)} \\ &= x_1'x_2'x_3 && \text{(Complementarity \& Identity Laws)} \end{aligned}$$

4. Identification of Essential Prime Implicants (EPIs)

A Prime Implicant (PI) becomes an **Essential Prime Implicant** if it is the *only* possible group that covers a specific minterm. To find the minimal Sum of Products (SOP), we must include all EPIs.

Let us evaluate the mapped groups to find the essential ones:

Group Identifier	Simplified Term	Minterms Covered	Distinguishing Minterm
G_1 (4-corners of $x_1 = 0$)	$x_1'x_3'x_5'$	$0, 2, 8, 10$	**8**
G_2 (Full row in $x_1 = 0$)	$x_1'x_2'x_3$	$4, 5, 6, 7$	**7**
G_3 (Full column in $x_1 = 0$)	$x_1'x_4x_5'$	$2, 6, 10, 14$	**14**
G_4 (Spans both maps)	$x_2'x_4x_5'$	$2, 6, 18, 22$ (d)	**18**
G_5 (Pair in $x_1 = 1$)	$x_1x_2'x_4'x_5$	$17, 21$	**17**
G_6 (Pair in $x_1 = 1$)	$x_1x_2x_3x_5$	$29, 31$	**31**

Every group listed above contains at least one minterm (highlighted in the final column) that cannot be covered by any other valid grouping. Therefore, **all six of these groups are Essential Prime Implicants**.

5. Final Minimal SOP Expression

We must now verify if any valid 1s remain uncovered by the established EPIs. The set of minterms covered by the union of G_1 through G_6 is:

$$\{0, 2, 8, 10\} \cup \{4, 5, 6, 7\} \cup \{2, 6, 10, 14\} \cup \{2, 6, 18\} \cup \{17, 21\} \cup \{29, 31\}$$
$$= \{0, 2, 4, 5, 6, 7, 8, 10, 14, 17, 18, 21, 29, 31\}$$

This perfectly matches our original function's minterm list. Because there are no remaining uncovered minterms, the minimal expression is simply the logical OR (sum) of all the Essential Prime Implicants.

Minimal SOP Expression:

$$f(x_1, x_2, x_3, x_4, x_5) = \mathbf{x_1'x_3'x_5' + x_1'x_2'x_3 + x_1'x_4x_5'} + \mathbf{x_2'x_4x_5' + x_1x_2'x_4'x_5 + x_1x_2x_3x_5}$$

Q6. Using K-maps, find the simplified product of sums form of the following Boolean function:

$$f(w, x, y, z) = \Sigma\,(1, 5, 8, 12, 14, 15), \quad \text{don't-care minterms: } d = \Sigma\,(3, 11)$$

(10 marks) **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

1. Problem Statement & Maxterm Identification

We are tasked with finding the simplified **Product of Sums (POS)** form for the 4-variable Boolean function:

$$f(w, x, y, z) = \Sigma\,(1, 5, 8, 12, 14, 15), \quad d = \Sigma\,(3, 11)$$

To find the POS form, we must group the **Maxterms** (the 0s) in the Karnaugh map. The total number of combinations for 4 variables is 16 (from 0 to 15). The Maxterms are the missing combinations not covered by the given minterms (1s) or don't-care conditions (ds):

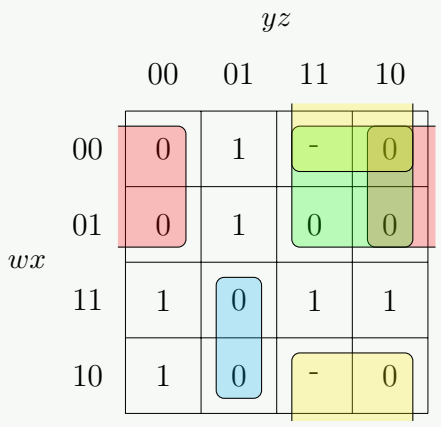
$$\begin{aligned} \text{Total Set} &= \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15\} \\ \text{Minterms (1s)} &= \{1, 5, 8, 12, 14, 15\} \\ \text{Don't-cares } (d\text{s}) &= \{3, 11\} \\ \text{Maxterms } (\Pi) &= \text{Total Set} - (\text{Minterms} \cup \text{Don't-cares}) \\ \Pi &= \{0, 2, 4, 6, 7, 9, 10, 13\} \end{aligned}$$

Thus, the function in standard POS form (with don't-cares) is:

$$f(w, x, y, z) = \Pi\,(0, 2, 4, 6, 7, 9, 10, 13) \cdot d\,(3, 11)$$

2. Karnaugh Map Construction

We populate the K-map by placing 0s at the Maxterm locations, ds at the don't-care locations, and 1s elsewhere. To simplify for POS, we group the 0s. We can use the ds to make our groupings larger, which further minimizes the Boolean terms.



3. Derivation of Sum Terms (POS Logic)
By grouping the 0s, we are fundamentally finding the minimal Sum of Products (SOP) for the inverted function, f'. Applying De Morgan's Law $(f')' = f$ yields the Product of Sums. A direct rule for reading POS from a K-map of 0s is: **Read the common variables for a group, but invert the literals** (i.e., if a common variable is 0, write it as an uncomplemented variable; if it is 1, write it as a complemented variable), and sum them together.
Let us evaluate each essential group of 0s:

Group	Cells Covered	Common Variables	Resulting Sum Term
G_1 (Quad)	$0, 2, 4, 6$ (Left & Right edges)	$w = 0,\ z = 0$	$(\mathbf{w + z})$
G_2 (Quad)	$2, 3, 6, 7$ (Top-right block)	$w = 0,\ y = 1$	$(\mathbf{w + y'})$
G_3 (Quad)	$2, 3, 10, 11$ (Top & Bottom edges)	$x = 0,\ y = 1$	$(\mathbf{x + y'})$
G_4 (Pair)	$9, 13$ (Vertical pair)	$w = 1,\ y = 0,\ z = 1$	$(\mathbf{w' + y + z'})$

Verification Step: All 0s $(0, 2, 4, 6, 7, 9, 10, 13)$ have been successfully covered by these four groups. The don't-cares $(3, 11)$ were strategically utilized to expand pairs into quads (G_2 and G_3), simplifying the resulting terms by dropping one additional variable.

4. Final Minimal POS Expression
Combining the derived sum terms together via logical AND (product), we arrive at the simplified Product of Sums expression for the Boolean function:

Simplified POS Expression:

$$f(w, x, y, z) = (\mathbf{w + z})(\mathbf{w + y'})(\mathbf{x + y'})(\mathbf{w' + y + z'})$$

Q7. Using K-maps, find the simplified sum of products form of the following Boolean function:

$$f(a, b, c, d, e, f) = \Sigma\,(1, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31)$$

Don't-care minterms: $d = \Sigma\,(6, 20, 24, 25, 26, 27)$. **(12 marks)** **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

1. Variable Analysis & Dimension Reduction
We are given a 6-variable Boolean function $f(a, b, c, d, e, f)$ with minterms up to 31. The total number of minterms for a 6-variable function spans from 0 to 63. Since the highest minterm (31) corresponds to the binary representation 011111_2, the most significant bit (MSB), variable a, is 0 for all active minterms and don't-care conditions.
Consequently, we can factor out a' and express the function as:

$$f(a, b, c, d, e, f) = a' \cdot g(b, c, d, e, f)$$

where $g(b, c, d, e, f)$ is a 5-variable function. We can plot g using two 4x4 Karnaugh maps representing the $b = 0$ and $b = 1$ planes. The variables c, d map to the rows and e, f map to the columns.

2. 5-Variable Karnaugh Map Construction
For the $b = 0$ map, the cell indices match the global minterm values directly (0 to 15). For the $b = 1$ map, the local indices are shifted by 16 (i.e., local cell i corresponds to global minterm $i + 16$).

- **Map 0 ($b = 0$):** Minterms: $1, 2, 3, 5, 7, 11, 13$. Don't cares: 6.
- **Map 1 ($b = 1$):** Minterms: $17, 19, 23, 29, 31$. Don't cares: $20, 24, 25, 26, 27$.
  (Local map indices for $b = 1$: minterms $1, 3, 7, 13, 15$; don't cares $4, 8, 9, 10, 11$).

Map 0: $a = 0, b = 0$

cd \ ef	00	01	11	10
00	0	1	1	1
01	0	1	1	-
11	0	1	0	0
10	0	0	1	0

Map 1: $a = 0, b = 1$

cd \ ef	00	01	11	10
00	0	1	1	0
01	-	0	1	0
11	0	1	1	0
10	-	-	-	-

3. Prime Implicants & Boolean Term Derivation
We identify the optimal groupings to cover all 1s. Groups spanning across both Map 0 and Map 1 (3D groups) eliminate the b variable. Note that every derived term is fundamentally intersected with a' because $a = 0$.

Group	Spanning Cells (Global Index)	Constant Variables	Term
G_1 (Quad)	Map 0: $2, 3, d_6, 7$	$a = 0, b = 0, c = 0, e = 1$	$a'b'c'e$
G_2 (3D Quad)	Map 0: $3, 11$ & Map 1: $19, d_{27}$	$a = 0, d = 0, e = 1, f = 1$	$a'd'ef$
G_3 (3D Quad)	Map 0: $1, 3$ & Map 1: $17, 19$	$a = 0, c = 0, d = 0, f = 1$	$a'c'd'f$
G_4 (Quad)	Map 0: $1, 3, 5, 7$	$a = 0, b = 0, c = 0, f = 1$	$a'b'c'f$
G_5 (3D Pair)	Map 0: 13 & Map 1: 29	$a = 0, c = 1, d = 1, e = 0, f = 1$	$a'cde'f$
G_6 (Quad)	Map 1: $19, 23, d_{27}, 31$	$a = 0, b = 1, e = 1, f = 1$	$a'bef$

Note: There are alternative minimal choices for covering isolated minterms like m_5 and m_{13} (e.g., pairing m_5 with m_{13}), but the selected combination maintains mathematical minimalism with 6 product terms.

4. Final Simplified SOP Expression
Combining all the identified prime implicants, we yield the fully minimized Sum of Products (SOP) expression for the Boolean function:

Simplified SOP Expression:

$$f(a, b, c, d, e, f) = \mathbf{a'b'c'e + a'd'ef + a'c'd'f + a'b'c'f + a'cde'f + a'bef}$$

Q8. Simplify the following Boolean functions using K-map:

(a) $F(A, B, C, D) = AB'C + A'B'D' + ABD + ACD' + AB'C' + A'BC'D$

(b) $F(A, B, C, D) = \Sigma\,(0, 2, 5, 7, 8, 15)$ with don't-care conditions $d(A, B, C, D) = \Sigma\,(10, 14)$

(15 marks) [CSE 205 | L-2/T-1 | 2013–14]

Answer

Part 1: Simplification of the first Boolean function

Step 1: Expand terms to standard minterms (Canonical SOP)
Given the function:

$$F(A, B, C, D) = AB'C + A'B'D' + ABD + ACD' + AB'C' + A'BC'D$$

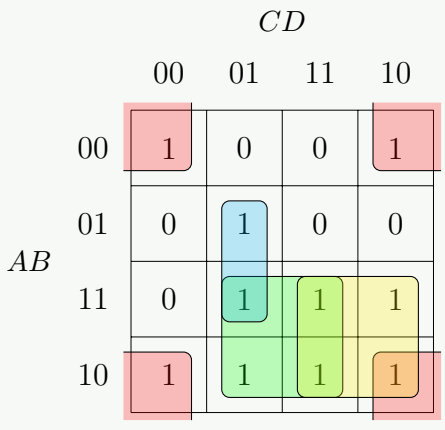
We expand each product term to its full minterms by applying the Boolean property $X + X' = 1$:

$$\begin{aligned}
AB'C &= AB'C(D + D') = AB'CD + AB'CD' && \rightarrow m_{11}(1011), m_{10}(1010)\\
A'B'D' &= A'B'(C + C')D' = A'B'CD' + A'B'C'D' && \rightarrow m_2(0010), m_0(0000)\\
ABD &= AB(C + C')D = ABCD + ABC'D && \rightarrow m_{15}(1111), m_{13}(1101)\\
ACD' &= A(B + B')CD' = ABCD' + AB'CD' && \rightarrow m_{14}(1110),\ (m_{10} \text{ already listed})\\
AB'C' &= AB'C'(D + D') = AB'C'D + AB'C'D' && \rightarrow m_9(1001), m_8(1000)\\
A'BC'D &= A'BC'D && \rightarrow m_5(0101)
\end{aligned}$$

Combining these, the function can be expressed as a sum of minterms:

$$F(A, B, C, D) = \Sigma\,(0, 2, 5, 8, 9, 10, 11, 13, 14, 15)$$

Step 2: Karnaugh Map Plotting and Grouping



Essential Prime Implicants Analysis:

- **Corners (Quad):** Cells $0, 2, 8, 10$ form a quad. Constants: $B = 0, D = 0 \implies$ **B′D′**.
- **Central Quad:** Cells $9, 11, 13, 15$ form a quad. Constants: $A = 1, D = 1 \implies$ **AD**.
- **Right-side Quad:** Cells $10, 11, 14, 15$ form a quad. Constants: $A = 1, C = 1 \implies$ **AC**.
- **Vertical Pair:** Cells $5, 13$ form a pair. Constants: $B = 1, C = 0, D = 1 \implies$ **BC′D**.

Note: The potential quad of $8, 9, 10, 11$ *(*AB'*) is redundant because all its* 1*s are already covered by the* $B'D'$*,* AD*, and* AC *groups.*

Final Simplified Expression 1:

$$F(A, B, C, D) = \mathbf{B'D' + AD + AC + BC'D}$$

Part 2: Simplification with Don't-Care Conditions

Step 1: K-Map Formulation
We are given:

$$F(A, B, C, D) = \Sigma\,(0, 2, 5, 7, 8, 15)$$
$$d(A, B, C, D) = \Sigma\,(10, 14)$$

We plot the 1s for the minterms and Xs for the don't-cares on the 4-variable K-map. Don't-cares are strategically included in groupings only if they help yield larger groups, thereby further reducing literal counts.

AB \ CD	00	01	11	10
00	1	0	0	1
01	0	1	1	0
11	0	0	1	-
10	1	0	0	-

Step 2: Grouping Strategy

- **Corners (Quad):** We group m_0, m_2, m_8, and use the don't-care d_{10}. This forms a quad.
  Constants: $B = 0, D = 0 \implies$ **B′D′**.
- **Horizontal Pair:** m_5 and m_7 form an isolated pair.
  Constants: $A = 0, B = 1, D = 1 \implies$ **A′BD**.
- **Vertical Pair:** m_{15} must be covered. We can pair it with m_7 vertically.
  Constants: $B = 1, C = 1, D = 1 \implies$ **BCD**.
  (Alternatively, m_{15} *could pair with the don't care* d_{14} *to yield* **ABC***. Both solutions are minimal and equally valid).*

Final Simplified Expression 2:

$$F(A, B, C, D) = \mathbf{B'D' + A'BD + BCD}$$

Q9. Minimize the following function in both SOP and POS forms using Karnaugh map:

$$f(A, B, C, D) = \Sigma\, m(0, 4, 7, 9, 13) + d(5, 8, 15)$$

(12 marks) **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

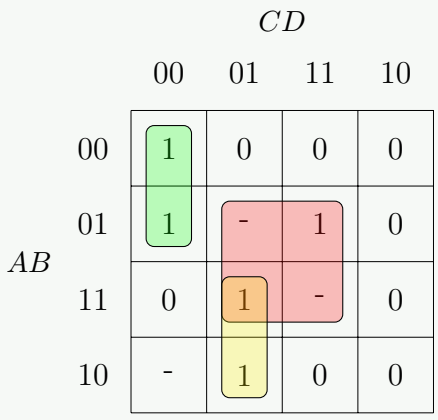
Part 1: Sum of Products (SOP) Minimization

Step 1: Identify Minterms and Don't Cares
The Boolean function is given in terms of minterms (1s) and don't-care conditions (Xs):

$$f(A, B, C, D) = \Sigma\, m(0, 4, 7, 9, 13) + d(5, 8, 15)$$

Step 2: Karnaugh Map Plotting and Grouping for SOP

We plot the 1s for the minterms and Xs for the don't-cares on a 4-variable K-map. We aim to form the largest possible groups (powers of 2) of 1s, incorporating Xs only if they help increase the group size.



Essential Prime Implicants Analysis:

- **Quad:** Cells $5, 7, 13, 15$ form a 2×2 square.
  Constant variables: $B = 1, D = 1 \implies \mathbf{BD}$.
- **Vertical Pair 1:** Cells $0, 4$ form a pair.
  Constant variables: $A = 0, C = 0, D = 0 \implies \mathbf{A'C'D'}$.
- **Vertical Pair 2:** Cells $9, 13$ form a pair.
  Constant variables: $A = 1, C = 0, D = 1 \implies \mathbf{AC'D}$.
  (Note: Minterm m_9 could alternatively be paired with d_8 to yield $AB'C'$. Both result in a minimal 3-literal term, making either choice perfectly valid.)

Final Simplified SOP Expression:

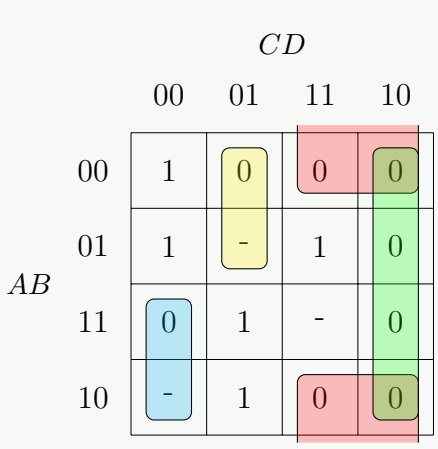
$$f_{SOP}(A, B, C, D) = \mathbf{BD + A'C'D' + AC'D}$$

Part 2: Product of Sums (POS) Minimization

Step 1: Identify Maxterms
To find the POS form, we focus on the 0s of the function. The maxterms (0s) are the missing indices from the universal set of 16 cells, excluding the don't-cares:

$$f(A, B, C, D) = \Pi\, M(1, 2, 3, 6, 10, 11, 12, 14) \cdot \Pi_d(5, 8, 15)$$

Step 2: Karnaugh Map Plotting and Grouping for POS
We plot the 0s and Xs. For POS minimization, each grouped block of 0s directly yields a sum term. A variable that remains constant as a 0 gives the uncomplemented literal (e.g., A), and a 1 gives the complemented literal (e.g., A').



Groupings for POS Sum Terms:

- **Wrap-around Quad:** Cells $2, 3, 10, 11$ form a quad by wrapping the top and bottom rows.
  Constants: $B = 0, C = 1 \implies (\mathbf{B + C'})$.
- **Column Quad:** Cells $2, 6, 14, 10$ form a full vertical column.
  Constants: $C = 1, D = 0 \implies (\mathbf{C' + D})$.

- **Vertical Pair 1:** Cells $1, 5$ form a pair (using d_5).
  Constants: $A = 0, C = 0, D = 1 \implies (\mathbf{A + C + D'})$.
  (Alternatively, M_1 could pair with M_3 to yield $(A + B + D')$. Both are valid 3-literal sums.)

- **Vertical Pair 2:** Cells $8, 12$ form a pair (using d_8).
  Constants: $A = 1, C = 0, D = 0 \implies (\mathbf{A' + C + D})$.
  (Alternatively, M_{12} could pair with M_{14} to yield $(A' + B' + D)$. Both are valid.)

Final Simplified POS Expression:

$$f_{POS}(A, B, C, D) = (\mathbf{B + C'})(\mathbf{C' + D})(\mathbf{A + C + D'})(\mathbf{A' + C + D})$$

Tabulation Method (Quine-McCluskey)

Q1. Simplify the following Boolean function to product-of-sums form:

$$F = \Sigma\,(3, 4, 6, 7, 9, 11, 12, 14, 15)$$

What is the main purpose of the tabulation method (Quine-McCluskey method) in digital logic design, and how does it systematically help in minimizing Boolean expressions compared to the Karnaugh map method? **(15 marks)** **[CSE 205 | L-2/T-1 | 2024–25]**

Answer

Part 1: Product-of-Sums (POS) Simplification

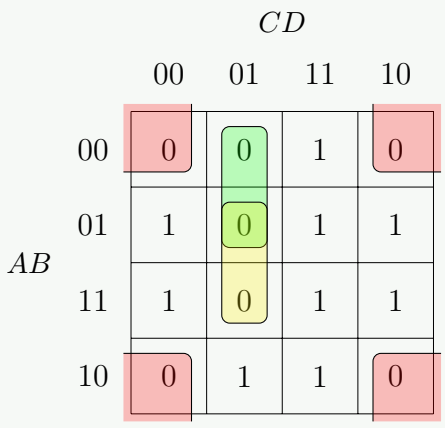
Step 1: Identify the Maxterms
The Boolean function is given in Sum of Minterms (SOP) format:

$$F(A, B, C, D) = \Sigma\, m(3, 4, 6, 7, 9, 11, 12, 14, 15)$$

To simplify into Product-of-Sums (POS) form, we must focus on the maxterms (0s) of the function. The maxterms are the indices missing from the universal set of 16 cells (from 0 to 15):

$$F(A, B, C, D) = \Pi\, M(0, 1, 2, 5, 8, 10, 13)$$

Step 2: Karnaugh Map Plotting and Grouping
We plot the 0s on a 4-variable K-map. For POS minimization, each valid group of 0s directly translates to a sum term. A variable that remains constant as a 0 yields the uncomplemented literal (e.g., A), and a 1 yields the complemented literal (e.g., A').



Essential Prime Implicant Analysis for Maxterms:

- **Corner Quad:** Cells $0, 2, 8, 10$ form a quad at the four corners.
  Constant variables: $B = 0, D = 0 \implies (\mathbf{B + D})$.

- **Vertical Pair 1:** Cell 13 is isolated and can only group with cell 5.
  Constant variables: $B = 1, C = 0, D = 1 \implies (\mathbf{B' + C + D'})$.

- **Vertical Pair 2:** Cell 1 must be covered. It can group with cell 5 (or cell 0). Grouping with 5 forms the pair $(1, 5)$.
  Constant variables: $A = 0, C = 0, D = 1 \implies (\mathbf{A + C + D'})$.
  (Note: Grouping cell 1 with cell 0 yields $(A + B + C)$, which is an equally valid minimal term. Both selections produce a minimal POS expression.)

Final Simplified POS Expression:

$$F_{POS}(A, B, C, D) = (\mathbf{B + D})(\mathbf{B' + C + D'})(\mathbf{A + C + D'})$$

Part 2: The Quine-McCluskey (Tabulation) Method

Main Purpose

The Quine-McCluskey (Q-M) method is an exact, algorithmic tabular technique used for minimizing Boolean functions. Its primary purpose is to systematically find the most simplified Boolean expression, particularly when functions contain more than four or five variables.

Systematic Advantages over the Karnaugh Map Method:

(a) **Algorithmic vs. Visual:** The K-map relies heavily on human visual pattern recognition to identify adjacent cells. While efficient for up to 4 variables, it becomes extremely error-prone and visually unwieldy for 5 or more variables. The Q-M method replaces graphical grouping with a strict algebraic tabulation, eliminating visual errors.

(b) **Exhaustive Prime Implicant Generation:** The Q-M method systematically groups minterms by their number of 1s (index) and compares every term against others differing by exactly one bit. This ensures *every single* Prime Implicant is discovered without relying on geometric intuition.

(c) **Prime Implicant Chart (Coverage Table):** Once all Prime Implicants are found, the method uses a coverage chart to cleanly separate *Essential Prime Implicants* from redundant ones. It systematically resolves overlapping terms using techniques like row/column dominance and Petrick's Method to guarantee an absolute minimal SOP or POS equation.

(d) **Computer Programmability:** Because it is a deterministic, step-by-step mathematical algorithm, the tabulation method is highly suitable for software implementation. Modern logic synthesis tools (e.g., in EDA software for FPGA/ASIC design) rely on algorithms derived from Quine-McCluskey and Espresso rather than heuristic graphical maps.

Q2. Using tabular method, find the prime implicants and essential prime implicants of the following Boolean function:

$$F(A, B, C, D) = \Sigma\,(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

(15 marks) **[CSE 205 | L-2/T-1 | 2022–23]**

Answer

Step 1: Binary Representation and Initial Grouping (Column I)

First, we list all the given minterms and convert them to their 4-bit binary representations (A, B, C, D). We then group them based on the number of 1s in their binary form (their index).

Group	Minterm	A B C D	Used
0	m_0	0 0 0 0	✓
1	m_2	0 0 1 0	✓
	m_8	1 0 0 0	✓
2	m_3	0 0 1 1	✓
	m_5	0 1 0 1	✓
	m_9	1 0 0 1	✓
	m_{10}	1 0 1 0	✓
3	m_7	0 1 1 1	✓
	m_{11}	1 0 1 1	✓
	m_{13}	1 1 0 1	✓
4	m_{15}	1 1 1 1	✓

Step 2: Combine Pairs of Minterms (Column II)

We compare minterms from adjacent groups (Group i with Group $i+1$). If they differ by exactly one bit, we combine them, place a dash (-) in the differing bit position, and mark the original minterms as used (✓).

Group	Combined Minterms	A B C D	Used
0-1	(0, 2)	0 0 - 0	✓
	(0, 8)	- 0 0 0	✓
1-2	(2, 3)	0 0 1 -	✓
	(2, 10)	- 0 1 0	✓
	(8, 9)	1 0 0 -	✓
	(8, 10)	1 0 - 0	✓
2-3	(3, 7)	0 - 1 1	✓
	(3, 11)	- 0 1 1	✓
	(5, 7)	0 1 - 1	✓
	(5, 13)	- 1 0 1	✓
	(9, 11)	1 0 - 1	✓
	(9, 13)	1 - 0 1	✓
	(10, 11)	1 0 1 -	✓
3-4	(7, 15)	- 1 1 1	✓
	(11, 15)	1 - 1 1	✓
	(13, 15)	1 1 - 1	✓

Step 3: Combine Quads of Minterms (Column III)

Now, we compare the terms from adjacent groups in Column II. They can be combined if their dashes align and they differ by exactly one numeric bit. Uncombined terms from Column II would be Prime Implicants, but here all terms find a match.

Group	Combined Minterms	A B C D	Prime Implicant (Literal)
0-1-2	(0, 2, 8, 10)	- 0 - 0	**PI**$_1 = B'D'$
	(0, 8, 2, 10)	*- 0 - 0*	*Duplicate*
1-2-3	(2, 3, 10, 11)	- 0 1 -	**PI**$_2 = B'C$
	(2, 10, 3, 11)	*- 0 1 -*	*Duplicate*
	(8, 9, 10, 11)	1 0 - -	**PI**$_3 = AB'$
	(8, 10, 9, 11)	*1 0 - -*	*Duplicate*
2-3-4	(3, 7, 11, 15)	- - 1 1	**PI**$_4 = CD$
	(3, 11, 7, 15)	*- - 1 1*	*Duplicate*
	(5, 7, 13, 15)	- 1 - 1	**PI**$_5 = BD$
	(5, 13, 7, 15)	*- 1 - 1*	*Duplicate*
	(9, 11, 13, 15)	1 - - 1	**PI**$_6 = AD$
	(9, 13, 11, 15)	*1 - - 1*	*Duplicate*

Since no further combinations can be made, the unique terms in Column III represent all the **Prime Implicants**.

Step 4: Prime Implicant Chart

We construct a chart mapping the Prime Implicants to the minterms they cover. We look for columns containing a single 'X'. These minterms can only be covered by one specific Prime Implicant, making that Prime Implicant **essential**.

PI	Term	0	2	3	5	7	8	9	10	11	13	15
PI$_1$*	**B'D'**	**X**	X				X		X			
PI$_2$	B'C		X	X					X	X		
PI$_3$	AB'						X	X	X	X		
PI$_4$	CD			X		X				X		X
PI$_5$*	**BD**				**X**	X					X	X
PI$_6$	AD							X		X	X	X

Observation from the Chart:

- Minterm **0** is covered *only* by PI$_1$ ($B'D'$).
- Minterm **5** is covered *only* by PI$_5$ (BD).

These isolated columns signify that PI$_1$ and PI$_5$ are absolutely required to cover those minterms.

Final Result Summary

All Prime Implicants: $B'D'$ (PI$_1$), $B'C$ (PI$_2$), AB' (PI$_3$), CD (PI$_4$), BD (PI$_5$), AD (PI$_6$).

Essential Prime Implicants: **B'D'** and **BD**

Q3. Simplify the following Boolean function into (a) sum-of-products form and (b) product-of-sums form:

$$F(A, B, C, D) = \Sigma\,(0, 1, 2, 5, 8, 9, 10)$$

(14 marks) **[CSE 205 | L-2/T-1 | 2022–23]**

Answer

Part (a): Sum-of-Products (SOP) Minimization

Step 1: Identify Minterms
The Boolean function is given directly as a sum of minterms:

$$F(A, B, C, D) = \Sigma\, m(0, 1, 2, 5, 8, 9, 10)$$

Step 2: Karnaugh Map Plotting and Grouping for SOP
We plot the 1s on a 4-variable K-map and group them to find the essential prime implicants.

AB \ CD	00	01	11	10
00	1	1	0	1
01	0	1	0	0
11	0	0	0	0
10	1	1	0	1

Essential Prime Implicants Analysis:

- **Wrap-around Quad:** Cells $0, 1, 8, 9$ form a quad spanning the top and bottom rows.
  Constant variables: $B = 0, C = 0 \implies \mathbf{B'C'}$.
- **Corner Quad:** Cells $0, 2, 8, 10$ form a quad at the four corners.
  Constant variables: $B = 0, D = 0 \implies \mathbf{B'D'}$.
- **Vertical Pair:** Cells $1, 5$ form a pair.
  Constant variables: $A = 0, C = 0, D = 1 \implies \mathbf{A'C'D}$.

Final Simplified SOP Expression:

$$F_{SOP}(A, B, C, D) = \mathbf{B'C' + B'D' + A'C'D}$$

Part (b): Product-of-Sums (POS) Minimization

Step 1: Identify Maxterms
To find the POS form, we identify the maxterms (0s) by finding the missing indices from the 16 possible combinations (0 to 15):

$$F(A, B, C, D) = \Pi\, M(3, 4, 6, 7, 11, 12, 13, 14, 15)$$

Step 2: Karnaugh Map Plotting and Grouping for POS
We plot the 0s on the K-map. For POS, each grouped block of 0s yields a sum term. Constant 0s yield uncomplemented variables, and constant 1s yield complemented variables.

AB \ CD	00	01	11	10
00	1	1	0	1
01	0	1	0	0
11	0	0	0	0
10	1	1	0	1

Groupings for POS Sum Terms:

- **Horizontal Quad (Row 11):** Cells $12, 13, 14, 15$ form a complete row.
  Constant variables: $A = 1, B = 1 \implies (\mathbf{A' + B'})$.
- **Vertical Quad (Column 11):** Cells $3, 7, 11, 15$ form a complete column.
  Constant variables: $C = 1, D = 1 \implies (\mathbf{C' + D'})$.
- **Side-Edge Quad:** Cells $4, 6, 12, 14$ form a quad by wrapping the left and right edges across two rows.
  Constant variables: $B = 1, D = 0 \implies (\mathbf{B' + D})$.

Final Simplified POS Expression:

$$F_{POS}(A, B, C, D) = (\mathbf{A' + B'})(\mathbf{C' + D'})(\mathbf{B' + D})$$

Q4. Determine the prime implicants of the following function using the tabulation (tabular) method:

$$F(w, x, y, z) = \Sigma\,(0, 1, 2, 8, 10, 11, 14, 15)$$

(15 marks) **[CSE 205 | L-2/T-1 | 2021–22]**

Answer

Tabulation (Quine-McCluskey) Method

To determine the prime implicants of the Boolean function $F(w, x, y, z) = \Sigma\,(0, 1, 2, 8, 10, 11, 14, 15)$, we will systematically group the minterms based on the number of 1s in their binary representation and then progressively combine them.

Step 1: Binary Representation and Initial Grouping (Column I)

We list all the given minterms, represent them in 4-bit binary format (w, x, y, z), and group them by their index (number of 1s).

Group	Minterm	w x y z	Used
0	m_0	0 0 0 0	✓
1	m_1	0 0 0 1	✓
	m_2	0 0 1 0	✓
	m_8	1 0 0 0	✓
2	m_{10}	1 0 1 0	✓
3	m_{11}	1 0 1 1	✓
	m_{14}	1 1 1 0	✓
4	m_{15}	1 1 1 1	✓

Step 2: Combine Pairs of Minterms (Column II)

We compare minterms from adjacent groups (Group i and Group $i + 1$). If they differ by exactly one bit, we combine them, place a dash (-) in the differing position, and mark the original minterms as used (✓). Terms that cannot be combined with any term in the next stage are marked as Prime Implicants (PI).

Group	Combined Minterms	w x y z	Status / PI
0-1	(0, 1)	0 0 0 -	$\mathbf{PI_1}$ (Uncombined)
	(0, 2)	0 0 - 0	✓
	(0, 8)	- 0 0 0	✓
1-2	(2, 10)	- 0 1 0	✓
	(8, 10)	1 0 - 0	✓
2-3	(10, 11)	1 0 1 -	✓
	(10, 14)	1 - 1 0	✓
3-4	(11, 15)	1 - 1 1	✓
	(14, 15)	1 1 1 -	✓

Note: The pair $(0, 1)$ cannot be combined with any term in the next stage because there is no matching term in Group 1-2 with a dash in the last position. Therefore, it becomes our first Prime Implicant.

Step 3: Combine Quads of Minterms (Column III)

Now, we compare the paired terms from adjacent groups in Column II. They can be combined if their dashes align and they differ by exactly one numeric bit.

Group	Combined Minterms	w x y z	Status / PI
0-1-2	(0, 2, 8, 10)	- 0 - 0	$\mathbf{PI_2}$
	(0, 8, 2, 10)	*- 0 - 0*	*Duplicate*
2-3-4	(10, 11, 14, 15)	1 - 1 -	$\mathbf{PI_3}$
	(10, 14, 11, 15)	*1 - 1 -*	*Duplicate*

Since no further groups can be compared, the process stops here. The unique terms in Column III, along with the uncombined term from Column II, form the complete set of Prime Implicants.

Final List of Prime Implicants

Translating the binary combinations back to Boolean variables ($0 \rightarrow$ complemented, $1 \rightarrow$ uncomplemented, $- \rightarrow$ omitted variable):

PI$_1$ from $(0, 1) \implies 000- \implies \mathbf{w'x'y'}$

PI$_2$ from $(0, 2, 8, 10) \implies -0-0 \implies \mathbf{x'z'}$

PI$_3$ from $(10, 11, 14, 15) \implies 1-1- \implies \mathbf{wy}$

Optional Verification (Prime Implicant Chart):
Although the question only asks for the prime implicants, constructing the PI chart verifies coverage and essentiality:

PI	Term	0	1	2	8	10	11	14	15
PI$_1$*	**w'x'y'**	X	**X**						
PI$_2$*	**x'z'**	X		**X**	**X**	X			
PI$_3$*	**wy**					X	**X**	**X**	**X**

*Every column has at least one minterm covered by only one PI (denoted by bold **X**s in isolated columns). Thus, all three Prime Implicants are also Essential Prime Implicants.*

The Prime Implicants are:
w'x'y', x'z', wy

Q5. Compare the Tabulation method and the Karnaugh map for minimizing Boolean functions based on their merits and demerits. **(15 marks)** **[CSE 205 | L-2/T-1 | 2019–20]**

Answer

Comparison of Karnaugh Map and Tabulation (Quine-McCluskey) Methods

Both the Karnaugh Map (K-map) and the Tabulation method are widely used techniques for simplifying Boolean expressions to their minimal Sum-of-Products (SOP) or Product-of-Sums (POS) forms. However, they operate on entirely different principles—one being graphical and heuristic, the other being algebraic and algorithmic.

1. Karnaugh Map (K-map) Method

The Karnaugh map is a graphical representation of a truth table where adjacent cells differ by exactly one variable (Gray code sequence).
Merits:

- **Visual Simplicity:** It relies on the human brain's ability to recognize geometric patterns, making it highly intuitive and fast for manual solving.
- **Immediate Results:** Grouping directly yields the simplified prime implicants without tedious intermediate algebraic steps.
- **Don't Care Handling:** "Don't care" conditions can be effortlessly visualized and included in groups to maximize their size.
- **Optimal for Small Variables:** It is generally the fastest and most efficient manual method for logic functions with 2, 3, or 4 variables.

Demerits:

- **Scalability Issues:** It becomes exceedingly complex and visualization breaks down for 5 variables (requiring 3D visualization) and is practically unusable for 6 or more variables.
- **Human Error:** Because it relies on human visual inspection, it is easy to miss optimal groupings or redundant terms, potentially yielding a sub-optimal result.
- **Not Programmable:** The graphical, heuristic nature of K-maps makes them highly unsuitable for implementation in computer software.

2. Tabulation (Quine-McCluskey) Method

The Tabulation method is a systematic, step-by-step algorithmic procedure based on the exhaustive algebraic application of the theorem $XY + XY' = X$.
Merits:

- **Highly Scalable:** It can seamlessly handle any number of variables (5, 6, or more) without a breakdown in methodology.

- **Algorithmic and Programmable:** Because it uses strict tabular procedures, it is easily coded into software. Modern Electronic Design Automation (EDA) synthesis tools utilize algorithms derived from this method (e.g., the Espresso algorithm).
- **Guaranteed Minimization:** It exhaustively generates all prime implicants and systematically uses a prime implicant chart to guarantee the absolute minimal expression. Visual errors are completely eliminated.

Demerits:

- **Tedious Manual Process:** For a human, the exhaustive listing, comparing, and checking of minterms is heavily time-consuming and prone to arithmetic or clerical errors.
- **Computational Complexity:** As the number of variables n increases, the number of minterms grows exponentially (2^n). The algorithm's memory and processing requirements expand rapidly, making it computationally expensive for very large circuits.

Summary Comparison Table

Feature	Karnaugh Map	Tabulation Method
Approach	Graphical / Visual mapping	Algorithmic / Tabular step-by-step
Ideal Number of Variables	Up to 4 (maximum 5)	5 or more (scales to any number)
Error Susceptibility	Prone to visual omission errors	Prone to clerical tracking errors manually
Computer Automation	Highly difficult / Impractical	Highly suitable and widely implemented
Guarantee of Minimum	Relies on user's visual accuracy	Mathematically guaranteed
Process Speed (Manual)	Very fast for small functions	Very slow and tedious

Q6. Using the Tabulation method, simplify the following Boolean function by first finding the essential prime implicants:

$$F(A, B, C, D) = \Sigma\,(1, 3, 4, 5, 10, 11, 12, 13, 14)$$

(28 marks) **[CSE 205 | L-2/T-1 | 2019–20]**

Answer

Tabulation (Quine-McCluskey) Method

Step 1: Binary Representation and Initial Grouping (Column I)
List all minterms, convert them to 4-bit binary format (A, B, C, D), and group them by the number of 1s (index).

Group	Minterm	A B C D	Used
1	m_1	0 0 0 1	✓
	m_4	0 1 0 0	✓
2	m_3	0 0 1 1	✓
	m_5	0 1 0 1	✓
	m_{10}	1 0 1 0	✓
	m_{12}	1 1 0 0	✓
3	m_{11}	1 0 1 1	✓
	m_{13}	1 1 0 1	✓
	m_{14}	1 1 1 0	✓

Step 2: Combine Pairs of Minterms (Column II)
Compare minterms from adjacent groups. If they differ by exactly one bit, combine them, place a dash (-) in the differing position, and mark the original minterms as used (✓).

Group	Combined Minterms	A B C D	Status / PI
1-2	(1, 3)	0 0 - 1	**PI**$_2$ $(A'B'D)$
	(1, 5)	0 - 0 1	**PI**$_3$ $(A'C'D)$
	(4, 5)	0 1 0 -	✓
	(4, 12)	- 1 0 0	✓
2-3	(3, 11)	- 0 1 1	**PI**$_4$ $(B'CD)$
	(5, 13)	- 1 0 1	✓
	(10, 11)	1 0 1 -	**PI**$_5$ $(AB'C)$
	(10, 14)	1 - 1 0	**PI**$_6$ (ACD')
	(12, 13)	1 1 0 -	✓
	(12, 14)	1 1 - 0	**PI**$_7$ (ABD')

Terms that do not combine further into Column III are marked as Prime Implicants (PIs).

Step 3: Combine Quads of Minterms (Column III)
Compare the paired terms from adjacent groups in Column II.

Group	Combined Minterms	A B C D	Status / PI
1-2-3	(4, 5, 12, 13)	- 1 0 -	**PI**$_1$ (BC')
	(4, 12, 5, 13)	*- 1 0 -*	*Duplicate*

Step 4: Prime Implicant Chart
Construct a chart to map the Prime Implicants to the minterms they cover. Look for columns with a single 'X' to find the **Essential Prime Implicants (EPIs)**.

PI	Term	1	3	4	5	10	11	12	13	14
PI$_1$*	**BC'**			**X**	X			X	**X**	
PI$_2$	A'B'D	X	X							
PI$_3$	A'C'D	X			X					
PI$_4$	B'CD		X				X			
PI$_5$	AB'C					X	X			
PI$_6$	ACD'					X				X
PI$_7$	ABD'							X		X

Analysis of the Chart:

- Minterms **4** and **13** are covered *only* by **PI**$_1$. Thus, **BC′** is an Essential Prime Implicant.
- Selecting PI$_1$ covers minterms 4, 5, 12, and 13.

Step 5: Reduced Prime Implicant Chart
After removing the covered minterms (4, 5, 12, 13) and PI$_1$, we construct a reduced chart for the remaining minterms (1, 3, 10, 11, 14):

PI	Term	1	3	10	11	14
PI$_2$	A'B'D	X	X			
PI$_3$	A'C'D	X				
PI$_4$	B'CD		X		X	
PI$_5$	AB'C			X	X	
PI$_6$	ACD'			X		X
PI$_7$	ABD'					X

Applying Row Dominance:

- **PI$_2$ dominates PI$_3$**: PI$_2$ covers 1, 3 while PI$_3$ covers only 1. We can safely discard PI$_3$.
- **PI$_6$ dominates PI$_7$**: PI$_6$ covers 10, 14 while PI$_7$ covers only 14. We can safely discard PI$_7$.

After discarding PI$_3$ and PI$_7$, minterm **1** is covered only by **PI$_2$**, and minterm **14** is covered only by **PI$_6$**. Therefore, PI$_2$ and PI$_6$ must be selected.

- Selecting PI$_2$ ($A'B'D$) and PI$_6$ (ACD') covers minterms 1, 3, 10, and 14.
- The only remaining uncovered minterm is **11**.
- Minterm 11 can be covered by either **PI$_4$** ($B'CD$) OR **PI$_5$** ($AB'C$). Both options yield a minimal expression with the same number of literals.

Final Minimized Boolean Expression

There are two equally valid minimal Sum-of-Products (SOP) expressions:

$$F(A, B, C, D) = \mathbf{BC' + A'B'D + ACD' + B'CD}$$

OR

$$F(A, B, C, D) = \mathbf{BC' + A'B'D + ACD' + AB'C}$$

Q7. Minimize the following function using the Quine-McCluskey (Q-M) method. Find essential prime implicants and all prime implicants:

$$f(A, B, C, D, E) = \Sigma\,(0, 1, 2, 8, 9, 15, 17, 21, 24, 25, 27, 31)$$

(20 marks) **[CSE 205 | L-2/T-1 | 2016–17]**

Answer

Quine-McCluskey (Tabulation) Method

We are given a 5-variable Boolean function:

$$f(A, B, C, D, E) = \Sigma\,(0, 1, 2, 8, 9, 15, 17, 21, 24, 25, 27, 31)$$

Step 1: Binary Representation and Grouping (Column I)

We list all minterms, convert them to 5-bit binary representations, and group them according to the number of 1s (index) they contain.

Group	Minterm	A B C D E	Used
0	m_0	0 0 0 0 0	✓
1	m_1	0 0 0 0 1	✓
	m_2	0 0 0 1 0	✓
	m_8	0 1 0 0 0	✓
2	m_9	0 1 0 0 1	✓
	m_{17}	1 0 0 0 1	✓
	m_{24}	1 1 0 0 0	✓
3	m_{21}	1 0 1 0 1	✓
	m_{25}	1 1 0 0 1	✓
4	m_{15}	0 1 1 1 1	✓
	m_{27}	1 1 0 1 1	✓
5	m_{31}	1 1 1 1 1	✓

Step 2: Combine Pairs of Minterms (Column II)

We compare minterms from adjacent groups. If they differ by exactly one bit, we combine them, place a dash (-) in the differing position, and mark the original minterms as used (✓). Terms that cannot be combined further are Prime Implicants (PIs).

Group	Combined Minterms	A B C D E	Status / PI
0-1	(0, 1)	0 0 0 0 -	✓
	(0, 2)	0 0 0 - 0	**PI**$_1$ ($A'B'C'E'$)
	(0, 8)	0 - 0 0 0	✓
1-2	(1, 9)	0 - 0 0 1	✓
	(1, 17)	- 0 0 0 1	✓
	(8, 9)	0 1 0 0 -	✓
	(8, 24)	- 1 0 0 0	✓
2-3	(9, 25)	- 1 0 0 1	✓
	(17, 21)	1 0 - 0 1	**PI**$_2$ ($AB'D'E$)
	(17, 25)	1 - 0 0 1	✓
	(24, 25)	1 1 0 0 -	✓
3-4	(25, 27)	1 1 0 - 1	**PI**$_3$ ($ABC'E$)
4-5	(15, 31)	- 1 1 1 1	**PI**$_4$ ($BCDE$)
	(27, 31)	1 1 - 1 1	**PI**$_5$ ($ABDE$)

Step 3: Combine Quads of Minterms (Column III)

We now compare the paired terms from adjacent groups in Column II.

Group	Combined Minterms	A B C D E	Status / PI
0-1-2	(0, 1, 8, 9)	0 - 0 0 -	**PI**$_6$ ($A'C'D'$)
	(0, 8, 1, 9)	*0 - 0 0 -*	*Duplicate*
1-2-3	(1, 9, 17, 25)	- - 0 0 1	**PI**$_7$ ($C'D'E$)
	(1, 17, 9, 25)	*- - 0 0 1*	*Duplicate*
	(8, 9, 24, 25)	- 1 0 0 -	**PI**$_8$ ($BC'D'$)
	(8, 24, 9, 25)	*- 1 0 0 -*	*Duplicate*

Step 4: Prime Implicant Chart

We construct a chart to map all Prime Implicants (PIs) to the minterms they cover. Columns with a single 'X' denote **Essential Prime Implicants (EPIs)**.

PI	Term	0	1	2	8	9	15	17	21	24	25	27	31
PI_1*	**A'B'C'E'**	X		**X**									
PI_2*	**AB'D'E**							X	**X**				
PI_3	ABC'E										X	X	
PI_4*	**BCDE**						**X**						X
PI_5	ABDE											X	X
PI_6	A'C'D'	X	X		X	X							
PI_7	C'D'E		X			X		X			X		
PI_8*	**BC'D'**				X	X				**X**	X		

Analysis of the Chart:

- **Minterm 2** is covered only by PI_1.
- **Minterm 21** is covered only by PI_2.
- **Minterm 15** is covered only by PI_4.
- **Minterm 24** is covered only by PI_8.

Thus, **PI_1, PI_2, PI_4, and PI_8** are **Essential Prime Implicants**. Selecting these covers minterms $0, 2, 8, 9, 15, 17, 21, 24, 25,$ and 31.

Step 5: Covering Remaining Minterms

The only uncovered minterms are **1** and **27**.

- Minterm **1** can be covered by either PI_6 ($A'C'D'$) or PI_7 ($C'D'E$). Both have 3 literals.
- Minterm **27** can be covered by either PI_3 ($ABC'E$) or PI_5 ($ABDE$). Both have 4 literals.

Final Results

1. List of All Prime Implicants:

$$PI_1 = A'B'C'E' \qquad PI_2 = AB'D'E \qquad PI_3 = ABC'E \qquad PI_4 = BCDE$$
$$PI_5 = ABDE \qquad PI_6 = A'C'D' \qquad PI_7 = C'D'E \qquad PI_8 = BC'D'$$

2. List of Essential Prime Implicants:

$$\mathbf{A'B'C'E'},\quad \mathbf{AB'D'E},\quad \mathbf{BCDE},\quad \mathbf{BC'D'}$$

3. Final Minimized Boolean Function (One of the 4 optimal solutions):

$$f(A, B, C, D, E) = \mathbf{A'B'C'E' + AB'D'E + BCDE + BC'D' + A'C'D' + ABC'E}$$

Note: Replacing $A'C'D'$ with $C'D'E$ and/or replacing $ABC'E$ with $ABDE$ yields three other equally minimal, mathematically correct expressions.

Q8. When is the tabulation method more preferable than the K-Map method for simplifying a Boolean function? **(5 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

Preferability of the Tabulation (Quine-McCluskey) Method

While the Karnaugh Map (K-Map) is an excellent intuitive tool for Boolean simplification, the Tabulation method (also known as the Quine-McCluskey method) becomes significantly more preferable—and often mandatory—under the following distinct scenarios in digital logic design:

1. Functions with a Large Number of Variables ($n \geq 5$)

- **K-Map Limitation:** K-Maps rely entirely on human visual pattern recognition. They are highly efficient and reliable for $2, 3,$ and 4 variables. However, a 5-variable K-Map requires visualizing relationships across two separate 16-cell sub-maps, and a 6-variable map requires four 16-cell sub-maps. Beyond 4 variables, human spatial visualization breaks down, making the process painfully slow and highly prone to oversight.
- **Tabulation Advantage:** The Tabulation method does not depend on graphical or geometric visualization. It utilizes a rigorous, tabular, algebraic algorithm that scales identically regardless of the number of variables. It is the definitive choice for manual simplification of 5 or more variables.

2. Requirement for Computer Automation (Programmability)

- **K-Map Limitation:** Because the K-Map is a heuristic, graphical method dependent on human spatial reasoning (looping adjacent 1s), it is inherently difficult to translate directly into efficient computer code.
- **Tabulation Advantage:** The Quine-McCluskey method is strictly algorithmic, systematic, and deterministic. It consists of well-defined, repetitive procedural steps (exhaustive list generation, binary matching, and prime implicant chart reduction). This makes it perfectly suited for software implementation. Modern Electronic Design Automation (EDA) software and logic synthesizers rely heavily on algorithms derived from this method (such as the Espresso algorithm).

3. Guaranteeing Absolute Minimization without Human Error

- **K-Map Limitation:** For moderately complex functions (especially those heavily populated with 1s and "don't care" conditions), a human designer using a K-Map might easily overlook the optimal grouping. Selecting a redundant group or missing a larger, non-obvious geometric grouping results in a sub-optimal circuit.
- **Tabulation Advantage:** The Tabulation method exhaustively generates *all* possible Prime Implicants (PIs) and then systematically isolates the Essential Prime Implicants (EPIs) using a rigorous cross-referencing chart. This mathematical rigor guarantees that the final Boolean Sum-of-Products (SOP) or Product-of-Sums (POS) expression is the absolute minimum, entirely eliminating the variable of human visual error.

Summary Conclusion

In professional and academic environments, the K-Map remains an excellent pedagogical tool and is preferred for quick, manual simplification of small logic circuits ($n \leq 4$). However, the **Tabulation method is explicitly preferable whenever a function exceeds 4 variables, when mathematical proof of minimal form is required, or when the minimization process is handled by a computer algorithm.**

Q9. Simplify the following Boolean function by using the tabulation method:

$$F(A, B, C, D) = \Sigma\,(0, 1, 2, 8, 10, 11, 14, 15)$$

(15 marks) [CSE 205 | L-2/T-1 | 2011–12]

Answer

Tabulation (Quine-McCluskey) Method

We are given the 4-variable Boolean function:

$$F(A, B, C, D) = \Sigma\,(0, 1, 2, 8, 10, 11, 14, 15)$$

Step 1: Binary Representation and Initial Grouping (Column I)

Group the minterms based on the number of 1s (index) in their binary representation.

Group	Minterm	A B C D	Used
0	m_0	0 0 0 0	✓
1	m_1	0 0 0 1	✓
	m_2	0 0 1 0	✓
	m_8	1 0 0 0	✓
2	m_{10}	1 0 1 0	✓
3	m_{11}	1 0 1 1	✓
	m_{14}	1 1 1 0	✓
4	m_{15}	1 1 1 1	✓

Step 2: Combine Pairs of Minterms (Column II)

Combine minterms from adjacent groups that differ by exactly one bit. Place a dash (-) in the position of change and check off (✓) the combined minterms.

Group	Combined Minterms	A B C D	Status / PI
0-1	(0, 1)	0 0 0 -	$\mathbf{PI}_1$ $(A'B'C')$
	(0, 2)	0 0 - 0	✓
	(0, 8)	- 0 0 0	✓
1-2	(2, 10)	- 0 1 0	✓
	(8, 10)	1 0 - 0	✓
2-3	(10, 11)	1 0 1 -	✓
	(10, 14)	1 - 1 0	✓
3-4	(11, 15)	1 0 1 1 is $1-11$ with 15 $\implies$ 1 - 1 1	✓
	(14, 15)	1 1 1 -	✓

Note: Minterm pair (0, 1) cannot combine further and forms Prime Implicant 1.

Step 3: Combine Quads of Minterms (Column III)
Combine pairs from Column II that have matching dash (-) positions and differ by exactly one bit.

Group	Combined Minterms	A B C D	Status / PI
0-1-2	(0, 2, 8, 10)	- 0 - 0	**PI**$_2$ ($B'D'$)
	(0, 8, 2, 10)	*- 0 - 0*	*Duplicate*
1-2-3	(2, 10, 8, 10)	No valid distinct match with same dash	
	(8, 10, 10, 14)	Does not match dash position	
2-3-4	(10, 11, 14, 15)	1 - 1 -	**PI**$_3$ (AC)
	(10, 14, 11, 15)	*1 - 1 -*	*Duplicate*

Let us review any uncombined pairs from Column II:

- (2, 10) → (- 0 1 0) combined with (8, 10) → (1 0 - 0) is invalid because dashes are in different positions. However, (0, 2) → (0 0 - 0) and (8, 10) → (1 0 - 0) combine to form (- 0 - 0).
- (2, 10) → (- 0 1 0) combines with (14, 15)? No. Let's check (2, 10) and (14, 15). No.
- Are there any leftover terms from Column II? Every single pair except (0,1) was covered by the quads. Let's double check:
  - (0,2), (0,8), (2,10), (8,10) form the quad (0,2,8,10).
  - (10,11), (10,14), (11,15), (14,15) form the quad (10,11,14,15).

Thus, we have successfully identified all Prime Implicants.

List of Prime Implicants (PIs):

- $\text{PI}_1 = 000- \implies A'B'C'$
- $\text{PI}_2 = -0-0 \implies B'D'$
- $\text{PI}_3 = 1-1- \implies AC$

Step 4: Prime Implicant Chart
Construct the selection chart to identify Essential Prime Implicants (EPIs).

PI	Term	0	1	2	8	10	11	14	15
PI$_1$*	**A'B'C'**	X	**X**						
PI$_2$*	**B'D'**	X		X	X	X			
PI$_3$*	**AC**					X	**X**	**X**	X

EPI Analysis:

- Minterm **1** is covered uniquely by **PI**$_1$. Thus, $A'B'C'$ is an EPI.
- Minterms **11** and **14** are covered uniquely by **PI**$_3$. Thus, AC is an EPI.
- Selecting PI_1 and PI_3 leaves minterms **0, 2, 8, 10** uncovered.
- Minterms **0, 2, 8, 10** are all covered by **PI**$_2$ ($B'D'$). Therefore, **PI**$_2$ is also essential to complete the coverage of the function.

Since all minterms are covered by selecting all three Prime Implicants, they are all Essential Prime Implicants.

Final Minimized Boolean Expression

$$F(A, B, C, D) = \mathbf{A'B'C' + B'D' + AC}$$

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Combinational Logic

Adders, Subtractors, Comparators, Decoders, Encoders, MUX, DEMUX

S4 Combinational Logic

Code Converter

Q1. Design a magnitude comparator for comparing two 3-bit binary numbers X and Y. **(14 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Problem Formulation
A magnitude comparator is a combinational logic circuit that compares two binary numbers and determines their relative magnitudes. We are given two 3-bit binary numbers, X and Y:

$$X = X_2X_1X_0$$
$$Y = Y_2Y_1Y_0$$

where X_2 and Y_2 are the Most Significant Bits (MSBs), and X_0 and Y_0 are the Least Significant Bits (LSBs).
The circuit will output three mutually exclusive signals:

- E (Equal): High (1) if $X = Y$.
- G (Greater): High (1) if $X > Y$.
- L (Less): High (1) if $X < Y$.

Since there are 6 input variables in total, a standard truth table would require $2^6 = 64$ rows. To simplify the design process, we derive the Boolean expressions directly using algorithmic logic based on bit significance.

2. Intermediate Bit-Level Functions
We first define the bit-wise comparison terms for any arbitrary bit position $i \in \{0, 1, 2\}$:

- **Bit Equality (e_i):** The i-th bits are equal ($X_i = Y_i$) if both are 1 or both are 0. This is represented by the XNOR operation:

$$e_i = X_iY_i + \overline{X_i}\overline{Y_i} = X_i \odot Y_i$$

- **Bit Greater (g_i):** The bit X_i is strictly greater than Y_i only when $X_i = 1$ and $Y_i = 0$.

$$g_i = X_i\overline{Y_i}$$

- **Bit Less (l_i):** The bit X_i is strictly less than Y_i only when $X_i = 0$ and $Y_i = 1$.

$$l_i = \overline{X_i}Y_i$$

3. Final Output Equations
Using the intermediate signals, we can construct the logic for the entire 3-bit word by evaluating from the MSB ($i = 2$) down to the LSB ($i = 0$).
Equality (E): For the two numbers to be equal, all corresponding bits must be identical simultaneously.

$$E = e_2 \cdot e_1 \cdot e_0$$
$$E = (X_2 \odot Y_2)(X_1 \odot Y_1)(X_0 \odot Y_0)$$

Greater Than (G): The number X is greater than Y if: 1. $X_2 > Y_2$ (MSB is greater), OR 2. $X_2 = Y_2$ AND $X_1 > Y_1$, OR 3. $X_2 = Y_2$ AND $X_1 = Y_1$ AND $X_0 > Y_0$. Expressing this logically using our intermediate terms:

$$G = g_2 + e_2g_1 + e_2e_1g_0$$
$$G = X_2\overline{Y_2} + (X_2 \odot Y_2)X_1\overline{Y_1} + (X_2 \odot Y_2)(X_1 \odot Y_1)X_0\overline{Y_0}$$

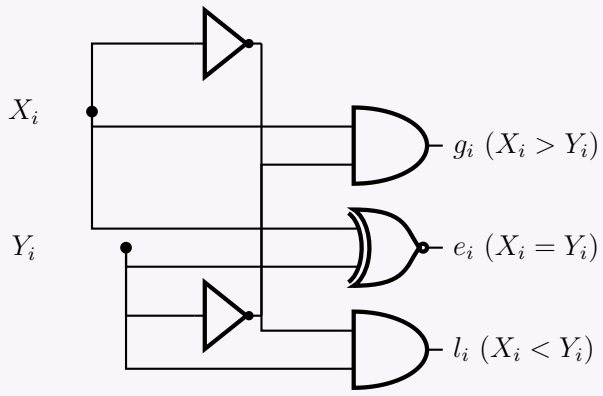
Less Than (L): Similarly, X is less than Y if the corresponding condition holds for the lesser bits:

$$L = l_2 + e_2l_1 + e_2e_1l_0$$
$$L = \overline{X_2}Y_2 + (X_2 \odot Y_2)\overline{X_1}Y_1 + (X_2 \odot Y_2)(X_1 \odot Y_1)\overline{X_0}Y_0$$

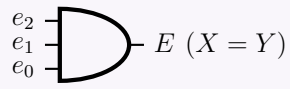
Note: L can also be obtained using a NOR gate since the states are mutually exclusive: $L = \overline{G + E}$.

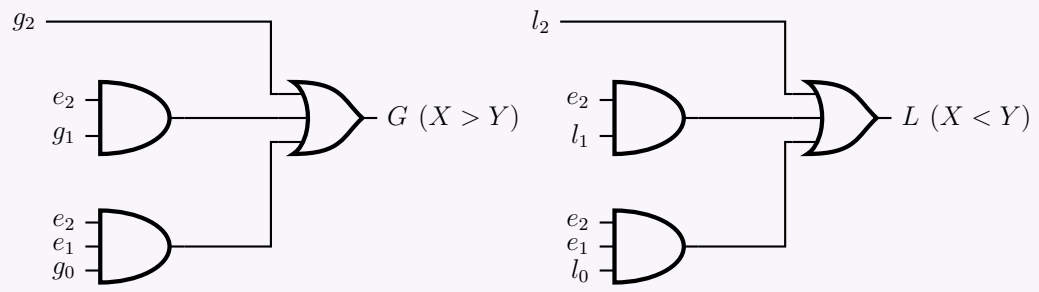
4. Logic Circuit Implementation
To ensure high readability and avoid overlapping routing, the complete logic circuit is separated into two hierarchical tiers. The first tier generates the intermediate bit-level signals (e_i, g_i, l_i). The second tier uses these signals to compute the final outputs (E, G, L).

Tier 1: Bit-Level Signal Generation (Shown for bit i)



Tier 2: Aggregate Output Generation (E, G, L)





Q2. A consensus circuit is a combinational circuit whose output is equal to 1 if exactly two inputs are 1, and 0 otherwise. Design a 3-input consensus circuit by finding the circuit's truth table, simplified Boolean equation and a circuit diagram. **(12 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Problem Formulation and Truth Table

A consensus circuit evaluates its inputs and produces a logic HIGH (1) if and only if exactly two of its inputs are HIGH. For a 3-input circuit with inputs A, B, and C, and output Y, we systematically evaluate all $2^3 = 8$ possible input combinations.

Inputs			Output	
A	B	C	Y	*Condition Met?*
0	0	0	0	None are 1
0	0	1	0	Only one is 1
0	1	0	0	Only one is 1
0	1	1	1	**Exactly two are 1** (B, C)
1	0	0	0	Only one is 1
1	0	1	1	**Exactly two are 1** (A, C)
1	1	0	1	**Exactly two are 1** (A, B)
1	1	1	0	Three are 1

From the truth table, the sum-of-minterms expression is:

$$Y(A, B, C) = \sum m(3, 5, 6)$$

2. Boolean Equation Simplification

To find the simplified Boolean equation, we map the minterms onto a 3-variable Karnaugh Map.

		BC = 00	01	11	10
A	0	0	0	1	0
	1	0	1	0	1

Analysis of the K-Map: Observing the plotted 1s at cells 3 (011), 5 (101), and 6 (110), there are no adjacent cells containing a 1. Because no groups of two, four, or eight can be formed, the Boolean expression cannot be further reduced using standard Sum-of-Products (SOP) grouping.

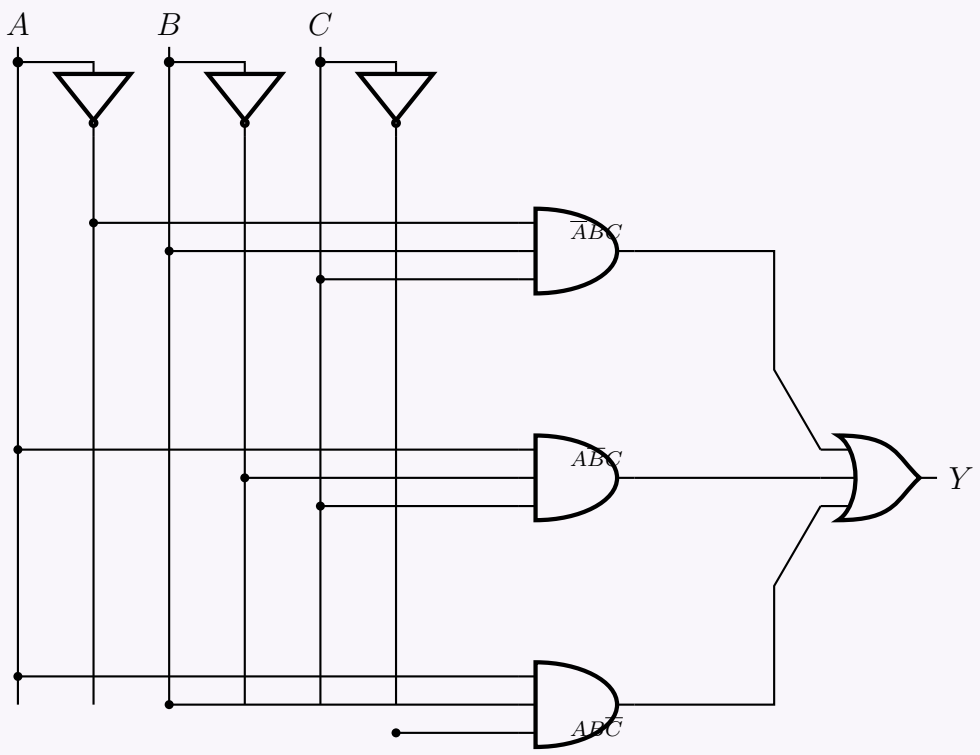
Thus, the minimal SOP Boolean equation is directly the canonical sum of products:

$$Y = m_3 + m_5 + m_6$$
$$Y = \overline{A}BC + A\overline{B}C + AB\overline{C}$$

Note: While not reducible in pure SOP form, algebraic manipulation using XOR logic is possible (e.g., $Y = C(A \oplus B) + AB\overline{C}$), but standard combinational design typically defaults to the minimal SOP utilizing AND, OR, and NOT gates.

3. Logic Circuit Diagram

We implement the minimal SOP expression using a standard logic bus architecture. Three 3-input AND gates generate the required product terms, which are then summed by a 3-input OR gate.



Q3. Design a combinational circuit that converts a three-bit gray code to a three-bit binary number. Use only a minimum number of gates in your design. **(20 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Answer

1. Truth Table Formulation

A Gray code to binary converter takes a Gray code input (G_2, G_1, G_0) and produces a binary number output (B_2, B_1, B_0). We evaluate all 8 possible combinations in standard binary order for the inputs to easily derive our K-maps.

Gray Code Input			**Binary Output**		
G_2	G_1	G_0	B_2	B_1	B_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	0	0
1	1	1	1	0	1

2. Karnaugh Map Analysis & Boolean Simplification

We derive the simplified Boolean expressions for each output bit (B_2, B_1, B_0).
For MSB (B_2): By direct inspection of the truth table, B_2 exactly matches the input G_2.

$$B_2 = G_2$$

For Middle Bit (B_1): We plot the minterms for B_1 ($\Sigma m(2, 3, 4, 5)$) on a K-map.

		G_1G_0 = 00	01	11	10
G_2	0	0	0	1	1
	1	1	1	0	0

$$B_1 = \overline{G_2}G_1 + G_2\overline{G_1}$$
$$B_1 = G_2 \oplus G_1 \quad \text{(XOR Definition)}$$

For LSB (B_0): We plot the minterms for B_0 ($\Sigma m(1, 2, 4, 7)$) on a K-map.

G_2 \ G_1G_0	00	01	11	10
0	0	1	0	1
1	1	0	1	0

The resulting "checkerboard" pattern indicates a multi-variable XOR relationship:

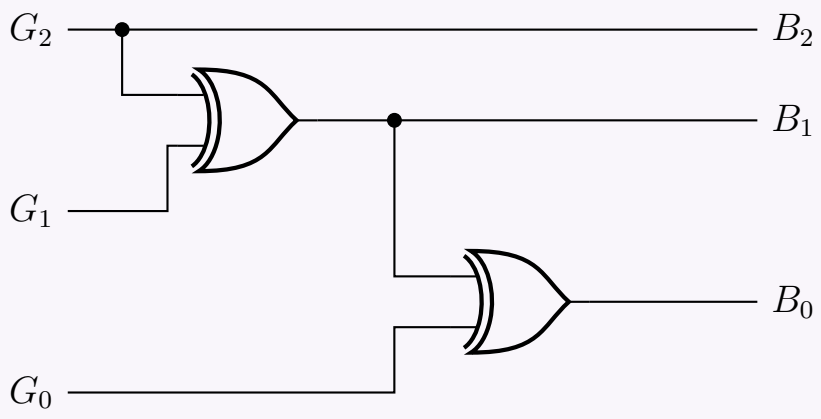
$$B_0 = \overline{G_2G_1}G_0 + \overline{G_2}G_1\overline{G_0} + G_2\overline{G_1G_0} + G_2G_1G_0$$
$$B_0 = \overline{G_2}(\overline{G_1}G_0 + G_1\overline{G_0}) + G_2(\overline{G_1G_0} + G_1G_0) \quad \text{(Distributive Law)}$$
$$B_0 = \overline{G_2}(G_1 \oplus G_0) + G_2(\overline{G_1 \oplus G_0}) \quad \text{(XOR and XNOR Definitions)}$$
$$B_0 = G_2 \oplus G_1 \oplus G_0 \quad \text{(XOR Definition)}$$

3. Logic Circuit Implementation (Minimized Gate Count)

To minimize the number of gates, we can cascade 2-input XOR gates by reusing the generated B_1 signal for the calculation of B_0:

$$B_0 = (G_2 \oplus G_1) \oplus G_0 = B_1 \oplus G_0$$

This reduces the circuit requirement to exactly **two 2-input XOR gates**.



Q4. Design a combinational circuit that converts a decimal digit represented with "Random Code" to "BCD Code". Use the K-Map method for logic simplification and show the circuit diagram.

Decimal Digit	Random Code ABCD	BCD Code wxyz
0	0000	0000
1	0011	0001
2	0101	0010
3	0110	0011
4	1010	0100
5	1011	0101
6	1100	0110
7	1101	0111
8	1110	1000
9	1111	1001

(20 marks) **[CSE 205 | L-2/T-1 | 2021–22]**

Answer

1. Identification of Minterms and Don't Cares

From the provided table, we define the combinational circuit with inputs A, B, C, D and outputs w, x, y, z. The input is a 4-bit "Random Code". Only 10 specific input combinations are valid (representing decimal digits 0-9). The remaining 6 combinations are invalid and will be treated as **Don't Cares (d)**.

- **Valid Inputs (Minterms):** $m_0, m_3, m_5, m_6, m_{10}, m_{11}, m_{12}, m_{13}, m_{14}, m_{15}$
- **Don't Cares (d):** $\sum d(1, 2, 4, 7, 8, 9)$

We can now list the sum of minterms for each output bit of the BCD code (w, x, y, z) by observing when each output is '1' in the truth table:

$$w(A,B,C,D) = \sum m(14,15) + \sum d(1,2,4,7,8,9)$$
$$x(A,B,C,D) = \sum m(10,11,12,13) + \sum d(1,2,4,7,8,9)$$
$$y(A,B,C,D) = \sum m(5,6,12,13) + \sum d(1,2,4,7,8,9)$$
$$z(A,B,C,D) = \sum m(3,6,11,13,15) + \sum d(1,2,4,7,8,9)$$

2. K-Map Simplification

We use 4-variable Karnaugh Maps to find the minimal Sum of Products (SOP) expressions for each output.

K-Map for w

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	1	1
10	0	0	0	0

Group 1 (Red): ABC
Simplified: $\mathbf{w = ABC}$

K-Map for x

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	0	0
10	0	0	1	1

Group 1 (Red): $A\overline{C}$
Group 2 (Green): $A\overline{B}$
Simplified: $\mathbf{x = A\overline{C} + A\overline{B}}$

K-Map for y

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	0	1
11	1	1	0	0
10	0	0	0	0

Group 1 (Red): $B\overline{C}$
Group 2 (Green): $\overline{A}B$
Simplified: $\mathbf{y = B\overline{C} + \overline{A}B}$

K-Map for z

AB \ CD	00	01	11	10
00	0	0	1	0
01	0	0	0	1
11	0	1	1	0
10	0	0	1	0

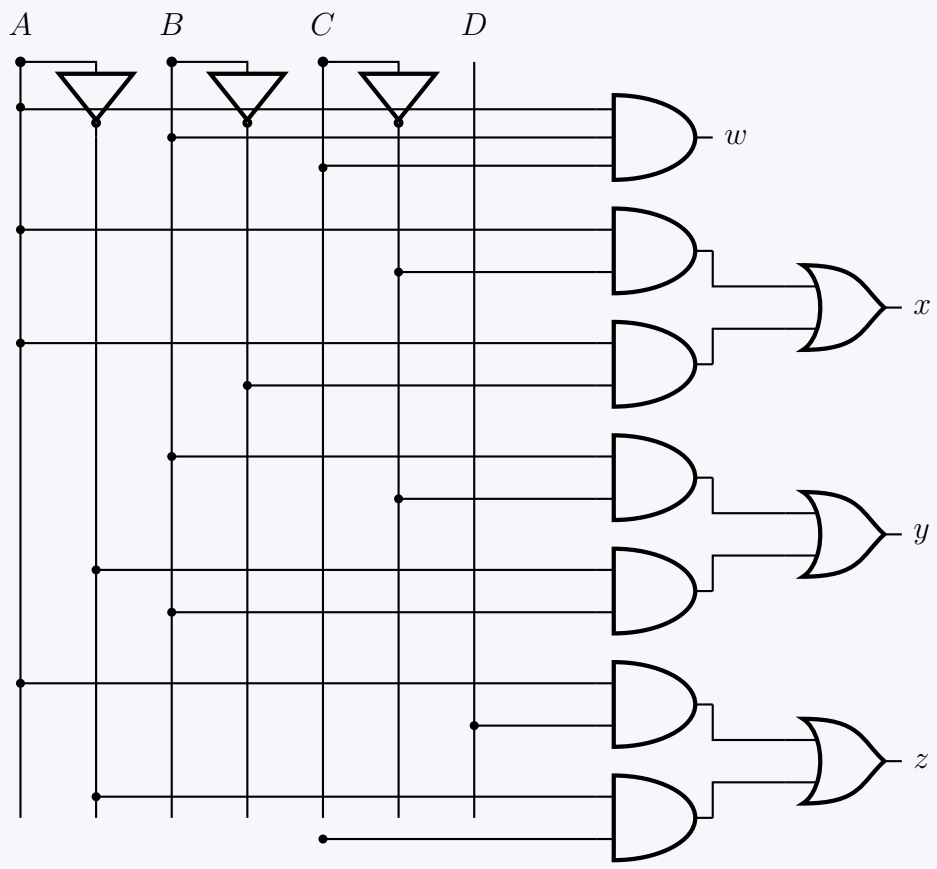
Group 1 (Red): AD
Group 2 (Green): $\overline{A}C$
Simplified: $\mathbf{z = AD + \overline{A}C}$

3. Logic Circuit Implementation

Using the derived minimized equations:

$$w = ABC$$
$$x = A\overline{C} + A\overline{B}$$
$$y = B\overline{C} + \overline{A}B$$
$$z = AD + \overline{A}C$$

We construct the circuit diagram utilizing logic gates. The input bus is organized with vertical lines for each variable and their inverted forms.



Number System & Arithmetic Operations

Q1. Describe the steps to find $M - N$ using $(r-1)$'s complement, where M and N are two n-digit unsigned numbers and r is the base. **(13 marks)** **[CSE 205 | L-2/T-1 | 2022–23, 2021–22]**

Answer

Theoretical Foundation

Subtraction of two n-digit unsigned numbers, M (the minuend) and N (the subtrahend), in base r can be efficiently performed using the $(r-1)$**'s complement** method. The $(r-1)$'s complement of a number N is mathematically defined as $(r^n - 1) - N$. This method simplifies hardware implementation by replacing subtraction operations with addition and simple complementation logic.

Algorithmic Steps for computing $M - N$

(a) **Find the $(r-1)$'s complement of the subtrahend N:** Compute the $(r-1)$'s complement of N. For a base r and n digits, this is algebraically represented as:

$$N_{(r-1)'s} = (r^n - 1) - N$$

Note: In practice (e.g., base 2 or base 10), this is achieved by subtracting each digit of N from $(r-1)$. In binary (1's complement), it is simply bitwise inversion.

(b) **Add the minuend M to the $(r-1)$'s complement of N:** Perform standard base-r addition of M and the result from Step 1. Let this intermediate sum be S:

$$\begin{aligned} S &= M + N_{(r-1)'s} \\ S &= M + (r^n - 1) - N \\ S &= (M - N) + r^n - 1 \end{aligned}$$

(c) **Inspect the result for an end carry (carry-out from the most significant digit):** The evaluation of the intermediate sum S splits into two distinct cases depending on whether an end carry is produced.

- **Case A: An end carry occurs (implies $M \geq N$)**

  If an end carry of 1 is generated, it indicates that the r^n term is present and the overall result is positive.

  i. Remove the end carry (which mathematically subtracts r^n).
  ii. Add 1 to the least significant digit (LSD) of the result. This operation is known as an **end-around carry**.

  Proof:

$$\begin{aligned} \text{Final Result} &= (S - r^n) + 1 \\ &= \big((M - N) + r^n - 1 - r^n\big) + 1 \\ &= (M - N) - 1 + 1 \\ &= M - N \end{aligned}$$

- **Case B: No end carry occurs (implies $M < N$)**

  If no end carry is generated, it indicates that the result is inherently negative, and the sum S is currently stored in its $(r-1)$'s complement form.

  i. Take the $(r-1)$'s complement of the intermediate sum S.
  ii. Affix a negative sign to the final magnitude.

  Proof:

$$\begin{aligned}\text{Final Magnitude} &= (r^n - 1) - S \\ &= (r^n - 1) - \big((M - N) + r^n - 1\big) \\ &= -(M - N) \\ &= N - M\end{aligned}$$

  Affixing the negative sign correctly yields $-(N - M) = M - N$.

Q2. Explain the difference between binary and gray encoding and the benefits of each. **(9 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Answer

1. Fundamental Differences

The primary difference between Standard Binary and Gray encoding lies in their structural properties regarding bit weighting and bit transitions between consecutive numbers.

- **Standard Binary Code:** This is a *weighted* (positional) number system. Each bit position has a specific mathematical weight (e.g., $2^0, 2^1, 2^2, \dots$). When transitioning between consecutive numbers, multiple bits can change simultaneously. For example, changing from decimal 3 to 4 requires three bit changes (**011** $\rightarrow$ **100**).
- **Gray Code:** This is an *unweighted* number system. It is a cyclic code where successive values differ in exactly **one bit position**. For example, transitioning from decimal 3 to 4 requires only one bit change (**010** $\rightarrow$ **110**).

Decimal	Binary Code	Gray Code	Bits Changed (Gray)
0	000	000	-
1	001	001	1
2	010	011	1
3	011	010	1
4	100	110	1
5	101	111	1
6	110	101	1
7	111	100	1

2. Benefits of Binary Encoding

Binary encoding is the foundational language of digital logic systems due to its direct mapping to base-2 mathematics.

- **Mathematical Computations:** Because it is a weighted system, standard arithmetic operations (addition, subtraction, multiplication, division) are natively straightforward to implement using standard combinational logic (e.g., adders and multipliers).
- **Magnitude Comparison:** Comparing two binary numbers (greater than, less than) is simple because the position of the bits directly indicates their magnitude.
- **Interfacing:** Most microprocessors, memories, and digital buses inherently process data in standard binary formats, requiring no conversion overhead for internal data routing.

3. Benefits of Gray Encoding

Gray code is primarily used to prevent spurious outputs from electromechanical switches and digital logic elements during state transitions.

- **Glitch and Error Minimization:** Since only one bit changes between adjacent values, the system is immune to intermediate transient states (glitches). In standard binary, varying signal propagation delays during the transition from 011 to 100 might momentarily read as 111 or 000. Gray code entirely eliminates this multi-bit race condition.
- **Electromechanical Rotary Encoders:** Gray code absolute encoders prevent angular position read errors. If the sensor is exactly on the boundary of two positions, the physical uncertainty only affects one bit, ensuring the read value is at most one position off.
- **Logic Minimization (K-Maps):** Karnaugh Maps utilize Gray code ordering for their axes. The single-bit change ensures that physically adjacent cells represent logically adjacent minterms, allowing for visual identification of prime implicants.
- **Clock Domain Crossing (CDC):** In asynchronous FIFO designs, read and write pointers are often converted to Gray code before being synchronized into different clock domains. Because only one bit changes at a time, the receiving clock domain will either see the old pointer value or the new pointer value, preventing data corruption.

Q3. Write the rules of subtraction for two n-digit numbers M and N in base r. Consider two cases: $M \geq N$ and $M < N$. Find the value of $9 - 6.9 - 9.8$ using the complement rule where the base is Hexadecimal. **(2+5 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Rules of Subtraction using r's Complement

To subtract two n-digit unsigned numbers, $M - N$, in a given base r, the r's complement method replaces subtraction with addition.
The r's complement of the subtrahend N is mathematically defined as $r^n - N$.
The subtraction procedure follows these steps:

(a) **Addition Phase:** Add the minuend M to the r's complement of the subtrahend N.

$$S = M + (r^n - N) = r^n + (M - N)$$

(b) **Case 1:** $M \geq N$

- The addition will produce a sum $S \geq r^n$, generating an **end carry** out of the most significant digit.
- **Rule:** Discard the end carry (which effectively subtracts r^n). The remaining result is the positive value $(M - N)$.

(c) **Case 2:** $M < N$

- The addition will produce a sum $S < r^n$, and **no end carry** is generated.
- **Rule:** The absence of a carry indicates a negative result. To find the magnitude, take the r's complement of the sum S and place a negative sign in front of it.

$$-[r^n - S] = -[r^n - (r^n + M - N)] = -(N - M) = M - N$$

2. Evaluation of $9 - 6.9 - 9.8$ in Hexadecimal (Base 16)

We evaluate the expression strictly left-to-right: $(9_{16} - 6.9_{16}) - 9.8_{16}$.
To ensure proper alignment for carries, we pad the numbers to 2 integer digits and 1 fractional digit (`XX.Y`).

- $M = 09.0_{16}$
- $N_1 = 06.9_{16}$
- $N_2 = 09.8_{16}$

Step 2A: First Subtraction, $A = 09.0_{16} - 06.9_{16}$
1. Find the 16's complement of $N_1 = 06.9_{16}$.

$$\text{15's complement of } 06.9_{16} = (F - 0)(F - 6).(F - 9) = F9.6_{16}$$
$$\text{16's complement of } 06.9_{16} = F9.6_{16} + 00.1_{16} = F9.7_{16}$$

2. Add M to the 16's complement of N_1:

$$09.0_{16} + F9.7_{16} = 102.7_{16}$$

3. An end carry of 1 is generated (Case $M \geq N$). Discard the carry.

$$A = 02.7_{16}$$

Step 2B: Second Subtraction, $B = A - N_2 = 02.7_{16} - 09.8_{16}$
1. Find the 16's complement of $N_2 = 09.8_{16}$.

$$\text{15's complement of } 09.8_{16} = (F - 0)(F - 9).(F - 8) = F6.7_{16}$$
$$\text{16's complement of } 09.8_{16} = F6.7_{16} + 00.1_{16} = F6.8_{16}$$

2. Add A to the 16's complement of N_2:

$$02.7_{16} + F6.8_{16} = F8.F_{16} \quad (\text{Note: } 7 + 8 = F_{16},\ 2 + 6 = 8_{16})$$

3. No end carry is generated (Case $M < N$), which signifies a negative result. To find the final magnitude, we must take the 16's complement of the sum $F8.F_{16}$.

$$\text{15's complement of } F8.F_{16} = (F - F)(F - 8).(F - F) = 07.0_{16}$$
$$\text{16's complement of } F8.F_{16} = 07.0_{16} + 00.1_{16} = 07.1_{16}$$

4. Affix the negative sign to the final magnitude:

$$B = -07.1_{16} = -7.1_{16}$$

Final Answer: The value of $9_{16} - 6.9_{16} - 9.8_{16}$ is $\mathbf{-7.1_{16}}$.

Q4. What is the difference between r's complement and $(r-1)$'s complement (where r is the base)? **(2 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Mathematical Definitions

Let N be a positive number in base r having n integer digits and m fractional digits.

- **$(r-1)$'s Complement (Diminished Radix Complement):** Mathematically, it is defined as:

$$(r-1)\text{'s Complement} = r^n - r^{-m} - N$$

  In practice, this is obtained by subtracting every single digit of the number N from the maximum possible digit value in that base, which is $(r-1)$.

- **r's Complement (Radix Complement):** Mathematically, it is defined as:

$$r\text{'s Complement} = r^n - N$$

  It can be straightforwardly obtained by finding the $(r-1)$'s complement and then adding 1 to the least significant digit (LSD), represented by r^{-m}.

$$r\text{'s Complement} = (r-1)\text{'s Complement} + r^{-m}$$

2. Key Operational Differences

Characteristic	$(r-1)$'s Complement	r's Complement
Computation Method	Simple digit-by-digit operation. For base-2, it is a simple bitwise inversion (NOT operation).	Requires generating the $(r-1)$'s complement first, then mathematically adding 1 to the LSD.
Hardware Complexity	Generating the complement is fast (no carry propagation), but arithmetic operations require an extra adder step.	Slower to generate (requires carry propagation through an adder), but simplifies arithmetic logic later.
Representation of Zero	Has **two** representations of zero: Positive Zero (+0) and Negative Zero (−0).	Has a **unique**, single representation of zero.
Subtraction Logic ($M-N$)	If an end carry is generated, it must be added back to the LSB. This is called an **end-around carry**.	If an end carry is generated, it is simply **discarded** (ignored).

3. Illustrative Example ($r=2$, Binary Base)

Consider the binary number $N = 10110_2$. Here, the base is $r = 2$, the number of integer digits is $n = 5$, and there are no fractional digits ($m = 0$, so $r^{-m} = 1$).

- **Finding the 1's Complement ($(r-1)$'s Complement):**
  Subtract each bit from 1 (which is equivalent to flipping every bit):

$$1\text{'s Complement} = 01001_2$$

- **Finding the 2's Complement (r's Complement):**
  Add 1 to the Least Significant Bit (LSB) of the 1's complement:

$$\begin{aligned} 2\text{'s Complement} &= 01001_2 + 1_2 \\ &= 01010_2 \end{aligned}$$

Q5. Convert $(0.625)_{10}$ to binary. Show every step of your calculation. **(4 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Method: Successive Multiplication by 2

To convert a decimal fraction to binary, we use the method of successive multiplication by the target base (which is 2 for binary). The procedure is as follows:

(a) Multiply the decimal fractional part by 2.

(b) Record the integer part of the resulting product. This integer becomes a digit in the binary fraction.

(c) Take the new fractional part and multiply it by 2 again.

(d) Repeat the process until the fractional part becomes 0.00 (exact conversion) or until the desired precision is reached.

(e) Read the recorded integer parts from **top to bottom** (first generated to last generated) to form the binary fraction.

2. Step-by-Step Calculation

We apply the successive multiplication method to $(0.625)_{10}$:

$$\begin{aligned} 0.625 \times 2 = \mathbf{1}.25 &\implies \text{Integer part: } \mathbf{1}, \quad \text{Remaining fraction: } 0.25 \\ 0.25 \times 2 = \mathbf{0}.50 &\implies \text{Integer part: } \mathbf{0}, \quad \text{Remaining fraction: } 0.50 \\ 0.50 \times 2 = \mathbf{1}.00 &\implies \text{Integer part: } \mathbf{1}, \quad \text{Remaining fraction: } 0.00 \end{aligned}$$

The fractional part has now reached 0.00, indicating that the conversion terminates precisely.

3. Final Result

Reading the generated integer parts from the first step down to the last step (top to bottom), we get the sequence $\mathbf{1}, \mathbf{0}, \mathbf{1}$.

Therefore, the decimal fraction $(0.625)_{10}$ is exactly equal to the binary fraction:

$$(0.625)_{10} = (0.101)_2$$

Q6. A decimal number N is represented in a weighted binary code as `0011 0100 1010 1100`. Write the decimal digits of N, if the following weights are used for the weighted binary code: 6, 3, 1, 1. **(4 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Method of Decoding

In a weighted binary code, each bit position in a sequence is assigned a specific numerical weight. For a 4-bit group $B = b_3b_2b_1b_0$ with corresponding weights $W = w_3, w_2, w_1, w_0$, the decimal equivalent digit D is calculated by computing the sum of the products of each bit and its respective weight:

$$D = (b_3 \times w_3) + (b_2 \times w_2) + (b_1 \times w_1) + (b_0 \times w_0)$$

Given the weights are $\mathbf{6}, \mathbf{3}, \mathbf{1}, \mathbf{1}$, the formula becomes:

$$D = (b_3 \times 6) + (b_2 \times 3) + (b_1 \times 1) + (b_0 \times 1)$$

The given 16-bit binary sequence is `0011 0100 1010 1100`. We will divide this sequence into four 4-bit nibbles, with each nibble representing a single decimal digit.

2. Step-by-Step Evaluation

We evaluate each 4-bit group from left to right:

- **First Digit (D_1):** `0011`

$$\begin{aligned} D_1 &= (0 \times 6) + (0 \times 3) + (1 \times 1) + (1 \times 1) \\ &= 0 + 0 + 1 + 1 \\ &= \mathbf{2} \end{aligned}$$

- **Second Digit (D_2):** `0100`

$$\begin{aligned} D_2 &= (0 \times 6) + (1 \times 3) + (0 \times 1) + (0 \times 1) \\ &= 0 + 3 + 0 + 0 \\ &= \mathbf{3} \end{aligned}$$

- **Third Digit (D_3):** `1010`

$$\begin{aligned} D_3 &= (1 \times 6) + (0 \times 3) + (1 \times 1) + (0 \times 1) \\ &= 6 + 0 + 1 + 0 \\ &= \mathbf{7} \end{aligned}$$

- **Fourth Digit (D_4):** `1100`

$$\begin{aligned} D_4 &= (1 \times 6) + (1 \times 3) + (0 \times 1) + (0 \times 1) \\ &= 6 + 3 + 0 + 0 \\ &= \mathbf{9} \end{aligned}$$

3. Summary Table

4-bit Group ($b_3b_2b_1b_0$)	Calculation ($6 \cdot b_3 + 3 \cdot b_2 + 1 \cdot b_1 + 1 \cdot b_0$)	Decimal Digit
0011	$0+0+1+1$	**2**
0100	$0+3+0+0$	**3**
1010	$6+0+1+0$	**7**
1100	$6+3+0+0$	**9**

Final Answer

The decimal digits of N are $2, 3, 7$, and 9. Therefore, the decimal number N is **2379**.

Q7. You are given two numbers: $(305.E)_{16}$ and $(109.6875)_{10}$.

(a) Convert the given numbers into binary.

(b) Subtract the latter number from the former using binary arithmetic.

(c) Convert the result of the subtraction into octal.

(15 marks) [CSE 205 | L-2/T-1 | 2013–14]

Answer

Part (a): Conversion to Binary

1. Converting $(305.E)_{16}$ to binary: Each hexadecimal digit corresponds directly to a 4-bit binary sequence.

$$3_{16} \longrightarrow 0011_2$$
$$0_{16} \longrightarrow 0000_2$$
$$5_{16} \longrightarrow 0101_2$$
$$E_{16} \longrightarrow 1110_2$$

Combining these nibbles gives:

$$(305.E)_{16} = (0011\,0000\,0101.1110)_2 = (\mathbf{1100000101.1110})_\mathbf{2}$$

2. Converting $(109.6875)_{10}$ to binary: We split the decimal number into its integer and fractional parts.
Integer Part (109_{10}): Successive division by 2.

$$\begin{aligned} 109 \div 2 = 54 \quad &\text{Remainder: } \mathbf{1} \text{ (LSB)} \\ 54 \div 2 = 27 \quad &\text{Remainder: } \mathbf{0} \\ 27 \div 2 = 13 \quad &\text{Remainder: } \mathbf{1} \\ 13 \div 2 = 6 \quad &\text{Remainder: } \mathbf{1} \\ 6 \div 2 = 3 \quad &\text{Remainder: } \mathbf{0} \\ 3 \div 2 = 1 \quad &\text{Remainder: } \mathbf{1} \\ 1 \div 2 = 0 \quad &\text{Remainder: } \mathbf{1} \text{ (MSB)} \end{aligned}$$

Reading from bottom to top, the integer part is $(1101101)_2$.
Fractional Part (0.6875_{10}): Successive multiplication by 2.

$$\begin{aligned} 0.6875 \times 2 = \mathbf{1}.375 \quad &\Longrightarrow \text{ Integer: } \mathbf{1} \text{ (MSB)} \\ 0.3750 \times 2 = \mathbf{0}.750 \quad &\Longrightarrow \text{ Integer: } \mathbf{0} \\ 0.7500 \times 2 = \mathbf{1}.500 \quad &\Longrightarrow \text{ Integer: } \mathbf{1} \\ 0.5000 \times 2 = \mathbf{1}.000 \quad &\Longrightarrow \text{ Integer: } \mathbf{1} \text{ (LSB)} \end{aligned}$$

Reading from top to bottom, the fractional part is $(.1011)_2$.
Combining both parts gives:

$$(109.6875)_{10} = (\mathbf{1101101.1011})_\mathbf{2}$$

Part (b): Binary Subtraction

We need to compute $X - Y$, where:

$$X = 1100000101.1110_2$$
$$Y = 1101101.1011_2$$

To perform the subtraction using the **2's complement method**, we first equalize the number of bits in X and Y by padding with leading zeros for the integer part and trailing zeros for the fractional part (if necessary). We will use a 14-bit representation (10 integer bits, 4 fractional bits).

$$X = 1100000101.1110$$
$$Y = 0001101101.1011$$

Step 1: Find the 2's complement of Y:

$$\begin{aligned} \text{1's complement of } Y &= 1110010010.0100 \\ \text{Add 1 to LSB } (0.0001) &= \underline{+\,0000000000.0001} \\ \text{2's complement of } Y &= 1110010010.0101 \end{aligned}$$

Step 2: Add X to the 2's complement of Y:

$$\begin{array}{rl} 1100000101.1110 & (X) \\ \underline{+\ 1110010010.0101} & (\text{2's complement of } Y) \\ \mathbf{1}\ 1010011000.0011 & (\text{Sum}) \end{array}$$

Step 3: Discard the end carry: Since an end carry is generated (**1**), the result is positive, and the carry is discarded.

$$X - Y = \mathbf{1010011000.0011_2}$$

Part (c): Conversion of Result to Octal

To convert the binary result 1010011000.0011_2 to octal, we group the bits into sets of three, starting from the binary point and moving outwards (left for the integer part, right for the fractional part). Pad with zeros where necessary.

$$\begin{aligned} \text{Integer part grouping: } & \underbrace{001}_{\mathbf{1}}\ \underbrace{010}_{\mathbf{2}}\ \underbrace{011}_{\mathbf{3}}\ \underbrace{000}_{\mathbf{0}} \\ \text{Fractional part grouping: } & .\underbrace{001}_{\mathbf{1}}\ \underbrace{100}_{\mathbf{4}} \end{aligned}$$

Replacing each 3-bit group with its corresponding octal digit yields the final answer:

$$(1010011000.0011)_2 = (\mathbf{1230.14})_\mathbf{8}$$

Q8. Perform the following binary subtraction: $11000011 - 10101010$. **(6 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

1. Method A: 2's Complement Subtraction (Standard Digital Logic Approach)

In digital hardware, subtraction $M - N$ is typically implemented using 2's complement addition: $M + (\text{2's complement of } N)$. Given:

$$\begin{aligned} M &= 11000011_2 \\ N &= 10101010_2 \end{aligned}$$

Step 1: Find the 2's complement of the subtrahend N

$$\begin{aligned} \text{1's complement of } N \text{ (invert all bits)} &= 01010101_2 \\ \text{Add 1 to the LSB} &= \underline{+\,00000001_2} \\ \text{2's complement of } N &= 01010110_2 \end{aligned}$$

Step 2: Add M and the 2's complement of N

$$\begin{array}{rl} 11000011_2 & (M) \\ \underline{+\ 01010110_2} & (\text{2's complement of } N) \\ \mathbf{1}\ 00011001_2 & (\text{Sum}) \end{array}$$

Step 3: Evaluate the end carry An end carry of **1** is generated, indicating that the result is positive. We discard the carry to obtain the final result.

$$\text{Result} = \mathbf{00011001_2}$$

2. Method B: Direct Binary Subtraction (Borrow Method)

We can also perform direct bit-by-bit subtraction. When subtracting 1 from 0, a borrow is required from the next most significant bit that contains a 1.

$$\begin{array}{rl} & 1\not{1}^{0}\not{0}^{1}\not{0}^{1}0011_2 \quad (\textit{Borrows propagate from bit 6 down to bit 3}) \\ - & 10101010_2 \\ \hline & \mathbf{00011001_2} \end{array}$$

Bit-by-bit breakdown:

- **Bit 0:** $1 - 0 = 1$
- **Bit 1:** $1 - 1 = 0$

- **Bit 2:** $0 - 0 = 0$
- **Bit 3:** $0 - 1 \implies$ Requires a borrow. The nearest 1 is at Bit 6. Borrowing leaves Bit 6 as 0, makes Bits 5 and 4 as 1, and gives Bit 3 a value of 2_{10} (10_2). $2 - 1 = 1$.
- **Bit 4:** $1(\text{borrowed}) - 0 = 1$
- **Bit 5:** $1(\text{borrowed}) - 1 = 0$
- **Bit 6:** $0(\text{after borrow out}) - 0 = 0$
- **Bit 7:** $1 - 1 = 0$

3. Verification (Decimal Conversion)

To ensure mathematical accuracy, we can verify the result using base-10 equivalent values:

$$\begin{aligned} M &= 11000011_2 = 128 + 64 + 2 + 1 = 195_{10} \\ N &= 10101010_2 = 128 + 32 + 8 + 2 = 170_{10} \\ M - N &= 195_{10} - 170_{10} = \mathbf{25_{10}} \end{aligned}$$

Converting the result back to binary verifies our answer:

$$25_{10} = 16 + 8 + 1 = \mathbf{00011001_2}$$

Final Answer: $11000011_2 - 10101010_2 = \mathbf{00011001_2}$

Q9. Using 2's complement, subtract decimal -70 from decimal -50. **(9 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

1. Problem Formulation and Bit Length

We are asked to evaluate $M - N$, where:

$$\begin{aligned} M &= -50_{10} \\ N &= -70_{10} \end{aligned}$$

The mathematical operation is $(-50_{10}) - (-70_{10})$, which is mathematically equivalent to $(-50_{10}) + (+70_{10})$.
To perform this in binary, we must determine the necessary bit width. The expected result is $+20_{10}$. The operands and the result all fall within the range $[-128, +127]$, which is exactly the representable range for **8-bit signed 2's complement** numbers. Therefore, we will use an 8-bit representation.

2. 8-Bit 2's Complement Representation of Operands

Step 2A: Representing $M = -50_{10}$

i. Find the binary magnitude of $+50$: $50 = 32 + 16 + 2 \implies \mathbf{00110010_2}$

ii. Find the 1's complement (invert bits): $\mathbf{11001101_2}$

iii. Find the 2's complement (add 1):

$$11001101_2 + 1_2 = \mathbf{11001110_2} \quad (\text{This is } -50_{10})$$

Step 2B: Representing $N = -70_{10}$

i. Find the binary magnitude of $+70$: $70 = 64 + 4 + 2 \implies \mathbf{01000110_2}$

ii. Find the 1's complement (invert bits): $\mathbf{10111001_2}$

iii. Find the 2's complement (add 1):

$$10111001_2 + 1_2 = \mathbf{10111010_2} \quad (\text{This is } -70_{10})$$

3. Subtraction using 2's Complement Addition

In digital logic, the subtraction $M - N$ is performed by adding the 2's complement of the subtrahend (N) to the minuend (M).

$$M - N = M + (\text{2's complement of } N)$$

Since $N = -70_{10}$, its 2's complement is simply $+70_{10}$, which we already found to be $\mathbf{01000110_2}$.
Now, we add M and the 2's complement of N:

$$\begin{array}{rll} & 11001110_2 & (M = -50_{10}) \\ + & 01000110_2 & (\text{2's comp of } N = +70_{10}) \\ \hline \mathbf{1} & \mathbf{00010100_2} & (\text{Sum}) \end{array}$$

4. Evaluation of Result

- **End Carry:** An end carry of **1** is generated out of the Most Significant Bit (MSB). In signed 2's complement arithmetic, when adding numbers of opposite signs, overflow is impossible, and any carry generated out of the sign bit is safely **discarded**.
- **Final Binary Result: $\mathbf{00010100_2}$**

Verification: Since the MSB (sign bit) of the result is 0, the number is positive. We convert the binary magnitude directly to decimal to verify correctness:

$$\begin{aligned} 00010100_2 &= (1 \times 2^4) + (1 \times 2^2) \\ &= 16 + 4 \\ &= +\mathbf{20_{10}} \end{aligned}$$

This mathematically matches the expected decimal calculation: $(-50) - (-70) = +20$.

Q10. Convert $(41.6875)_{10}$ to binary. **(5 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Integer Part Conversion: Successive Division

To convert the integer part $(41)_{10}$ to binary, we repeatedly divide by the target base (2) and record the remainders. The first remainder represents the Least Significant Bit (LSB) and the final remainder represents the Most Significant Bit (MSB).

$$\begin{aligned} 41 \div 2 = 20 \quad &\text{Remainder: } \mathbf{1} \quad \text{(LSB)} \\ 20 \div 2 = 10 \quad &\text{Remainder: } \mathbf{0} \\ 10 \div 2 = 5 \quad &\text{Remainder: } \mathbf{0} \\ 5 \div 2 = 2 \quad &\text{Remainder: } \mathbf{1} \\ 2 \div 2 = 1 \quad &\text{Remainder: } \mathbf{0} \\ 1 \div 2 = 0 \quad &\text{Remainder: } \mathbf{1} \quad \text{(MSB)} \end{aligned}$$

Reading the remainders from bottom to top (MSB to LSB), we obtain:

$$(41)_{10} = (101001)_2$$

2. Fractional Part Conversion: Successive Multiplication

To convert the fractional part $(0.6875)_{10}$ to binary, we repeatedly multiply by the target base (2) and record the integer portion of the product. The first generated integer represents the Most Significant Bit (MSB) of the fraction, closest to the radix point.

$$\begin{aligned} 0.6875 \times 2 = \mathbf{1}.3750 \quad &\Longrightarrow \text{ Integer part: } \mathbf{1} \quad \text{(MSB)} \\ 0.3750 \times 2 = \mathbf{0}.7500 \quad &\Longrightarrow \text{ Integer part: } \mathbf{0} \\ 0.7500 \times 2 = \mathbf{1}.5000 \quad &\Longrightarrow \text{ Integer part: } \mathbf{1} \\ 0.5000 \times 2 = \mathbf{1}.0000 \quad &\Longrightarrow \text{ Integer part: } \mathbf{1} \quad \text{(LSB)} \end{aligned}$$

The fractional part has reached exactly .0000, so the conversion terminates. Reading the integer parts from top to bottom, we obtain:

$$(0.6875)_{10} = (0.1011)_2$$

3. Final Result

Combining the binary integer and fractional parts together:

$$\begin{aligned} (41)_{10} + (0.6875)_{10} &= (101001)_2 + (0.1011)_2 \\ (41.6875)_{10} &= (\mathbf{101001.1011})_\mathbf{2} \end{aligned}$$

Q11. Perform the subtraction with the following binary unsigned numbers using (i) 2's complement, (ii) 1's complement: $11010 - 1101$. **(10 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Initialization and Bit Equalization

We are required to perform the subtraction $M - N$, where:

$$\begin{aligned} M &= 11010_2 \quad \text{(5-bit number)} \\ N &= 1101_2 \quad \text{(4-bit number)} \end{aligned}$$

Before applying complement arithmetic, both numbers must have the same number of bits. We pad the subtrahend N with a leading

zero to equalize the bit lengths to 5 bits.

$$M = 11010_2$$
$$N = 01101_2$$

Part (i): Subtraction using 2's Complement

The operation $M - N$ is evaluated as: $M + (\text{2's complement of } N)$.

Step 1: Find the 2's complement of N

$$\begin{aligned} \text{1's complement of } N \text{ (invert all bits)} &= 10010_2 \\ \text{Add 1 to the LSB} &= \underline{+\,00001_2} \\ \text{2's complement of } N &= 10011_2 \end{aligned}$$

Step 2: Add M to the 2's complement of N

$$\begin{array}{cll} & 11010_2 & (M) \\ + & 10011_2 & (\text{2's complement of } N) \\ \hline \mathbf{1} & \mathbf{01101_2} & (\text{Sum}) \end{array}$$

Step 3: Evaluate the end carry An end carry of **1** is generated. In 2's complement unsigned subtraction, a generated end carry indicates that the result is strictly positive and the carry is simply **discarded**.

$$\text{Final Result} = \mathbf{01101_2}$$

Part (ii): Subtraction using 1's Complement

The operation $M - N$ is evaluated as: $M + (\text{1's complement of } N)$, followed by an end-around carry logic step.

Step 1: Find the 1's complement of N

$$\text{1's complement of } N = 10010_2$$

Step 2: Add M to the 1's complement of N

$$\begin{array}{cll} & 11010_2 & (M) \\ + & 10010_2 & (\text{1's complement of } N) \\ \hline \mathbf{1} & 01100_2 & (\text{Intermediate Sum}) \end{array}$$

Step 3: Evaluate the end carry and perform End-Around Carry An end carry of **1** is generated. In 1's complement unsigned subtraction, this indicates a positive result, but the carry must be added back to the Least Significant Bit (LSB) of the intermediate sum. This is known as an **end-around carry**.

$$\begin{array}{cll} & 01100_2 & (\text{Intermediate Sum}) \\ + & 1_2 & (\text{End-Around Carry}) \\ \hline & \mathbf{01101_2} & (\text{Final Result}) \end{array}$$

3. Verification (Decimal Translation)

We can mathematically verify both methods by converting to decimal:

$$M = 11010_2 = 16 + 8 + 2 = 26_{10}$$
$$N = 01101_2 = 8 + 4 + 1 = 13_{10}$$
$$M - N = 26_{10} - 13_{10} = \mathbf{13_{10}}$$

Converting the decimal result back to binary:

$$13_{10} = 8 + 4 + 1 = \mathbf{01101_2}$$

Both methods successfully yield the correct binary result.

Binary Adder & Subtractor

Q1. Design a 4-bit binary subtractor circuit using full adders.

(a) Construct and draw the logic diagram of the subtractor circuit.

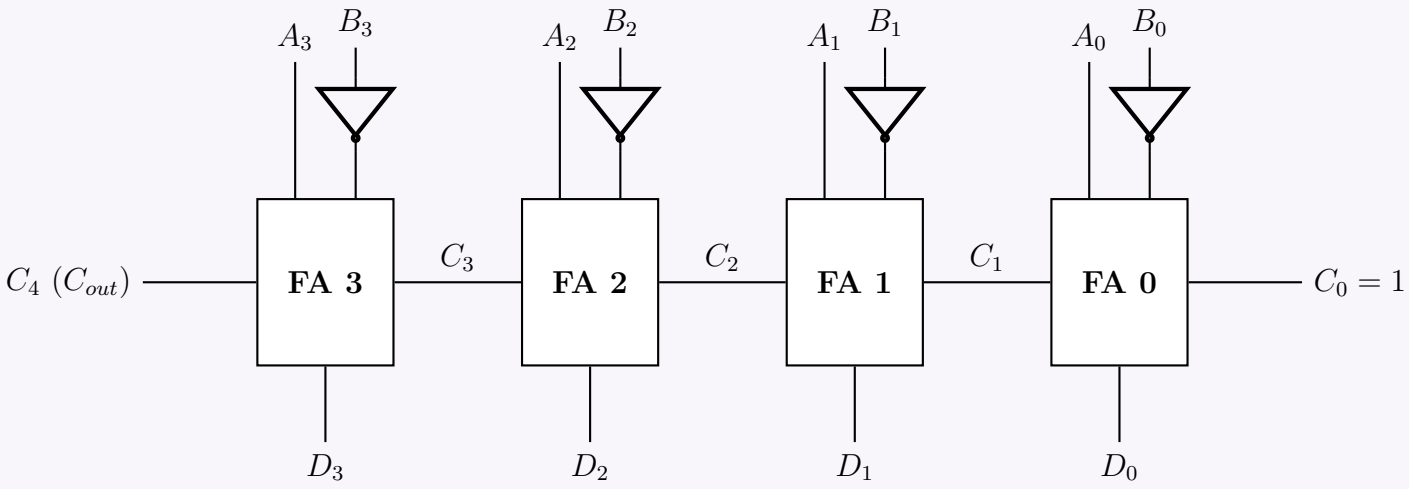
(b) Explain how subtraction is performed using 2's complement representation.(Understand level)

(15 marks) **[CSE 205 | L-2/T-1 | 2024–25]**

Answer

Part (a): Logic Diagram of a 4-Bit Binary Subtractor

To design a 4-bit binary subtractor using Full Adders (FAs), we utilize the principle of 2's complement addition. The circuit consists of four Full Adders in a ripple-carry configuration. The minuend bits (A_0 to A_3) are fed directly into the adders, while the subtrahend bits (B_0 to B_3) are passed through NOT gates to produce their 1's complement. The initial carry-in is set to logic 1 to complete the 2's complement representation.



Part (b): Explanation of Subtraction using 2's Complement

In digital systems, subtraction is typically implemented using addition hardware to reduce complexity. The operation $A - B$ is mathematically equivalent to adding the negative value of the subtrahend (B) to the minuend (A).

$$\begin{aligned} \text{Difference } (D) &= A - B \\ &= A + (-B) \end{aligned}$$

In the binary number system, the negative of a number is represented by its **2's complement**. The computation proceeds in the following logical steps:

(a) **Generating the 1's Complement:** The subtrahend bits ($B_3B_2B_1B_0$) are passed through NOT gates. This logically inverts every bit (changing 1s to 0s and 0s to 1s), producing the 1's complement of B, denoted as $\overline{B}$.

(b) **Forming the 2's Complement:** To convert the 1's complement to the 2's complement, we must mathematically add 1. In the circuit design above, this is achieved by permanently wiring the input carry (C_0) of the least significant Full Adder (FA 0) to a logic high (1).

$$-B = \overline{B} + 1$$

(c) **Parallel Addition:** The cascaded Full Adders now perform a standard parallel addition of three components: the minuend A, the inverted subtrahend $\overline{B}$, and the logic 1 from C_0.

$$D = A + \overline{B} + 1$$

Because $\overline{B} + 1$ is the 2's complement of B, the output naturally evaluates to the difference $A - B$.

(d) **Interpreting the Final Carry (C_4):**

- If $C_4 = 1$: The result is positive and is represented in its true binary form. The end carry is ignored/discarded.
- If $C_4 = 0$: The result is negative and is natively generated in its 2's complement form. To determine its true magnitude, one would need to take the 2's complement of the result.

Q2. Design a full subtractor circuit with three inputs x, y, B_{in} and two outputs Diff and B_{out}. The circuit subtracts $x - y - B_{in}$, where B_{in} is the input borrow and B_{out} is the output borrow. **(18 marks)** **[CSE 205 | L-2/T-1 | 2019–20]**

Answer

1. Truth Table Formulation

A full subtractor is a combinational circuit that performs subtraction between two bits (x and y), taking into account a borrow-in (B_{in}) from a lower significant stage. It produces two outputs: the Difference (Diff) and the Borrow-out (B_{out}). The mathematical operation is $x - y - B_{in}$.

Inputs			Outputs	
x (Minuend)	y (Subtrahend)	B_{in} (Borrow In)	Diff	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2. Boolean Expression Derivation

For Difference (Diff): The minterms for Diff are $\sum m(1, 2, 4, 7)$.

		yB_{in}: 00	01	11	10
x	0	0	1	0	1
	1	1	0	1	0

No adjacent cells can be grouped in the K-map. We write the sum of products (SOP) expression and simplify using Boolean algebra:

$$\begin{aligned}
\text{Diff} &= \bar{x}\bar{y}B_{in} + \bar{x}y\bar{B}_{in} + x\bar{y}\bar{B}_{in} + xyB_{in} \\
&= \bar{x}(\bar{y}B_{in} + y\bar{B}_{in}) + x(\bar{y}\bar{B}_{in} + yB_{in}) && \text{(Distributive Law)} \\
&= \bar{x}(y \oplus B_{in}) + x(\overline{y \oplus B_{in}}) && \text{(XOR and XNOR Definitions)} \\
&= x \oplus (y \oplus B_{in}) && \text{(XOR Definition)} \\
&= \mathbf{x \oplus y \oplus B_{in}}
\end{aligned}$$

For Borrow-out (B_{out}): The minterms for B_{out} are $\sum m(1, 2, 3, 7)$.

		yB_{in}: 00	01	11	10
x	0	0	1	1	1
	1	0	0	1	0

By grouping the 1s in the K-map, we obtain three pairs:

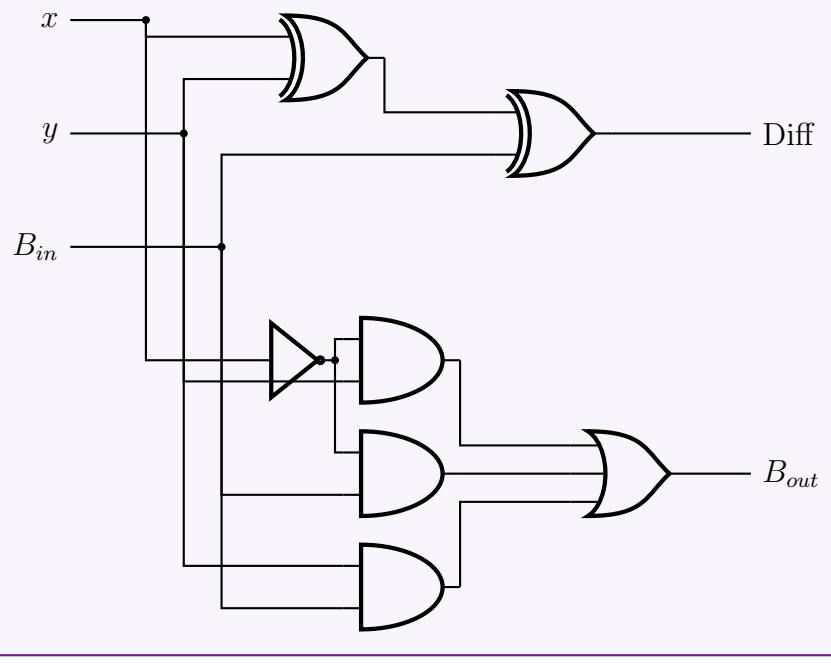
- Group 1 (cells 2, 3 $\rightarrow m_3, m_2$): yields $\mathbf{\bar{x}y}$
- Group 2 (cells 1, 2 $\rightarrow m_1, m_3$): yields $\mathbf{\bar{x}B_{in}}$
- Group 3 (cells 2, 6 $\rightarrow m_3, m_7$): yields $\mathbf{yB_{in}}$

Thus, the simplified Boolean expression is:

$$\mathbf{B_{out} = \bar{x}y + \bar{x}B_{in} + yB_{in}}$$

3. Logic Circuit Diagram

Using the simplified Boolean expressions, we can draw the full subtractor circuit using XOR, AND, OR, and NOT gates.



Q3. Write the advantages and disadvantages of serial and parallel adders. **(4 marks)** **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

1. Serial Adders

A serial adder processes binary addition one bit at a time, sequentially. It typically uses a single Full Adder (FA), a flip-flop (to store the carry bit for the next clock cycle), and shift registers to hold the data.

Advantages:

- **Minimal Hardware Complexity:** Requires only a single Full Adder and one D flip-flop, regardless of the size (n-bits) of the operands.
- **Low Cost and Reduced Area:** Because of the minimal gate count, it occupies very little space on a silicon die and is inexpensive to manufacture.
- **High Scalability:** Expanding the adder to handle larger operands does not require adding more logic gates; the system only requires running the clock for more cycles.

Disadvantages:

- **Slow Operational Speed:** The addition operation is inherently slow. To add two n-bit numbers, the system requires n clock cycles.
- **Complex Control Mechanism:** Requires additional control logic, shift registers, and a synchronized clock to feed the bits sequentially.

2. Parallel Adders

A parallel adder processes all bits of the corresponding operands simultaneously. Common implementations include the Ripple Carry Adder (RCA) and the Carry Lookahead Adder (CLA).

Advantages:

- **High Operational Speed:** Performs the arithmetic operation extremely fast, as all bits are added concurrently in a single operation phase.
- **Immediate Results:** The final sum bits and the final carry-out are available almost immediately, subject only to the combinational propagation delay of the logic gates.
- **Simpler Control Logic:** Does not depend on sequential shifting or multiple clock cycles to yield a single addition result.

Disadvantages:

- **High Hardware Overhead:** Requires n Full Adders to compute the sum of two n-bit operands, substantially increasing the gate count.
- **Increased Cost and Power Consumption:** Larger silicon area translates to higher production costs and greater power dissipation.
- **Propagation Delay (in RCA):** In basic parallel architectures like the Ripple Carry Adder, the carry bit must propagate from the Least Significant Bit (LSB) to the Most Significant Bit (MSB), creating a delay bottleneck for large bit-widths (though this is solved by using more hardware-intensive Lookahead architectures).

3. Summary Comparison Table

Parameter	Serial Adder	Parallel Adder
Operating Speed	Slow (Takes n clock cycles)	Fast (Combinational delay time)
Hardware Required	1 Full Adder, 1 Flip-Flop	n Full Adders (for n-bit addition)
Circuit Cost	Low	High
Silicon Area	Very small	Large
Data Application	Bits are fed serially (one by one)	Bits are fed simultaneously
Scalability overhead	Time (More cycles needed)	Hardware (More gates needed)

Q4. Design a 4-bit BCD adder using 4-bit binary adders and basic logic gates. Use a block diagram to draw a binary adder. **(15 marks)**
[CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Principle of Operation

A 4-bit Binary Coded Decimal (BCD) adder performs the addition of two BCD digits (ranging from 0 to 9) and produces a valid BCD sum. Since a 4-bit binary adder natively computes base-16 (from 0000 to 1111), its output must be corrected if the sum exceeds the valid BCD range (> 9) or if a carry is generated out of the most significant bit.
The conditions for a valid BCD sum requiring **no correction** are:

- The binary sum is less than or equal to 9 (1001_2).
- No final carry out is generated ($K = 0$).

The condition requiring **correction** (adding 6_{10} or 0110_2 to the sum) is met if the sum is invalid (> 9). This happens under three specific scenarios:

(a) A carry (K) is generated out of the MSB of the binary adder (e.g., $9 + 9 = 18$, carry generated).

(b) The sum bits $Z_3Z_2Z_1Z_0$ fall in the range 1010 to 1111. This occurs when:

- $Z_3 = 1$ and $Z_2 = 1$ (Values 12, 13, 14, 15).
- $Z_3 = 1$ and $Z_1 = 1$ (Values 10, 11).

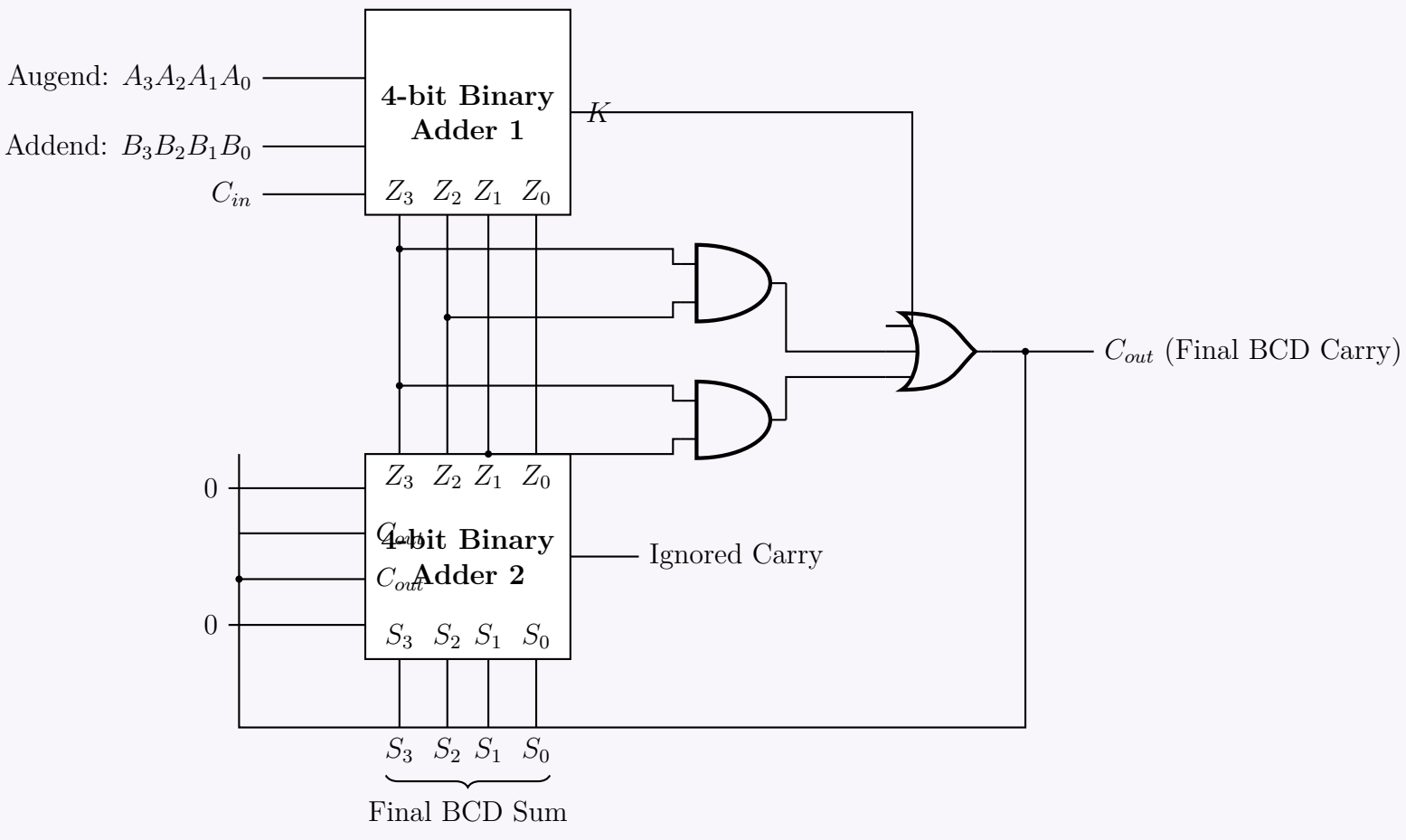
2. Correction Logic Derivation

Based on the conditions above, we can express the logic for the final carry-out (C_{out}) which triggers the addition of 0110_2:

$$C_{out} = K + (Z_3 \cdot Z_2) + (Z_3 \cdot Z_1)$$

When $C_{out} = 1$, the second 4-bit binary adder adds 0110_2 to the intermediate sum $Z_3Z_2Z_1Z_0$ to produce the final valid BCD sum $S_3S_2S_1S_0$. If $C_{out} = 0$, it adds 0000_2, leaving the valid sum unchanged.

3. BCD Adder Block Diagram



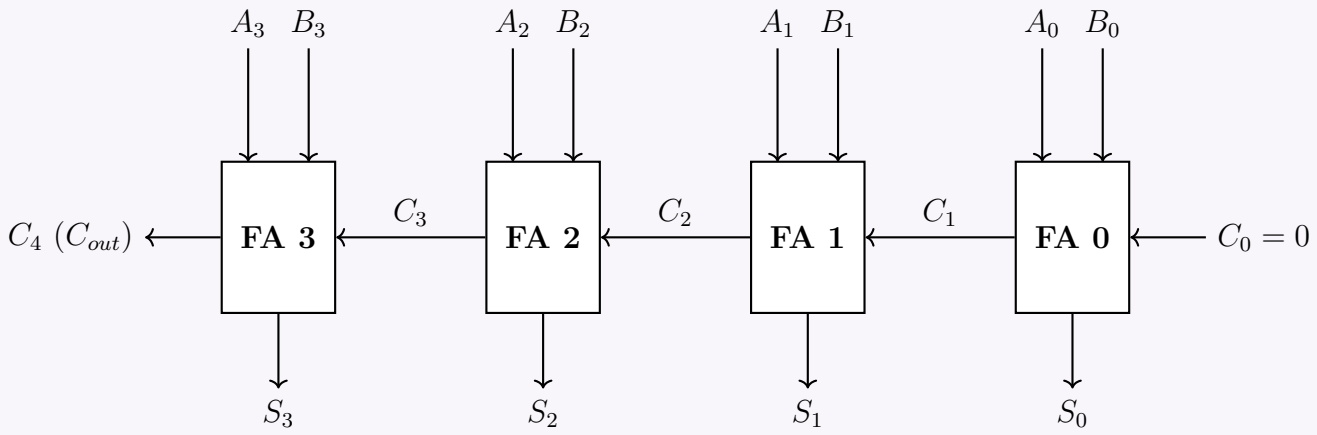
Q5. Design a four-bit binary adder using a 1-bit full-adder circuit. How can it be modified to use as a four-bit subtractor? **(15 marks)**
[CSE 205 | L-2/T-1 | 2013–14]

Answer

1. Design of a Four-Bit Binary Adder

A 4-bit binary adder can be constructed by cascading four 1-bit Full Adder (FA) circuits in a configuration known as a **Ripple Carry Adder (RCA)**. Each full adder computes the addition of two corresponding bits of the operands (A_i and B_i) along with the carry generated from the previous, less significant stage (C_i).
For two 4-bit numbers $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$:

- The Carry-In to the Least Significant Bit (LSB) stage, C_0, is tied to 0.
- The Carry-Out of each FA (C_{i+1}) is directly connected to the Carry-In of the next FA.
- The final Carry-Out (C_4) indicates an overflow or the 5th bit of the sum.



2. Modification to a Four-Bit Adder-Subtractor

To modify the binary adder to perform both addition and subtraction, we utilize the principle of **2's Complement Arithmetic**. Subtraction $A - B$ is mathematically equivalent to adding the 2's complement of B to A:

$$\begin{aligned} A - B &= A + (-B) \\ &= A + (\text{1's complement of } B) + 1 \\ &= A + \bar{B} + 1 \end{aligned}$$

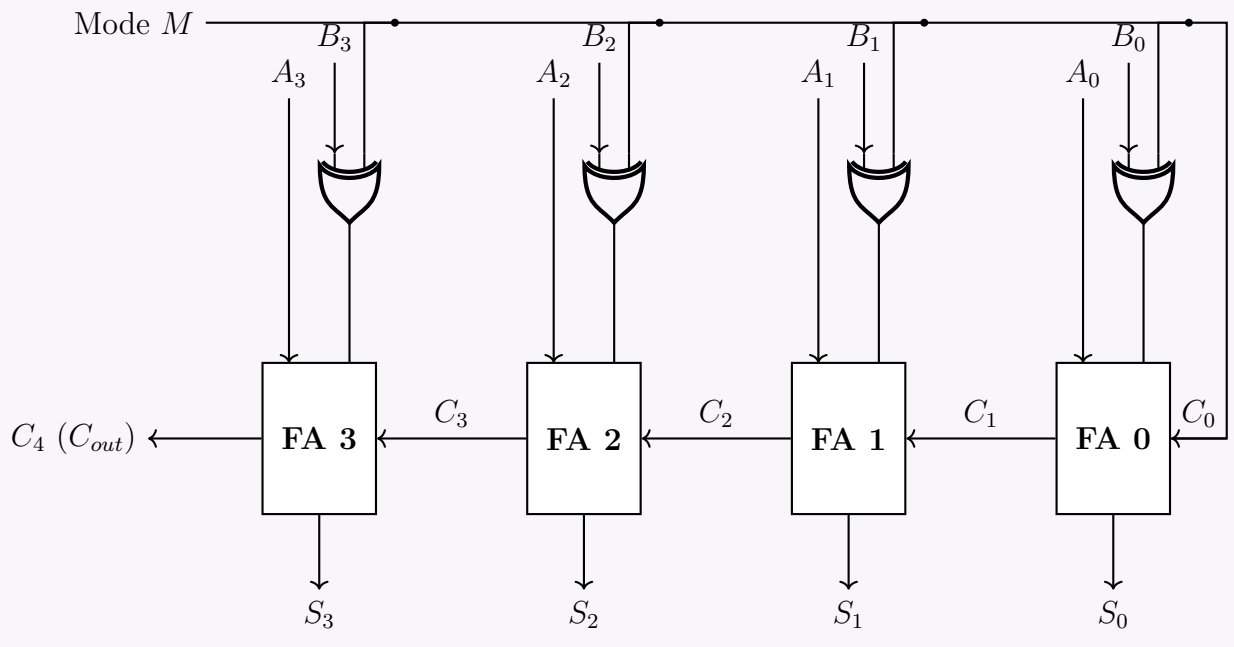
We introduce a control mode line, M, and add an **XOR gate** at the B input of each full adder. The properties of the XOR gate dictate:

$$\begin{aligned} B \oplus 0 &= B \quad \text{(Passes the bit unchanged)} \\ B \oplus 1 &= \bar{B} \quad \text{(Inverts the bit)} \end{aligned}$$

Operation Modes:

- **Addition ($M = 0$):** The XOR gates output $B \oplus 0 = B$. The initial carry-in C_0 receives $M = 0$. The circuit computes $S = A + B$.
- **Subtraction ($M = 1$):** The XOR gates output $B \oplus 1 = \bar{B}$. The initial carry-in C_0 receives $M = 1$. The circuit computes $S = A + \bar{B} + 1$, yielding the correct $A - B$.

3. Four-Bit Adder-Subtractor Circuit Diagram



Q6. Starting with a truth table, design a full subtractor. **(20 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

1. Truth Table

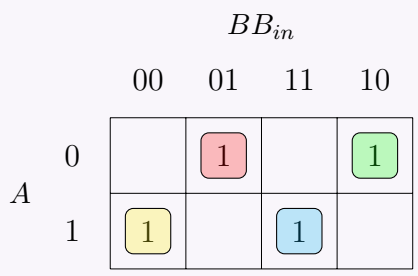
A Full Subtractor is a combinational circuit that performs subtraction between two bits (A and B), taking into account a borrow-in (B_{in}) from a lower significant stage. It produces two outputs: the Difference (D) and the Borrow-out (B_{out}).

Inputs			Outputs	
Minuend (A)	**Subtrahend (B)**	**Borrow-in (B_{in})**	**Difference (D)**	**Borrow-out (B_{out})**
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2. Karnaugh Maps and Boolean Derivation

Using the truth table, we can plot the Karnaugh maps for the outputs D and B_{out}.

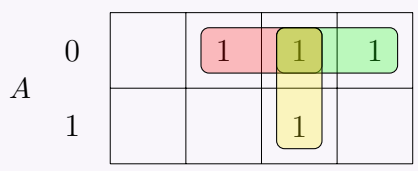
For Difference (D):
The minterms are $\sum m(1, 2, 4, 7)$.



No simplifications (groupings) are possible. We derive the expression mathematically:

$$\begin{aligned}
D &= \bar{A}\bar{B}B_{in} + \bar{A}B\overline{B_{in}} + A\bar{B}\overline{B_{in}} + ABB_{in} \\
&= \bar{A}(\bar{B}B_{in} + B\overline{B_{in}}) + A(\bar{B}\overline{B_{in}} + BB_{in}) && \text{(Distributive Law)} \\
&= \bar{A}(B \oplus B_{in}) + A(\overline{B \oplus B_{in}}) && \text{(XOR and XNOR definitions)} \\
&= A \oplus (B \oplus B_{in}) && \text{(XOR definition)} \\
&= A \oplus B \oplus B_{in}
\end{aligned}$$

For Borrow-out (B_{out}):
The minterms are $\sum m(1, 2, 3, 7)$.

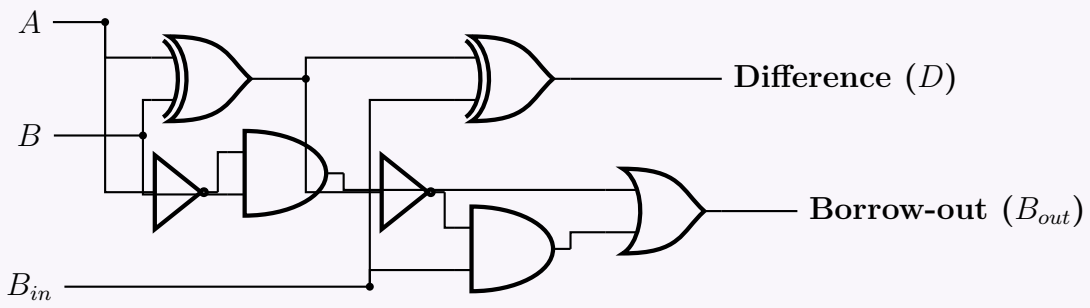


From the K-map groupings, the standard Sum of Products (SOP) is $B_{out} = \bar{A}B + \bar{A}B_{in} + BB_{in}$. To implement this efficiently using XOR gates (sharing logic with the Difference circuit), we algebraically manipulate the unsimplified minterms:

$$\begin{aligned}
B_{out} &= \bar{A}\bar{B}B_{in} + \bar{A}B\overline{B_{in}} + \bar{A}BB_{in} + ABB_{in} \\
&= \bar{A}B(\overline{B_{in}} + B_{in}) + \bar{A}\bar{B}B_{in} + ABB_{in} && \text{(Distributive Law)} \\
&= \bar{A}B(1) + B_{in}(\bar{A}\bar{B} + AB) && \text{(Complement Law \& Distributive Law)} \\
&= \bar{A}B + B_{in}(\overline{A \oplus B}) && \text{(XNOR Equivalence)}
\end{aligned}$$

3. Logic Circuit Implementation

The equations $D = A \oplus B \oplus B_{in}$ and $B_{out} = \bar{A}B + B_{in}(\overline{A \oplus B})$ indicate that a Full Subtractor can be elegantly constructed by cascading two Half-Subtractors and an OR gate.



BCD Adder-Subtractor

Q1. Design a BCD adder-subtractor which takes two 4-bit BCD numbers A and B. Addition and subtraction is selected by a selector bit S: when $S = 0$ the circuit produces $A + B$; when $S = 1$ it produces $A - B$. **(12 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Answer

1. Principle of Operation

A BCD (Binary Coded Decimal) adder-subtractor computes $A + B$ when the mode selector $S = 0$, and $A - B$ when $S = 1$. Both inputs A and B are 4-bit BCD digits. To perform subtraction, we utilize the **10's complement** arithmetic method:

$$\begin{aligned} A - B &= A + (\text{10's complement of } B) \\ &= A + (\text{9's complement of } B) + 1 \end{aligned}$$

We can achieve both operations seamlessly using a standard **BCD Adder** by feeding it A, a conditionally modified B (let's call it Y), and setting the initial carry-in $C_{in} = S$.

- **When $S = 0$ (Addition):** $Y = B$ and $C_{in} = 0$. The circuit computes $A + B + 0$.
- **When $S = 1$ (Subtraction):** $Y =$ 9's complement of B and $C_{in} = 1$. The circuit computes $A + (9 - B) + 1$, effectively performing $A - B$.

2. Conditional 9's Complementer Logic

We need a combinational logic circuit that takes $B_3B_2B_1B_0$ and selector S, and outputs $Y_3Y_2Y_1Y_0$.

- If $S = 0$, $Y_i = B_i$.
- If $S = 1$, $Y_i = (9 - B)_i$.

Let us analyze the relationships bit by bit for the $S = 1$ case (9's complement, $9 = 1001_2$):

Decimal	B_3	B_2	B_1	B_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1	0	0	1
1	0	0	0	1	1	0	0	0
2	0	0	1	0	0	1	1	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	0	1
5	0	1	0	1	0	1	0	0
6	0	1	1	0	0	0	1	1
7	0	1	1	1	0	0	1	0
8	1	0	0	0	0	0	0	1
9	1	0	0	1	0	0	0	0

By applying Boolean algebra, we derive the generalized equations for Y incorporating the mode S:

$$Y_0 = B_0 \oplus S \qquad \text{(Toggles for 9's complement)}$$
$$Y_1 = B_1 \qquad (Y_1 \text{ remains identical to } B_1 \text{ in valid BCD range})$$
$$Y_2 = B_2 \oplus (S \cdot B_1) \qquad \text{(Inverts } B_2 \text{ conditionally based on } B_1 \text{ and } S)$$
$$Y_3 = \bar{S}B_3 + S(\overline{B_3} \cdot \overline{B_2} \cdot \overline{B_1}) \qquad (Y_3 = 1 \text{ only for } B = 0, 1 \text{ during subtraction})$$

3. BCD Addition and Correction Logic

The core of the circuit is a standard BCD Adder. It consists of:

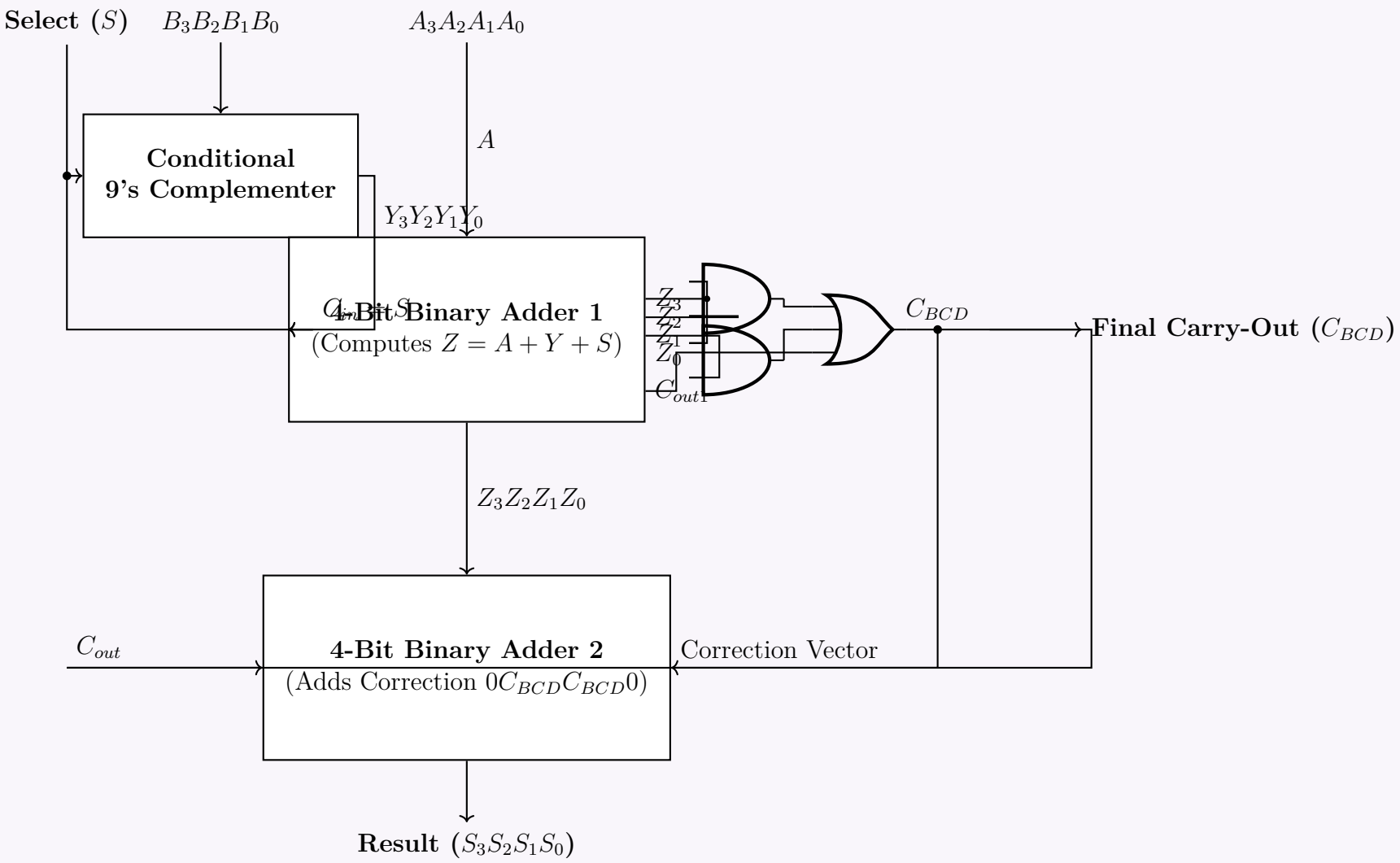
(a) **Binary Adder 1:** Adds the 4-bit numbers A, Y, and $C_{in} = S$. It produces a 4-bit binary sum $Z_3Z_2Z_1Z_0$ and a carry-out C_{out1}.

(b) **Correction Logic:** The binary sum may exceed 9 (1001_2), rendering it an invalid BCD digit. A correction factor of 6 (0110_2) must be added if the sum is > 9 or if a carry was generated. The condition for BCD correction C_{BCD} is:

$$C_{BCD} = C_{out1} + (Z_3 \cdot Z_2) + (Z_3 \cdot Z_1)$$

(c) **Binary Adder 2:** Adds the correction factor $0, C_{BCD}, C_{BCD}, 0$ to the intermediate sum Z. The final output is the valid BCD sum/difference $S_3S_2S_1S_0$.

4. Complete System Block Diagram



Q2. Design a BCD adder-subtractor for two BCD inputs A and B. Addition and subtraction is selected by a selector bit x: when $x = 0$ the circuit computes $A + B$; when $x = 1$ it computes $A - B$. Explain the operations $9 + 7$ and $9 - 7$ with your designed circuit. **(9+4 marks)** **[CSE 205 | L-2/T-1 | 2017–18]**

Answer

1. Design of the BCD Adder-Subtractor

A single-digit BCD Adder-Subtractor processes two 4-bit BCD inputs $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$, controlled by a mode selector x.

- **Addition ($x = 0$):** The circuit computes $A + B$. The carry-in is 0.
- **Subtraction ($x = 1$):** The circuit uses 10's complement arithmetic to compute $A - B = A + (9\text{'s complement of } B) + 1$. The $+1$ is elegantly achieved by tying the selector x directly to the initial carry-in C_{in}.

Conditional 9's Complementer: We need a combinational block that passes B unchanged when $x = 0$ and outputs the 9's complement of B $(9 - B)$ when $x = 1$. Let the output of this block be $Y = Y_3Y_2Y_1Y_0$. The logic equations derived from a truth table for 9's complement are:

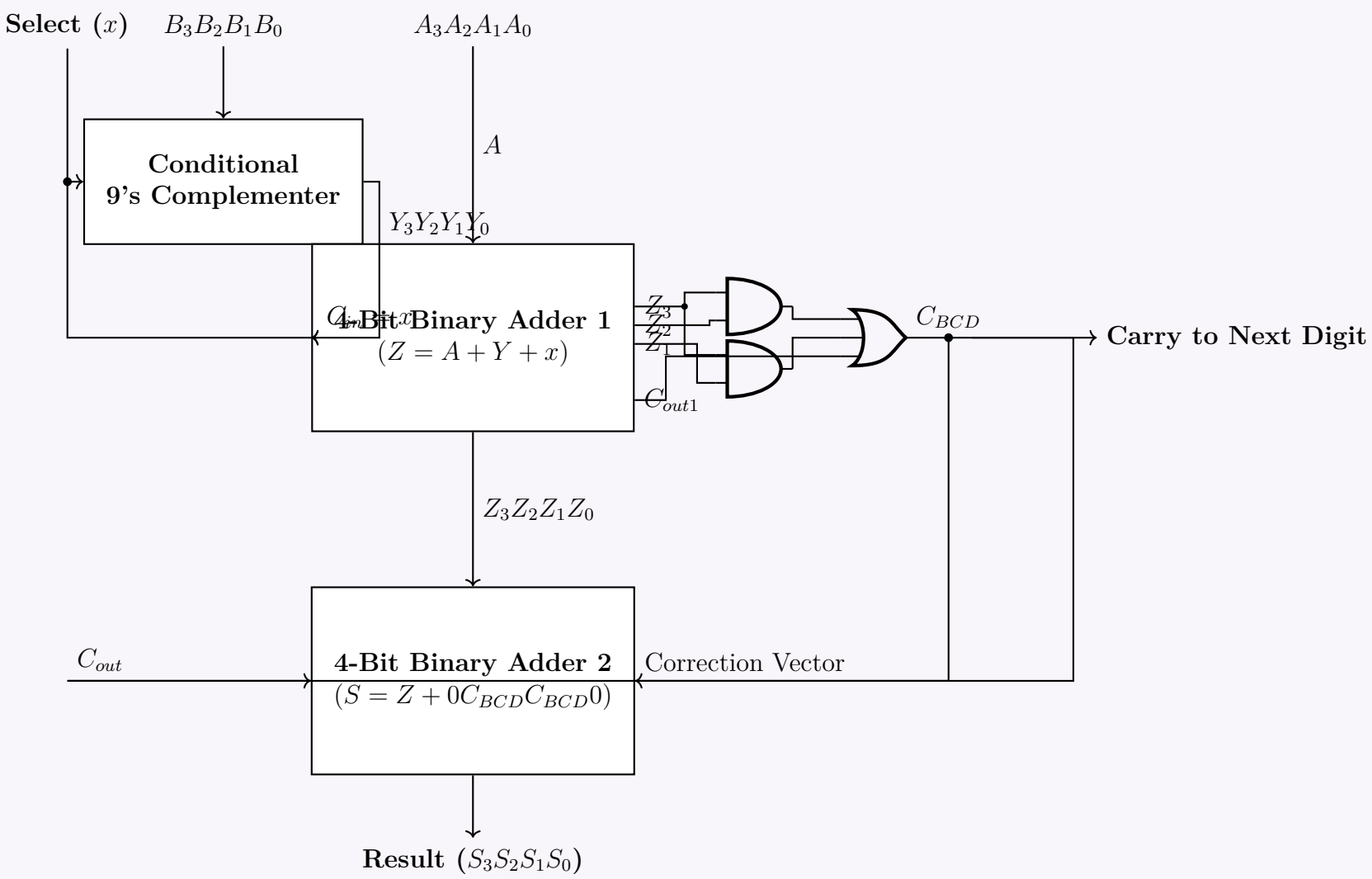
$$
\begin{aligned}
Y_0 &= B_0 \oplus x \\
Y_1 &= B_1 \quad \text{(The 2nd bit remains unchanged in 9's complement)} \\
Y_2 &= B_2 \oplus (B_1 \cdot x) \\
Y_3 &= B_3\bar{x} + x(\overline{B_3} \cdot \overline{B_2} \cdot \overline{B_1})
\end{aligned}
$$

BCD Adder Architecture: The modified input Y is added to A along with $C_{in} = x$ using a **4-bit Binary Adder**. This yields a binary sum Z and a carry C_{out1}. Because valid BCD ranges from 0 to 9, a result > 9 requires a correction. We add 6 (0110_2) to the sum if a correction condition C_{BCD} is met:

$$C_{BCD} = C_{out1} + Z_3Z_2 + Z_3Z_1$$

A second **4-bit Binary Adder** adds the correction vector $0, C_{BCD}, C_{BCD}, 0$ to Z to produce the final valid BCD result S.

2. Circuit Diagram



3. Explanation of Operations

Operation 1: Compute $9 + 7$ ($x = 0$)

- **Inputs:** $A = 1001_2$ (9), $B = 0111_2$ (7), $x = 0$.
- **Complementer Output:** Since $x = 0$, $Y = B = 0111_2$.
- **Adder 1:** Computes $A + Y + x \implies 1001_2 + 0111_2 + 0 = 10000_2$.
  - Intermediate sum $Z = 0000_2$.
  - Carry $C_{out1} = 1$.
- **Correction Logic:** $C_{BCD} = C_{out1} + Z_3Z_2 + Z_3Z_1 = 1 + 0 + 0 = 1$. A correction is required.
- **Adder 2:** Adds Z and the correction vector $0C_{BCD}C_{BCD}0$ (0110_2).
  - $S = 0000_2 + 0110_2 = 0110_2$ (6_{10}).
- **Final Result:** The carry to the next BCD digit is $C_{BCD} = 1$, and the sum is $S = 0110_2$ (6). Thus, the final BCD representation is 16, which is strictly correct $(9 + 7 = 16)$.

Operation 2: Compute $9 - 7$ ($x = 1$)

- **Inputs:** $A = 1001_2$ (9), $B = 0111_2$ (7), $x = 1$.
- **Complementer Output:** Since $x = 1$, the logic outputs the 9's complement of B. $Y = 9 - 7 = 2 \implies 0010_2$.
- **Adder 1:** Computes $A + Y + x$ (adding 1 implements the 10's complement).
  - $1001_2 + 0010_2 + 1 = 1100_2$.
  - Intermediate sum $Z = 1100_2$ (12_{10}).
  - Carry $C_{out1} = 0$.
- **Correction Logic:** Check if $Z > 9$.
  - $C_{BCD} = C_{out1} + Z_3Z_2 + Z_3Z_1 = 0 + (1 \cdot 1) + (1 \cdot 0) = 1$. A correction is required.
- **Adder 2:** Adds Z and the correction vector 0110_2.
  - $1100_2 + 0110_2 = 10010_2$.
  - The lower 4 bits are $S = 0010_2$ (2_{10}).
- **Final Result:** The sum $S = 0010_2$ gives the correct BCD difference of 2. The $C_{BCD} = 1$ acts as a no-borrow indicator (signifying a positive result in a 10's complement subtractor), which is discarded for single-digit subtraction. Result is correctly 2.

Q3. Design a circuit which computes $A - B$ and $A + B$ where A and B are 4-bit BCD values. Explain the design. **(15 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Theoretical Foundation

To design a unified circuit capable of computing both the sum $(A+B)$ and the difference $(A-B)$ of two 4-bit BCD (Binary Coded Decimal) numbers, we utilize a mode control bit M:

- **When $M = 0$ (Addition):** The circuit computes $A + B$.
- **When $M = 1$ (Subtraction):** The circuit computes $A - B$ using **10's complement arithmetic**. Subtraction is realized as $A +$ (9's complement of B) $+ 1$.

A fundamental property of BCD arithmetic is that standard binary addition can be used, provided we apply a **BCD correction**. If the result of a BCD digit addition exceeds 9 (i.e., $> 1001_2$) or generates a binary carry-out, it is an invalid BCD digit. Adding 6 (0110_2) corrects this invalid state. Remarkably, this exact same correction logic applies to 10's complement BCD subtraction.

2. Logic Design of Components

A. Conditional 9's Complementer

We require a combinational circuit that takes the BCD digit B and the mode bit M, outputting Y:

$$\begin{aligned} &\text{If } M = 0 \implies Y = B \\ &\text{If } M = 1 \implies Y = 9 - B \quad \text{(9's complement)} \end{aligned}$$

By analyzing the truth table for a 9's complementer ($9 = 1001_2$), we derive the boolean expressions for $Y_3Y_2Y_1Y_0$:

$$\begin{aligned} Y_0 &= B_0 \oplus M && \text{(Bit 0 toggles when subtracting)} \\ Y_1 &= B_1 && \text{(Bit 1 is unchanged for valid BCD inputs)} \\ Y_2 &= B_2 \oplus (B_1 \cdot M) && \text{(Bit 2 toggles if Bit 1 is high during subtraction)} \\ Y_3 &= B_3\overline{M} + M(\overline{B_3} \cdot \overline{B_2} \cdot \overline{B_1}) && \text{(Bit 3 logic for 9's complement)} \end{aligned}$$

B. Binary Addition and Correction Logic

The inputs A, Y, and M (acting as the $+1$ for 10's complement when $M = 1$) are fed into a standard **4-bit Binary Adder**.

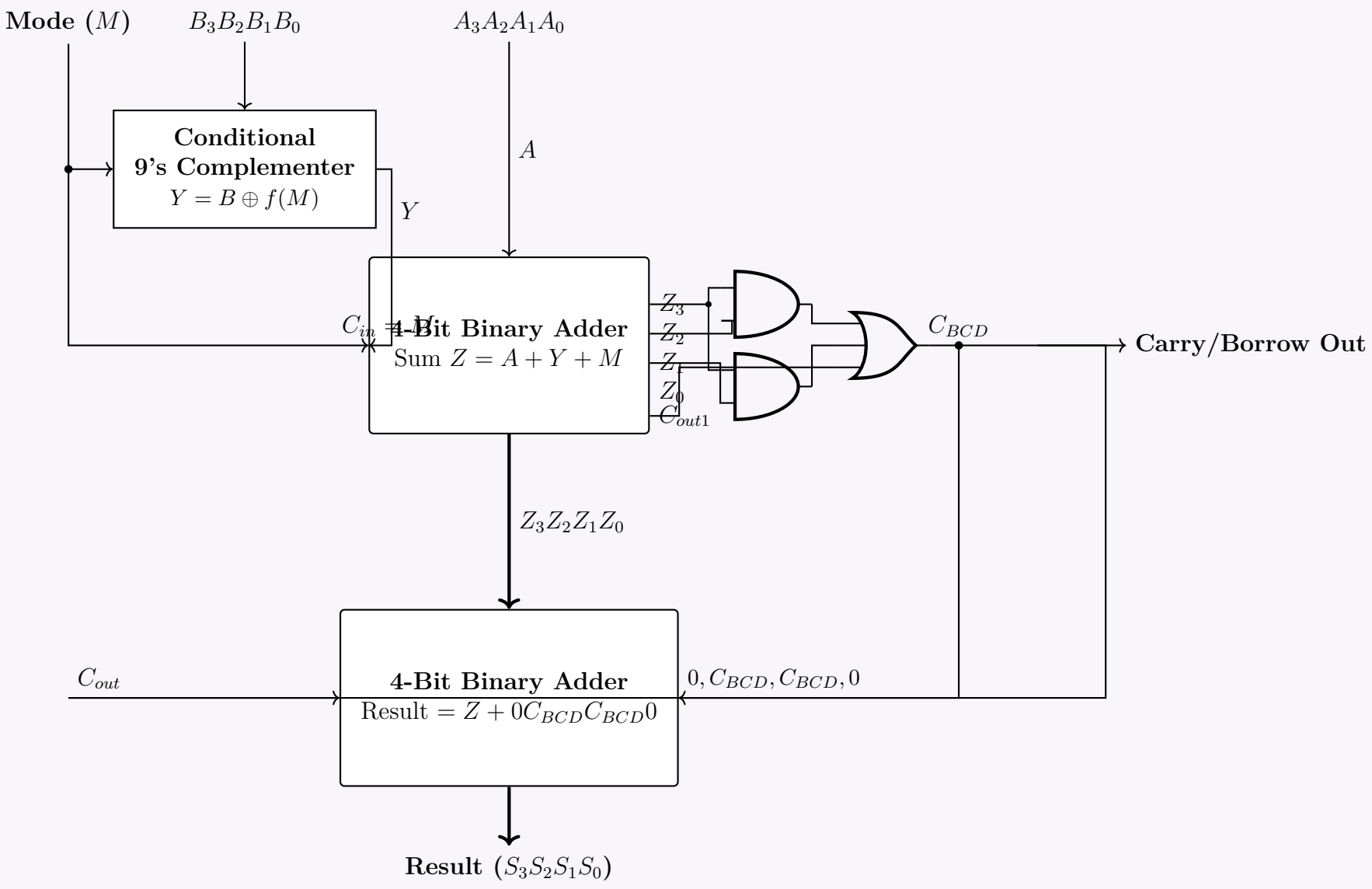
$$Z = A + Y + M \quad \text{with carry-out } C_{out1}$$

The correction condition C_{BCD} evaluates whether the binary sum $Z = Z_3Z_2Z_1Z_0$ requires the $+6$ adjustment:

$$C_{BCD} = C_{out1} + (Z_3 \cdot Z_2) + (Z_3 \cdot Z_1)$$

If $C_{BCD} = 1$, a second **4-bit Binary Adder** adds 0110_2 to Z. If $C_{BCD} = 0$, it adds 0000_2. The output of this second adder is the valid BCD result $S_3S_2S_1S_0$.

3. BCD Adder/Subtractor Circuit Diagram



Carry Look-Ahead Adder (CLA)

Q1. Explain the working principle of a Carry Look-Ahead (CLA) adder. Derive the expressions for carry generation (C_1, C_2, C_3, C_4) using generate and propagate functions. **(12 marks)** **[CSE 205 | L-2/T-1 | 2024–25]**

Answer

Working Principle of Carry Look-Ahead (CLA) Adder

In a standard Ripple Carry Adder (RCA), the carry bit from each full adder stage must propagate serially to the next stage before the subsequent addition can complete. This serial propagation introduces a significant time delay proportional to the number of bits in the adders.
The Carry Look-Ahead (CLA) adder overcomes this limitation by calculating the carry signals in advance, simultaneously for all stages, rather than waiting for them to ripple through. It achieves this by examining the inputs to determine whether a particular bit position will *generate* a carry on its own or *propagate* an incoming carry to the next stage. By decoupling the carry computation from the intermediate sum stages using high-speed two-level logic, the total addition time is vastly reduced.

Generate and Propagate Functions

For the i-th stage of the adder with individual inputs A_i and B_i, we define two Boolean functions:

- **Carry Generate (G_i):** A carry is unconditionally generated out of the i-th stage if both inputs A_i and B_i are logic 1, regardless of the incoming carry.

$$G_i = A_i \cdot B_i$$

- **Carry Propagate (P_i):** An incoming carry C_i is propagated to the next stage if either A_i or B_i is logic 1 (exclusive-OR is commonly used to simultaneously satisfy the standard sum definition).

$$P_i = A_i \oplus B_i$$

Using these definitions, the sum (S_i) and the outgoing carry (C_{i+1}) for the i-th stage can be elegantly expressed as:

$$S_i = P_i \oplus C_i$$
$$C_{i+1} = G_i + (P_i \cdot C_i)$$

Derivation of Carry Expressions (C_1, C_2, C_3, C_4)

To eliminate the serial dependency, we recursively expand the general carry equation $C_{i+1} = G_i + P_i \cdot C_i$. This allows us to express all future carries strictly in terms of the parallel generation functions (G_i), propagation functions (P_i), and the initial system carry-in (C_0).

$$\mathbf{C_1} = G_0 + P_0 \cdot C_0$$

$$\begin{aligned}
\mathbf{C_2} &= G_1 + P_1 \cdot C_1 \\
&= G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0) && \text{(Substitute expression for } C_1\text{)} \\
&= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0 && \text{(Distributive Law)}
\end{aligned}$$

$$\begin{aligned}
\mathbf{C_3} &= G_2 + P_2 \cdot C_2 \\
&= G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0) && \text{(Substitute expression for } C_2\text{)} \\
&= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0 && \text{(Distributive Law)}
\end{aligned}$$

$$\begin{aligned}
\mathbf{C_4} &= G_3 + P_3 \cdot C_3 \\
&= G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0) && \text{(Substitute expression for } C_3\text{)} \\
&= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0 && \text{(Distributive Law)}
\end{aligned}$$

Conclusion

The expanded expressions clearly demonstrate that C_1, C_2, C_3, and C_4 do not depend on the resolution of immediately preceding carry outputs (for instance, C_4 does not have to wait for C_3 to be physically calculated by the previous adder block). Instead, they depend entirely on the initial input carry C_0 and the external inputs (represented by P_i and G_i).
Because P_i and G_i are generated in parallel with a single gate delay, and the expanded carry expressions form a Sum-of-Products (SOP), the entire Look-Ahead Carry block can be implemented directly using a two-level AND-OR logic circuit. As a result, all intermediate carries are available simultaneously after just a few gate delays, drastically accelerating binary arithmetic operations in modern microprocessors.

Q2. What is the benefit of using an adder with carry lookahead logic? Derive expressions of different carry of a 4-bit adder with carry lookahead logic. **(15 marks)** **[CSE 205 | L-2/T-1 | 2023–24]**

Answer

1. Benefit of Using an Adder with Carry Lookahead Logic

In a conventional n-bit Ripple Carry Adder (RCA), the computation of the final sum requires the carry bit to propagate sequentially through every full-adder stage. This creates a linear time delay of $\mathcal{O}(n)$, which significantly slows down arithmetic operations in processing units.
The primary **benefit of a Carry Lookahead Adder (CLA)** is its ability to vastly reduce this computation time by calculating the carry-in signals for all stages simultaneously. Instead of waiting for the carry to "ripple" from the Least Significant Bit (LSB) to the Most Significant Bit (MSB), the CLA logic utilizes the input bits to dynamically predict whether a carry will be generated or propagated. This parallel evaluation using two-level logic (AND-OR) reduces the carry propagation delay to a constant theoretical time of $\mathcal{O}(1)$ (typically 2 to 3 gate delays), independent of the number of bits within the lookahead block, thereby maximizing the arithmetic processing speed.

2. Derivation of Carry Expressions for a 4-Bit CLA

To compute the carries independently of the sequential ripple, we first define two intermediate signals for the i-th bit stage based on inputs A_i and B_i:

- **Carry Generate (G_i):** Produces a carry regardless of the incoming carry.

$$G_i = A_i \cdot B_i$$

- **Carry Propagate (P_i):** Propagates an incoming carry to the next stage.

$$P_i = A_i \oplus B_i$$

The general Boolean equation for the outgoing carry of the i-th stage is:

$$C_{i+1} = G_i + (P_i \cdot C_i) \tag{1}$$

For a 4-bit adder, the bit stages are $i = 0, 1, 2, 3$. We iteratively substitute the previous stage's carry into equation (1) to express each carry purely in terms of the initial carry-in (C_0) and the parallel G_i and P_i signals.

Carry 1 (C_1): For $i = 0$

$$\mathbf{C_1 = G_0 + P_0 \cdot C_0}$$

Carry 2 (C_2): For $i = 1$

$$\begin{aligned} C_2 &= G_1 + P_1 \cdot C_1 \\ &= G_1 + P_1 \cdot (G_0 + P_0 \cdot C_0) && \text{(Substitute } C_1\text{)} \\ \mathbf{C_2} &\mathbf{= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0} && \text{(Distributive Law)} \end{aligned}$$

Carry 3 (C_3): For $i = 2$

$$\begin{aligned} C_3 &= G_2 + P_2 \cdot C_2 \\ &= G_2 + P_2 \cdot (G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0) && \text{(Substitute } C_2\text{)} \\ \mathbf{C_3} &\mathbf{= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0} && \text{(Distributive Law)} \end{aligned}$$

Carry 4 (C_4): For $i = 3$

$$\begin{aligned} C_4 &= G_3 + P_3 \cdot C_3 \\ &= G_3 + P_3 \cdot (G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0) && \text{(Substitute } C_3\text{)} \\ \mathbf{C_4} &\mathbf{= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0} && \text{(Distributive Law)} \end{aligned}$$

Summary of Results: The derivations yield the final expressions mathematically structured as Sum-of-Products (SOP). None of the carry signals (C_1, C_2, C_3, C_4) depend sequentially on the preceding full-adder outputs; they only require the base input signals (G_i, P_i, and C_0), allowing them to be synthesized simultaneously via rapid two-level logic hardware.

Q3. Design a 4 bit Carry Look Ahead (CLA) adder which will take $X_3X_2X_1X_0$ and $Y_3Y_2Y_1Y_0$ as input, and produce their sum. Show the steps and the role of different *carries* of this CLA. **(13 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Answer

1. Design Overview of a 4-bit Carry Look-Ahead (CLA) Adder

A standard ripple-carry adder suffers from propagation delay because each stage must wait for the carry from the previous stage. A Carry Look-Ahead (CLA) adder eliminates this sequential dependency by calculating all carry bits simultaneously. The 4-bit CLA takes inputs $X_3X_2X_1X_0$ and $Y_3Y_2Y_1Y_0$ along with an initial carry C_0, and produces the sum $S_3S_2S_1S_0$ and the final carry-out C_4. The design follows three distinct stages:

(a) **Generate and Propagate Logic:** Computes the auxiliary signals for each bit.

(b) **Carry Look-Ahead Logic:** Computes the intermediate carries (C_1, C_2, C_3, C_4) in parallel.

(c) **Sum Logic:** Computes the final sum bits (S_0, S_1, S_2, S_3).

2. Step-by-Step Derivation and Logic Design

Step A: Propagate (P_i) and Generate (G_i) Functions
For each bit position $i \in \{0, 1, 2, 3\}$, we define two signals:

$$G_i = X_i \cdot Y_i \qquad \text{(Carry Generate: produces a carry if both inputs are 1)}$$
$$P_i = X_i \oplus Y_i \qquad \text{(Carry Propagate: passes an incoming carry to the next stage)}$$

Step B: The Role and Derivation of Different Carries
The core equation for any carry is $C_{i+1} = G_i + P_i \cdot C_i$. The fundamental *role* of the different carries (C_1, C_2, C_3, C_4) in a CLA is to pre-compute the carry status for all stages independently of the intermediate sums, functioning strictly on the basis of P_i, G_i, and the initial C_0. By expanding the core equation, we express every carry solely as a two-level Sum-of-Products (SOP), reducing the delay to just $\mathcal{O}(1)$ gate levels.
The mathematical derivations are as follows:

$$\mathbf{C_1 = G_0 + P_0 \cdot C_0}$$

$$C_2 = G_1 + P_1 \cdot C_1$$
$$\mathbf{C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0} \qquad \text{(Substitute } C_1 \text{ \& Apply Distributive Law)}$$

$$C_3 = G_2 + P_2 \cdot C_2$$
$$\mathbf{C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0} \qquad \text{(Substitute } C_2 \text{ \& Apply Distributive Law)}$$

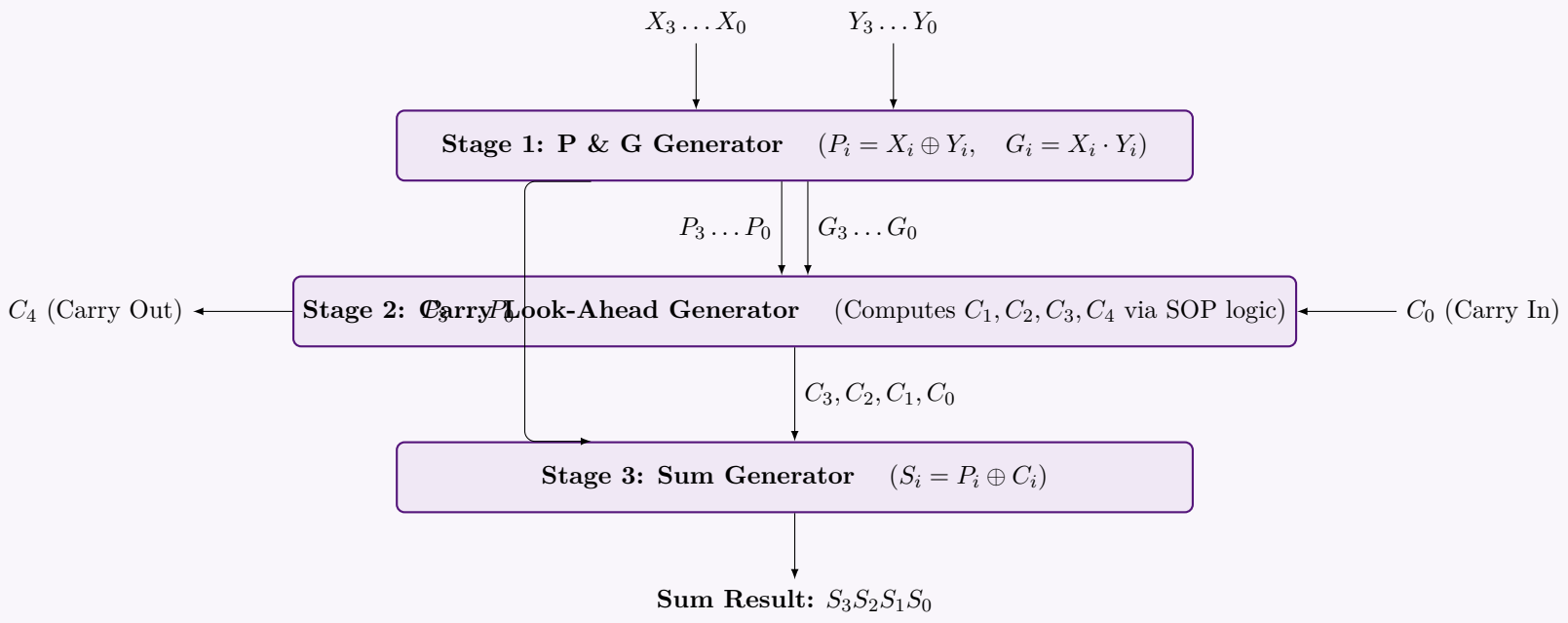
$$C_4 = G_3 + P_3 \cdot C_3$$
$$\mathbf{C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0} \qquad \text{(Substitute } C_3 \text{ \& Distribute)}$$

Step C: Sum Generation
Once the carries are evaluated in parallel, the final sum is calculated as:

$$S_i = P_i \oplus C_i \quad \text{for } i = 0, 1, 2, 3$$

3. Structural Block Diagram of the 4-Bit CLA Adder

To implement this design reliably, the hardware is structured into three parallel modules based on the equations above.



Conclusion on the Role of Carries: In this architecture, the C_1, C_2, C_3, and C_4 logic gates (comprising Stage 2) are completely decoupled from the S_i calculations of preceding bits. They serve the critical role of flattening the serial propagation path. By utilizing direct inputs and pre-computed P_i and G_i values, they feed the correct carry bit into Stage 3 almost instantaneously, empowering the adder to process the 4-bit numbers at a fraction of the time required by standard adders.

Q4. Design a 3-bit CLA circuit and show all types of "carry" for the addition $3 + 4$ with your designed CLA. **(8+4 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

A Carry Lookahead Adder (CLA) significantly reduces carry propagation delay by computing carries in parallel using early generated and propagated carry signals.

1. Mathematical Derivation for a 3-bit CLA

For any bit position i, the logic evaluates two intermediate signals:

- **Propagate Carry** ($\mathbf{P_i}$): Indicates if an incoming carry will be passed to the next bit. $P_i = A_i \oplus B_i$
- **Generate Carry** ($\mathbf{G_i}$): Indicates if a carry is directly created at this bit regardless of the incoming carry. $G_i = A_i \cdot B_i$

The sum and lookahead carry equations are given by:

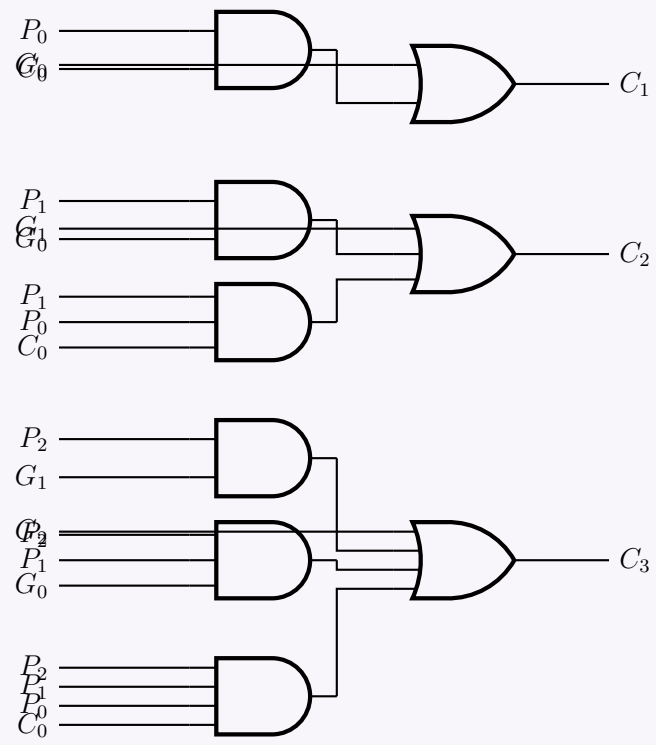
$$S_i = P_i \oplus C_i$$
$$C_{i+1} = G_i + P_i \cdot C_i$$

Unrolling the carry equations for a 3-bit adder yields the **Carry Lookahead logic**:

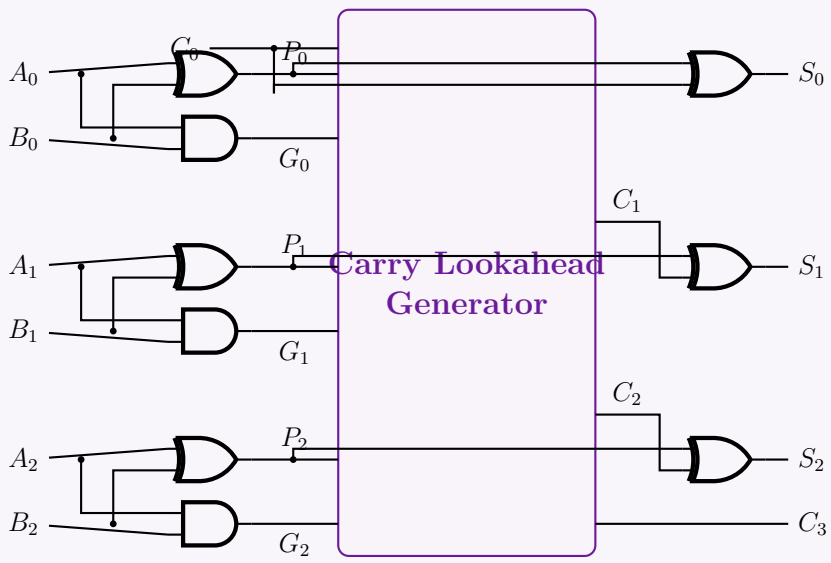
$$\begin{aligned}
\textbf{Carry 1:} \quad & C_1 = G_0 + P_0C_0 \\
\textbf{Carry 2:} \quad & C_2 = G_1 + P_1C_1 \\
& = G_1 + P_1(G_0 + P_0C_0) \\
& = G_1 + P_1G_0 + P_1P_0C_0 \\
\textbf{Carry 3:} \quad & C_3 = G_2 + P_2C_2 \\
& = G_2 + P_2(G_1 + P_1G_0 + P_1P_0C_0) \\
& = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0
\end{aligned}$$

2. 3-Bit CLA Circuit Design

Below is the gate-level implementation of the **Carry Lookahead Generator** that computes C_1, C_2, and C_3 in parallel.



The fully integrated structural top-level design incorporates the initial PG Generators, the CLA unit (designed above), and the final Sum XOR gates:



3. Application: Adding $3 + 4$

We are performing the addition $A + B$, where:

- $A = 3_{10} = 011_2 \implies A_2 = 0, A_1 = 1, A_0 = 1$
- $B = 4_{10} = 100_2 \implies B_2 = 1, B_1 = 0, B_0 = 0$
- Assume an initial carry input $C_0 = 0$.

Step 1: Calculate Generate (G_i) and Propagate (P_i) Carries

$$\begin{aligned} \text{Bit 0:} \quad & P_0 = A_0 \oplus B_0 = 1 \oplus 0 = \mathbf{1} & \quad & G_0 = A_0 \cdot B_0 = 1 \cdot 0 = \mathbf{0} \\ \text{Bit 1:} \quad & P_1 = A_1 \oplus B_1 = 1 \oplus 0 = \mathbf{1} & \quad & G_1 = A_1 \cdot B_1 = 1 \cdot 0 = \mathbf{0} \\ \text{Bit 2:} \quad & P_2 = A_2 \oplus B_2 = 0 \oplus 1 = \mathbf{1} & \quad & G_2 = A_2 \cdot B_2 = 0 \cdot 1 = \mathbf{0} \end{aligned}$$

Step 2: Calculate Internal Lookahead Carries (C_i)

$$\begin{aligned} C_1 &= G_0 + P_0 C_0 = 0 + (1 \cdot 0) = \mathbf{0} \\ C_2 &= G_1 + P_1 C_1 = 0 + (1 \cdot 0) = \mathbf{0} \\ C_3 &= G_2 + P_2 C_2 = 0 + (1 \cdot 0) = \mathbf{0} \end{aligned}$$

Note: Because bits of A and B do not simultaneously contain '1' in any position, $G_i = 0$ across all bits. As the starting carry $C_0 = 0$, there are no incoming carries to propagate either. Thus, all internal lookahead carries evaluate to zero.

Step 3: Calculate Final Sum (S_i)

$$\begin{aligned} S_0 &= P_0 \oplus C_0 = 1 \oplus 0 = \mathbf{1} \\ S_1 &= P_1 \oplus C_1 = 1 \oplus 0 = \mathbf{1} \\ S_2 &= P_2 \oplus C_2 = 1 \oplus 0 = \mathbf{1} \end{aligned}$$

Summary of all signal types (Showing all "carries"):

Bit (i)	A_i	B_i	**Generate (G_i)**	**Propagate (P_i)**	**Carry In (C_i)**	**Sum (S_i)**
0	1	0	0	1	$C_0 = 0$	1
1	1	0	0	1	$C_1 = 0$	1
2	0	1	0	1	$C_2 = 0$	1
					Final Carry Out (C_3)	**0**

Final Result: $C_3 S_2 S_1 S_0 = 0111_2 = \mathbf{7_{10}}$.

Q5. For a carry look-ahead adder circuit, what advantage do we get if we use it instead of a 4-bit simple full adder? For a NOT gate with 2 ns propagation delay and other basic gates with 4 ns propagation delay, what will be the total propagation delay? **(5 marks)** **[CSE 205 | L-2/T-1 | 2016–17]**

Answer

Part 1: Advantage of a Carry Look-Ahead Adder (CLA)

In a simple 4-bit full adder (often called a **Ripple Carry Adder** or RCA), the carry bit ripples from the least significant bit (LSB) to the most significant bit (MSB). Consequently, the sum for bit i cannot be computed until the carry-out from bit $i - 1$ is ready. This creates a linear propagation delay, $O(n)$, which becomes unacceptably slow for larger bit-widths.

The primary advantage of the **Carry Look-Ahead Adder (CLA)** is that it completely eliminates this ripple effect. By calculating intermediary **Generate (G_i)** and **Propagate (P_i)** signals, the CLA computes all carry signals (C_1, C_2, C_3, C_4) **in parallel** using 2-level Sum-of-Products (SOP) logic.

- **Speed:** The carry generation delay becomes constant (theoretical $O(1)$ gate levels), drastically reducing the overall addition time.
- **Independence:** Each carry bit is calculated directly from the inputs (A, B) and the initial carry-in (C_0), without waiting for the intermediate sum/carry of preceding bits.

Part 2: Total Propagation Delay Calculation

The problem explicitly provides the delay of a NOT gate (2 ns) and other basic gates (4 ns). Standard "basic gates" include AND and OR gates. Because the NOT gate delay is distinctly specified, we must decompose the XOR gate (which is a derived gate) into its constituent basic gates to accurately trace the delay.

1. Delay of an XOR Gate The XOR operation $X \oplus Y = X\bar{Y} + \bar{X}Y$ is constructed using NOT, AND, and OR gates:

$$\begin{aligned} \text{NOT gates } (\bar{X}, \bar{Y} \text{ delay}): &\quad 2 \text{ ns} \\ \text{AND layer } (X\bar{Y}, \bar{X}Y \text{ delay}): &\quad 2 \text{ ns} + 4 \text{ ns} = 6 \text{ ns} \\ \text{OR layer (Final XOR delay)}: &\quad 6 \text{ ns} + 4 \text{ ns} = \mathbf{10} \text{ ns} \end{aligned}$$

Thus, any XOR operation introduces a 10 ns delay from the time its latest input is available.

2. Stage-by-Stage Delay Trace for the CLA Assume all primary inputs (A_i, B_i, C_0) arrive simultaneously at $t = 0$ ns.
Stage 1: Generate (G_i) and Propagate (P_i) Signals
The signals are computed in parallel for all bits:

$$\begin{aligned} G_i = A_i \cdot B_i &\quad (1 \text{ AND gate}) \implies \text{Valid at } t = 0 + 4 = \mathbf{4} \text{ ns} \\ P_i = A_i \oplus B_i &\quad (1 \text{ XOR logic}) \implies \text{Valid at } t = 0 + 10 = \mathbf{10} \text{ ns} \end{aligned}$$

Stage 2: Carry Look-Ahead Logic (C_i)
The carry bits are evaluated using 2-level AND-OR logic. For example, C_4:

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

- The inputs to this block are the G_i signals (arriving at 4 ns) and P_i signals (arriving at 10 ns).
- **AND Layer:** Must wait for the slowest input, which is P_i at 10 ns.
  Outputs of the AND gates are valid at $t = 10 \text{ ns} + 4 \text{ ns} = \mathbf{14}$ ns.
- **OR Layer:** Takes the outputs of the AND layer.
  Outputs of the OR gates (all C_i) are valid at $t = 14 \text{ ns} + 4 \text{ ns} = \mathbf{18}$ ns.

Stage 3: Final Sum Generation (S_i)
The sum is computed as $S_i = P_i \oplus C_i = P_i\bar{C}_i + \bar{P}_iC_i$.
We track the arrival times of the inputs to this final XOR logic:

- P_i is available at 10 ns.
- C_i is available at 18 ns.
- **NOT Layer:** Computes $\bar{P}_i$ and $\bar{C}_i$.
  - $\bar{P}_i$ is valid at $10 \text{ ns} + 2 \text{ ns} = 12$ ns.
  - $\bar{C}_i$ is valid at $18 \text{ ns} + 2 \text{ ns} = \mathbf{20}$ ns.
- **AND Layer ($P_i\bar{C}_i$ and $\bar{P}_iC_i$):** The slowest input is $\bar{C}_i$ at 20 ns.
  The AND gates evaluate at $t = 20 \text{ ns} + 4 \text{ ns} = \mathbf{24}$ ns.
- **OR Layer (Final Sum):**
  The final OR gate evaluates at $t = 24 \text{ ns} + 4 \text{ ns} = \mathbf{28 \text{ ns}}$.

Conclusion: The total propagation delay to compute the final sum and carry-out in this Carry Look-Ahead Adder is **28 ns**.

Note: If a simpler architecture is assumed where the XOR gate is treated intrinsically as a standard basic gate with a 4 ns delay (ignoring the provided NOT gate delay parameter), the calculation simplifies to: P_i ready at 4 ns → Carry ANDs at 8 ns → Carry ORs at 12 ns → Final Sum XOR at 16 ns. However, utilizing the explicitly provided NOT delay requires the standard decomposition shown above.

Q6. What is the problem with the binary parallel adder circuit? Design and implement a 4-bit carry look-ahead adder circuit. If the propagation delay is 1 ns for a NOT gate and 2 ns for other basic gates, what is the propagation delay of the CLA adder to produce its final output? **(18 marks)** **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

1. The Problem with the Binary Parallel Adder

A standard Binary Parallel Adder (also known as a **Ripple Carry Adder** or RCA) cascades multiple 1-bit full adders to add n-bit numbers. The primary problem with this architecture is the **carry propagation delay**. In an RCA, the sum S_i and the carry-out C_{i+1} for any bit position i cannot be correctly computed until the carry-in C_i from the previous stage has been evaluated. Consequently, the carry must "ripple" sequentially from the least significant bit (LSB) to the most significant bit (MSB). For an n-bit adder, the worst-case propagation delay is $O(n)$, which acts as a major bottleneck in high-speed computing systems.

2. Design and Implementation of a 4-Bit Carry Look-Ahead Adder

A Carry Look-Ahead Adder (CLA) solves the ripple delay by computing all carry signals simultaneously using two intermediate signals for each bit:

- **Generate ($G_i = A_i \cdot B_i$):** Produces a carry out regardless of the carry in.
- **Propagate ($P_i = A_i \oplus B_i$):** Propagates a carry in to the carry out.

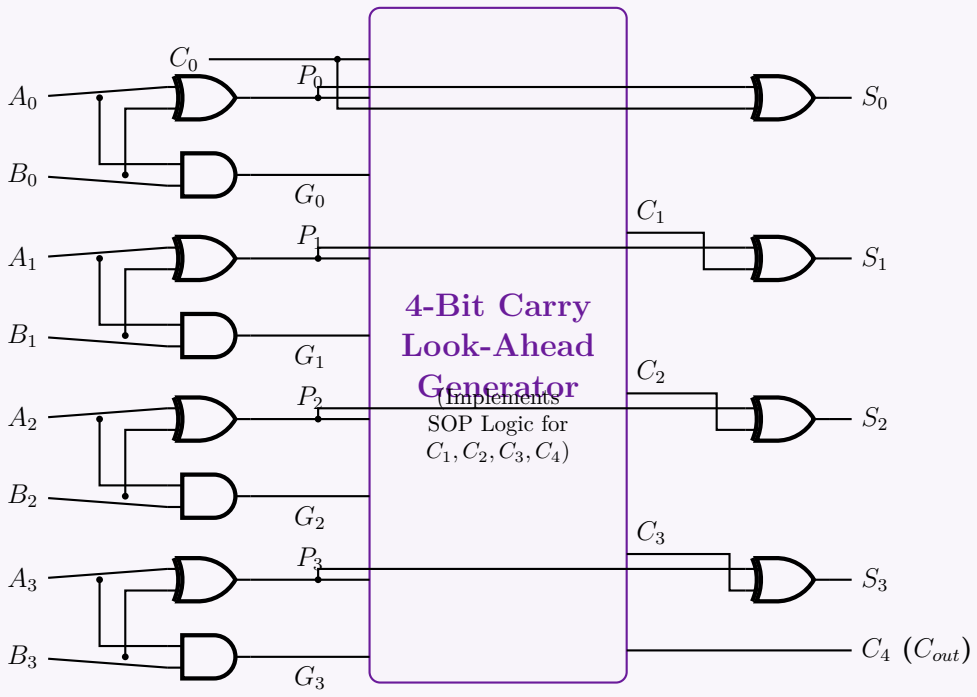
The standard sum and carry formulas are:

$$S_i = P_i \oplus C_i$$
$$C_{i+1} = G_i + P_i \cdot C_i$$

By unrolling the carry equations, we derive independent carry logic for the 4-bit CLA:

$$C_1 = G_0 + P_0C_0$$
$$C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0$$
$$C_3 = G_2 + P_2C_2 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$
$$C_4 = G_3 + P_3C_3 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

Structural Block Diagram of the 4-Bit CLA:



3. Propagation Delay Calculation

We are given the delay of a **NOT gate (1 ns)** and **other basic gates (2 ns)** (i.e., AND/OR gates). Because the NOT gate delay is specifically provided, we must decompose the complex XOR gates into their basic gate equivalents: $X \oplus Y = X\bar{Y} + \bar{X}Y$.

$$\begin{aligned} \text{NOT layer } (\bar{X}, \bar{Y}): &\quad 1 \text{ ns} \\ \text{AND layer } (X\bar{Y}, \bar{X}Y): &\quad 1 \text{ ns} + 2 \text{ ns} = 3 \text{ ns} \\ \text{OR layer (Final XOR delay)}: &\quad 3 \text{ ns} + 2 \text{ ns} = \mathbf{5} \text{ ns} \end{aligned}$$

Thus, an XOR operation yields a delay of 5 ns. Now, assuming all inputs (A_i, B_i, C_0) arrive at $t = 0$ ns, we trace the CLA critical path:

Stage 1: Generate (G_i) and Propagate (P_i) Signals

- $G_i = A_i \cdot B_i$ uses 1 AND gate $\implies$ Ready at $t = 0 + 2 = \mathbf{2}$ ns.
- $P_i = A_i \oplus B_i$ uses 1 XOR gate $\implies$ Ready at $t = 0 + 5 = \mathbf{5}$ ns.

Stage 2: Carry Look-Ahead Logic (C_1, C_2, C_3, C_4) The logic evaluates 2-level Sum-of-Products (SOP) equations (AND gates followed by OR gates).

- **AND Layer:** Takes inputs G_i (2 ns), P_i (5 ns), and C_0 (0 ns). It must wait for the slowest input (P_i at 5 ns).
  Output of AND layer $\implies$ 5 ns + 2 ns = **7** ns.
- **OR Layer:** Takes the output of the AND layer.
  Output of OR layer (all C_i and C_4) $\implies$ 7 ns + 2 ns = **9** ns.

*(Note: The final carry out C_4 is completely evaluated and valid at **9 ns**.)*

Stage 3: Final Sum Generation (S_i) The final sum is $S_i = P_i \oplus C_i = P_i\bar{C}_i + \bar{P}_iC_i$.

- **Inputs Available:** P_i is ready at 5 ns, and C_i is ready at 9 ns.
- **NOT Layer:** $\bar{P}_i$ is ready at $5 + 1 = 6$ ns, and $\bar{C}_i$ is ready at $9 + 1 = \mathbf{10}$ ns.
- **AND Layer:** Must wait for the slowest input ($\bar{C}_i$ at 10 ns).
  Evaluation finishes at $t = 10$ ns + 2 ns = **12** ns.
- **OR Layer:** Generates the final Sum.
  Evaluation finishes at $t = 12$ ns + 2 ns = **14 ns**.

Conclusion: The propagation delay for the CLA adder to produce its final output (the final Sum bit) is **14 ns**.

Q7. What are the advantages of a CLA over a normal binary four-bit adder? Assume that the exclusive-OR gate has a propagation delay of 10 ns and that the AND or OR gates have a propagation delay of 5 ns. What is the total propagation delay time in the four-bit CLA adder? **(10 marks)** **[CSE 205 | L-2/T-1 | 2013–14]**

Answer

1. Advantages of a CLA over a Normal Binary Four-Bit Adder

A "normal" binary four-bit adder, typically implemented as a **Ripple Carry Adder (RCA)**, cascades single-bit full adders in a series. The primary advantages of replacing this with a **Carry Look-Ahead Adder (CLA)** are:

- **Elimination of the Ripple Effect:** In an RCA, the carry propagates sequentially from the LSB to the MSB, resulting in a linear delay $O(n)$. A CLA precomputes the carry signals in parallel using Generate (G_i) and Propagate (P_i) logic, ensuring that the MSB does not have to wait for the carry to ripple through all preceding bits.
- **Constant Carry Delay:** Regardless of the bit-width of the adder block (up to a fan-in limit), the carry outputs (C_1, C_2, C_3, C_4) are generated simultaneously via a 2-level Sum-of-Products (SOP) circuit. This significantly boosts the clock speed capability of the CPU's Arithmetic Logic Unit (ALU).
- **Faster Overall Addition:** Because the carries arrive much earlier, the final Sum ($S_i = P_i \oplus C_i$) bits are also evaluated much faster compared to a standard RCA.

---

2. Total Propagation Delay Calculation

We are given the following inherent gate delays:

- **XOR gate delay:** 10 ns
- **AND gate delay:** 5 ns
- **OR gate delay:** 5 ns

Assume all primary inputs (A_i, B_i, and initial carry C_0) are applied simultaneously at $t = 0$ ns. The calculation proceeds stage by stage based on the critical path of the CLA architecture.

Stage 1: Generate (G_i) and Propagate (P_i) Signals For each bit i, these signals are calculated directly from the inputs in parallel:

$$G_i = A_i \cdot B_i \qquad \text{(1 AND gate)} \implies \text{Valid at } t = 0 + 5 = \mathbf{5} \text{ ns}$$
$$P_i = A_i \oplus B_i \qquad \text{(1 XOR gate)} \implies \text{Valid at } t = 0 + 10 = \mathbf{10} \text{ ns}$$

Stage 2: Carry Look-Ahead Logic (C_1, C_2, C_3, C_4) The look-ahead logic utilizes a 2-level AND-OR structure (Sum-of-Products) for all carries. For example, the longest expression is:

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

- **AND Layer:** The inputs to the AND gates are G_i (arriving at 5 ns), P_i (arriving at 10 ns), and C_0 (0 ns). The AND gates must wait for the slowest input, which is P_i at 10 ns.
  Evaluating the AND gates adds 5 ns: $t = 10 \text{ ns} + 5 \text{ ns} = \mathbf{15}$ ns.
- **OR Layer:** The OR gates sum the outputs of the AND layer (which arrive at 15 ns) and the standalone G_i signals (which arrive early at 5 ns). It waits for the slowest input at 15 ns.
  Evaluating the OR gates adds 5 ns: $t = 15 \text{ ns} + 5 \text{ ns} = \mathbf{20}$ ns.

All internal carry bits (C_1, C_2, C_3) and the final carry-out (C_4) are valid at 20 ns.

Stage 3: Final Sum Generation (S_i) The final sum for each bit is calculated using the equation:

$$S_i = P_i \oplus C_i$$

- **Inputs:** The XOR gate receives P_i (ready at 10 ns) and C_i (ready at 20 ns).
- **Evaluation:** The XOR gate must wait for the slowest input, which is C_i at 20 ns.
  Evaluating the XOR gate adds 10 ns: $t = 20 \text{ ns} + 10 \text{ ns} = \mathbf{30 \text{ ns}}$.

Conclusion:
The total propagation delay time for the four-bit CLA adder to stabilize all outputs (including the Sum bits and the final Carry-out) is **30 ns**.

2's Complement Circuit

Q1. Using only half adders, design a circuit that outputs the 1's complement of a 4-bit input when the control input C is 1, and passes the input unchanged when C is 0. **(6 marks)** **[CSE 205 | L-2/T-1 | 2023–24]**

Answer

1. Logical Analysis and Formulation

Let the 4-bit input be $A = A_3A_2A_1A_0$ and the required 4-bit output be $Y = Y_3Y_2Y_1Y_0$. We are given a control signal C which determines the operation:

- When $C = 0$, the output passes the input unchanged: $Y_i = A_i$.
- When $C = 1$, the output is the 1's complement of the input: $Y_i = A_i'$.

We can formulate the truth table for a single bit i (where $i \in \{0, 1, 2, 3\}$):

Control (C)	Input (A_i)	Desired Output (Y_i)	Description
0	0	0	Pass Unchanged ($Y_i = A_i$)
0	1	1	
1	0	1	1's Complement ($Y_i = A_i'$)
1	1	0	

Extracting the Boolean expression from the truth table via sum-of-products for Y_i, we get:

$$Y_i = C'A_i + CA_i'$$
$$Y_i = C \oplus A_i \quad \text{(Exclusive-OR Operation)}$$

2. Implementation using Half Adders

The problem restricts us to using **only Half Adders (HA)**. Let's review the Boolean equations for a standard Half Adder with inputs X and Y:

$$S \text{ (Sum)} = X \oplus Y$$
$$C_{out} \text{ (Carry)} = X \cdot Y$$

Notice that the Sum (S) output of a Half Adder inherently performs an XOR operation on its two inputs. Therefore, if we assign the Half Adder inputs as:

- $X = A_i$ (Data bit)
- $Y = C$ (Control signal)

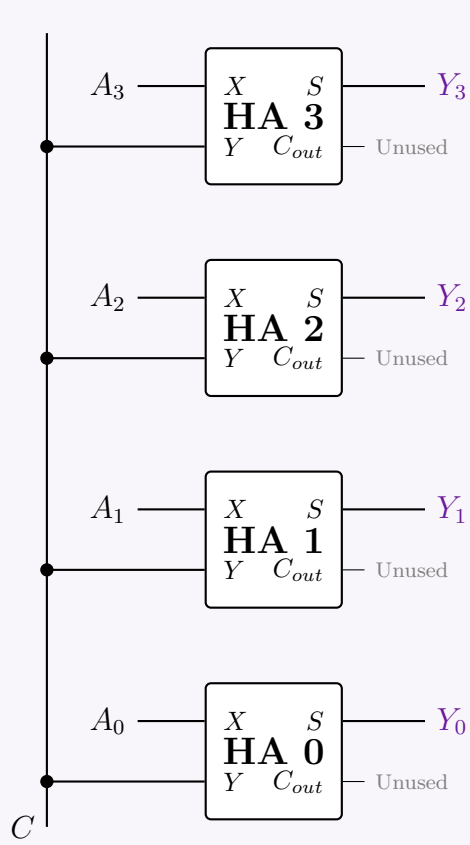
Then the Sum output becomes:

$$S_i = A_i \oplus C = Y_i$$

The Carry output ($C_{out} = A_i \cdot C$) is not needed for this operation and will simply be left unconnected (unused). To process a 4-bit input, we will employ exactly four Half Adders in parallel.

3. Schematic Diagram

The following circuit diagram illustrates the 4-bit conditional complementer constructed entirely from Half Adders.



Q2. Design a circuit that takes a 4-bit number as input and produces the 2's complement of the given number. **(20 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Logical Formulation and Algorithm

To design a circuit that generates the 2's complement of a 4-bit binary input $A = A_3A_2A_1A_0$, we can utilize the well-known "look-ahead" or bit-scan algorithm for 2's complement:

> *Starting from the least significant bit (LSB) up to the most significant bit (MSB), leave all bits unchanged until the first '1' is encountered. Leave that first '1' unchanged, and then invert all subsequent higher-order bits.*

We can implement this conditionally using Exclusive-OR (XOR) gates. An XOR gate acts as a controlled inverter:

- $X \oplus 0 = X$ (Passes unchanged)
- $X \oplus 1 = X'$ (Inverts)

We will generate a sequence of "Enable Inversion" signals, denoted as E_i, where $E_i = 1$ if any bit from position 0 to $i-1$ is 1.

2. Boolean Equations

Let the 4-bit input be $A_3A_2A_1A_0$ and the 2's complement output be $Y_3Y_2Y_1Y_0$.

- **Bit 0 (LSB):** Never inverted.

$$E_0 = 0$$
$$Y_0 = A_0 \oplus 0 = A_0$$

- **Bit 1:** Inverted only if $A_0 = 1$.

$$E_1 = A_0$$
$$Y_1 = A_1 \oplus E_1$$

- **Bit 2:** Inverted if either A_0 or A_1 is 1.

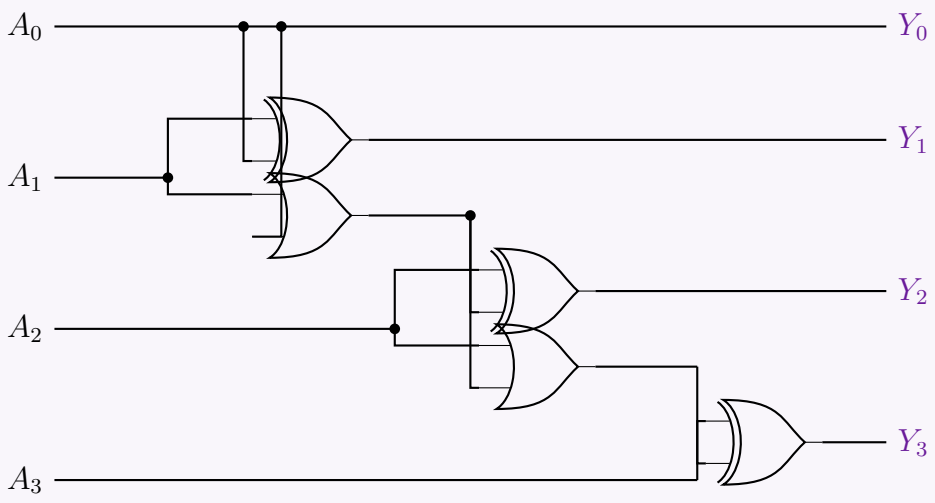
$$E_2 = E_1 \vee A_1 \quad \text{(Generated using an OR gate)}$$
$$Y_2 = A_2 \oplus E_2$$

- **Bit 3 (MSB):** Inverted if A_0, A_1, or A_2 is 1.

$$E_3 = E_2 \vee A_2 \quad \text{(Generated using an OR gate)}$$
$$Y_3 = A_3 \oplus E_3$$

3. Logic Circuit Diagram

Using the derived Boolean equations, we can construct the logic circuit using cascading OR gates to propagate the E_i signals, and XOR gates to generate the final Y_i output bits.



Note: An alternative method is to simply use four NOT gates to find the 1's complement of the inputs and then feed those inverted signals into a 4-bit Parallel Adder integrated circuit (such as the 7483) with the initial carry-in (C_{in}) permanently tied to logic 1. However, the dedicated combinatorial network shown above directly achieves the target with minimal propagation delay and gate count.

Overflow Detection

Q1. What is overflow in binary arithmetic? Explain how overflow is detected in signed addition and signed subtraction. Provide suitable examples. **(11 marks)** **[CSE 205 | L-2/T-1 | 2024–25]**

Answer

1. Definition of Overflow

In binary arithmetic, **overflow** is an error condition that occurs when the true mathematical result of an arithmetic operation exceeds the maximum or minimum numerical value that the designated fixed-width hardware register can represent.
In an n-bit signed 2's complement representation system, the range of valid integers is bounded by:

$$[-2^{n-1}, \quad 2^{n-1} - 1]$$

When an operation yields a result outside this window, an overflow occurs, leading to corrupted data (e.g., adding two positive numbers yields a negative value, or vice versa).

> **Crucial Distinction (Carry vs. Overflow):** A *carry-out* from the most significant bit (MSB) simply indicates that the unsigned magnitude exceeded the capacity, which is expected and normal in signed arithmetic. An *overflow* indicates that the *signed* value is mathematically invalid for the given bit-width.

—

2. Overflow Detection Methods

A. Signed Addition

There are two primary techniques used to detect overflow during the addition of two signed binary numbers ($A + B = Y$):

(a) **Sign Bit Analysis Rule:** Overflow can *only* happen when adding two numbers of the same sign.

- Positive + Positive = Negative $\implies$ **Overflow**
- Negative + Negative = Positive $\implies$ **Overflow**

Adding a positive number and a negative number can never cause an overflow because the absolute magnitude of the sum is guaranteed to be smaller than or equal to the largest operand.

(b) **The Carry XOR Rule (Hardware Implementation):** Let C_{n-1} be the internal carry generated *into* the MSB (sign bit position), and C_n be the final carry-out generated *out* of the MSB. The overflow hardware status flag (V) is directly computed via an Exclusive-OR operation:

$$V = C_{n-1} \oplus C_n$$

If $V = 1$, an overflow has occurred.

B. Signed Subtraction

Signed subtraction ($A - B$) is conventionally converted into an addition problem by taking the 2's complement of the subtrahend (B) and adding it to the minuend (A), such that $A - B = A + (-B)$.

(a) **Sign Bit Analysis Rule:** Overflow occurs if and only if the operands have opposite signs and the result matches the sign of the subtrahend:

- Positive − Negative = Negative $\implies$ **Overflow**
- Negative − Positive = Positive $\implies$ **Overflow**

(b) **Hardware Implementation:** Because the subtraction is physically transformed into an addition operation inside the Arithmetic Logic Unit (ALU), the exact same hardware check ($V = C_{n-1} \oplus C_n$) evaluates overflow seamlessly using the intermediate carries of the adder block.

—

3. Illustrative Examples (4-Bit Signed System)

In a 4-bit signed 2's complement system, the range of valid numbers is $[-2^3, 2^3 - 1] = [-8, +7]$.

Example 1: Signed Addition Overflow (Positive + Positive)

Compute $(+5) + (+4)$. The true mathematical answer is $+9$, which is outside the range $[-8, +7]$.

Step	Carry Line	Bit 3 (MSB)	Bit 2	Bit 1	Bit 0	Signed Value
Carries:	**C_4 = 0**	**C_3 = 1**	0	0		
Operand 1 (A):		0	1	0	1	+5
Operand 2 (B):	+	0	1	0	0	+4
Result (Y):		**1**	**0**	**0**	**1**	**−7 (Incorrect)**

- **Sign Check:** Two positive numbers ($A_3 = 0, B_3 = 0$) yielded a negative result ($Y_3 = 1$). This reveals an overflow condition.
- **XOR Rule Evaluation:**

$$V = C_3 \oplus C_4$$
$$V = 1 \oplus 0 = \mathbf{1} \quad \text{(Overflow Detected)}$$

Example 2: Signed Subtraction Overflow (Negative - Positive)

Compute $(-6) - (+3)$. The true mathematical answer is -9, which falls outside $[-8, +7]$. We transform this into addition: $(-6) + (-3)$.

- -6 in 4-bit 2's complement form is 1010_2.
- -3 in 4-bit 2's complement form is 1101_2.

Step	Carry Line	Bit 3 (MSB)	Bit 2	Bit 1	Bit 0	Signed Value
Carries:	**C_4 = 1**	**C_3 = 0**	0	0		
Minuend (A):		1	0	1	0	-6
$-$Subtrahend ($-B$):	+	1	1	0	1	-3
Result (Y):		**0**	**1**	**1**	**1**	$+7$ **(Incorrect)**

- **Sign Check:** A negative number minus a positive number yielded a positive result ($Y_3 = 0$). This reveals an overflow condition.
- **XOR Rule Evaluation:**

$$V = C_3 \oplus C_4$$
$$V = 0 \oplus 1 = \mathbf{1} \quad \text{(Overflow Detected)}$$

Q2. How do you check for overflow in the case of addition of two binary numbers in both signed and unsigned representations? **(5 marks)**
[CSE 205 | L-2/T-1 | 2013–14]

Answer

1. Overflow in Unsigned Binary Addition

In an n-bit unsigned binary system, all bits are used to represent the magnitude of a positive number. The valid range of representable integers is:

$$[0, \quad 2^n - 1]$$

Overflow Condition: An overflow occurs when the sum of two unsigned numbers exceeds the maximum capacity $(2^n - 1)$, meaning the result requires $n + 1$ bits to be represented correctly.
Detection Mechanism: In hardware, unsigned overflow is detected simply by examining the **final carry-out** (C_{out}) from the Most Significant Bit (MSB) position.

- If $C_{out} = 1$, an **overflow has occurred**.
- If $C_{out} = 0$, the sum is valid and bounded within n bits.

Example (4-bit Unsigned, Range: 0 to 15): Add 12_{10} and 5_{10}.

Step	Carry	Bit 3	Bit 2	Bit 1	Bit 0	Decimal
Carries:	**1**	0	0	0		
Operand A:		1	1	0	0	12
Operand B:	+	0	1	0	1	5
Result (S):	$C_{out} = 1$	**0**	**0**	**0**	**1**	1 **(Incorrect)**

The true sum is 17, which exceeds 15. The $C_{out} = 1$ correctly flags this overflow.
—

2. Overflow in Signed Binary Addition (2's Complement)

In an n-bit signed 2's complement system, the MSB acts as the sign bit (0 for positive, 1 for negative). The valid range of representable integers is:

$$[-2^{n-1}, \quad 2^{n-1} - 1]$$

Overflow Condition: An overflow occurs when the sum of two numbers with the same sign yields a result that falls outside this valid range, causing the sign bit of the sum to flip incorrectly. *Note: Adding a positive and a negative number can NEVER cause an overflow.*
Detection Mechanisms: There are two primary ways to detect signed overflow:
Method A: Sign-Bit Observation (Logical Rule) Let the sign bits of the addends be A_{n-1} and B_{n-1}, and the sign bit of the sum be S_{n-1}. Overflow occurs if the addends have the same sign, but the sum has a different sign:

$$\text{Overflow } (V) = A_{n-1}B_{n-1}S'_{n-1} + A'_{n-1}B'_{n-1}S_{n-1}$$

Method B: Carry XOR (Hardware Implementation) Let C_{n-1} be the internal carry *into* the MSB, and C_n be the carry *out* of the MSB. The overflow flag (V) is strictly the Exclusive-OR (XOR) of these two carries:

$$V = C_{n-1} \oplus C_n$$

If $V = 1$, an overflow has occurred. (The carry out is ignored for the actual arithmetic value, but crucial for this XOR check).
Example (4-bit Signed, Range: -8 to $+7$): Add $+5_{10}$ and $+4_{10}$.

Step	Carry	Bit 3 (Sign)	Bit 2	Bit 1	Bit 0	Decimal
Carries:	$C_4 = 0$	$C_3 = 1$	0	0		
Operand A:		0	1	0	1	$+5$
Operand B:	$+$	0	1	0	0	$+4$
Result (S):		**1**	**0**	**0**	**1**	-7 **(Incorrect)**

The true sum is $+9$, which exceeds $+7$. We evaluate the hardware overflow condition:

$$V = C_3 \oplus C_4$$
$$V = 1 \oplus 0 = \mathbf{1} \quad \text{(Overflow Detected)}$$

Additionally, observing the sign bits (Method A): $+5$ and $+4$ are positive (0), but the sum appears negative (1). This confirms the overflow.

Binary Multiplier

Q1. Discuss and design a 2-bit by 2-bit binary multiplier. **(15 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Theoretical Foundation and Logical Formulation

A 2-bit by 2-bit binary multiplier takes two 2-bit unsigned inputs:

- **Multiplicand:** $A = A_1A_0$
- **Multiplier:** $B = B_1B_0$

The maximum possible value for both A and B is 3_{10} (11_2). Therefore, the maximum product is $3 \times 3 = 9_{10}$, which requires 4 bits to represent (1001_2). Let the 4-bit product be $P = P_3P_2P_1P_0$.
The binary multiplication process is identical to decimal multiplication, relying on the generation of *partial products* and their subsequent addition. In binary, a partial product is simply the logical AND of two bits.

		A_1		A_0	
$\times$		B_1		B_0	
		A_1B_0	A_0B_0		(Partial Product 1)
$+$	A_1B_1	A_0B_1			(Partial Product 2, shifted)
P_3	P_2	P_1	P_0		(Final Product)

2. Boolean Equations for Product Bits

From the columnar addition in the tableau above, we can derive the exact logical requirements for each product bit P_i.

Bit 0 (P_0): The least significant bit is formed directly by the AND operation of the LSBs.

$$P_0 = A_0 \cdot B_0$$

Bit 1 (P_1): Requires adding two partial product bits: A_1B_0 and A_0B_1. This is done using a Half Adder.

$$P_1 = (A_1 \cdot B_0) \oplus (A_0 \cdot B_1) \quad \text{(Sum output of HA1)}$$
$$C_1 = (A_1 \cdot B_0) \cdot (A_0 \cdot B_1) \quad \text{(Carry output of HA1)}$$

Bit 2 (P_2): Requires adding the next partial product bit (A_1B_1) and the carry C_1 from the previous stage. We use a second Half Adder.

$$P_2 = (A_1 \cdot B_1) \oplus C_1 \quad \text{(Sum output of HA2)}$$
$$C_2 = (A_1 \cdot B_1) \cdot C_1 \quad \text{(Carry output of HA2)}$$

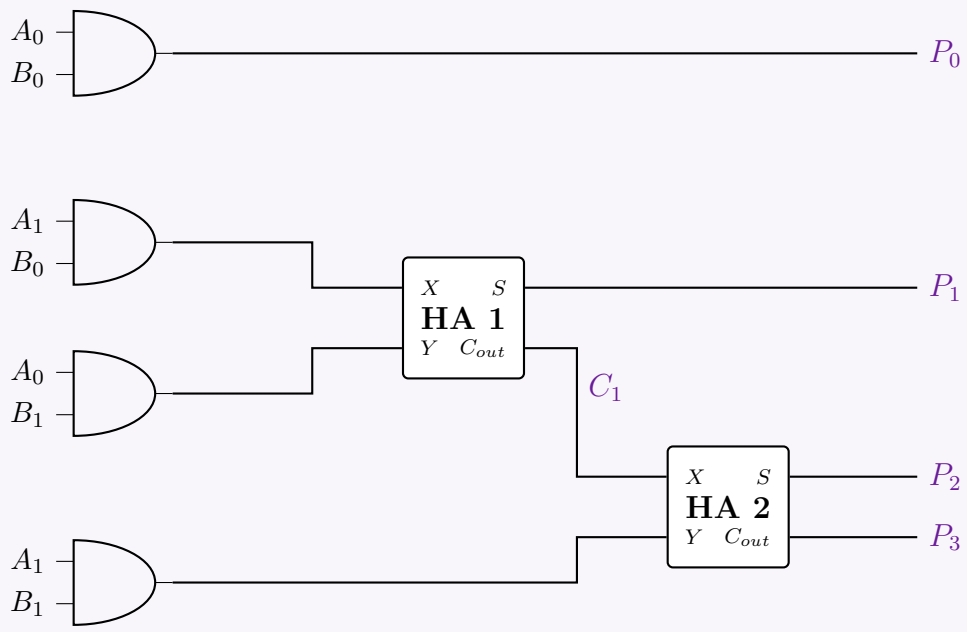
Bit 3 (P_3): The most significant bit is simply the final carry out.

$$P_3 = C_2$$

3. Circuit Implementation

To implement this design, we require:

- **Four 2-input AND gates** to generate the partial products: A_0B_0, A_1B_0, A_0B_1, and A_1B_1.
- **Two Half Adders (HA)** to sum the aligned columns and propagate carries.



Q2. Design a circuit with a 4-bit binary adder which returns $A \times B$ where A and B are 3-bit binary numbers. **(10 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Answer

1. Logical Formulation of 3×3 Multiplication

Let the two 3-bit unsigned binary numbers be the Multiplicand $A = A_2A_1A_0$ and the Multiplier $B = B_2B_1B_0$. The maximum value for each is 7_{10} (111_2), meaning the maximum product is 49_{10} (110001_2). Thus, the final product requires 6 bits: $P = P_5P_4P_3P_2P_1P_0$. The multiplication is achieved by generating three *partial products* (PP_0, PP_1, PP_2) using 2-input AND gates, and then summing them based on their positional weights.

			A_2	A_1		A_0	
$\times$			B_2	B_1		B_0	
				A_2B_0	A_1B_0	A_0B_0	(PP_0)
			A_2B_1	A_1B_1	A_0B_1		(PP_1, shifted)
$+$		A_2B_2	A_1B_2	A_0B_2			(PP_2, shifted)
	P_5	P_4	P_3	P_2	P_1	P_0	(Final Product)

Because there are three partial products to add, a purely combinational implementation requires **two addition stages**. We will construct this using **two 4-bit binary parallel adders**.

2. Stage-by-Stage Adder Configuration

Stage 1 (Adder 1): Adds the upper bits of PP_0 with PP_1.

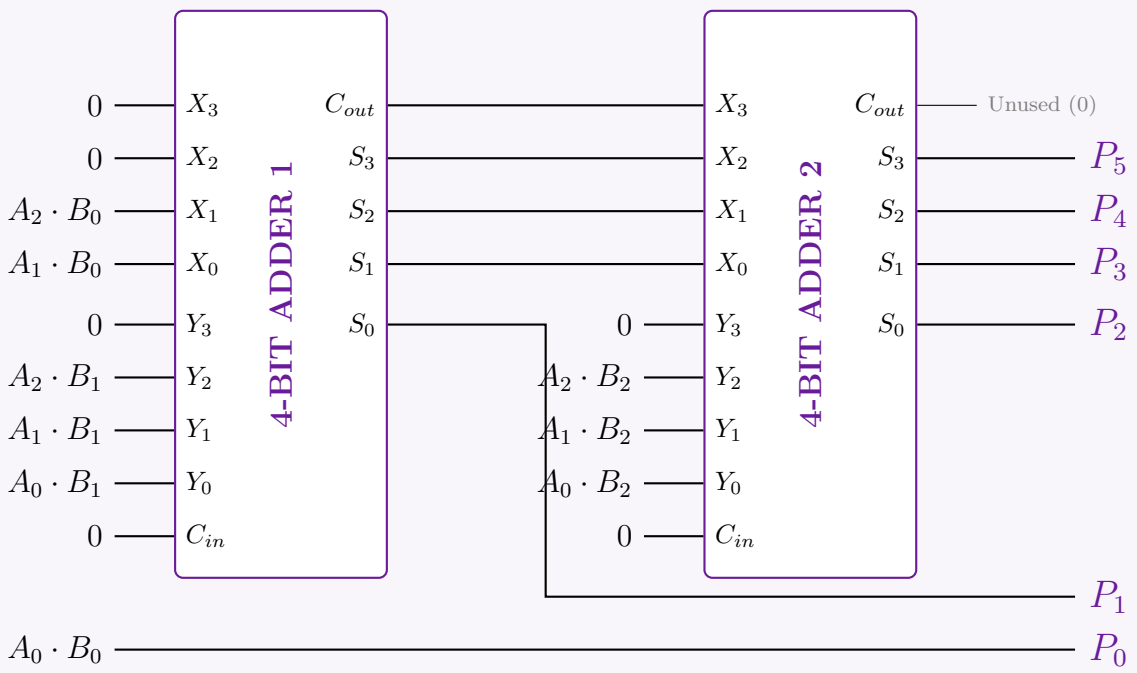
- P_0 is simply A_0B_0 and requires no addition. It bypasses the adders.
- **Input X** (PP_0 components): $X_3 = 0, X_2 = 0, X_1 = A_2B_0, X_0 = A_1B_0$.
- **Input Y** (PP_1 components): $Y_3 = 0, Y_2 = A_2B_1, Y_1 = A_1B_1, Y_0 = A_0B_1$.
- **Output:** Yields sum bits S_3, S_2, S_1, S_0 and carry C_{out1}.
- The least significant sum bit represents $P_1 = S_0$.

Stage 2 (Adder 2): Adds the shifted intermediate sum from Stage 1 to PP_2.

- **Input X** (Shifted Sum 1): $X_3 = C_{out1}, X_2 = S_3, X_1 = S_2, X_0 = S_1$.
- **Input Y** (PP_2 components): $Y_3 = 0, Y_2 = A_2B_2, Y_1 = A_1B_2, Y_0 = A_0B_2$.
- **Output:** Yields sum bits S'_3, S'_2, S'_1, S'_0. The carry C_{out2} is always 0.
- These sum bits directly form the remaining product bits: $P_5 = S'_3, P_4 = S'_2, P_3 = S'_1, P_2 = S'_0$.

3. Logic Circuit Schematic

The following diagram illustrates the complete 3x3 multiplier utilizing two 4-bit adders. *(Note: The 9 logical products like $A_0 \cdot B_0$ are assumed to be pre-generated via a parallel bank of standard 2-input AND gates).*



Q3. Using 4-bit adders, design a 3-bit multiplier for 3-bit variables $X(X_1, X_2, X_3)$ and $Y(Y_1, Y_2, Y_3)$. Explain the design. **(15 marks)**
[CSE 205 | L-2/T-1 | 2016–17]

Answer

1. Mathematical Formulation of the 3×3 Multiplier

Let the two 3-bit variables be represented as:

- **Multiplicand:** $X = X_3X_2X_1$ (where X_3 is the MSB and X_1 is the LSB)
- **Multiplier:** $Y = Y_3Y_2Y_1$ (where Y_3 is the MSB and Y_1 is the LSB)

Multiplying a 3-bit number by a 3-bit number yields a maximum product of $7 \times 7 = 49_{10}$ (110001_2), which requires 6 bits. Let the final product be $P = P_6P_5P_4P_3P_2P_1$.
We generate partial products by multiplying the multiplicand by each bit of the multiplier using AND gates, and then we sum these partial products according to their binary weights.

				X_3	X_2		X_1
$\times$				Y_3	Y_2		Y_1
				X_3Y_1	X_2Y_1	X_1Y_1	(PP_1, weight 2^0)
			X_3Y_2	X_2Y_2	X_1Y_2		(PP_2, weight 2^1)
$+$		X_3Y_3	X_2Y_3	X_1Y_3			(PP_3, weight 2^2)
	P_6	P_5	P_4	P_3	P_2	P_1	(Final Product)

2. Design Strategy Using 4-Bit Adders

To sum these three rows of partial products, we will cascade **two 4-bit binary adders**. A standard 4-bit adder has inputs $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$, generating sums $S_3S_2S_1S_0$ and a carry-out C_{out}. We map the partial products to the adder inputs strictly based on their positional weights.

Stage 1: Adding PP_1 and PP_2 (Adder 1)

We align PP_1 and PP_2 into the first 4-bit adder.

- **Weight 2^0:** $A_0 = X_1Y_1$, $B_0 = 0$. The sum S_0 becomes the final product bit $\mathbf{P_1}$.
- **Weight 2^1:** $A_1 = X_2Y_1$, $B_1 = X_1Y_2$. The sum S_1 becomes the final product bit $\mathbf{P_2}$.
- **Weight 2^2:** $A_2 = X_3Y_1$, $B_2 = X_2Y_2$. The sum is S_2.
- **Weight 2^3:** $A_3 = 0$, $B_3 = X_3Y_2$. The sum is S_3.
- **Carry Out:** C_{out1} has a positional weight of 2^4.

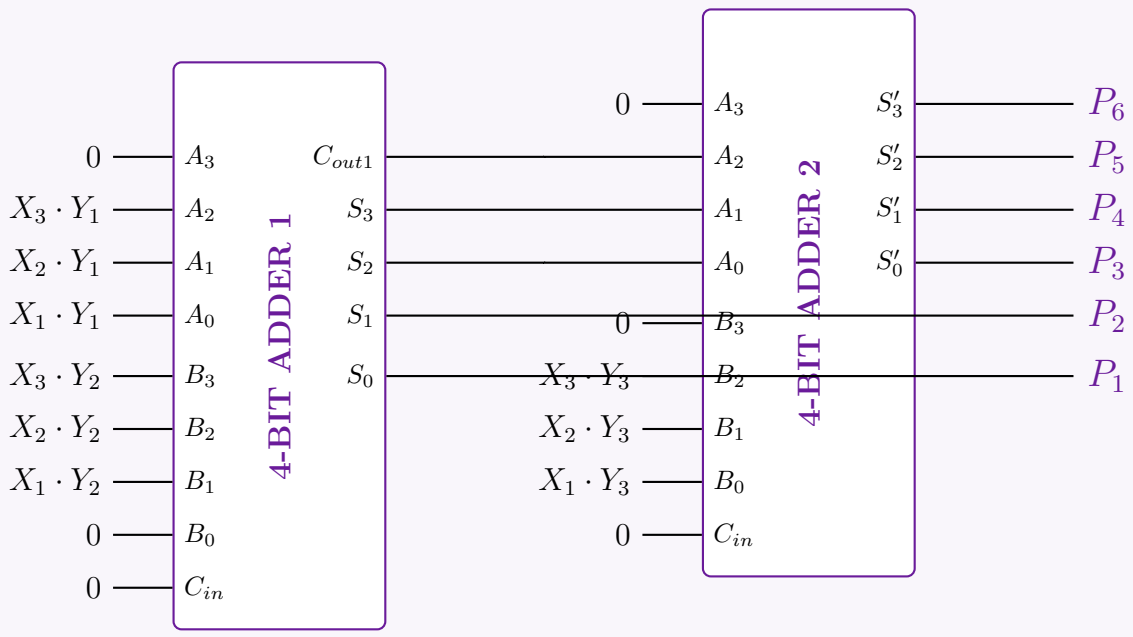
Stage 2: Adding the Intermediate Sum to PP_3 (Adder 2)

We take the unresolved upper bits from Adder 1 (S_2, S_3, C_{out1}) and add them to PP_3. Notice that PP_3 starts at weight 2^2.

- **Weight 2^2:** $A_0 = S_2$ (from Adder 1), $B_0 = X_1Y_3$. The sum S'_0 becomes $\mathbf{P_3}$.
- **Weight 2^3:** $A_1 = S_3$ (from Adder 1), $B_1 = X_2Y_3$. The sum S'_1 becomes $\mathbf{P_4}$.
- **Weight 2^4:** $A_2 = C_{out1}$ (from Adder 1), $B_2 = X_3Y_3$. The sum S'_2 becomes $\mathbf{P_5}$.
- **Weight 2^5:** $A_3 = 0$, $B_3 = 0$. The sum S'_3 absorbs the carry and becomes $\mathbf{P_6}$.

3. Logic Circuit Schematic

The circuit diagram below illustrates the exact routing of the partial products into the two cascaded 4-bit adders. *(Assume all 9 partial products X_iY_j are generated using a parallel bank of 2-input AND gates prior to the adder stages).*



Q4. How would you implement a one-bit by one-bit binary multiplier using basic gates? **(5 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Answer

1. Problem Definition and Truth Table

A one-bit by one-bit binary multiplier takes two single-bit inputs:

- **Multiplicand:** $A \in \{0, 1\}$
- **Multiplier:** $B \in \{0, 1\}$

Since the maximum possible value for both inputs is 1, the maximum possible product is $1 \times 1 = 1$. Therefore, the product P can be represented using a single bit.
To determine the logical relationship between the inputs and the output, we construct the truth table for all possible combinations of A and B:

Multiplicand (A)	**Multiplier (B)**	**Product (P)**
0	0	0
0	1	0
1	0	0
1	1	**1**

2. Boolean Expression Derivation

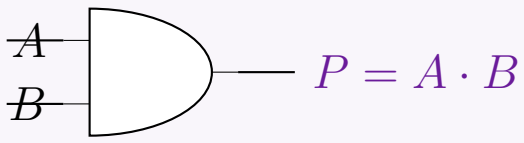
From the truth table, we observe that the output P is high (1) if and only if both input A is 1 **and** input B is 1.
Extracting the logical expression using the Sum of Products (SOP) form based on the single minterm where the output is 1:

$$P = m_3$$
$$P = A \cdot B$$

This equation is the fundamental definition of the logical **AND** operation. Thus, the entire process of 1-bit by 1-bit binary multiplication is functionally identical to passing the two bits through a standard 2-input AND gate.

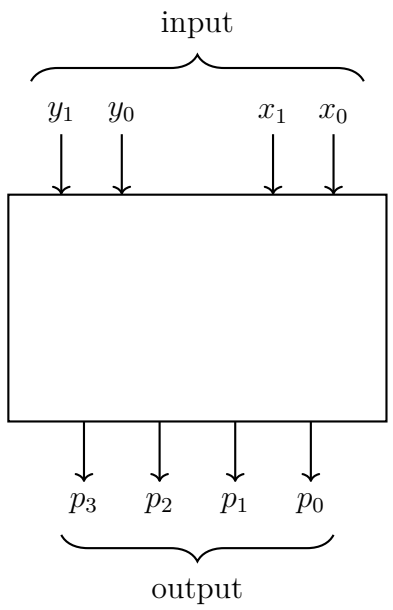
3. Logic Circuit Implementation

The hardware implementation of this multiplier is extremely straightforward. It requires exactly one basic logic gate: a 2-input AND gate.



1-bit x 1-bit Multiplier

Q5. The figure below represents a multiplier that takes two 2-bit binary numbers x_1x_0 and y_1y_0 as inputs, and produces an output binary number $p_3p_2p_1p_0$ equal to the arithmetic product of the two input numbers. Design the logic circuit for the multiplier. **(20 marks)**
[CSE 205 | L-2/T-1 | 2012–13]



Answer

1. Mathematical Formulation of the 2×2 Multiplier

Let the two 2-bit input variables be the Multiplicand $X = x_1x_0$ and the Multiplier $Y = y_1y_0$. The maximum value for each input is 3_{10} (11_2), meaning the maximum possible product is $3 \times 3 = 9_{10}$ (1001_2). Thus, the final product requires 4 bits: $P = p_3p_2p_1p_0$. The multiplication is performed by generating partial products using logical AND operations and summing them based on their binary positional weights.

			x_1	x_0
$\times$			y_1	y_0
			x_1y_0	x_0y_0
$+$		x_1y_1	x_0y_1	
	p_3	p_2	p_1	p_0

2. Boolean Logic Equations

From the positional addition in the table above, we can derive the boolean expressions for each bit of the product:

- **Bit p_0 (Weight 2^0):** This is exactly the first partial product.

$$p_0 = x_0 \cdot y_0$$

- **Bit p_1 (Weight 2^1):** This is the sum of x_1y_0 and x_0y_1. We can use a **Half Adder (HA1)** for this addition.

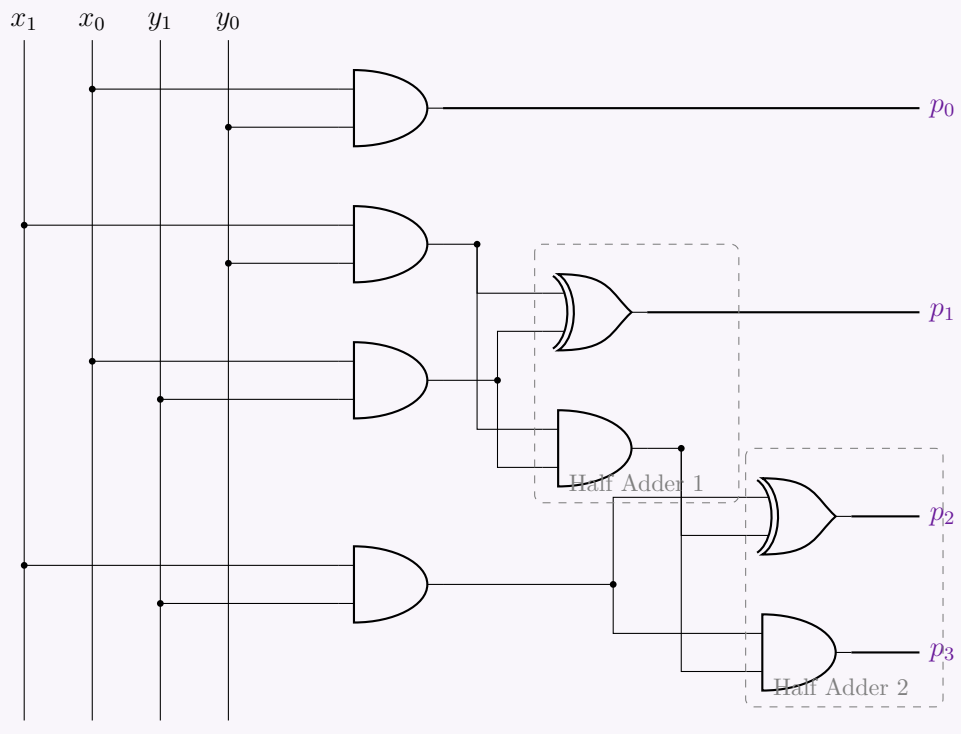
$$p_1 = (x_1 \cdot y_0) \oplus (x_0 \cdot y_1) \quad \text{(Sum from HA1)}$$
$$c_1 = (x_1 \cdot y_0) \cdot (x_0 \cdot y_1) \quad \text{(Carry from HA1)}$$

- **Bits p_2 and p_3 (Weights 2^2 and 2^3):** p_2 is the sum of the partial product x_1y_1 and the carry c_1 from the previous column. This requires a second **Half Adder (HA2)**. The carry from this addition forms the most significant bit, p_3.

$$p_2 = (x_1 \cdot y_1) \oplus c_1 \quad \text{(Sum from HA2)}$$
$$p_3 = (x_1 \cdot y_1) \cdot c_1 \quad \text{(Carry from HA2)}$$

3. Gate-Level Circuit Diagram

The circuit is constructed using four 2-input AND gates to generate the partial products, and two Half Adders (each consisting of an XOR gate and an AND gate) to perform the summation.



Incrementer

Q1. Design a 4-bit combinational circuit incrementer (a circuit that adds 1 to a 4-bit binary number) using only half adders. **(10 marks)**
[CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Mathematical Derivation of the Incrementer

An incrementer is a combinational circuit that adds a constant 1 to a given binary number. Let the 4-bit input be $A = A_3A_2A_1A_0$. We are performing the arithmetic addition:

$$A + 0001_2$$

In a standard addition process, each bit position i uses a Full Adder with inputs A_i, B_i, and C_i (Carry-in). A Full Adder performs the following operations:

$$S_i = A_i \oplus B_i \oplus C_i$$
$$C_{i+1} = (A_i \cdot B_i) + C_i \cdot (A_i \oplus B_i)$$

Let us evaluate this for our specific addition where the second operand $B = 0001_2$.
Stage 0 (Least Significant Bit)

For the LSB ($i = 0$), we are adding A_0 and $B_0 = 1$. The initial carry-in $C_0 = 0$.

$$S_0 = A_0 \oplus 1 \oplus 0 = A_0 \oplus 1 \qquad \text{(Equivalent to } \overline{A_0}\text{)}$$
$$C_1 = (A_0 \cdot 1) + 0 \cdot (A_0 \oplus 1) = A_0 \cdot 1$$

These equations, $S_0 = A_0 \oplus 1$ and $C_1 = A_0 \cdot 1$, perfectly match the boolean expressions for a **Half Adder** where the two inputs are A_0 and 1.
Stages 1, 2, and 3 (Higher-Order Bits)

For the remaining bits ($i = 1, 2, 3$), the bit from the second operand is always $B_i = 0$. Substituting $B_i = 0$ into the Full Adder equations yields:

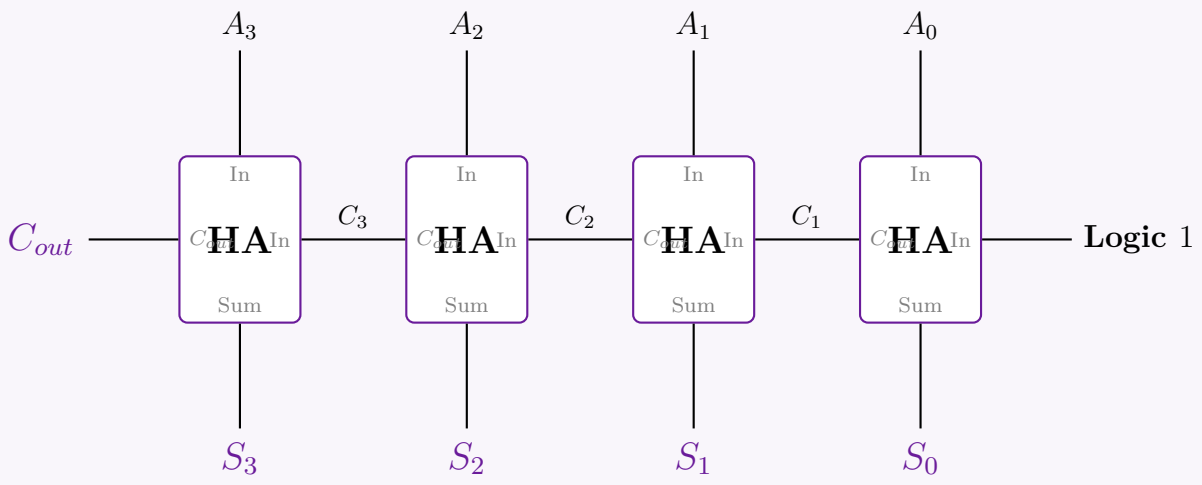
$$S_i = A_i \oplus 0 \oplus C_i = A_i \oplus C_i$$
$$C_{i+1} = (A_i \cdot 0) + C_i \cdot (A_i \oplus 0) = A_i \cdot C_i$$

Notice that these reduced equations, $S_i = A_i \oplus C_i$ and $C_{i+1} = A_i \cdot C_i$, are functionally identical to a **Half Adder** where the two inputs are A_i and C_i.

Conclusion: Because one of the operands is fixed to zero for all positions except the LSB, every Full Adder stage degrades gracefully into a Half Adder stage. Therefore, a 4-bit incrementer can be implemented using exactly four cascaded Half Adders.

2. Logic Circuit Diagram

The circuit routes the incoming carry from the previous Half Adder directly into the next Half Adder. The LSB Half Adder receives a constant Logic High (1) as its secondary input.



Q2. Design a 4-bit incrementer circuit using 4 half-adders. Note that the incrementer circuit adds 1 to a binary number. **(10 marks)**
[CSE 205 | L-2/T-1 | 2011–12]

Answer

1. Mathematical Derivation of the Incrementer

An incrementer circuit adds a constant value of 1 to a given binary number. Let the 4-bit input number be represented as $A = A_3A_2A_1A_0$. We wish to perform the arithmetic operation:

$$A + 0001_2$$

In a general binary addition, each bit position i utilizes a Full Adder with inputs A_i, B_i, and C_i (Carry-in). A Full Adder performs:

$$S_i = A_i \oplus B_i \oplus C_i$$
$$C_{i+1} = (A_i \cdot B_i) + C_i \cdot (A_i \oplus B_i)$$

Let us evaluate these equations for our specific increment operation, where the second operand is $B = 0001_2$.
Stage 0 (Least Significant Bit)

For the LSB ($i = 0$), we are adding A_0 and $B_0 = 1$. The initial carry-in to the circuit is $C_0 = 0$.

$$S_0 = A_0 \oplus 1 \oplus 0 = A_0 \oplus 1 \qquad \text{(Equivalent to } \overline{A_0}\text{)}$$
$$C_1 = (A_0 \cdot 1) + 0 \cdot (A_0 \oplus 1) = A_0 \cdot 1$$

These expressions, $S_0 = A_0 \oplus 1$ and $C_1 = A_0 \cdot 1$, precisely define a **Half Adder** whose two inputs are A_0 and a constant logic 1.
Stages 1, 2, and 3 (Higher-Order Bits)

For all remaining bits ($i = 1, 2, 3$), the bit from the second operand is zero ($B_i = 0$). Substituting $B_i = 0$ into the Full Adder equations yields:

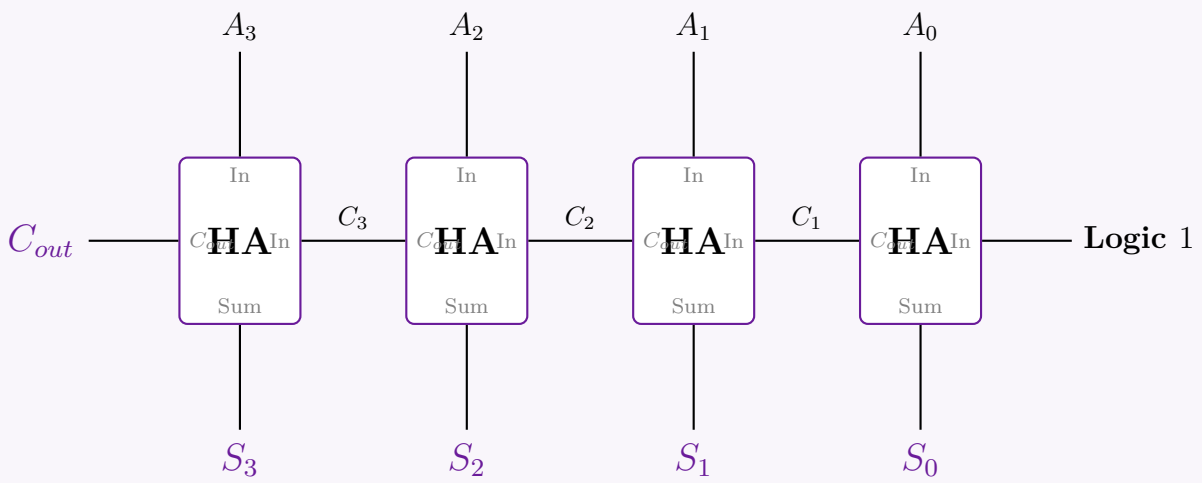
$$S_i = A_i \oplus 0 \oplus C_i = A_i \oplus C_i$$
$$C_{i+1} = (A_i \cdot 0) + C_i \cdot (A_i \oplus 0) = A_i \cdot C_i$$

Notice that these reduced equations, $S_i = A_i \oplus C_i$ and $C_{i+1} = A_i \cdot C_i$, are functionally identical to a **Half Adder** where the two inputs are A_i and the carry C_i from the preceding stage.

Conclusion: Since the operand added is fixed to zero for all positions except the LSB, every Full Adder stage reduces to a Half Adder. Thus, a 4-bit incrementer can be elegantly implemented using a cascade of exactly four Half Adders.

2. Logic Circuit Diagram

The circuit diagram below illustrates the implementation. The incoming carry from the previous Half Adder is routed directly into one of the inputs of the next Half Adder. The LSB Half Adder receives a constant Logic High (1) as its secondary input to inject the +1 value.



Encoders & Decoders

Q1. Design a light sensor based controller circuit for accessing eight servers A, B, C, D, E, F, G, H. The light sensors are numbered as $S_1, S_2, \ldots$ and so on. The sensors of the controller will be ON to show which server is active. The servers have some priority order: C, A, E, H, D, G, F, B (C has the highest priority and B has the lowest priority). **(15 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. System Architecture & Priority Mapping

The problem requires a controller that manages access to eight servers based on a strict priority hierarchy and indicates the active server using light sensors. Since multiple servers might request access simultaneously, we must resolve these requests using a **Priority Encoder**. The encoder will generate a unique binary code for the highest-priority active request. This code is then fed into a **Decoder**, which activates exactly one output line to turn on the corresponding light sensor.
Let the requests from the servers be $R_C, R_A, R_E, R_H, R_D, R_G, R_F, R_B$. We map these to the inputs D_7 to D_0 of an 8-to-3 Priority Encoder, where D_7 has the highest priority and D_0 has the lowest. The outputs of the 3-to-8 Decoder (Y_7 to Y_0) will drive the light sensors S_1 to S_8.

Priority	Server	Encoder Input	Decoder Output	Active Sensor
7 (Highest)	**C**	$D_7 = R_C$	Y_7	S_1
6	**A**	$D_6 = R_A$	Y_6	S_2
5	**E**	$D_5 = R_E$	Y_5	S_3
4	**H**	$D_4 = R_H$	Y_4	S_4
3	**D**	$D_3 = R_D$	Y_3	S_5
2	**G**	$D_2 = R_G$	Y_2	S_6
1	**F**	$D_1 = R_F$	Y_1	S_7
0 (Lowest)	**B**	$D_0 = R_B$	Y_0	S_8

2. Truth Table & Boolean Equations for the Priority Encoder

The 8-to-3 Priority Encoder generates a 3-bit binary output $A_2A_1A_0$ and a Valid Bit (V) which is high (1) if at least one request is active. We use 'X' to denote "Don't Care" conditions for lower-priority inputs when a higher-priority input is active.

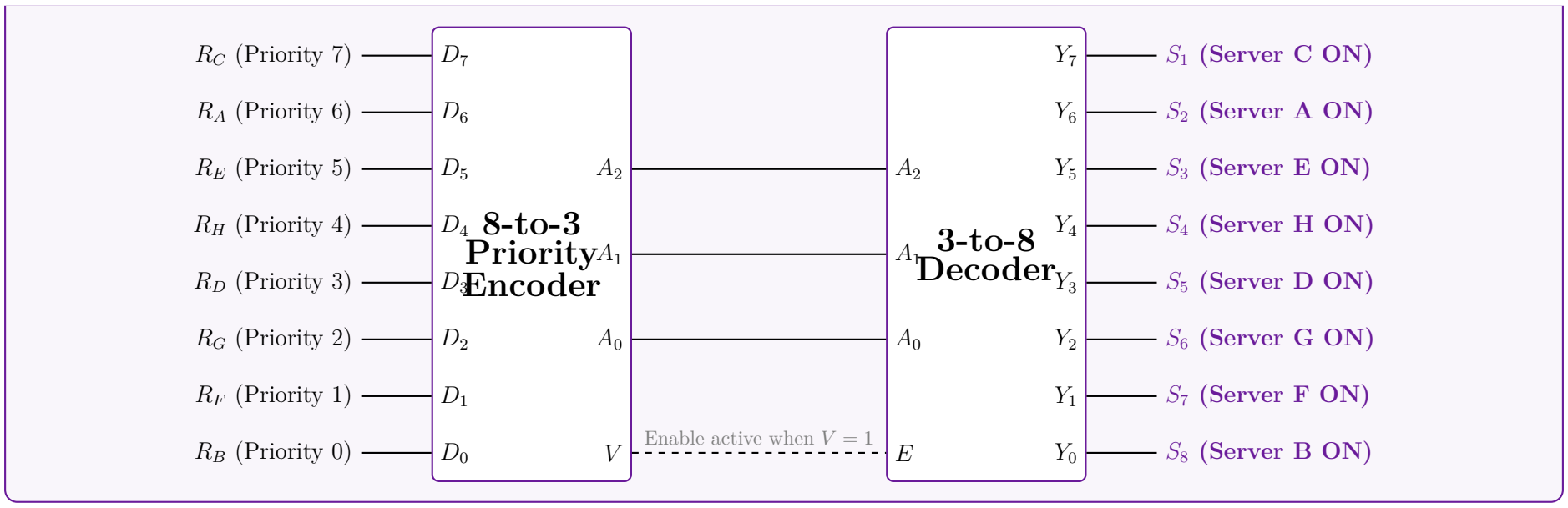
Server Requests (Encoder Inputs)								**Outputs**			
D_7 (C)	D_6 (A)	D_5 (E)	D_4 (H)	D_3 (D)	D_2 (G)	D_1 (F)	D_0 (B)	A_2	A_1	A_0	V (Enable)
0	0	0	0	0	0	0	0	0	0	0	**0**
1	X	X	X	X	X	X	X	1	1	1	**1**
0	1	X	X	X	X	X	X	1	1	0	**1**
0	0	1	X	X	X	X	X	1	0	1	**1**
0	0	0	1	X	X	X	X	1	0	0	**1**
0	0	0	0	1	X	X	X	0	1	1	**1**
0	0	0	0	0	1	X	X	0	1	0	**1**
0	0	0	0	0	0	1	X	0	0	1	**1**
0	0	0	0	0	0	0	1	0	0	0	**1**

Using Boolean algebra, we can extract the Sum-of-Products (SOP) expressions for the encoder outputs. A specific output bit is high if the highest active priority input corresponds to a '1' in that bit's binary weight:

$$
\begin{aligned}
V &= D_7 + D_6 + D_5 + D_4 + D_3 + D_2 + D_1 + D_0 \\
A_2 &= D_7 + D_6 + D_5 + D_4 \\
A_1 &= D_7 + D_6 + \overline{D_5} \cdot \overline{D_4} \cdot D_3 + \overline{D_5} \cdot \overline{D_4} \cdot D_2 \\
A_0 &= D_7 + \overline{D_6} \cdot D_5 + \overline{D_6} \cdot \overline{D_4} \cdot D_3 + \overline{D_6} \cdot \overline{D_4} \cdot \overline{D_2} \cdot D_1
\end{aligned}
$$

3. Controller Logic Circuit Diagram

The controller is implemented using Medium Scale Integration (MSI) blocks. The Valid Bit (V) from the Priority Encoder acts as the Enable (E) pin for the 3-to-8 Decoder. This ensures that no light sensor turns on if no server is requested.



Q2. Show the truth table of an 8-to-3 line priority encoder, where higher priority is given to lower numbered input. **(9 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Problem Analysis and Priority Definition

An 8-to-3 line priority encoder compresses eight inputs (D_0 to D_7) into a 3-bit binary output (A_2, A_1, A_0). It also typically includes a Valid output bit (V) to distinguish between the state where input D_0 is active versus the state where no inputs are active.
In a standard priority encoder, higher-numbered inputs usually have higher priority. However, this specific problem dictates a **reversed priority scheme**: **the lower-numbered input has the higher priority**.

- **Highest Priority:** D_0
- **Lowest Priority:** D_7

This implies that if D_0 is active (1), the encoder will output the binary equivalent of 0 (000_2), completely ignoring the states of all other inputs D_1 through D_7. The system only considers D_1 if $D_0 = 0$, and so on.

2. Truth Table Construction

In the table below, 'X' represents a **Don't Care** condition, meaning the input can be either 0 or 1 without affecting the output, due to a higher-priority input already dictating the result.

Inputs (Highest to Lowest Priority)								**Outputs**			
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	A_2	A_1	A_0	**Valid (V)**
0	0	0	0	0	0	0	0	0	0	0	**0**
1	X	X	X	X	X	X	X	0	0	0	**1**
0	1	X	X	X	X	X	X	0	0	1	**1**
0	0	1	X	X	X	X	X	0	1	0	**1**
0	0	0	1	X	X	X	X	0	1	1	**1**
0	0	0	0	1	X	X	X	1	0	0	**1**
0	0	0	0	0	1	X	X	1	0	1	**1**
0	0	0	0	0	0	1	X	1	1	0	**1**
0	0	0	0	0	0	0	1	1	1	1	**1**

3. Boolean Output Equations (For Reference)

Although not strictly asked, defining the Boolean equations based on the truth table demonstrates a complete understanding of the circuit's underlying logic. The outputs are only valid when $V = 1$.

$$V = D_0 + D_1 + D_2 + D_3 + D_4 + D_5 + D_6 + D_7$$
$$A_2 = \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot (D_4 + D_5 + D_6 + D_7)$$
$$A_1 = \overline{D_0} \cdot \overline{D_1} \cdot (D_2 + D_3) + \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot (D_6 + D_7)$$
$$A_0 = \overline{D_0} \cdot D_1 + \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot D_3 + \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot D_5 + \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_6} \cdot D_7$$

Q3. After a school final result, Ishrat applied for college admission and gave her choices from eight colleges indexed A to H, following the priority order B, A, D, E, F, H, G, C (B has the highest priority, C has the lowest priority). Find the Boolean function of her choices and show the design steps. **(15 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Answer

1. Problem Definition and Priority Mapping

The problem describes a system where eight choices (colleges A through H) are ranked by priority. The system must output a unique binary code corresponding to the highest-priority college chosen. This requires an **8-to-3 Priority Encoder**.
First, we assign priority levels 7 (highest) down to 0 (lowest) and map each college to an input variable $I_7 \dots I_0$:

Priority	**College**	**Input Var**	**Output Code ($Y_2Y_1Y_0$)**
7 (Highest)	**B**	I_7	1 1 1
6	**A**	I_6	1 1 0
5	**D**	I_5	1 0 1
4	**E**	I_4	1 0 0
3	**F**	I_3	0 1 1
2	**H**	I_2	0 1 0
1	**G**	I_1	0 0 1
0 (Lowest)	**C**	I_0	0 0 0

2. Truth Table Construction

A valid bit (V) is introduced to differentiate the condition where college C (000) is chosen from the condition where *no* college is chosen. In the table below, 'X' denotes a "Don't Care" condition, meaning lower priority inputs are ignored when a higher priority input is active.

College Inputs (Highest to Lowest Priority)								**Binary Output**			
B (I_7)	**A** (I_6)	**D** (I_5)	**E** (I_4)	**F** (I_3)	**H** (I_2)	**G** (I_1)	**C** (I_0)	Y_2	Y_1	Y_0	V
0	0	0	0	0	0	0	0	0	0	0	**0**
1	X	X	X	X	X	X	X	1	1	1	**1**
0	1	X	X	X	X	X	X	1	1	0	**1**
0	0	1	X	X	X	X	X	1	0	1	**1**
0	0	0	1	X	X	X	X	1	0	0	**1**
0	0	0	0	1	X	X	X	0	1	1	**1**
0	0	0	0	0	1	X	X	0	1	0	**1**
0	0	0	0	0	0	1	X	0	0	1	**1**
0	0	0	0	0	0	0	1	0	0	0	**1**

3. Boolean Function Derivation

From the truth table, we derive the Boolean expressions. A specific output bit is high if the highest active priority input possesses a logic '1' in its corresponding binary weight. Lower priority inputs are masked by negating the higher priority inputs.
1. Valid Bit (V): Active if any college is selected.

$$V = B + A + D + E + F + H + G + C$$

2. Most Significant Bit (Y_2): High for I_7, I_6, I_5, I_4.

$$\begin{aligned} Y_2 &= I_7 + \overline{I_7} I_6 + \overline{I_7 I_6} I_5 + \overline{I_7 I_6 I_5} I_4 \\ &= I_7 + I_6 + I_5 + I_4 \quad \text{(By sequential application of } X + \overline{X}Y = X + Y\text{)} \end{aligned}$$
$$Y_2 = B + A + D + E$$

3. Middle Bit (Y_1): High for I_7, I_6, I_3, I_2.

$$\begin{aligned} Y_1 &= I_7 + \overline{I_7} I_6 + \overline{I_7 I_6 I_5 I_4} I_3 + \overline{I_7 I_6 I_5 I_4 I_3} I_2 \\ &= I_7 + I_6 + \overline{I_5 I_4} I_3 + \overline{I_5 I_4} I_2 \quad \text{(Absorption of } I_7, I_6 \text{ into subsequent terms)} \end{aligned}$$
$$Y_1 = B + A + \overline{DE} F + \overline{DE} H$$

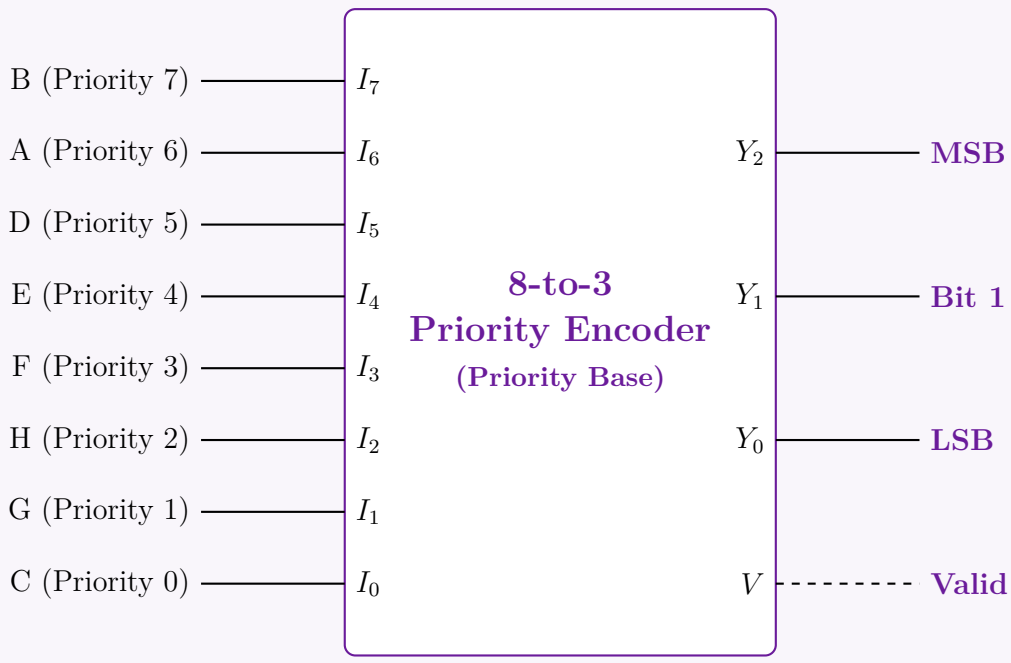
4. Least Significant Bit (Y_0): High for I_7, I_5, I_3, I_1.

$$\begin{aligned} Y_0 &= I_7 + \overline{I_7}I_6I_5 + \overline{I_7I_6I_5I_4}I_3 + \overline{I_7I_6I_5I_4I_3I_2}I_1 \\ &= I_7 + \overline{I_6}I_5 + \overline{I_6I_5I_4}I_3 + \overline{I_6I_5I_4I_3I_2}I_1 \\ &= I_7 + \overline{I_6}(I_5 + \overline{I_5I_4}I_3) + \dots \\ &= I_7 + \overline{I_6}I_5 + \overline{I_6I_4}I_3 + \overline{I_6I_4I_2}I_1 \quad \text{(Using Distributive and Boolean logic reductions)} \end{aligned}$$

$Y_0 = B + \overline{A}D + \overline{AE}F + \overline{AEH}G$

4. Logic Circuit Implementation (Block Diagram)

The derived Boolean functions can be implemented using standard NOT, AND, and OR logic gates. Structurally, they are synthesized inside an 8-to-3 Priority Encoder macro cell.



Q4. Design an 8-to-3 priority encoder with the priority order: 5, 0, 7, 4, 6, 3, 2, 1 (5 has highest priority, 1 has lowest priority). **(13 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Answer

1. Priority Mapping and System Definition

An 8-to-3 priority encoder takes 8 inputs (D_0 to D_7) and compresses them into a 3-bit binary output (Y_2, Y_1, Y_0) based on a strict priority hierarchy. It also outputs a Valid bit (V) to indicate if at least one input is active.
The problem specifies an irregular priority order. We assign priority levels from 7 (highest) down to 0 (lowest) and map them to their respective inputs and expected binary outputs (the binary index of the active input).

Priority Level	Input Variable	Active Index	Binary Output ($Y_2Y_1Y_0$)
7 (Highest)	D_5	5	1 0 1
6	D_0	0	0 0 0
5	D_7	7	1 1 1
4	D_4	4	1 0 0
3	D_6	6	1 1 0
2	D_3	3	0 1 1
1	D_2	2	0 1 0
0 (Lowest)	D_1	1	0 0 1

2. Truth Table Construction

In the truth table, we evaluate the inputs in decreasing order of priority. An 'X' signifies a "Don't Care" condition, indicating that lower-priority inputs are ignored when a higher-priority input is active (Logic 1).

Inputs (Highest to Lowest Priority)								Outputs			
D_5 (P7)	D_0 (P6)	D_7 (P5)	D_4 (P4)	D_6 (P3)	D_3 (P2)	D_2 (P1)	D_1 (P0)	Y_2	Y_1	Y_0	V
0	0	0	0	0	0	0	0	0	0	0	**0**
1	X	X	X	X	X	X	X	1	0	1	**1**
0	1	X	X	X	X	X	X	0	0	0	**1**
0	0	1	X	X	X	X	X	1	1	1	**1**
0	0	0	1	X	X	X	X	1	0	0	**1**
0	0	0	0	1	X	X	X	1	1	0	**1**
0	0	0	0	0	1	X	X	0	1	1	**1**
0	0	0	0	0	0	1	X	0	1	0	**1**
0	0	0	0	0	0	0	1	0	0	1	**1**

3. Mathematical Derivation of Boolean Equations

To extract the Boolean expressions, we define mutually exclusive intermediate activation signals (E_i) for each input, ensuring it only triggers if all higher-priority inputs are 0.

$$
\begin{aligned}
E_5 &= D_5 \\
E_0 &= \overline{D_5} \cdot D_0 \\
E_7 &= \overline{D_5} \cdot \overline{D_0} \cdot D_7 \\
E_4 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot D_4 \\
E_6 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot D_6 \\
E_3 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot D_3 \\
E_2 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot \overline{D_3} \cdot D_2 \\
E_1 &= \overline{D_5} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_6} \cdot \overline{D_3} \cdot \overline{D_2} \cdot D_1
\end{aligned}
$$

The **Valid Bit** (V) is 1 if any input is active:

$$V = D_5 + D_0 + D_7 + D_4 + D_6 + D_3 + D_2 + D_1$$

Most Significant Bit (Y_2): Active for D_5 (101), D_7 (111), D_4 (100), D_6 (110).

$$
\begin{aligned}
Y_2 &= E_5 + E_7 + E_4 + E_6 \\
&= D_5 + \overline{D_5 D_0} D_7 + \overline{D_5 D_0 D_7} D_4 + \overline{D_5 D_0 D_7 D_4} D_6 \\
&= D_5 + \overline{D_5 D_0} \left(D_7 + \overline{D_7} D_4 + \overline{D_7 D_4} D_6 \right) \quad \text{(Factoring)} \\
&= D_5 + \overline{D_0} \left(D_7 + D_4 + D_6 \right) \quad \text{(Using Distributive Law: } A + \overline{A}B = A + B \text{ repeatedly)}
\end{aligned}
$$

$$Y_2 = D_5 + \overline{D_0}(D_7 + D_4 + D_6)$$

Middle Bit (Y_1): Active for D_7 (111), D_6 (110), D_3 (011), D_2 (010).

$$
\begin{aligned}
Y_1 &= E_7 + E_6 + E_3 + E_2 \\
&= \overline{D_5 D_0} D_7 + \overline{D_5 D_0 D_7 D_4} D_6 + \overline{D_5 D_0 D_7 D_4 D_6} D_3 + \overline{D_5 D_0 D_7 D_4 D_6 D_3} D_2 \\
&= \overline{D_5 D_0} \left(D_7 + \overline{D_7 D_4} D_6 + \overline{D_7 D_4 D_6} D_3 + \overline{D_7 D_4 D_6 D_3} D_2 \right) \\
&= \overline{D_5 D_0} \left(D_7 + \overline{D_4}(D_6 + \overline{D_6} D_3 + \overline{D_6 D_3} D_2) \right) \quad \text{(Applying } A + \overline{A}B = A + B)
\end{aligned}
$$

$$Y_1 = \overline{D_5 D_0}(D_7 + \overline{D_4}(D_6 + D_3 + D_2))$$

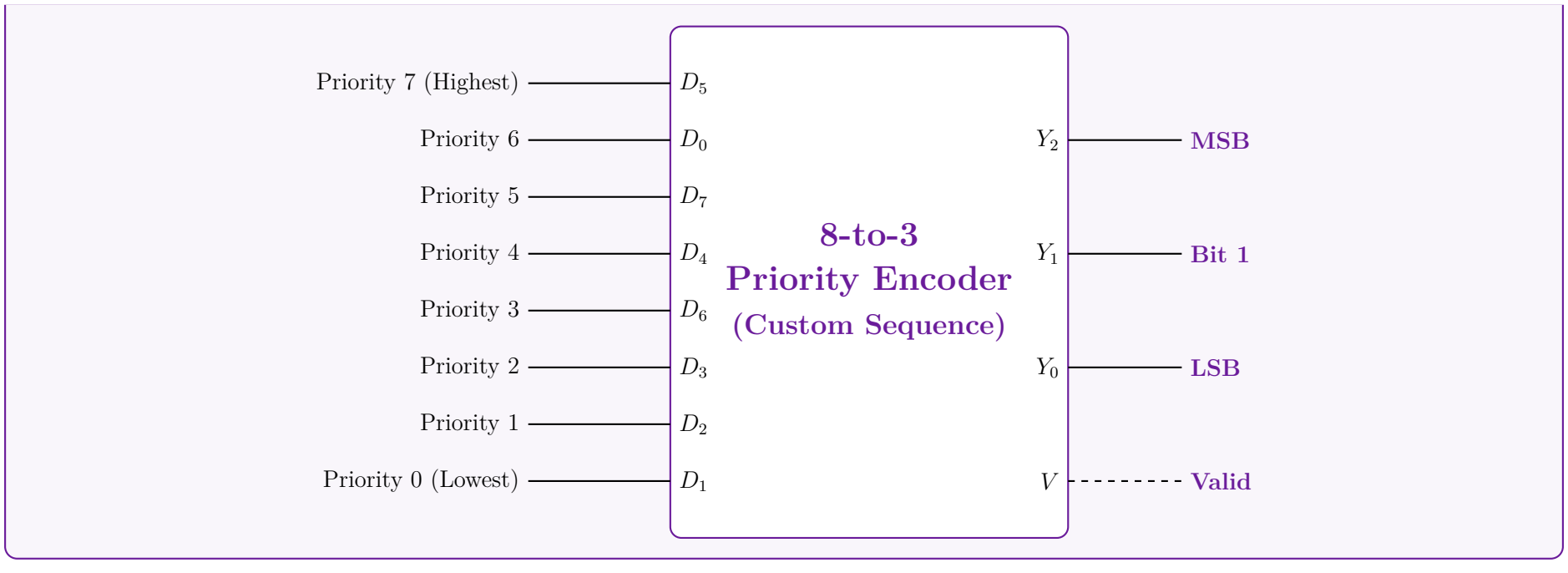
Least Significant Bit (Y_0): Active for D_5 (101), D_7 (111), D_3 (011), D_1 (001).

$$
\begin{aligned}
Y_0 &= E_5 + E_7 + E_3 + E_1 \\
&= D_5 + \overline{D_5 D_0} D_7 + \overline{D_5 D_0 D_7 D_4 D_6} D_3 + \overline{D_5 D_0 D_7 D_4 D_6 D_3 D_2} D_1 \\
&= D_5 + \overline{D_5 D_0} \left(D_7 + \overline{D_7 D_4 D_6} D_3 + \overline{D_7 D_4 D_6 D_3 D_2} D_1 \right) \\
&= D_5 + \overline{D_0} \left(D_7 + \overline{D_4 D_6}(D_3 + \overline{D_3 D_2} D_1) \right) \quad \text{(Reduction via Distributive Law)}
\end{aligned}
$$

$$Y_0 = D_5 + \overline{D_0}(D_7 + \overline{D_4 D_6}(D_3 + \overline{D_2} D_1))$$

4. Logic Circuit Implementation (Block Diagram)

The gate-level schematic is constructed directly from the simplified Boolean equations above using AND, OR, and NOT gates. Below is the structural representation of the custom encoder logic block.



Q5. Design an 8-to-3 priority encoder with the priority order: 6, 0, 7, 4, 5, 3, 1, 2 (6 has highest priority, 2 has lowest priority). **(13 marks)**

[CSE 205 | L-2/T-1 | 2017–18]

Answer

1. System Definition and Priority Mapping

An 8-to-3 priority encoder compresses eight inputs (D_0 through D_7) into a 3-bit binary output (Y_2, Y_1, Y_0) according to a strict priority hierarchy. It also generates a Valid bit (V) to indicate if any input is active.
The problem specifies an irregular priority order. We assign priority levels from 7 (highest) down to 0 (lowest). The binary output generated corresponds to the numerical index of the highest-priority active input.

Priority Level	**Input Variable**	**Active Index**	**Binary Output ($Y_2Y_1Y_0$)**
7 (Highest)	D_6	6	1 1 0
6	D_0	0	0 0 0
5	D_7	7	1 1 1
4	D_4	4	1 0 0
3	D_5	5	1 0 1
2	D_3	3	0 1 1
1	D_1	1	0 0 1
0 (Lowest)	D_2	2	0 1 0

2. Truth Table Construction

The truth table is constructed by evaluating the inputs in decreasing order of priority. An 'X' signifies a "Don't Care" condition, meaning that lower-priority inputs are completely ignored when a higher-priority input is active (Logic 1).

Inputs (Highest to Lowest Priority)								**Outputs**			
D_6 (P7)	D_0 (P6)	D_7 (P5)	D_4 (P4)	D_5 (P3)	D_3 (P2)	D_1 (P1)	D_2 (P0)	Y_2	Y_1	Y_0	V
0	0	0	0	0	0	0	0	0	0	0	**0**
1	X	X	X	X	X	X	X	1	1	0	**1**
0	1	X	X	X	X	X	X	0	0	0	**1**
0	0	1	X	X	X	X	X	1	1	1	**1**
0	0	0	1	X	X	X	X	1	0	0	**1**
0	0	0	0	1	X	X	X	1	0	1	**1**
0	0	0	0	0	1	X	X	0	1	1	**1**
0	0	0	0	0	0	1	X	0	0	1	**1**
0	0	0	0	0	0	0	1	0	1	0	**1**

3. Mathematical Derivation of Boolean Equations

To synthesize the boolean equations, we establish mutually exclusive intermediate activation signals (E_i) for each input. An input's activation signal is true only if it is active *and* all higher-priority inputs are inactive (0).

$$
\begin{aligned}
E_6 &= D_6 \\
E_0 &= \overline{D_6} \cdot D_0 \\
E_7 &= \overline{D_6} \cdot \overline{D_0} \cdot D_7 \\
E_4 &= \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot D_4 \\
E_5 &= \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot D_5 \\
E_3 &= \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot D_3 \\
E_1 &= \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_3} \cdot D_1 \\
E_2 &= \overline{D_6} \cdot \overline{D_0} \cdot \overline{D_7} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_3} \cdot \overline{D_1} \cdot D_2
\end{aligned}
$$

The **Valid Bit** (V) is 1 if any of the input signals is high:

$$V = D_6 + D_0 + D_7 + D_4 + D_5 + D_3 + D_1 + D_2$$

Most Significant Bit (Y_2): Active for inputs D_6 (110), D_7 (111), D_4 (100), D_5 (101).

$$
\begin{aligned}
Y_2 &= E_6 + E_7 + E_4 + E_5 \\
&= D_6 + \overline{D_6 D_0} D_7 + \overline{D_6 D_0 D_7} D_4 + \overline{D_6 D_0 D_7 D_4} D_5 \\
&= D_6 + \overline{D_6 D_0} \left(D_7 + \overline{D_7} D_4 + \overline{D_7 D_4} D_5 \right) \quad \text{(Factoring } \overline{D_6 D_0}\text{)} \\
&= D_6 + \overline{D_0} \left(D_7 + D_4 + D_5 \right) \quad \text{(Using Distributive Law: } A + \overline{A}B = A + B \text{ repeatedly)}
\end{aligned}
$$

$$Y_2 = D_6 + \overline{D_0}(D_7 + D_4 + D_5)$$

Middle Bit (Y_1): Active for inputs D_6 (110), D_7 (111), D_3 (011), D_2 (010).

$$
\begin{aligned}
Y_1 &= E_6 + E_7 + E_3 + E_2 \\
&= D_6 + \overline{D_6 D_0} D_7 + \overline{D_6 D_0 D_7 D_4 D_5} D_3 + \overline{D_6 D_0 D_7 D_4 D_5 D_3 D_1} D_2 \\
&= D_6 + \overline{D_6 D_0} \left[D_7 + \overline{D_7 D_4 D_5} D_3 + \overline{D_7 D_4 D_5 D_3 D_1} D_2 \right] \\
&= D_6 + \overline{D_0} \left[D_7 + \overline{D_4 D_5} \left(D_3 + \overline{D_3 D_1} D_2 \right) \right] \quad \text{(Applying } A + \overline{A}B = A + B\text{)}
\end{aligned}
$$

$$Y_1 = D_6 + \overline{D_0} \left[D_7 + \overline{D_4 D_5}(D_3 + \overline{D_1} D_2) \right]$$

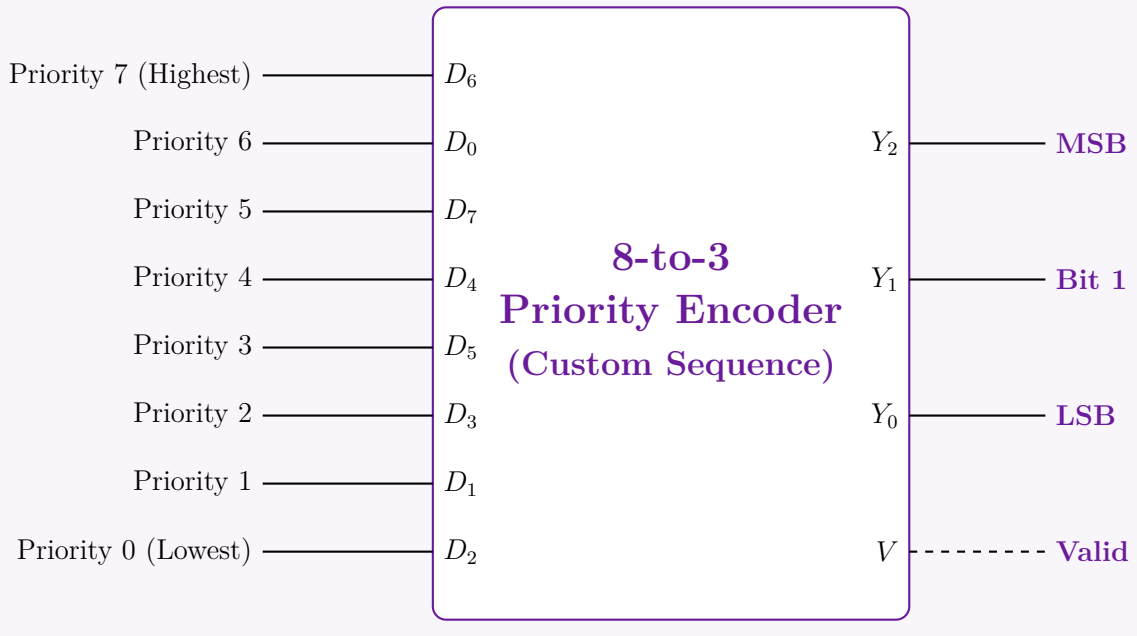
Least Significant Bit (Y_0): Active for inputs D_7 (111), D_5 (101), D_3 (011), D_1 (001).

$$
\begin{aligned}
Y_0 &= E_7 + E_5 + E_3 + E_1 \\
&= \overline{D_6 D_0} D_7 + \overline{D_6 D_0 D_7 D_4} D_5 + \overline{D_6 D_0 D_7 D_4 D_5} D_3 + \overline{D_6 D_0 D_7 D_4 D_5 D_3} D_1 \\
&= \overline{D_6 D_0} \left[D_7 + \overline{D_7 D_4} D_5 + \overline{D_7 D_4 D_5} D_3 + \overline{D_7 D_4 D_5 D_3} D_1 \right] \\
&= \overline{D_6 D_0} \left[D_7 + \overline{D_4} \left(D_5 + \overline{D_5} D_3 + \overline{D_5 D_3} D_1 \right) \right] \quad \text{(Reduction via Distributive Law)}
\end{aligned}
$$

$$Y_0 = \overline{D_6 D_0} \left[D_7 + \overline{D_4}(D_5 + D_3 + D_1) \right]$$

4. Logic Circuit Implementation (Block Diagram)

The gate-level schematic can be mapped directly from the simplified Boolean expressions using standard AND, OR, and NOT gates. Below is the structural representation of the custom encoder logic block, capturing the custom priority assignment.



Q6. "A lower-order code will get priority" — Design and explain a priority encoder with negative enable for 9 bits. **(10 marks)** [CSE

205 | L-2/T-1 | 2016–17]

Answer

1. System Definition and Analysis

A **9-to-4 Priority Encoder** is required to encode 9 input bits (D_0 through D_8) into a 4-bit binary output code (Y_3, Y_2, Y_1, Y_0). Since $2^3 < 9 \leq 2^4$, a 4-bit output is the minimum required to uniquely represent all 9 possible input activations.
The design specifications define two critical rules:

(a) **Negative Enable ($\overline{E}$)**: The encoder is active when the enable pin is logic '0'. When $\overline{E} = 1$, the encoder is disabled, forcing outputs to '0'.

(b) **Lower-order Priority**: Input D_0 possesses the highest priority, and D_8 possesses the lowest priority.

To distinguish between the state where D_0 is selected (outputting 0000) and the state where no inputs are active or the chip is disabled, we introduce a **Valid bit (V)**. $V = 1$ if the chip is enabled and at least one input is active.

2. Truth Table Construction

In the table below, 'X' denotes a "Don't Care" condition. When a higher-priority input (lower index) is active ('1'), the states of lower-priority inputs (higher indices) do not affect the output.

Enable	**Inputs (Highest Priority → Lowest Priority)**									**Outputs**				
$\overline{E}$	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8	Y_3	Y_2	Y_1	Y_0	V
1	X	X	X	X	X	X	X	X	X	0	0	0	0	**0**
0	0	0	0	0	0	0	0	0	0	0	0	0	0	**0**
0	1	X	X	X	X	X	X	X	X	0	0	0	0	**1**
0	0	1	X	X	X	X	X	X	X	0	0	0	1	**1**
0	0	0	1	X	X	X	X	X	X	0	0	1	0	**1**
0	0	0	0	1	X	X	X	X	X	0	0	1	1	**1**
0	0	0	0	0	1	X	X	X	X	0	1	0	0	**1**
0	0	0	0	0	0	1	X	X	X	0	1	0	1	**1**
0	0	0	0	0	0	0	1	X	X	0	1	1	0	**1**
0	0	0	0	0	0	0	0	1	X	0	1	1	1	**1**
0	0	0	0	0	0	0	0	0	1	1	0	0	0	**1**

3. Mathematical Derivation of Boolean Equations

Let us define an active-high internal enable signal $En = \overline{(\overline{E})}$. To prevent logic hazards and simplify the expressions, we define mutually exclusive intermediate activation signals (A_i) for each input. An input A_i is true only if the encoder is enabled, D_i is active, and all higher-priority inputs (D_0 to D_{i-1}) are inactive.

$$
\begin{aligned}
A_0 &= En \cdot D_0 \\
A_1 &= En \cdot \overline{D_0} \cdot D_1 \\
A_2 &= En \cdot \overline{D_0} \cdot \overline{D_1} \cdot D_2 \\
A_3 &= En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot D_3 \\
A_4 &= En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot D_4 \\
A_5 &= En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot D_5 \\
A_6 &= En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot D_6 \\
A_7 &= En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_6} \cdot D_7 \\
A_8 &= En \cdot \overline{D_0} \cdot \overline{D_1} \cdot \overline{D_2} \cdot \overline{D_3} \cdot \overline{D_4} \cdot \overline{D_5} \cdot \overline{D_6} \cdot \overline{D_7} \cdot D_8
\end{aligned}
$$

Using these intermediate signals, we map the bits of the binary output corresponding to the active index.
Valid Bit (V): Asserted if the chip is enabled and any input is active.

$$V = En \cdot (D_0 + D_1 + D_2 + D_3 + D_4 + D_5 + D_6 + D_7 + D_8)$$

Most Significant Bit (Y_3): Logic 1 only for decimal index 8 (1000_2).

$$Y_3 = A_8$$

Bit 2 (Y_2): Logic 1 for indices 4, 5, 6, and 7.

$$Y_2 = A_4 + A_5 + A_6 + A_7$$

Bit 1 (Y_1): Logic 1 for indices 2, 3, 6, and 7.

$$Y_1 = A_2 + A_3 + A_6 + A_7$$

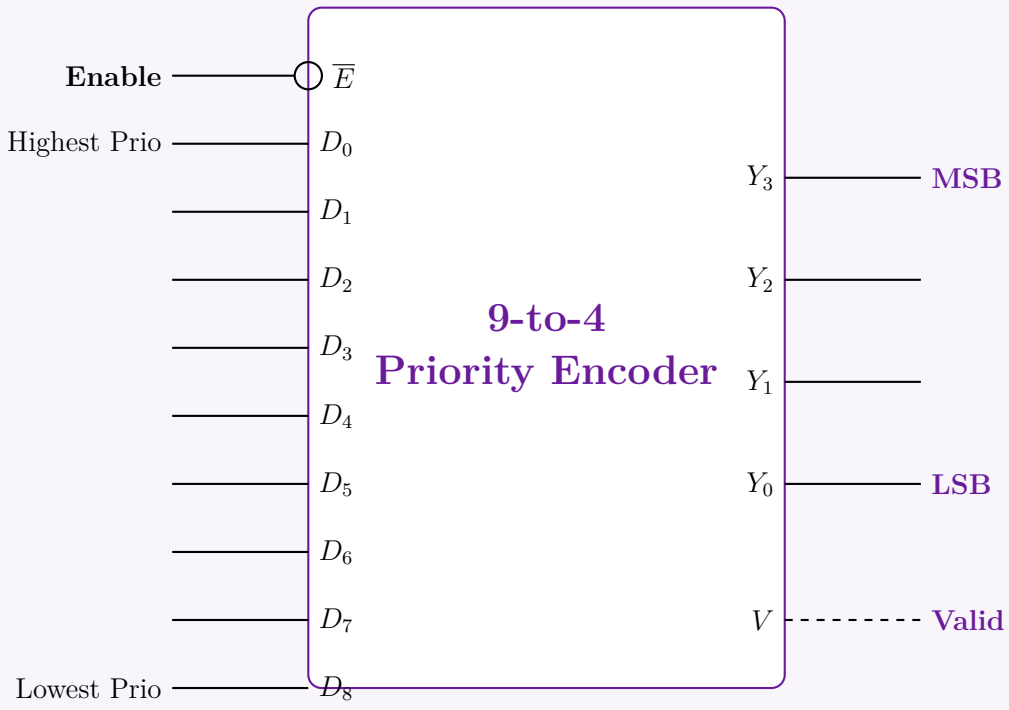
Least Significant Bit (Y_0): Logic 1 for odd indices 1, 3, 5, and 7.

$$Y_0 = A_1 + A_3 + A_5 + A_7$$

Note: A_8 is not included in Y_2, Y_1, Y_0 because decimal 8 (1000_2) sets those bits to 0.

4. Logic Structural Diagram

Below is the block diagram representation of the encoder highlighting the priority hierarchy and the negative enable input.



Q7. Design a full adder with a decoder and basic logic gates. **(12 marks)** **[CSE 205 | L-2/T-1 | 2014–15]**

Answer

1. Problem Analysis and Truth Table

A Full Adder is a combinational arithmetic circuit that adds three single-bit numbers: A, B, and a carry-in C_{in}. It produces two outputs: a Sum (S) and a Carry-out (C_{out}).
To design this using a decoder, we first construct the truth table to determine the active minterms for each output.

Inputs			**Outputs**		**Minterm (m_i)**
A	B	C_{in}	S (Sum)	C_{out} (Carry)	
0	0	0	0	0	m_0
0	0	1	1	0	m_1
0	1	0	1	0	m_2
0	1	1	0	1	m_3
1	0	0	1	0	m_4
1	0	1	0	1	m_5
1	1	0	0	1	m_6
1	1	1	1	1	m_7

2. Boolean Expressions in Minterm Form

From the truth table, we extract the standard Sum of Products (SOP) form for both outputs by identifying the rows where the output is logic ‘1‘.

$$S(A, B, C_{in}) = \sum m(1, 2, 4, 7)$$
$$C_{out}(A, B, C_{in}) = \sum m(3, 5, 6, 7)$$

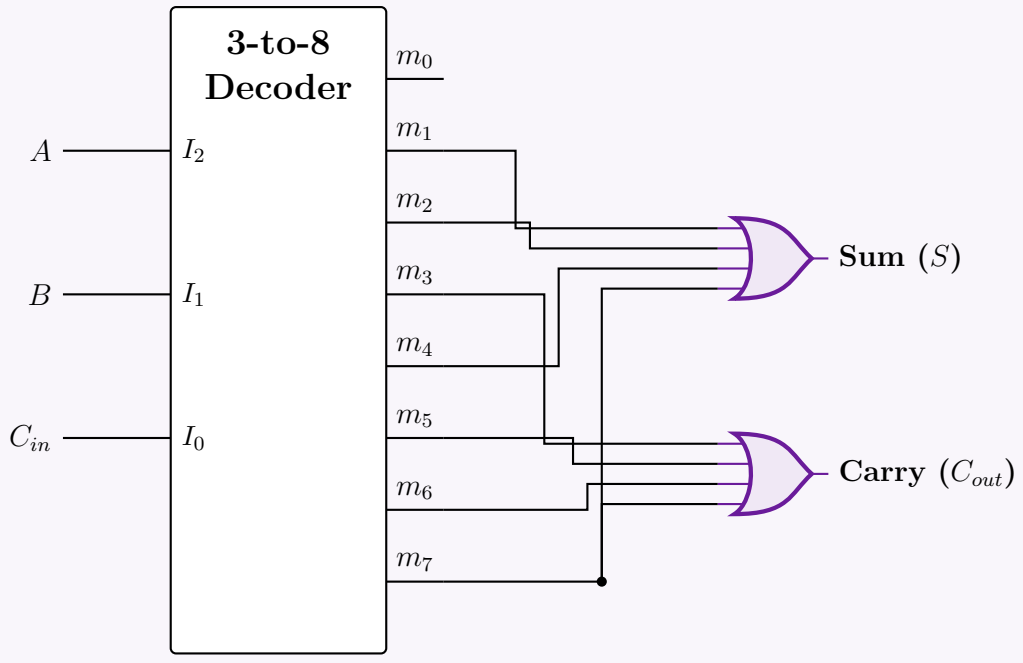
3. Decoder Implementation Strategy

A 3-to-8 line decoder activates exactly one of its 8 output lines (representing minterms m_0 to m_7) based on the 3-bit binary input. By connecting the inputs A, B, C_{in} to the select lines of the decoder, the decoder automatically generates all 8 minterms.
To implement the Full Adder logic:

- We use a **4-input OR gate** to logically sum the activated minterms m_1, m_2, m_4, and m_7 to generate the **Sum** (S).
- We use a second **4-input OR gate** to sum the activated minterms m_3, m_5, m_6, and m_7 to generate the **Carry** (C_{out}).

4. Logic Circuit Diagram

Below is the complete schematic utilizing a 3-to-8 decoder and standard OR gates.



Q8. An Octal-to-Binary Encoder may cause ambiguities if more than one input is simultaneously set to 1. How can this ambiguity be resolved? **(4 marks)** **[CSE 205 | L-2/T-1 | 2014–15]**

Answer

1. Understanding the Ambiguity in a Standard Encoder

A standard 8-to-3 (Octal-to-Binary) encoder assumes that exactly one input is active at any given time. If this assumption is violated and multiple inputs are set to Logic '1' simultaneously, two major ambiguities arise:

(a) **Invalid Output Generation:** The standard encoder uses simple OR gates to generate the outputs. For instance, input D_3 produces the binary output 011_2, and input D_5 produces 101_2. If both D_3 and D_5 are HIGH simultaneously, the hardware performs a bitwise OR operation:

$$011_2 \text{ (from } D_3) \vee 101_2 \text{ (from } D_5) = 111_2$$

The output 111_2 corresponds to D_7, falsely indicating that the seventh input is active, even though it is not.

(b) **Zero-State Ambiguity:** A standard encoder outputs 000_2 when input D_0 is HIGH. However, it also outputs 000_2 when *all* inputs are LOW. There is no way to distinguish whether D_0 is genuinely active or if the system is simply idle.

2. Resolving the Ambiguity: The Priority Encoder

To resolve these issues, we replace the standard encoder with a **Priority Encoder**. A priority encoder imposes a strict hierarchy among the inputs and introduces a validation signal:

- **Priority Hierarchy:** If two or more inputs are HIGH at the same time, the input with the highest designated priority takes precedence. The binary output will correspond solely to this highest-priority active input, ignoring all lower-priority inputs. (Usually, the input with the highest subscript, D_7, is given the highest priority).
- **Valid Bit (V):** An additional output line, V, is introduced. $V = 1$ if one or more inputs are active, and $V = 0$ if all inputs are 0. This resolves the zero-state ambiguity for D_0.

3. Priority Encoder Truth Table

Assuming D_7 has the highest priority and D_0 has the lowest priority, we can construct the truth table. The symbol 'X' represents a "Don't Care" condition, clearly demonstrating that lower-priority inputs are ignored when a higher-priority input is active.

Inputs (Highest to Lowest Priority)								**Outputs**			
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	Y_2	Y_1	Y_0	V **(Valid)**
0	0	0	0	0	0	0	0	0	0	0	**0**
1	X	X	X	X	X	X	X	1	1	1	**1**
0	1	X	X	X	X	X	X	1	1	0	**1**
0	0	1	X	X	X	X	X	1	0	1	**1**
0	0	0	1	X	X	X	X	1	0	0	**1**
0	0	0	0	1	X	X	X	0	1	1	**1**
0	0	0	0	0	1	X	X	0	1	0	**1**
0	0	0	0	0	0	1	X	0	0	1	**1**
0	0	0	0	0	0	0	1	0	0	0	**1**

4. Boolean Equations for the Priority Encoder

By applying Boolean logic to the truth table, we can derive the equations that physically resolve the hardware ambiguity. An input is only acknowledged if all higher-priority inputs are FALSE ($\overline{D_n}$).
Valid Bit (V):

$$V = D_7 + D_6 + D_5 + D_4 + D_3 + D_2 + D_1 + D_0$$

Most Significant Bit (Y_2):

$$Y_2 = D_7 + \overline{D_7}D_6 + \overline{D_7 D_6}D_5 + \overline{D_7 D_6 D_5}D_4$$
$$Y_2 = D_7 + D_6 + D_5 + D_4 \quad \text{(Simplified via } A + \overline{A}B = A + B\text{)}$$

Middle Bit (Y_1):

$$Y_1 = D_7 + \overline{D_7}D_6 + \overline{D_7 D_6 D_5 D_4}D_3 + \overline{D_7 D_6 D_5 D_4 D_3}D_2$$
$$Y_1 = D_7 + D_6 + \overline{D_5 D_4}D_3 + \overline{D_5 D_4}D_2$$

Least Significant Bit (Y_0):

$$Y_0 = D_7 + \overline{D_7 D_6}D_5 + \overline{D_7 D_6 D_5 D_4}D_3 + \overline{D_7 D_6 D_5 D_4 D_3 D_2}D_1$$
$$Y_0 = D_7 + \overline{D_6}D_5 + \overline{D_6 D_4}D_3 + \overline{D_6 D_4 D_2}D_1$$

By incorporating these negated terms ($\overline{D_n}$), the combinational circuit inherently locks out lower-order inputs whenever a higher-order input is detected, successfully resolving both multiple-input and zero-state ambiguities.

Q9. What is a priority encoder? Design a four-input priority encoder with inputs I_0, I_1, I_2, I_3, where the priority order is $I_2 > I_3 > I_1 > I_0$. **(10 marks)** **[CSE 205 | L-2/T-1 | 2013–14]**

Answer

1. Definition of a Priority Encoder

A **priority encoder** is a combinational logic circuit that resolves multiple simultaneous active inputs by assigning a predefined priority level to each input. If two or more inputs are asserted (Logic '1') at the same time, the encoder generates a binary output corresponding solely to the input with the highest designated priority, ignoring all lower-priority inputs. It typically includes a **Valid bit (V)** output to differentiate between an active input mapping to zero (e.g., 00_2) and the idle state where no inputs are active.

2. Priority Mapping and Truth Table

For this 4-to-2 priority encoder, the priority order is explicitly given as $I_2 > I_3 > I_1 > I_0$. The output binary code (Y_1, Y_0) will correspond to the subscript index of the highest-priority active input.

- **Highest Priority (I_2):** Outputs 10_2 (decimal 2)
- **Second Priority (I_3):** Outputs 11_2 (decimal 3)
- **Third Priority (I_1):** Outputs 01_2 (decimal 1)
- **Lowest Priority (I_0):** Outputs 00_2 (decimal 0)

We construct the truth table listing inputs in descending order of priority. An 'X' represents a "Don't Care" condition, emphasizing that lower-order inputs are ignored.

Inputs (Highest → Lowest)				**Outputs**		
I_2 (P3)	I_3 (P2)	I_1 (P1)	I_0 (P0)	Y_1	Y_0	V
0	0	0	0	0	0	**0**
1	X	X	X	1	0	**1**
0	1	X	X	1	1	**1**
0	0	1	X	0	1	**1**
0	0	0	1	0	0	**1**

3. Mathematical Derivation of Boolean Equations

Valid Bit (V): The valid bit is HIGH if any of the inputs are HIGH.

$$V = I_2 + I_3 + I_1 + I_0$$

Most Significant Bit (Y_1): From the truth table, Y_1 is '1' for the rows where I_2 is active or I_3 is active (and I_2 is not).

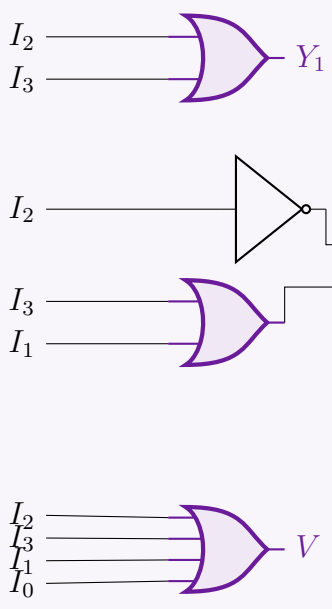
$$\begin{aligned} Y_1 &= I_2 + \overline{I_2} I_3 \\ Y_1 &= I_2 + I_3 \quad \text{(Applying the Distributive Law: } A + \overline{A}B = A + B\text{)} \end{aligned}$$

Least Significant Bit (Y_0): Y_0 is '1' for the row where I_3 is active (with $I_2 = 0$), and the row where I_1 is active (with $I_2 = 0$ and $I_3 = 0$).

$$\begin{aligned} Y_0 &= \overline{I_2} I_3 + \overline{I_2}\,\overline{I_3} I_1 \\ &= \overline{I_2}(I_3 + \overline{I_3} I_1) \quad \text{(Factoring out } \overline{I_2}\text{)} \\ Y_0 &= \overline{I_2}(I_3 + I_1) \quad \text{(Applying } A + \overline{A}B = A + B \text{ inside the parenthesis)} \end{aligned}$$

4. Logic Circuit Implementation

The derived equations use minimal standard logic gates. We require one NOT gate, two OR gates, and one AND gate for the output mapping, alongside a 4-input OR gate for the valid bit.



Q10. You need a full adder for an experiment but in the lab you could only find plenty of 3×8 decoders. Can any such decoder be used as a full adder? Explain your answer with a truth table. **(15 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

1. Direct Answer and Theoretical Justification

Yes, a 3×8 decoder can be used to implement a full adder, provided that we also have access to basic logic gates (specifically, OR gates) to combine the outputs.
Reasoning: An $n \times 2^n$ decoder functions as a universal minterm generator for n input variables. Since a full adder is a combinational circuit with 3 inputs (A, B, C_{in}), a 3×8 decoder will generate all $2^3 = 8$ possible minterms (m_0 to m_7). Any Boolean function of 3 variables can be expressed in standard Sum of Products (SOP) form. Therefore, by logically ORing the specific minterms that produce a logic '1' for the Sum and Carry outputs, we can fully realize the full adder circuit.

2. Truth Table Analysis

To determine which minterms to combine, we construct the truth table for a full adder. It adds two significant bits (A, B) and a carry-in bit (C_{in}), producing a Sum (S) and a Carry-out (C_{out}).

Inputs			**Outputs**		**Minterm Designation**
A	B	C_{in}	**Sum** (S)	**Carry** (C_{out})	
0	0	0	0	0	m_0
0	0	1	**1**	0	m_1
0	1	0	**1**	0	m_2
0	1	1	0	**1**	m_3
1	0	0	**1**	0	m_4
1	0	1	0	**1**	m_5
1	1	0	0	**1**	m_6
1	1	1	**1**	**1**	m_7

3. Boolean Minterm Equations

From the truth table, we extract the Canonical Sum of Products (SOP) form by identifying the rows where the respective outputs evaluate to logic '1'.
Sum Output (S):

$$S(A, B, C_{in}) = \sum m(1, 2, 4, 7)$$
$$S = m_1 + m_2 + m_4 + m_7$$

Carry Output (C_{out}):

$$C_{out}(A, B, C_{in}) = \sum m(3, 5, 6, 7)$$
$$C_{out} = m_3 + m_5 + m_6 + m_7$$

4. Logic Circuit Implementation

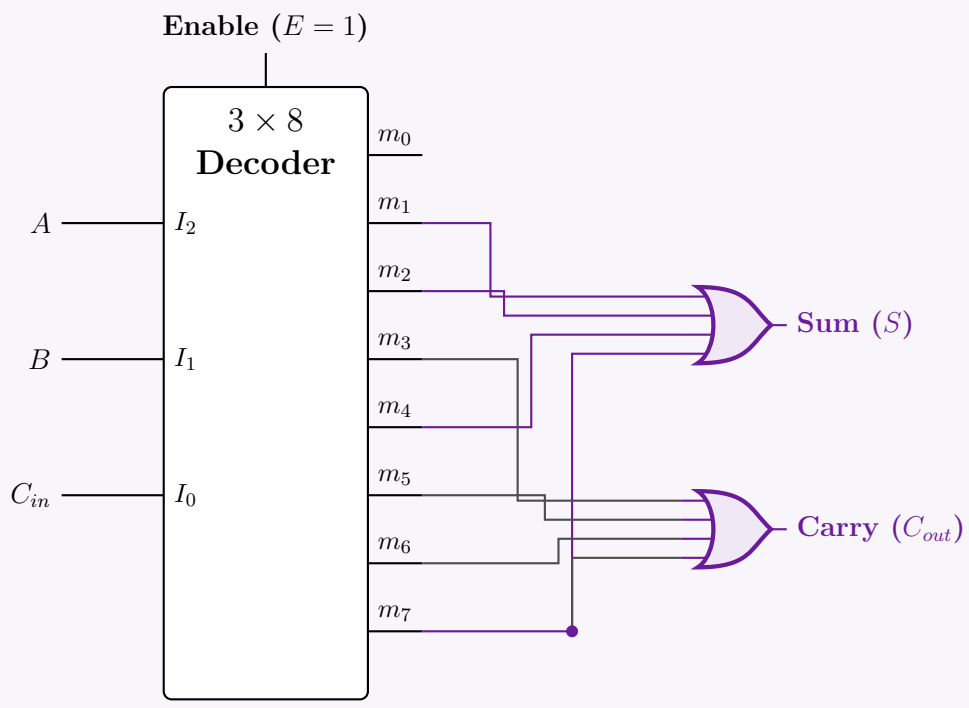
To implement the circuit:

(a) Connect the inputs A, B, and C_{in} to the select lines I_2, I_1, and I_0 of the 3×8 decoder.

(b) Assume the decoder has an active-high enable ($E = 1$) and active-high outputs.

(c) Route the output pins corresponding to m_1, m_2, m_4, and m_7 into a 4-input OR gate to generate the **Sum** (S).

(d) Route the output pins corresponding to m_3, m_5, m_6, and m_7 into a second 4-input OR gate to generate the **Carry** (C_{out}).



Note: If the 3×8 decoder in the lab features active-low outputs (such as the standard 74LS138), NAND gates must be used in place of the OR gates to properly assert the Sum and Carry logic levels according to De Morgan's Laws.

Q11. Design a 4-input priority encoder and show the gate-level circuit diagram. **(12 marks)** **[CSE 205 | L-2/T-1 | 2011–12, 2013–14]**

Answer

1. Priority Specification and Truth Table

A 4-input priority encoder has four inputs (D_3, D_2, D_1, D_0) and outputs a binary code (Y_1, Y_0) indicating the active input with the highest priority. It also includes a Valid bit (V) that is logic '1' when at least one input is active.
We will assume the standard priority ordering where D_3 **has the highest priority** and D_0 **has the lowest priority**. If a higher-priority input is active, the states of all lower-priority inputs are treated as "Don't Cares" (denoted by 'X').

Inputs				**Outputs**		
D_3 (Highest)	D_2	D_1	D_0 (Lowest)	Y_1	Y_0	V (**Valid**)
0	0	0	0	X	X	**0**
1	X	X	X	1	1	**1**
0	1	X	X	1	0	**1**
0	0	1	X	0	1	**1**
0	0	0	1	0	0	**1**

2. Boolean Logic Derivation

We derive the Boolean equations from the truth table by identifying the conditions where each output is '1'.
Valid Bit (V): The valid bit is HIGH if any of the four inputs is HIGH.

$$V = D_3 + D_2 + D_1 + D_0$$

Most Significant Output Bit (Y_1): Y_1 is HIGH when D_3 is HIGH, or when D_2 is HIGH (provided D_3 is LOW).

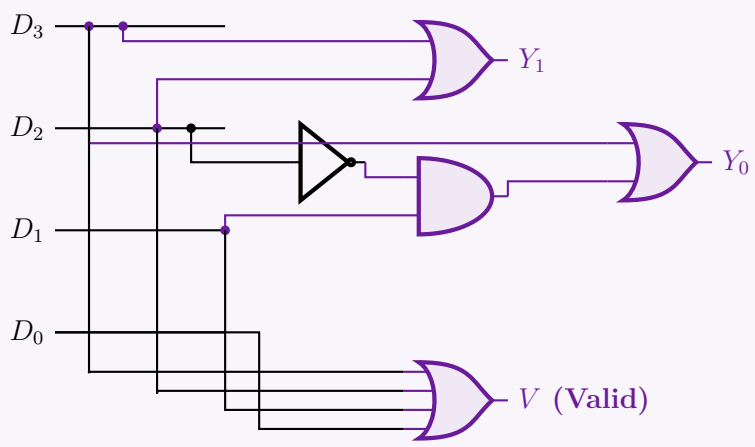
$$Y_1 = D_3 + \overline{D_3} D_2$$
$$Y_1 = D_3 + D_2 \quad \text{(Applied Distributive Law: } A + \overline{A}B = A + B\text{)}$$

Least Significant Output Bit (Y_0): Y_0 is HIGH when D_3 is HIGH, or when D_1 is HIGH (provided both D_3 and D_2 are LOW).

$$Y_0 = D_3 + \overline{D_3 D_2} D_1$$
$$Y_0 = D_3 + \overline{D_2} D_1 \quad \text{(Applied Distributive Law: } A + \overline{A}X = A + X\text{, where } X = \overline{D_2}D_1\text{)}$$

3. Gate-Level Circuit Diagram

Using the simplified Boolean equations, the priority encoder can be realized using basic logic gates (NOT, AND, OR).



Q12. Design a 3×8 decoder using two 2×4 decoder circuits. **(8 marks)** **[CSE 205 | L-2/T-1 | 2011–12]**

Answer

1. Design Principle

A 3×8 decoder takes 3 input lines (A_2, A_1, A_0) and activates exactly one of its 8 output lines (Y_0 to Y_7) based on the binary value of the inputs. We can construct this by cascading two 2×4 decoders (each possessing an Enable pin, E) using the following logic partition:

- **Common Inputs (LSBs):** The two least significant bits, A_1 and A_0, are connected in parallel to the selection inputs of both 2×4 decoders. They dictate which of the four outputs within an active decoder is asserted.
- **Selection Input (MSB):** The most significant bit, A_2, acts as a block selector. We connect A_2 directly to the enable pin of the second decoder and route it through a NOT gate to the enable pin of the first decoder.

- **Operation:**
  - When $A_2 = 0$, the **Top Decoder** is enabled. It handles input combinations 000_2 to 011_2, asserting outputs Y_0 through Y_3.
  - When $A_2 = 1$, the **Bottom Decoder** is enabled. It handles input combinations 100_2 to 111_2, asserting outputs Y_4 through Y_7.

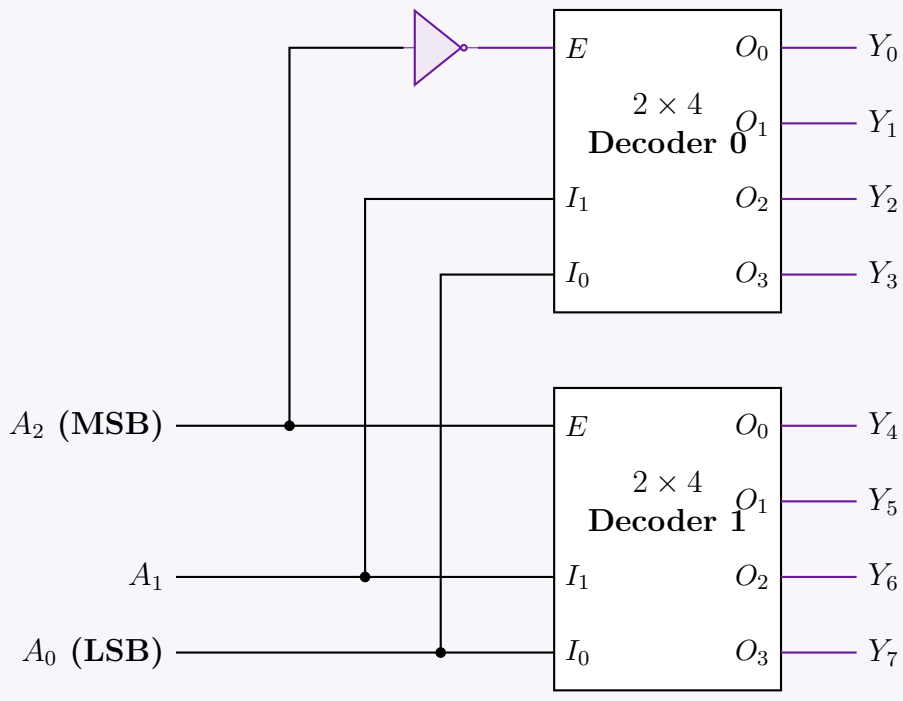
2. Truth Table Partitioning

The truth table clearly demonstrates how the MSB (A_2) partitions the outputs into two mutually exclusive sets, mapping perfectly to the two 2×4 decoders.

Inputs			Outputs								Active Hardware Block
A_2	A_1	A_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7	
0	0	0	**1**	0	0	0	0	0	0	0	Decoder 0 ($E = \overline{A_2}$)
0	0	1	0	**1**	0	0	0	0	0	0	
0	1	0	0	0	**1**	0	0	0	0	0	
0	1	1	0	0	0	**1**	0	0	0	0	
1	0	0	0	0	0	0	**1**	0	0	0	Decoder 1 ($E = A_2$)
1	0	1	0	0	0	0	0	**1**	0	0	
1	1	0	0	0	0	0	0	0	**1**	0	
1	1	1	0	0	0	0	0	0	0	**1**	

3. Circuit Diagram

Assumption: The 2×4 decoders utilize active-high Enable (E) pins and active-high outputs.



Function Implementation using Decoder

Q1. Design a 1-bit full subtractor using a 4 to 16 line decoder that has active low outputs and two active low enables $\overline{EN_1}$ and $\overline{EN_2}$. **(12 marks)** **[CSE 205 | L-2/T-1 | 2024–25]**

Answer

1. Full Subtractor Truth Table and Minterm Extraction

A 1-bit full subtractor subtracts a subtrahend bit (Y) and a borrow-in bit (B_{in}) from a minuend bit (X), producing a Difference (D) and a Borrow-out (B_{out}). We begin by defining its truth table.

Inputs			**Outputs**		**Minterm**
X (Minuend)	Y (Subtrahend)	B_{in} (Borrow In)	**Difference (D)**	**Borrow Out (B_{out})**	
0	0	0	0	0	m_0
0	0	1	**1**	**1**	m_1
0	1	0	**1**	**1**	m_2
0	1	1	0	**1**	m_3
1	0	0	**1**	0	m_4
1	0	1	0	0	m_5
1	1	0	0	0	m_6
1	1	1	**1**	**1**	m_7

2. Boolean Expressions and De Morgan's Law Application

From the truth table, we extract the canonical Sum of Products (SOP) expressions for D and B_{out}:

$$D(X, Y, B_{in}) = \sum m(1, 2, 4, 7) = m_1 + m_2 + m_4 + m_7$$
$$B_{out}(X, Y, B_{in}) = \sum m(1, 2, 3, 7) = m_1 + m_2 + m_3 + m_7$$

The problem specifies that the decoder has **active-low outputs**. This means the decoder outputs are the complements of the standard minterms ($\overline{m_0}, \overline{m_1}, \ldots, \overline{m_{15}}$).
To implement a logical OR operation using active-low inputs, we apply **De Morgan's Law** ($\overline{A \cdot B} = \overline{A} + \overline{B}$). By feeding the active-low outputs into a **NAND gate**, we reconstruct the SOP form:

$$D = m_1 + m_2 + m_4 + m_7 = \overline{\overline{m_1} \cdot \overline{m_2} \cdot \overline{m_4} \cdot \overline{m_7}}$$
$$\Rightarrow D = \text{NAND}(\overline{m_1}, \overline{m_2}, \overline{m_4}, \overline{m_7})$$

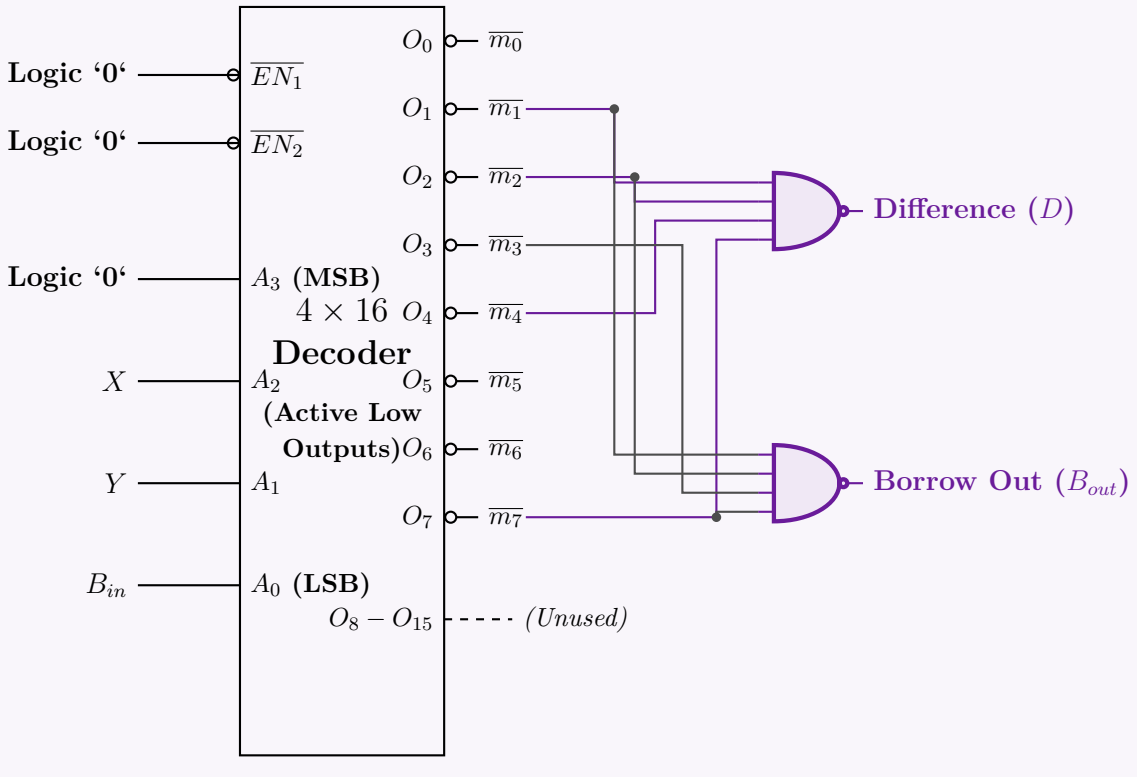
$$B_{out} = m_1 + m_2 + m_3 + m_7 = \overline{\overline{m_1} \cdot \overline{m_2} \cdot \overline{m_3} \cdot \overline{m_7}}$$
$$\Rightarrow B_{out} = \text{NAND}(\overline{m_1}, \overline{m_2}, \overline{m_3}, \overline{m_7})$$

3. Decoder Configuration

To adapt a 4×16 decoder into our required 3×8 configuration:

- **Data Inputs:** Connect the subtractor inputs X, Y, B_{in} to the lower three decoder inputs A_2, A_1, A_0.
- **MSB Grounding:** Tie the most significant bit input (A_3) to Logic '0' (Ground). This disables outputs $\overline{m_8}$ through $\overline{m_{15}}$, restricting operation to the first 8 outputs.
- **Enable Pins:** Connect the two active-low enables ($\overline{EN_1}$ and $\overline{EN_2}$) to Logic '0' (Ground) to keep the chip permanently active.

4. Logic Circuit Implementation



Q2. Design a 4-to-16-line decoder by using only the minimum number of 2-to-4-line decoders. Assume that the 2-to-4-line decoders have an enable input ('1' = enabled) but the designed 4-to-16-line decoder does not have an enable input. **(12 marks)** **[CSE 205 | L-2/T-1 | 2023–24]**

Answer

1. Design Strategy and Component Count

To design a 4-to-16-line decoder, we must decode a 4-bit binary input (A_3, A_2, A_1, A_0) into 16 distinct outputs (D_0 to D_{15}). Given that we are restricted to 2-to-4-line decoders (which have 2 inputs, 4 outputs, and 1 enable pin), we need to determine the minimum number of components:

- **Output Stage:** To generate 16 outputs, we need at least $16 \div 4 = 4$ decoders.
- **Input/Decoding Stage (Root):** To select which of these 4 output decoders is active at any given time, we need a mechanism to decode the two Most Significant Bits (MSBs). This requires exactly **1** additional 2-to-4-line decoder.

Therefore, the **minimum number of 2-to-4-line decoders required is 5**.

2. Input and Output Mapping

We partition the 4-bit input $A_3A_2A_1A_0$ as follows:

- **Least Significant Bits (A_1, A_0):** These are connected in parallel to the I_1 and I_0 inputs of all four output-stage decoders. They determine which specific line (0 to 3) is activated *within* the enabled decoder.
- **Most Significant Bits (A_3, A_2):** These are connected to the I_1 and I_0 inputs of the root decoder. The root decoder will assert exactly one of its outputs based on A_3 and A_2.
- **Enable Logic (E):** The outputs of the root decoder (O_0, O_1, O_2, O_3) are connected to the Enable (E) inputs of the four output-stage decoders, respectively. Because the final 4-to-16-line decoder is specified to have **no enable input**, the root decoder's enable input is permanently tied to **Logic '1' (HIGH)**.

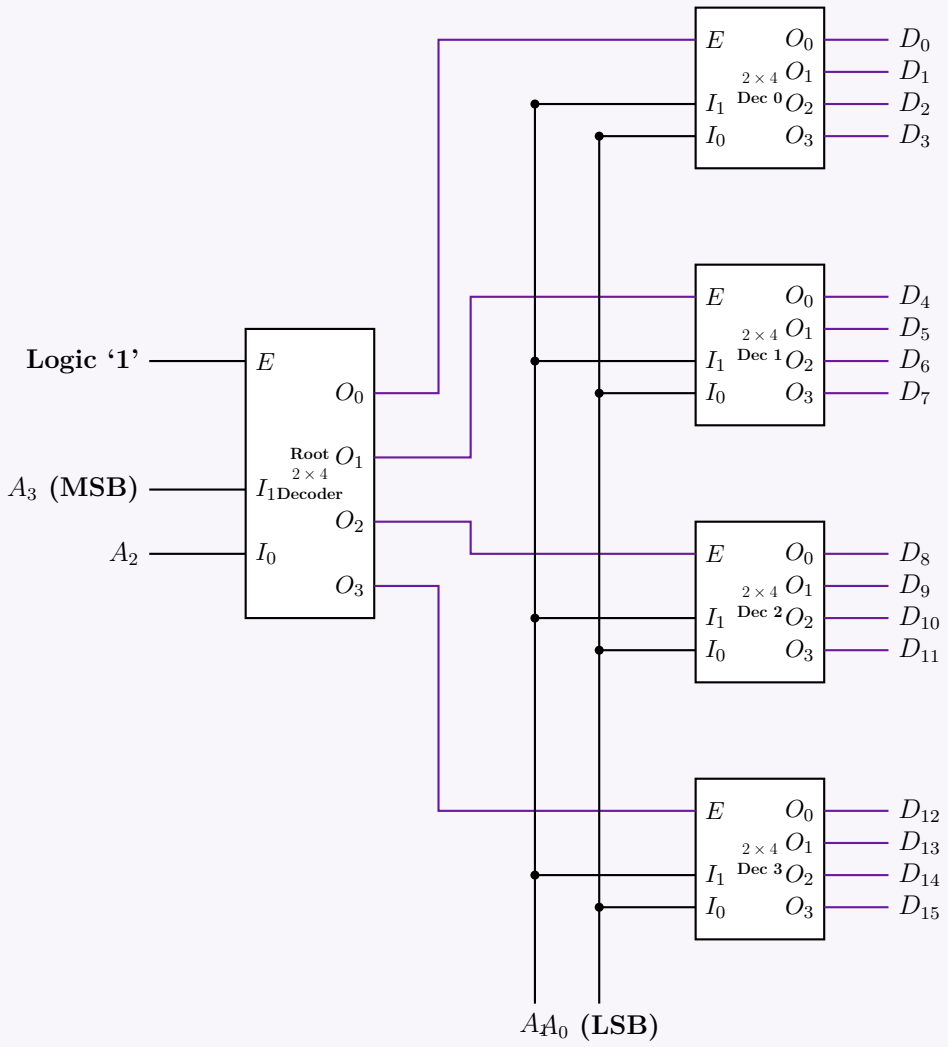
3. Truth Table Partitioning

The functional partition behaves according to the state of the MSBs:

A_3	A_2	**Active Output Decoder**	**Active Output Range (based on A_1, A_0)**
0	0	**Decoder 0** (Root $O_0 = 1$)	D_0, D_1, D_2, D_3
0	1	**Decoder 1** (Root $O_1 = 1$)	D_4, D_5, D_6, D_7
1	0	**Decoder 2** (Root $O_2 = 1$)	D_8, D_9, D_{10}, D_{11}
1	1	**Decoder 3** (Root $O_3 = 1$)	$D_{12}, D_{13}, D_{14}, D_{15}$

4. Logic Circuit Implementation

Note: All 2-to-4-line decoders shown below feature active-high inputs and outputs, and an active-high Enable (E) pin as per the problem description.



Q3. Design a full adder circuit using a 3×8 decoder and two OR gates. **(10 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Answer

1. Full Adder Truth Table and Minterm Extraction

A Full Adder is a combinational logic circuit that performs the arithmetic sum of three bits: two significant bits (A and B) and a carry-in bit (C_{in}). It produces two outputs: a Sum (S) and a Carry-out (C_{out}).
To design the circuit using a decoder, we first map the input-output relationship using a truth table and extract the canonical minterms.

Inputs			**Outputs**		**Minterm**
A	B	C_{in}	**Sum** (S)	**Carry** (C_{out})	
0	0	0	0	0	m_0
0	0	1	**1**	0	m_1
0	1	0	**1**	0	m_2
0	1	1	0	**1**	m_3
1	0	0	**1**	0	m_4
1	0	1	0	**1**	m_5
1	1	0	0	**1**	m_6
1	1	1	**1**	**1**	m_7

2. Boolean Logic Derivation

A 3×8 decoder generates all $2^3 = 8$ possible minterms for 3 input variables. Each output pin of the decoder represents exactly one minterm. Any Boolean function can be expressed in the Canonical Sum of Products (SOP) form. Thus, by identifying the minterms where the outputs are logic '1', we can form the functions:
Sum Output (S):

$$S(A, B, C_{in}) = \sum m(1, 2, 4, 7)$$
$$S = m_1 + m_2 + m_4 + m_7$$

Carry Output (C_{out}):

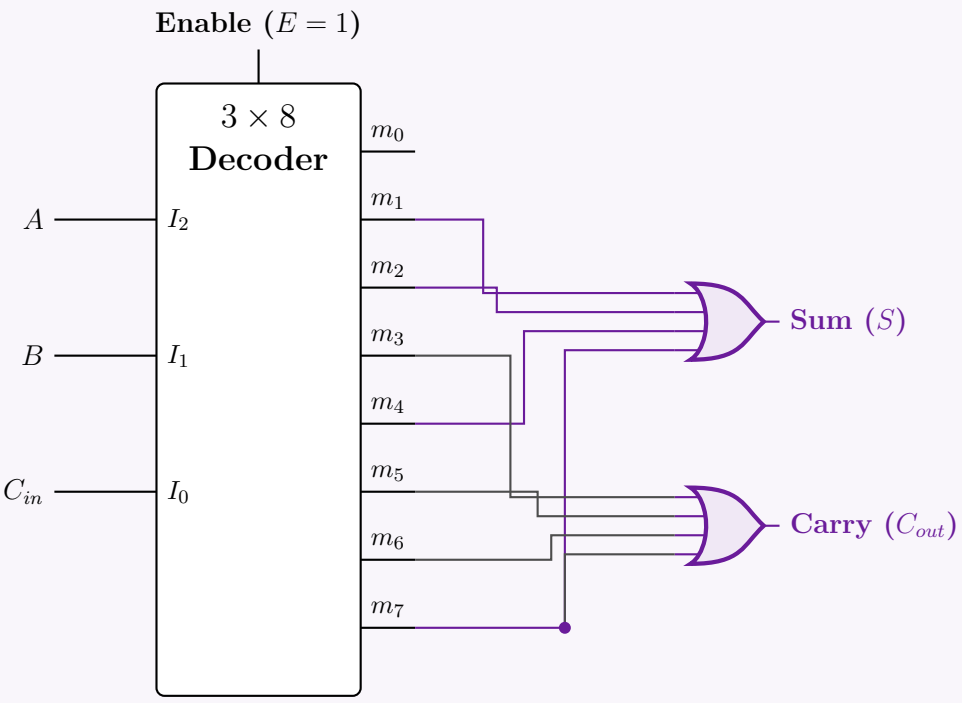
$$C_{out}(A, B, C_{in}) = \sum m(3, 5, 6, 7)$$
$$C_{out} = m_3 + m_5 + m_6 + m_7$$

3. Circuit Implementation Strategy

The implementation requires a 3×8 decoder and two 4-input OR gates:

- Connect the inputs A, B, and C_{in} to the select lines I_2, I_1, and I_0 of the decoder, respectively.
- Tie the enable pin (E) of the decoder to logic HIGH ('1').
- Route the outputs corresponding to m_1, m_2, m_4, and m_7 to the first OR gate to yield the **Sum** (S).
- Route the outputs corresponding to m_3, m_5, m_6, and m_7 to the second OR gate to yield the **Carry** (C_{out}).

4. Gate-Level Circuit Diagram



Q4. Implement $F(x, y, z) = \Sigma\,(3, 5, 6, 7)$ using an active low 3×8 decoder and AND gates. **(10 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Truth Table and Boolean Function Analysis

We are given the Boolean function $F(x, y, z) = \Sigma\,(3, 5, 6, 7)$. Let us first analyze the truth table to identify the minterms (where $F = 1$) and maxterms (where $F = 0$).

Inputs			Output F	Minterm Notation
x	y	z		
0	0	0	0	m_0
0	0	1	0	m_1
0	1	0	0	m_2
0	1	1	**1**	m_3
1	0	0	0	m_4
1	0	1	**1**	m_5
1	1	0	**1**	m_6
1	1	1	**1**	m_7

2. Mathematical Derivation for Active-Low Decoder

A standard sum-of-products (SOP) expression for F uses the active minterms:

$$\begin{aligned} F(x, y, z) &= \Sigma\, m(3, 5, 6, 7) \\ &= m_3 + m_5 + m_6 + m_7 \end{aligned}$$

However, we must implement this using a decoder with **active-low outputs** and **AND gates**. An active-low decoder outputs the *complement* of the minterms (i.e., $O_i = \overline{m_i}$).
To match the available hardware (AND gates and complemented minterms), we can express the function F using its Product of Sums (POS) form, which groups the *maxterms* (the rows where $F = 0$):

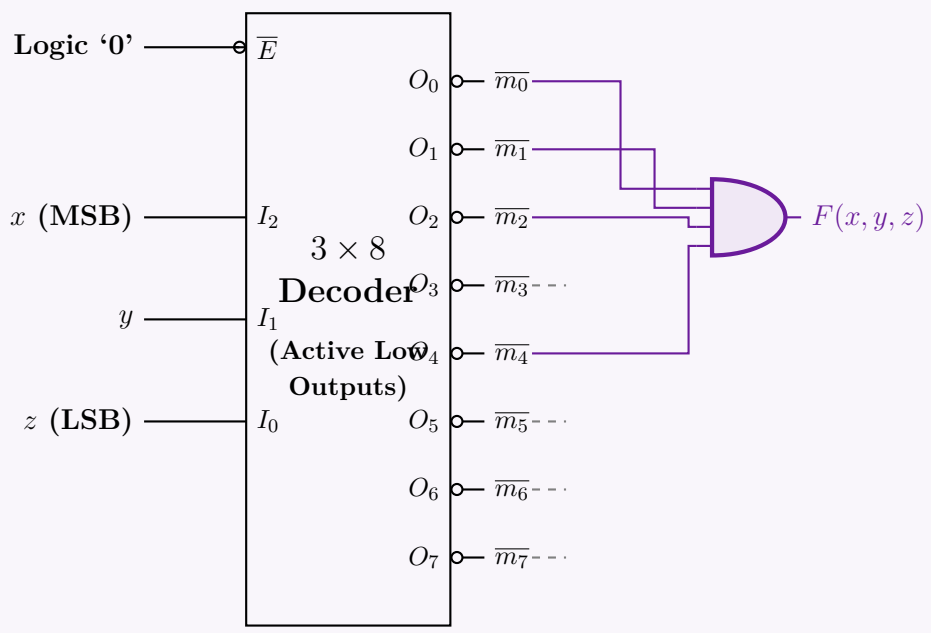
$$\begin{aligned} F(x, y, z) &= \Pi\, M(0, 1, 2, 4) \\ &= M_0 \cdot M_1 \cdot M_2 \cdot M_4 \quad \text{(Product of Maxterms)} \end{aligned}$$

By definition, a maxterm is the exact complement of a minterm ($M_i = \overline{m_i}$). Substituting this relationship into our equation yields:

$$F(x, y, z) = \overline{m_0} \cdot \overline{m_1} \cdot \overline{m_2} \cdot \overline{m_4}$$

Conclusion: Since the active-low decoder directly outputs $\overline{m_0}, \overline{m_1}, \overline{m_2}$, and $\overline{m_4}$, we simply connect these specific four output lines to a single **4-input AND gate** to realize the function F.

3. Logic Circuit Implementation



Q5. Design a logic circuit which takes two 2-bit numbers X_1X_0 and Y_1Y_0 and finds their sum using a 2×4 line decoder with inputs I_0, I_1, active-low outputs O_3, O_2, O_1, O_0, and active-low enables $\overline{EN_1}$ and $\overline{EN_2}$. (Use the minimum number of extra logic gates.) **(15 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Answer

1. Functional Partitioning of the 2-Bit Adder

The addition of two 2-bit numbers, X_1X_0 and Y_1Y_0, yields a 3-bit sum $S_2S_1S_0$. We can partition this operation into two distinct stages:

- **LSB Stage (Half Adder):** Adds X_0 and Y_0 to produce Sum S_0 and Carry-out C_1. We will implement this stage using the provided 2×4 decoder to minimize extra gates.
- **MSB Stage (Full Adder):** Adds X_1, Y_1, and C_1 to produce Sum S_1 and the final Carry-out S_2. This will be implemented using standard logic gates.

2. LSB Implementation using the 2×4 Decoder

The 2×4 decoder has inputs I_0, I_1 and active-low outputs O_0, O_1, O_2, O_3. To enable the decoder, both active-low enables ($\overline{EN_1}$ and $\overline{EN_2}$) must be tied to Logic '0' (Ground).
We connect the LSBs to the decoder inputs: $I_0 = X_0$ and $I_1 = Y_0$. The decoder generates the active-low minterms of these variables: $O_i = \overline{m_i}$.
The Half Adder Boolean equations are:

$$\text{Sum: } S_0 = X_0 \oplus Y_0 = m_1 + m_2$$
$$\text{Carry: } C_1 = X_0 \cdot Y_0 = m_3$$

Using De Morgan's Law ($\overline{A \cdot B} = \overline{A} + \overline{B}$), we can synthesize these using the active-low decoder outputs with minimum extra gates:

$$S_0 = \overline{\overline{m_1} \cdot \overline{m_2}} = \text{NAND}(O_1, O_2) \quad \text{(Requires 1 NAND gate)}$$
$$C_1 = \overline{\overline{m_3}} = \text{NOT}(O_3) \quad \text{(Requires 1 NOT gate)}$$

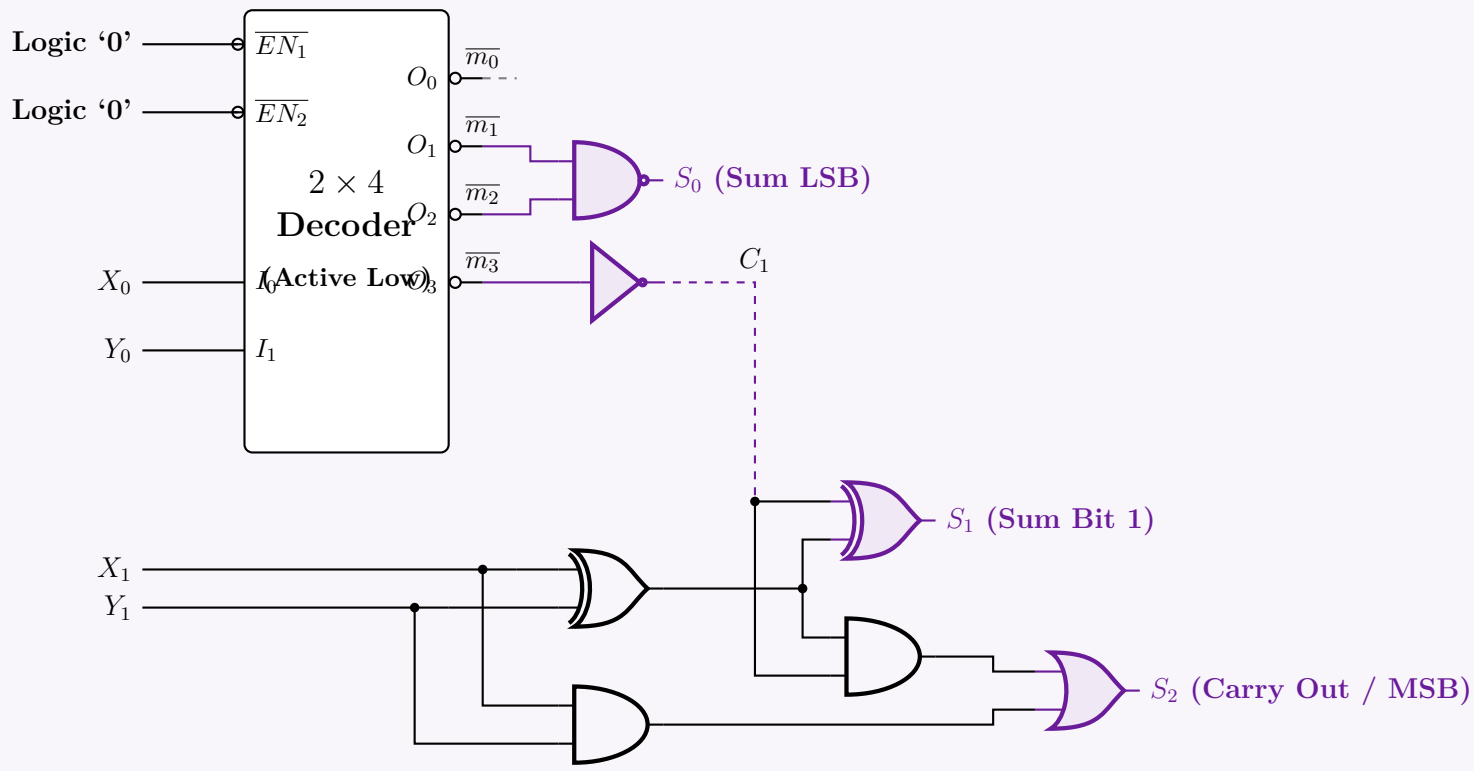
3. MSB Implementation (Full Adder)

For the MSB, we use standard logic gates. To achieve minimum gate count, we share the XOR product $(X_1 \oplus Y_1)$:

$$S_1 = (X_1 \oplus Y_1) \oplus C_1$$
$$S_2 = \text{Carry-out} = (X_1 \cdot Y_1) + C_1 \cdot (X_1 \oplus Y_1)$$

This requires 2 XOR gates, 2 AND gates, and 1 OR gate.

4. Complete Logic Circuit Diagram



Q6. Design a logic circuit for $f(w, x, y, z) = \Sigma\,(0, 2, 6, 7, 10, 11, 12, 14)$ with a 2×4 line decoder that has inputs A, B, active-low enables $\overline{EN_1}$ and $\overline{EN_2}$. (Use the minimum number of extra logic gates.) **(14 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Answer

1. Shannon Expansion & Functional Partitioning

To implement the 4-variable function $f(w, x, y, z) = \Sigma\,(0, 2, 6, 7, 10, 11, 12, 14)$ using a 2×4 line decoder, we must strategically partition the function. A 2×4 decoder yields the four minterms of its two input variables. By assigning inputs $A = y$ and $B = z$, the decoder's active-high outputs will generate $\overline{y}\overline{z}$, $\overline{y}z$, $y\overline{z}$, and yz.
We apply **Shannon's Expansion Theorem** to expand $f(w, x, y, z)$ around variables y and z:

$$f(w, x, y, z) = \overline{y}\overline{z} \cdot f(w, x, 0, 0) + \overline{y}z \cdot f(w, x, 0, 1) + y\overline{z} \cdot f(w, x, 1, 0) + yz \cdot f(w, x, 1, 1)$$

2. Evaluating the Residual Functions

We extract the active minterms from the function for each combination of (y, z):

- **For $y = 0, z = 0$ (Decoder Output O_0):**
  Minterms where $y = 0, z = 0$ are $0, 4, 8, 12$. In our function, only minterms 0(0000) and 12(1100) are present.
  $$f(w, x, 0, 0) = \overline{w}\overline{x} + wx = w \odot x \quad \text{(XNOR)}$$
- **For $y = 0, z = 1$ (Decoder Output O_1):**
  Minterms where $y = 0, z = 1$ are $1, 5, 9, 13$. None of these exist in f.
  $$f(w, x, 0, 1) = 0$$
- **For $y = 1, z = 0$ (Decoder Output O_2):**
  Minterms where $y = 1, z = 0$ are $2, 6, 10, 14$. All of these minterms $(0010, 0110, 1010, 1110)$ are present in f.
  $$f(w, x, 1, 0) = 1$$
- **For $y = 1, z = 1$ (Decoder Output O_3):**
  Minterms where $y = 1, z = 1$ are $3, 7, 11, 15$. In our function, only minterms 7(0111) and 11(1011) are present.
  $$f(w, x, 1, 1) = \overline{w}x + w\overline{x} = w \oplus x \quad \text{(XOR)}$$

3. Boolean Logic Minimization

Substituting these residual functions back into the Shannon expansion gives:

$$f(w, x, y, z) = (\overline{y}\overline{z}) \cdot (w \odot x) + (\overline{y}z) \cdot (0) + (y\overline{z}) \cdot (1) + (yz) \cdot (w \oplus x)$$

Let the outputs of the 2×4 decoder be $O_0 = \overline{y}\overline{z}$, $O_1 = \overline{y}z$, $O_2 = y\overline{z}$, and $O_3 = yz$. The equation maps directly to the decoder pins:

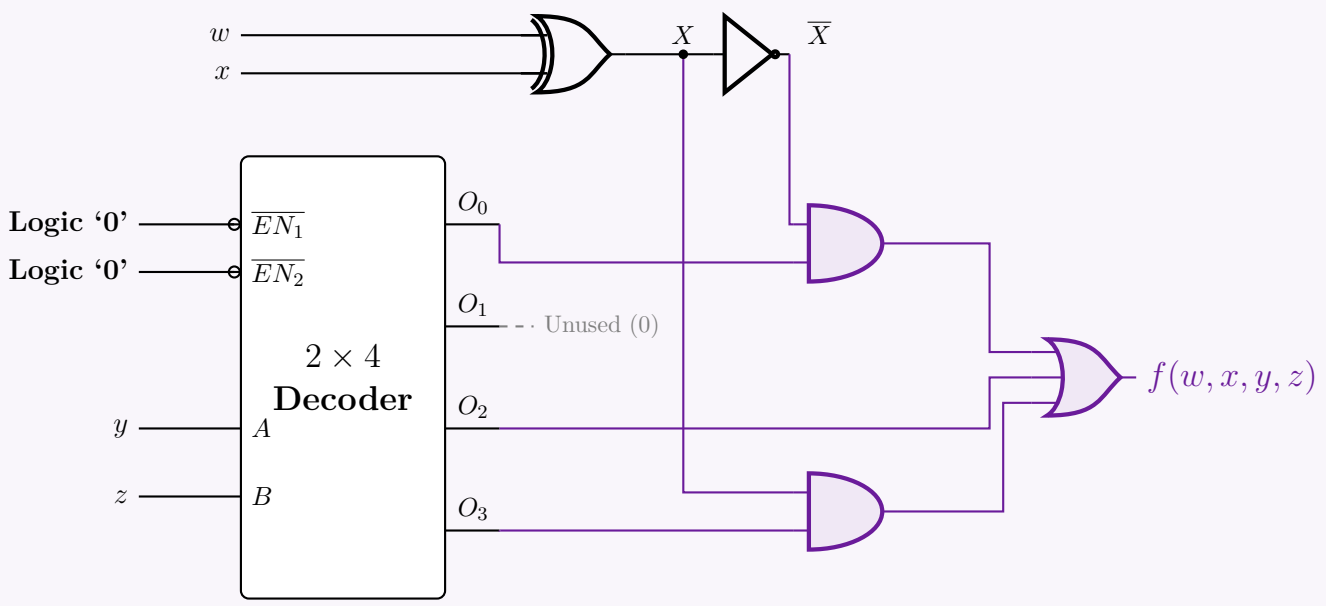
$$f(w, x, y, z) = O_0 \cdot (w \odot x) + O_2 + O_3 \cdot (w \oplus x)$$

To use the **minimum number of extra logic gates**, we note the complementary relationship: $(w \odot x) = \overline{w \oplus x}$. We can generate intermediate variable $X = w \oplus x$ using one XOR gate, and branch it through a NOT gate to yield $\overline{X} = w \odot x$.

$$f_{final} = O_0 \cdot \overline{X} + O_2 + O_3 \cdot X$$

Gate Count: 1 XOR, 1 NOT, 2 ANDs, and 1 3-input OR. The active-low enables $(\overline{EN_1}, \overline{EN_2})$ must be tied to Ground ('Logic 0') to keep the decoder permanently active.

4. Logic Circuit Implementation



Q7. Design a logic circuit for $f(w, x, y, z) = \text{SOP}(0, 2, 6, 7, 8, 10, 11, 13, 14)$ with a 2×4 line decoder that has inputs A, B, active-low enable $\overline{EN}$ and active-low outputs D'_0, D'_1, D'_2, D'_3. (Use the minimum number of extra logic gates.) **(12 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

1. Functional Partitioning via Shannon's Expansion

To implement the 4-variable boolean function $f(w, x, y, z)$ using a 2×4 line decoder, we partition the function using two variables. We choose the least significant variables, y and z, as the decoder inputs ($A = y$, $B = z$).
The decoder has active-low outputs D'_0, D'_1, D'_2, D'_3. These correspond to the complemented minterms of (y, z):

$$D'_0 = \overline{\overline{y}\,\overline{z}} \qquad D'_1 = \overline{\overline{y}\,z} \qquad D'_2 = \overline{y\,\overline{z}} \qquad D'_3 = \overline{y\,z}$$

Applying Shannon's Expansion Theorem around y and z:

$$\begin{aligned} f(w, x, y, z) &= (\overline{y}\,\overline{z}) \cdot I_0(w, x) + (\overline{y}\,z) \cdot I_1(w, x) + (y\,\overline{z}) \cdot I_2(w, x) + (y\,z) \cdot I_3(w, x) \\ &= \overline{D'_0} \cdot I_0(w, x) + \overline{D'_1} \cdot I_1(w, x) + \overline{D'_2} \cdot I_2(w, x) + \overline{D'_3} \cdot I_3(w, x) \end{aligned}$$

2. Evaluating the Residual Functions $I_i(w, x)$

We evaluate $f(w, x, y, z) = \sum(0, 2, 6, 7, 8, 10, 11, 13, 14)$ for each combination of y and z:

- **For $y = 0, z = 0$ (Minterms $0, 4, 8, 12$):**
  Given in f: $\{0, 8\}$.

$$I_0(w, x) = \overline{w}\,\overline{x} + w\overline{x} = \overline{x}(\overline{w} + w) = \overline{x} \quad \text{(Distributive \& Complement Laws)}$$

- **For $y = 0, z = 1$ (Minterms $1, 5, 9, 13$):**
  Given in f: $\{13\}$.

$$I_1(w, x) = wx$$

- **For $y = 1, z = 0$ (Minterms $2, 6, 10, 14$):**
  Given in f: $\{2, 6, 10, 14\}$.

$$I_2(w, x) = \overline{w}\,\overline{x} + \overline{w}x + w\overline{x} + wx = 1 \quad \text{(Identity)}$$

- **For $y = 1, z = 1$ (Minterms $3, 7, 11, 15$):**
  Given in f: $\{7, 11\}$.

$$I_3(w, x) = \overline{w}x + w\overline{x} = w \oplus x \quad \text{(XOR definition)}$$

3. Gate Minimization using De Morgan's Laws

Substitute the residual functions back into the expanded equation:

$$f(w, x, y, z) = \overline{D'_0} \cdot \overline{x} + \overline{D'_1} \cdot (wx) + \overline{D'_2} \cdot (1) + \overline{D'_3} \cdot (w \oplus x)$$

To minimize hardware with active-low decoder outputs, we apply **De Morgan's Involution Law** ($\overline{\overline{f}} = f$) to convert the outer Sum-of-Products into a NAND-NAND equivalent structure:

$$f(w, x, y, z) = \overline{\overline{\overline{D'_0} \cdot \overline{x}} \cdot \overline{\overline{D'_1} \cdot (wx)} \cdot \overline{\overline{D'_2}} \cdot \overline{\overline{D'_3} \cdot (w \oplus x)}}$$

Applying De Morgan's Law ($\overline{A \cdot B} = \overline{A} + \overline{B}$) to the inner terms:

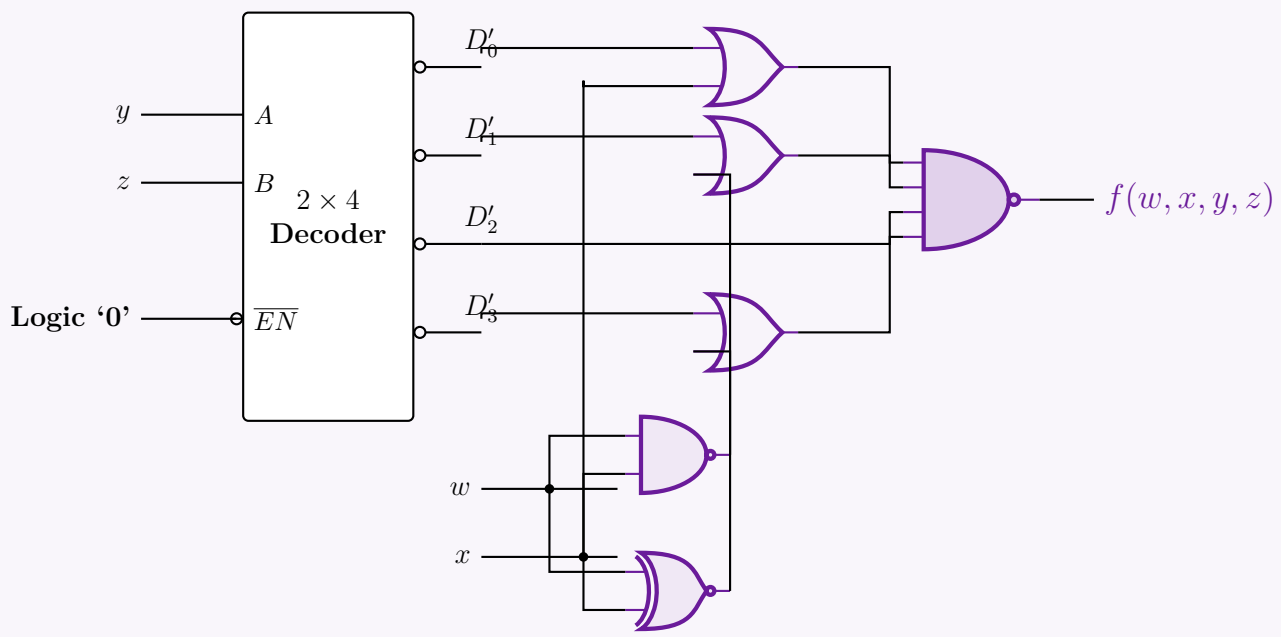
$$\overline{\overline{D'_0} \cdot \overline{x}} = D'_0 + x$$
$$\overline{\overline{D'_1} \cdot (wx)} = D'_1 + \overline{wx} = D'_1 + (w \text{ NAND } x)$$
$$\overline{\overline{D'_2}} = D'_2$$
$$\overline{\overline{D'_3} \cdot (w \oplus x)} = D'_3 + \overline{w \oplus x} = D'_3 + (w \text{ XNOR } x)$$

Final optimized Boolean equation mapping directly to gates:

$$f(w,x,y,z) = \text{NAND}_4\Big((D'_0 + x),\ (D'_1 + \text{NAND}(w,x)),\ D'_2,\ (D'_3 + \text{XNOR}(w,x))\Big)$$

Gate Count: 1 NAND2, 1 XNOR2, 3 OR2, and 1 NAND4 gate.

4. Logic Circuit Implementation



Q8. Using a 2×4 line decoder, design a half subtractor. (Decoder outputs D_0, D_1, D_2, D_3 are active high; enable $\overline{EN}$ is active low.) **(10 marks)** **[CSE 205 | L-2/T-1 | 2017–18]**

Answer

1. Half Subtractor Functional Analysis

A half subtractor is a combinational circuit that performs the subtraction of two single-bit numbers, yielding a Difference (D) and a Borrow (B). Let the minuend be X and the subtrahend be Y.

Truth Table:

Inputs		Outputs	
X	Y	Difference (D)	Borrow (B)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

From the truth table, we can extract the standard Sum-of-Products (SOP) expressions in terms of minterms $m(X,Y)$:

$$D(X,Y) = \overline{X}Y + X\overline{Y} = \Sigma m(1,2)$$
$$B(X,Y) = \overline{X}Y = \Sigma m(1)$$

2. Decoder Implementation Strategy

A 2×4 line decoder with active-high outputs takes two inputs (let's map $A = X, B = Y$) and asserts exactly one of its four outputs D_0, D_1, D_2, D_3 corresponding to the decimal equivalent of the binary input.
The active-high outputs generate the canonical minterms of the input variables:

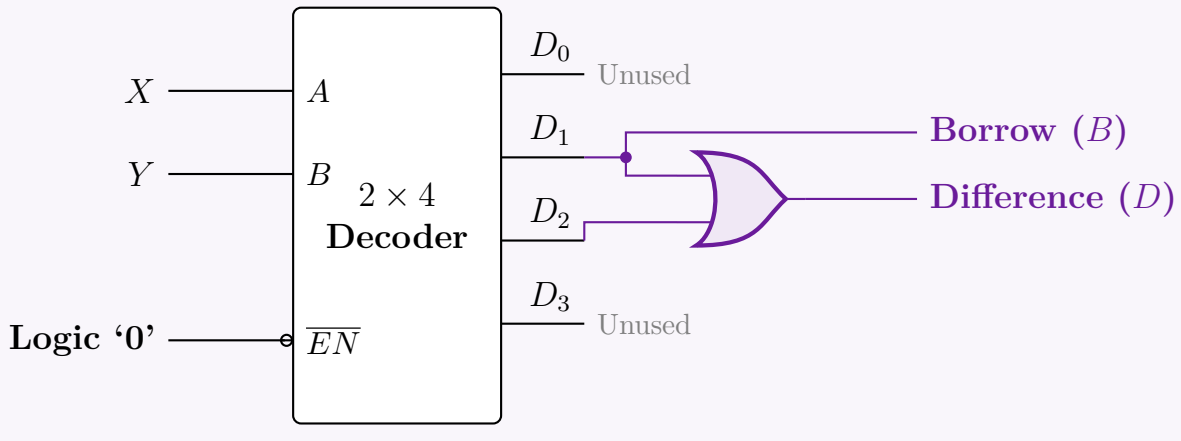
$$D_0 = m_0 = \overline{X}\,\overline{Y}$$
$$D_1 = m_1 = \overline{X}Y$$
$$D_2 = m_2 = X\overline{Y}$$
$$D_3 = m_3 = XY$$

By substituting the decoder outputs into the boolean expressions for the half subtractor, we obtain:

$$Difference(D) = D_1 + D_2 \qquad \text{(Requires one 2-input OR gate)}$$
$$Borrow(B) = D_1 \qquad \text{(Direct connection, no gates required)}$$

The decoder has an active-low enable ($\overline{EN}$). For the decoder to function normally, it must be permanently enabled by tying this pin to Logic '0' (Ground).

3. Logic Circuit Implementation



Q9. Using only a 2×4 line decoder, solve the function $f(A, B, C, D) = \pi\,(3, 5, 9, 15, 12, 4, 1)$. The 2×4 line decoder has an active-low enable and active-low outputs. (Use the minimum number of gates if required.) **(10 marks)** **[CSE 205 | L-2/T-1 | 2016–17]**

Answer

1. Standard Form Conversion (POS to SOP)

The given function is in Product of Sums (POS) form, specified by its maxterms:

$$f(A, B, C, D) = \pi\,(1, 3, 4, 5, 9, 12, 15)$$

For a 4-variable boolean function, the possible minterms/maxterms range from 0 to 15. The set of maxterms and minterms are mutually exclusive and collectively exhaustive. Therefore, we can express the function in its Sum of Products (SOP) form by listing the missing indices:

$$f(A, B, C, D) = \Sigma\, m(0, 2, 6, 7, 8, 10, 11, 13, 14)$$

2. Functional Partitioning via Shannon's Expansion

We use a 2×4 line decoder, requiring two select inputs. Let us partition the function using the least significant variables, C and D, as the decoder inputs ($S_1 = C$, $S_0 = D$).
The decoder features an active-low enable ($\overline{EN}$) and active-low outputs (D'_0, D'_1, D'_2, D'_3). When enabled ($\overline{EN} = 0$), the active-low outputs correspond to the complemented minterms of (C, D):

$$D'_0 = \overline{\overline{C}\,\overline{D}} \qquad D'_1 = \overline{\overline{C}D} \qquad D'_2 = \overline{C\overline{D}} \qquad D'_3 = \overline{CD}$$

Applying Shannon's Expansion Theorem with respect to C and D:

$$\begin{aligned} f(A, B, C, D) &= (\overline{C}\,\overline{D}) \cdot I_0(A, B) + (\overline{C}D) \cdot I_1(A, B) + (C\overline{D}) \cdot I_2(A, B) + (CD) \cdot I_3(A, B) \\ &= \overline{D'_0} \cdot I_0(A, B) + \overline{D'_1} \cdot I_1(A, B) + \overline{D'_2} \cdot I_2(A, B) + \overline{D'_3} \cdot I_3(A, B) \end{aligned}$$

3. Evaluating the Residual Functions $I_i(A, B)$

We group the SOP minterms $\{0, 2, 6, 7, 8, 10, 11, 13, 14\}$ based on the states of C and D:

- **For $C = 0, D = 0$ (D'_0 active):** Corresponds to minterms $\{0, 4, 8, 12\}$.
  Present in f: $\{0, 8\}$.

$$I_0(A, B) = \overline{A}\,\overline{B} + A\overline{B} = \overline{B}(\overline{A} + A) = \overline{B} \quad \text{(Distributive \& Complement Laws)}$$

- **For $C = 0, D = 1$ (D'_1 active):** Corresponds to minterms $\{1, 5, 9, 13\}$.
  Present in f: $\{13\}$.

$$I_1(A, B) = AB$$

- **For $C = 1, D = 0$ (D'_2 active):** Corresponds to minterms $\{2, 6, 10, 14\}$.
  Present in f: $\{2, 6, 10, 14\}$.

$$I_2(A, B) = \overline{A}\,\overline{B} + \overline{A}B + A\overline{B} + AB = 1 \quad \text{(Universal Coverage)}$$

- **For $C = 1, D = 1$ (D'_3 active):** Corresponds to minterms $\{3, 7, 11, 15\}$.
  Present in f: $\{7, 11\}$.

$$I_3(A, B) = \overline{A}B + A\overline{B} = A \oplus B \quad \text{(XOR Definition)}$$

4. Gate Minimization via De Morgan's Laws

Substitute the derived residual functions back into the expanded equation:

$$f(A,B,C,D) = \overline{D'_0} \cdot \overline{B} + \overline{D'_1} \cdot (AB) + \overline{D'_2} \cdot (1) + \overline{D'_3} \cdot (A \oplus B)$$

Because our decoder outputs are active-low, we use **De Morgan's Involution Law** ($\overline{\overline{f}} = f$) to shift the logic into an active-low/NAND framework, minimizing extra gates:

$$f(A,B,C,D) = \overline{\overline{\overline{D'_0} \cdot \overline{B}} \cdot \overline{\overline{D'_1} \cdot (AB)} \cdot \overline{\overline{D'_2}} \cdot \overline{\overline{D'_3} \cdot (A \oplus B)}}$$

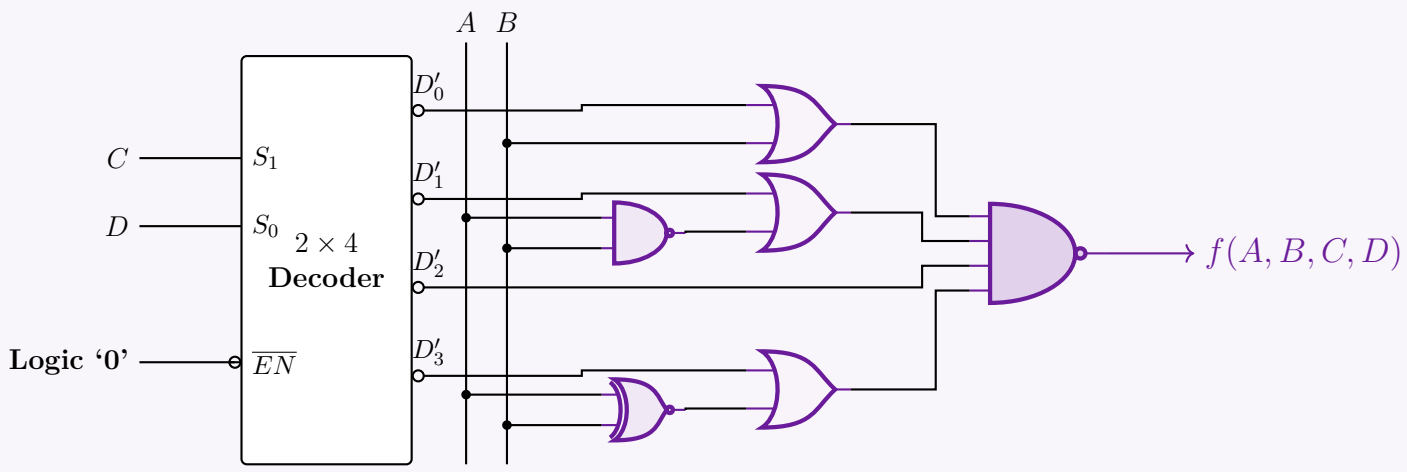
Applying De Morgan's Law ($\overline{X \cdot Y} = \overline{X} + \overline{Y}$) term by term:

$$\begin{aligned}
\overline{\overline{D'_0} \cdot \overline{B}} &= D'_0 + B \\
\overline{\overline{D'_1} \cdot (AB)} &= D'_1 + \overline{AB} = D'_1 + \text{NAND}(A,B) \\
\overline{\overline{D'_2}} &= D'_2 \\
\overline{\overline{D'_3} \cdot (A \oplus B)} &= D'_3 + \overline{A \oplus B} = D'_3 + \text{XNOR}(A,B)
\end{aligned}$$

Final optimized logic expression:

$$f(A,B,C,D) = \text{NAND}_4\Big((D'_0 + B),\ (D'_1 + \text{NAND}(A,B)),\ D'_2,\ (D'_3 + \text{XNOR}(A,B))\Big)$$

5. Logic Circuit Implementation



Q10. Consider a 3×8 decoder whose outputs are active low and which has one active high enable signal. Implement the following functions using the above-mentioned decoder:

(a) $f(a,b,c,d) = \Pi\,(3,5,14)$

(b) $f(a,b,c,d) = \Sigma\,(1,6,8,13)$

Note that you will need to implement both functions in a single circuit. You may use as many decoders as required, keeping the implementation as efficient as possible. Basic gates are also allowed. **(13 marks)** **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

1. 4-to-16 Line Decoder Expansion

To implement 4-variable functions (a,b,c,d), we must first construct a 4×16 decoder using two available 3×8 decoders. We partition the variables as follows:

- The three least significant bits (**LSBs**), b, c, and d, act as the common select inputs (S_2, S_1, S_0) for both decoders.
- The most significant bit (**MSB**), a, acts as the enable controller.
- **Decoder 0** is enabled when $a = 0$ (using an inverter since the enable is active-high) and generates minterms m_0 to m_7.
- **Decoder 1** is enabled when $a = 1$ (connected directly) and generates minterms m_8 to m_{15}.

Because the decoders have **active-low outputs**, they will output $\overline{m_i}$ for the selected minterm index i, and Logic '1' for all other unselected lines.

2. Mathematical Derivation and Minimization

Function 1: $f_1(a,b,c,d) = \Pi\,(3,5,14)$
The function is provided in Product of Sums (POS) form as the product of maxterms M_i. By definition, a maxterm is the exact

complement of a minterm ($M_i = \overline{m_i}$).

$$\begin{aligned} f_1(a,b,c,d) &= M_3 \cdot M_5 \cdot M_{14} \\ &= \overline{m_3} \cdot \overline{m_5} \cdot \overline{m_{14}} \quad \text{(Definition of Maxterms)} \end{aligned}$$

Since the decoder inherently produces the complemented minterms ($\overline{m_i}$) on its active-low output pins (D'_3 from Dec 0, D'_5 from Dec 0, and D'_6 from Dec 1, which corresponds to overall index 14), we simply need to **AND** these active-low signals together.

$$f_1(a,b,c,d) = \text{AND}(\overline{m_3}, \overline{m_5}, \overline{m_{14}})$$

Function 2: $f_2(a,b,c,d) = \Sigma\,(1,6,8,13)$
The function is given in Sum of Products (SOP) form.

$$f_2(a,b,c,d) = m_1 + m_6 + m_8 + m_{13}$$

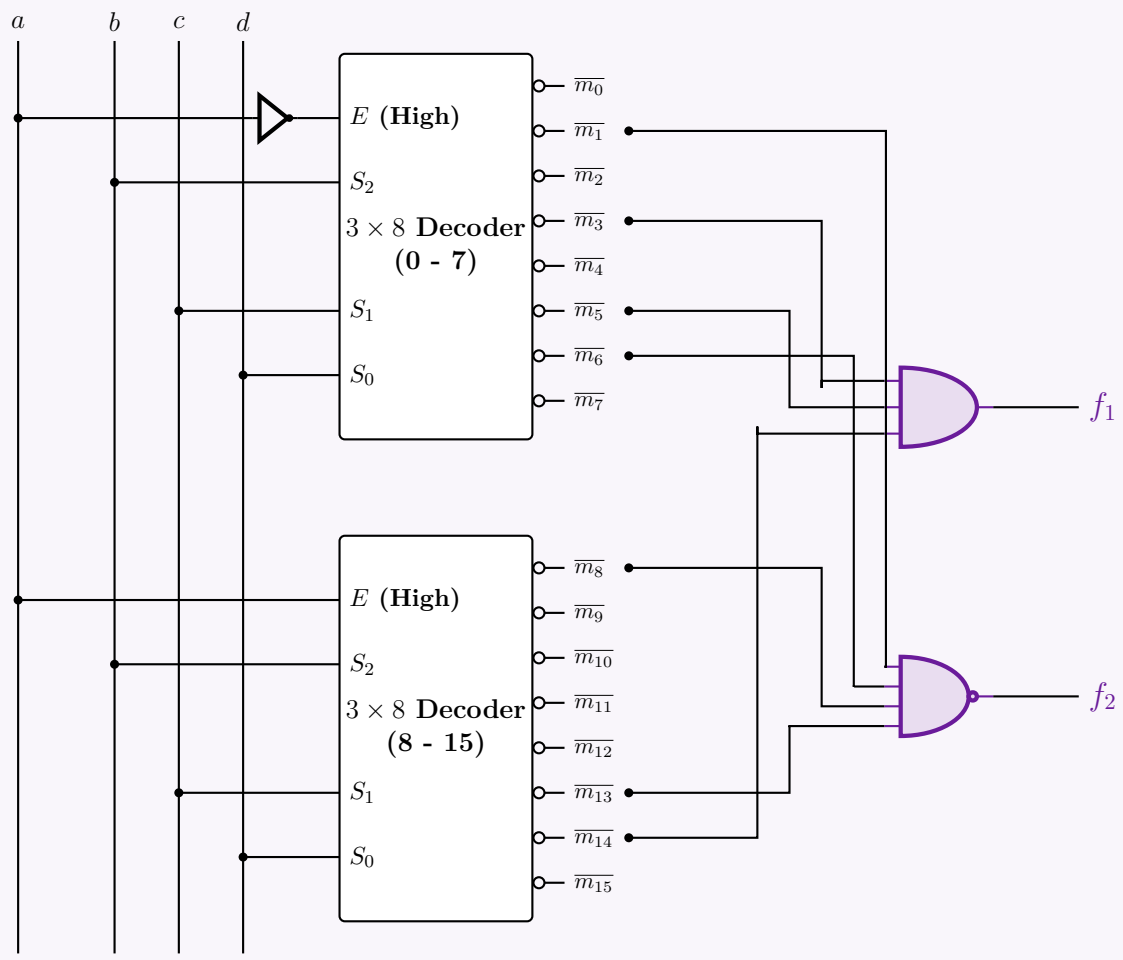
To map this to our active-low decoder outputs ($\overline{m_i}$), we apply **De Morgan's Involution Law** ($\overline{\overline{f}} = f$) followed by De Morgan's Law for addition ($\overline{X+Y} = \overline{X} \cdot \overline{Y}$):

$$\begin{aligned} f_2(a,b,c,d) &= \overline{\overline{m_1 + m_6 + m_8 + m_{13}}} \\ &= \overline{\overline{m_1} \cdot \overline{m_6} \cdot \overline{m_8} \cdot \overline{m_{13}}} \quad \text{(By De Morgan's Law)} \end{aligned}$$

This translates directly into a **NAND** operation applied to the active-low decoder outputs.

$$f_2(a,b,c,d) = \text{NAND}(\overline{m_1}, \overline{m_6}, \overline{m_8}, \overline{m_{13}})$$

3. Integrated Circuit Implementation



Decoder as Demultiplexer

Q1. What is a demultiplexer? Give an example. **(8 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Answer

1. Definition of a Demultiplexer

A **Demultiplexer** (often abbreviated as **DeMUX**) is a combinational logic circuit that performs the reverse operation of a multiplexer. It takes a single input data line and routes it to one of 2^n possible output lines.
The specific output line that receives the input data is determined by the binary value present on the n select (or control) lines. Because it distributes a single input across multiple outputs, a demultiplexer is frequently referred to as a **data distributor**.
Key Characteristics:

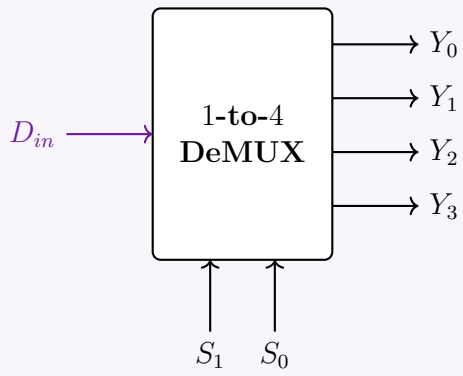
- **Data Inputs:** 1
- **Select Lines:** n

- **Data Outputs:** 2^n

2. Example: A 1-to-4 Line Demultiplexer

To illustrate, consider a 1-to-4 demultiplexer. It features 1 data input (D_{in}), $n = 2$ select lines (S_1, S_0), and $2^2 = 4$ outputs (Y_0, Y_1, Y_2, Y_3).

Block Diagram



Truth Table

The output that mirrors the input D_{in} is determined by the binary combination of S_1 and S_0. All other unselected outputs are held at logic 0.

Select Lines		**Outputs**			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	$\mathbf{D_{in}}$
0	1	0	0	$\mathbf{D_{in}}$	0
1	0	0	$\mathbf{D_{in}}$	0	0
1	1	$\mathbf{D_{in}}$	0	0	0

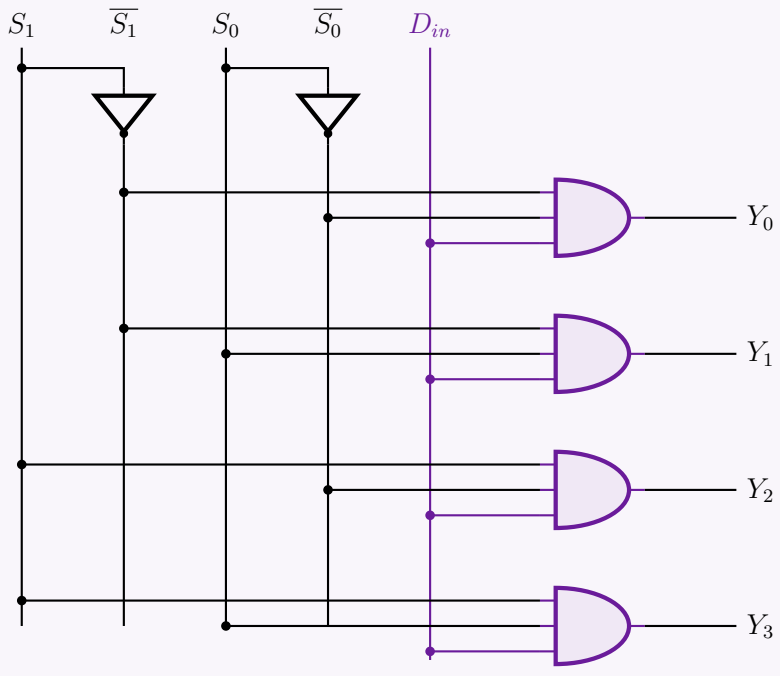
Boolean Expressions

From the truth table, we can directly extract the Sum-of-Products (SOP) expressions for each output line:

$$Y_0 = \overline{S_1} \cdot \overline{S_0} \cdot D_{in}$$
$$Y_1 = \overline{S_1} \cdot S_0 \cdot D_{in}$$
$$Y_2 = S_1 \cdot \overline{S_0} \cdot D_{in}$$
$$Y_3 = S_1 \cdot S_0 \cdot D_{in}$$

Logic Circuit Implementation

Using basic logic gates (NOT gates for the select lines and 3-input AND gates for the outputs), the 1-to-4 demultiplexer is implemented as follows:



Q2. Can a decoder act like a demultiplexer? If yes, explain how. **(5 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Answer

1. Direct Answer

Yes, a decoder can act as a demultiplexer. Specifically, any $N \times 2^N$ line decoder equipped with an **Enable** (EN) input can be directly used as a 1-to-2^N line demultiplexer without requiring any additional logic gates or structural modifications.

2. Conceptual Explanation & Pin Mapping

To understand how this conversion works, we must compare the functional definitions of both circuits:

- A **Decoder** activates one out of 2^N output lines based on the binary combination applied to its N input lines, provided the chip is enabled.
- A **Demultiplexer (DeMUX)** routes a single data input to one of 2^N output lines based on the binary combination applied to its N select lines.

By cleverly reassigning the roles of the decoder's pins, it functions perfectly as a demultiplexer:

$$\textbf{Decoder Enable Input } (EN) \implies \textbf{Demux Data Input } (D_{in})$$
$$\textbf{Decoder Input Lines } (A_{n-1} \dots A_0) \implies \textbf{Demux Select Lines } (S_{n-1} \dots S_0)$$
$$\textbf{Decoder Output Lines } (Y_{2^n-1} \dots Y_0) \implies \textbf{Demux Output Lines } (Y_{2^n-1} \dots Y_0)$$

Mechanism of Action: If the Enable pin (EN) is used as the data input (D_{in}):

(a) When $\mathbf{D_{in} = 0}$: The decoder is disabled. All outputs are forced to logic '0', regardless of the select line inputs. The '0' from the data input has been successfully routed to the output.

(b) When $\mathbf{D_{in} = 1}$: The decoder is enabled. The single output specified by the select lines goes to logic '1' (while all others remain '0'). The '1' from the data input has been successfully routed to the selected output.

In both cases, the value of D_{in} is accurately transferred to the selected output channel.

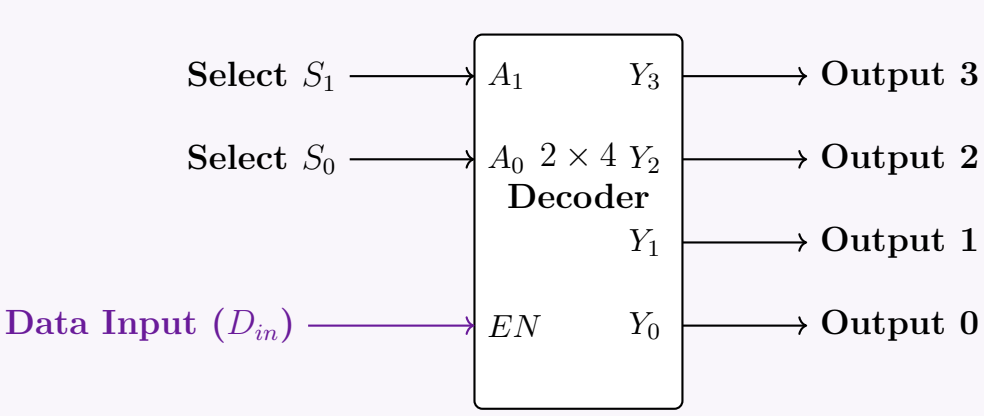
3. Example: 2×4 Decoder as a 1-to-4 Demultiplexer

Truth Table Verification

The following truth table demonstrates a 2×4 active-high decoder operating as a Demultiplexer. Notice how the targeted output perfectly mirrors the state of D_{in}.

Data Input	**Select Lines**		**Outputs**			
EN (D_{in})	A_1 (S_1)	A_0 (S_0)	Y_3	Y_2	Y_1	Y_0
0	X	X	0	0	0	0
1	0	0	0	0	0	**1**
1	0	1	0	0	**1**	0
1	1	0	0	**1**	0	0
1	1	1	**1**	0	0	0

Block Diagram Implementation



Standard 2×4 Decoder IC
(e.g., half of 74LS139 with
active-high logic assumed)

Q3. Show that a decoder can be used as a demultiplexer. **(8 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Answer

1. Theoretical Equivalence

A decoder with an **Enable** input is functionally and structurally identical to a demultiplexer. We can show this by directly mapping the terminals of an $N \times 2^N$ decoder to a 1-to-2^N demultiplexer:

- **Data Input Mapping:** The **Enable** (E) pin of the decoder is used as the single **Data Input** (D_{in}) of the demultiplexer.
- **Select Line Mapping:** The N **Input lines** ($A_{N-1}, \ldots, A_0$) of the decoder are used as the N **Select lines** ($S_{N-1}, \ldots, S_0$) of the demultiplexer.
- **Output Mapping:** The 2^N **Output lines** ($Y_{2^N-1}, \ldots, Y_0$) serve as the data outputs for both circuits.

When $E = 0$, the decoder is disabled, forcing all outputs to 0. This corresponds to routing a data bit of '0' to the output. When $E = 1$, the decoder is enabled, and the specific output addressed by the input lines goes to 1. This corresponds to routing a data bit of '1' to the selected output.

2. Mathematical and Truth Table Proof

Let us examine a 2-to-4 line decoder with an active-high Enable pin. The Boolean expressions for its outputs are:

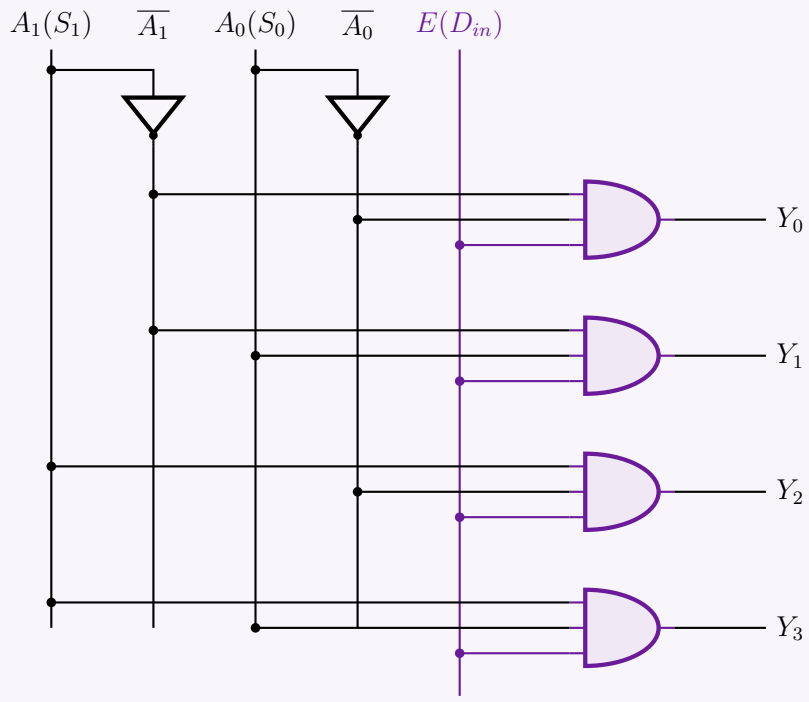
$$Y_0 = E \cdot \overline{A_1} \cdot \overline{A_0}$$
$$Y_1 = E \cdot \overline{A_1} \cdot A_0$$
$$Y_2 = E \cdot A_1 \cdot \overline{A_0}$$
$$Y_3 = E \cdot A_1 \cdot A_0$$

If we substitute $E \to D_{in}$, $A_1 \to S_1$, and $A_0 \to S_0$, we get the exact Boolean expressions of a 1-to-4 demultiplexer. The truth table below confirms this dual functionality:

Enable / Data In	**Inputs / Select**		**Outputs**				**Operation Description**
E (D_{in})	A_1 (S_1)	A_0 (S_0)	Y_3	Y_2	Y_1	Y_0	
0	0	0	0	0	0	**0**	$D_{in} = 0$ routed to Y_0
1	0	0	0	0	0	**1**	$D_{in} = 1$ routed to Y_0
0	0	1	0	0	**0**	0	$D_{in} = 0$ routed to Y_1
1	0	1	0	0	**1**	0	$D_{in} = 1$ routed to Y_1
0	1	0	0	**0**	0	0	$D_{in} = 0$ routed to Y_2
1	1	0	0	**1**	0	0	$D_{in} = 1$ routed to Y_2
0	1	1	**0**	0	0	0	$D_{in} = 0$ routed to Y_3
1	1	1	**1**	0	0	0	$D_{in} = 1$ routed to Y_3

3. Logic Circuit Demonstration

The underlying gate-level architecture perfectly illustrates how the Enable pin distributes its signal to the selected logic pathway.



Q4. How can a decoder be used as a demultiplexer? Explain with an example. **(5 marks)** **[CSE 205 | L-2/T-1 | 2016–17]**

Answer

1. The Core Principle

A standard $n \times 2^n$ line decoder can be used directly as a 1-to-2^n line demultiplexer simply by utilizing its **Enable (E)** input as the **Data Input (D_{in})**. No internal structural changes or extra logic gates are required.
To achieve this, the following pin mapping is established:

- **Enable Pin (E):** Acts as the **Data Input (D_{in})**. The signal applied here is the data to be distributed.
- **Decoder Inputs ($A_{n-1} \dots A_0$):** Act as the **Select Lines ($S_{n-1} \dots S_0$)**. They determine which output line will receive the data.
- **Decoder Outputs ($Y_{2^n-1} \dots Y_0$):** Serve directly as the **Demultiplexer Outputs**.

How it works: When the decoder is disabled (e.g., $E = 0$ for active-high enable), all outputs are forced to logic '0', successfully transferring the '0' from the input to the output. When the decoder is enabled ($E = 1$), the specific output addressed by the input lines goes to logic '1', while all other lines remain '0'. Thus, the logic '1' is successfully transferred to the selected output line.

2. Example: 2×4 Decoder as a 1-to-4 Demultiplexer

Let us configure a standard 2×4 decoder with an active-high Enable pin to function as a 1-to-4 demultiplexer.

Mathematical Verification

The Boolean equations for a 2×4 active-high decoder are:

$$Y_0 = E \cdot \overline{A_1} \cdot \overline{A_0}$$
$$Y_1 = E \cdot \overline{A_1} \cdot A_0$$
$$Y_2 = E \cdot A_1 \cdot \overline{A_0}$$
$$Y_3 = E \cdot A_1 \cdot A_0$$

By substituting the variables $E \to D_{in}$, $A_1 \to S_1$, and $A_0 \to S_0$, the equations perfectly match the standard Sum-of-Products (SOP) expressions of a 1-to-4 demultiplexer:

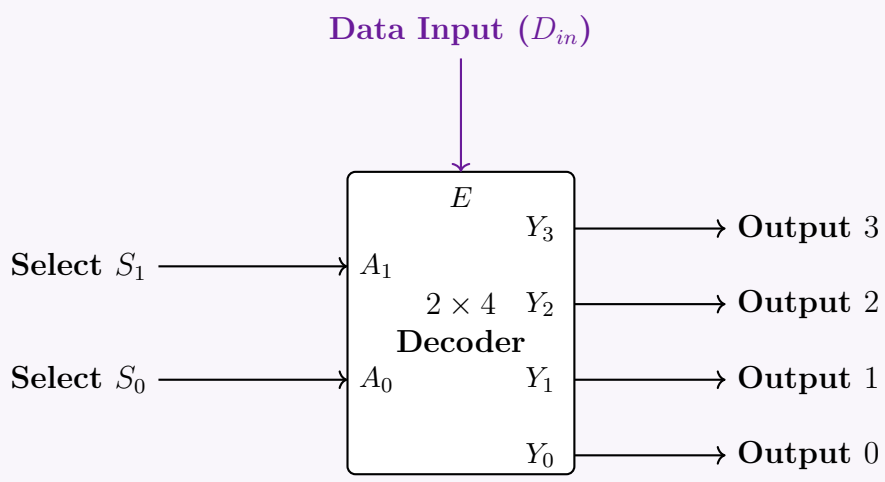
$$Y_0 = D_{in} \cdot \overline{S_1} \cdot \overline{S_0}$$
$$Y_3 = D_{in} \cdot S_1 \cdot S_0 \quad \text{(and similarly for } Y_1, Y_2\text{)}$$

Truth Table

The following truth table demonstrates that the selected output perfectly mirrors the state of the Enable / Data Input pin.

Enable / Data Input	**Decoder Inputs / Select**		**Outputs**			
E (D_{in})	A_1 (S_1)	A_0 (S_0)	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	**0**
1	0	0	0	0	0	**1**
0	0	1	0	0	**0**	0
1	0	1	0	0	**1**	0
0	1	0	0	**0**	0	0
1	1	0	0	**1**	0	0
0	1	1	**0**	0	0	0
1	1	1	**1**	0	0	0

Logic Block Diagram Configuration



BCD to 7-Segment Decoder & BCD Error Detector

Q1. Explain how the Ripple-blanking input (RBI) and Ripple-blanking output (RBO) pins of the BCD to 7-segment decoder (IC 7447) should be connected such that all but the LSB show blank for leading zeros. **(6 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

1. Concept of Ripple Blanking in IC 7447

The IC 7447 is a BCD to 7-segment decoder/driver that includes built-in logic for zero suppression (blanking leading or trailing zeros). This is managed by two active-low pins:

- **$\overline{\text{RBI}}$ (Ripple Blanking Input):** If $\overline{RBI}$ is LOW (Logic '0') and the BCD input is 0000, the display is completely blanked (all segments OFF), and the IC forces its $\overline{RBO}$ pin LOW.
- **$\overline{\text{RBO}}$ (Ripple Blanking Output):** Outputs a LOW (Logic '0') only if the display is blanked (i.e., $\overline{RBI}$ is LOW and the BCD input is 0000). Otherwise, it outputs a HIGH (Logic '1').

2. Connection Strategy for Leading Zero Suppression

To suppress leading zeros in a multi-digit display (e.g., displaying '0007' as ' 7'), while ensuring that an absolute zero value ('0000') displays as ' 0' (the LSB must not be blanked), the pins must be connected in a cascaded "ripple" configuration:

(a) **Most Significant Digit (MSD):** The $\overline{RBI}$ pin of the MSD is hardwired to **GND (Logic '0')**. If the MSD BCD input is zero, the digit is blanked, and its $\overline{RBO}$ becomes '0'. If the input is non-zero, it displays the digit, and its $\overline{RBO}$ becomes '1'.

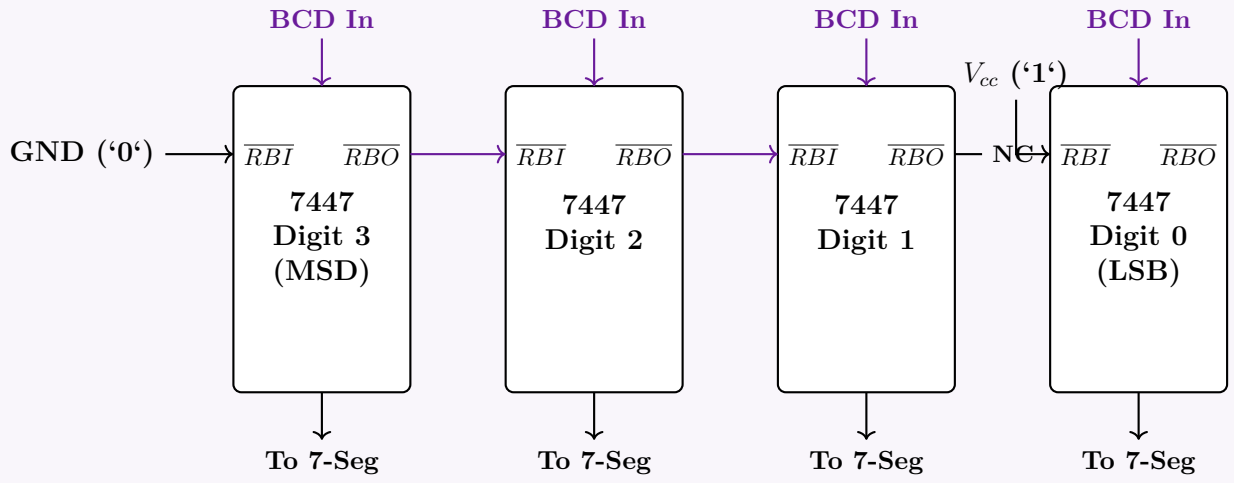
(b) **Intermediate Digits:** The $\overline{RBI}$ of each subsequent digit is connected directly to the $\overline{RBO}$ of the next most significant digit.

- If the previous higher-order digit was blanked, it passes a '0' to the current digit, allowing it to blank if its BCD input is also zero.
- If any higher-order digit was non-zero, it passes a '1' down the chain, forcing all subsequent digits to display normally (e.g., '0105' becomes ' 105', preserving the inner zero).

(c) **Least Significant Digit (LSB):** The $\overline{RBI}$ pin of the LSB is hardwired to V_{cc} **(Logic '1')**. This guarantees that the LSB will never be blanked, ensuring that a completely zero value will display a single '0'. The $\overline{RBO}$ of the previous digit is left unconnected (NC).

3. Block Diagram of a 4-Digit Implementation

The following logic circuit illustrates the connections for a 4-digit display system:



4. Truth Table Verification (Cascading Logic)

Consider the behavior of the chain for the number **0030**:

Stage	BCD Input	$\overline{RBI}$ (In)	$\overline{RBO}$ (Out)	Display Status
Digit 3 (MSD)	0000	**0 (GND)**	0	Blanked
Digit 2	0000	0 (from MSD)	0	Blanked
Digit 1	0011	0 (from Dig 2)	1	Displays '3' (forces $\overline{RBO}$ High)
Digit 0 (LSB)	0000	**1 (Vcc)**	1	Displays '0' (never blanks)

Q2. You are receiving BCD codes sent by a remote transmitter. The bits are $A_3A_2A_1A_0$ (A_0 being the LSB). Design a BCD-error-detector (BED) for your receiver that will produce a HIGH for any received code that is not a valid BCD code. (For example, if you receive 1010, the output of BED will be HIGH.) **(15 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

1. Problem Analysis and Truth Table

A valid Binary-Coded Decimal (BCD) digit is represented by a 4-bit sequence ranging from 0000_2 (0) to 1001_2 (9). Any 4-bit sequence from 1010_2 (10) to 1111_2 (15) is considered an invalid BCD code.
Let the 4-bit input be $A_3A_2A_1A_0$, where A_3 is the Most Significant Bit (MSB) and A_0 is the Least Significant Bit (LSB). We need to design a BCD Error Detector (BED) whose output E is HIGH (Logic '1') for invalid BCD codes and LOW (Logic '0') for valid BCD codes.

Input Bits				Output	Condition
A_3	A_2	A_1	A_0	E	
0	0	0	0	0	Valid (0)
0	0	0	1	0	Valid (1)
0	0	1	0	0	Valid (2)
0	0	1	1	0	Valid (3)
0	1	0	0	0	Valid (4)
0	1	0	1	0	Valid (5)
0	1	1	0	0	Valid (6)
0	1	1	1	0	Valid (7)
1	0	0	0	0	Valid (8)
1	0	0	1	0	Valid (9)
1	0	1	0	**1**	**Invalid (10)**
1	0	1	1	**1**	**Invalid (11)**
1	1	0	0	**1**	**Invalid (12)**
1	1	0	1	**1**	**Invalid (13)**
1	1	1	0	**1**	**Invalid (14)**
1	1	1	1	**1**	**Invalid (15)**

2. Karnaugh Map Minimization

We can extract the minterms for the error condition from the truth table:

$$E(A_3, A_2, A_1, A_0) = \sum m(10, 11, 12, 13, 14, 15)$$

We plot these minterms on a 4-variable Karnaugh Map to find the minimized Sum of Products (SOP) expression.

A_3A_2 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

Group 1: A_3A_2 (Minterms 12, 13, 14, 15)

Group 2: A_3A_1 (Minterms 10, 11, 14, 15)

From the Karnaugh map, the simplified Boolean expression is:

$$E = A_3A_2 + A_3A_1$$

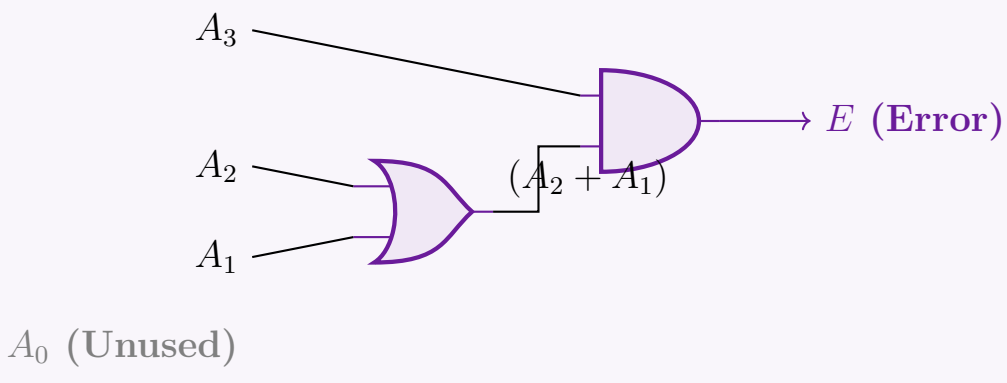
Using the **Distributive Law** of Boolean Algebra, we can factor out A_3 to reduce the gate count for an optimal hardware implementation:

$$E = A_3 \cdot (A_2 + A_1)$$

Note: The LSB (A_0) is entirely redundant for detecting an error, as its state does not dictate whether the code is beyond the valid BCD range.

3. Logic Circuit Implementation

The optimized expression requires only a single 2-input OR gate and a single 2-input AND gate.



Multiplexers & Demultiplexers

Q1. There is a 4×4 smart grid to serve as the electricity transmission facility. The grids will be activated based on the logic function $g(a, b, c, d) = \Sigma\,(1, 2, 5, 7, 12, 14)$. Design the logic circuit using a switching circuit that has 3-bit selector bits S_2, S_1, S_0 to select any of the eight input lines. The switching circuit is activated by two Enables — one active low $\overline{EN_0}$ and another active high EN_1. (Design with the minimum number of extra logic gates if required.) **(13 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Answer

1. Problem Analysis and Mux Configuration

The problem requires us to design a logic circuit for the function $g(a, b, c, d) = \Sigma\,(1, 2, 5, 7, 12, 14)$ using a switching circuit with 3 select bits. A switching circuit with 3 select lines is an **8-to-1 Multiplexer (MUX)**.
Since there are 4 input variables (a, b, c, d) and the multiplexer has 3 select lines (S_2, S_1, S_0), we will use the **variable mapping method**. We map the three most significant variables to the select lines:

$$S_2 = a$$
$$S_1 = b$$
$$S_0 = c$$

The remaining variable, d, (along with its complement $\overline{d}$, Logic 0, and Logic 1) will be applied to the data input lines D_0 through D_7.
The MUX features two enable pins which control its active state:

- $\overline{EN_0}$ is an **active-low** enable. It must be connected to **Logic 0 (Ground)** to be activated.
- EN_1 is an **active-high** enable. It must be connected to **Logic 1 (V_{cc})** to be activated.

2. Multiplexer Implementation Table

To determine the inputs for $D_0 \ldots D_7$, we construct an implementation table. The 16 minterms are distributed across the 8 data lines based on the decimal equivalent of the select bits a, b, c.

- The top row contains minterms where $d = 0$ (Even minterms).
- The bottom row contains minterms where $d = 1$ (Odd minterms).

We circle/highlight the minterms present in the given function $\Sigma\,(1, 2, 5, 7, 12, 14)$ and deduce the data line input.

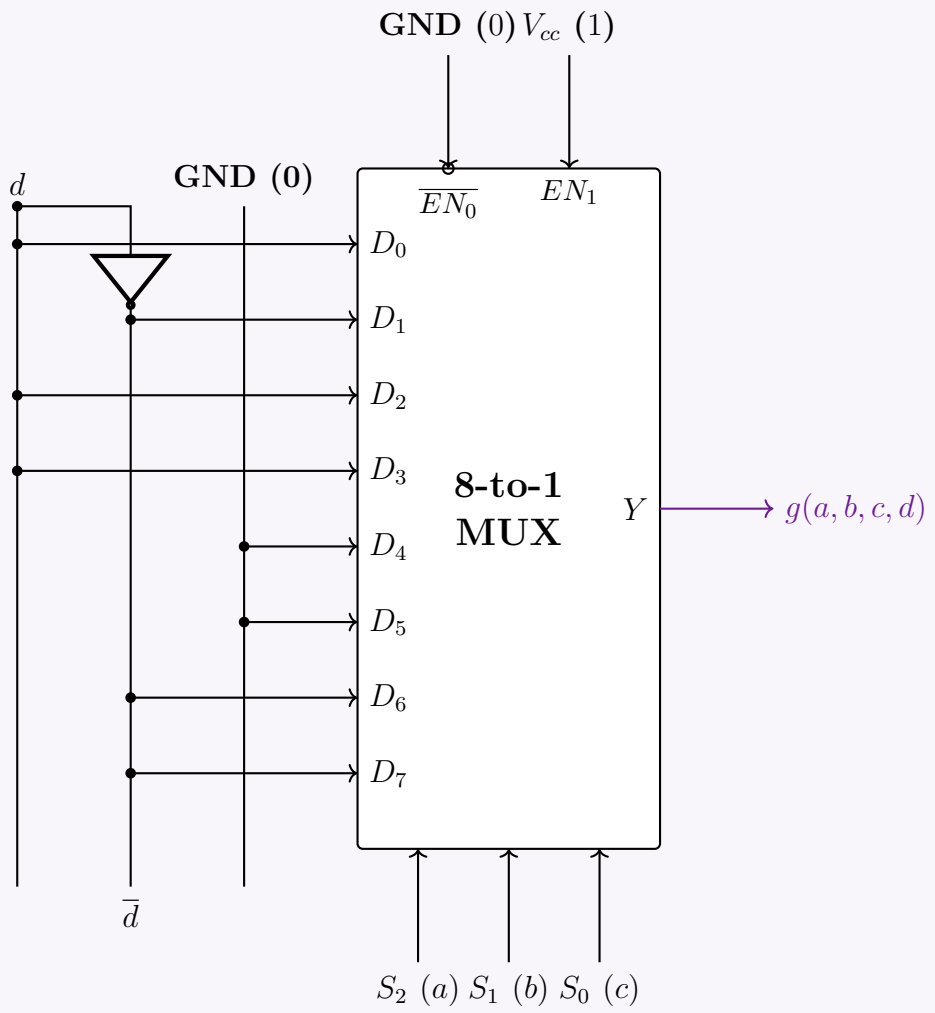
Variable	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
$\overline{d}$ $(d = 0)$	0	**2**	4	6	8	10	**12**	**14**
d $(d = 1)$	**1**	3	**5**	**7**	9	11	13	15
MUX Input	**d**	$\overline{\mathbf{d}}$	**d**	**d**	**0**	**0**	$\overline{\mathbf{d}}$	$\overline{\mathbf{d}}$

Derivation rules applied:

- For D_0, D_2, D_3: Only the minterm corresponding to $d = 1$ is present. Input is d.
- For D_1, D_6, D_7: Only the minterm corresponding to $d = 0$ is present. Input is $\overline{d}$.
- For D_4, D_5: Neither minterm is present. Input is Logic 0.

3. Circuit Diagram

The final 8-to-1 multiplexer circuit is implemented below. Only one NOT gate is required as the extra logic to generate $\overline{d}$.



Q2. Using the switching circuit defined above (5-bit selector lines A, B, C, D, E where A is MSB and E is LSB), design a 32×1 multiplexer. (Design with the minimum number of logic gates if required.) **(10 marks)** **[CSE 205 | L-2/T-1 | 2024–25]**

Answer

1. System Architecture

A 32×1 Multiplexer requires 32 data inputs, 5 select lines (A, B, C, D, E), and 1 output. Based on the provided switching circuit (an 8-to-1 MUX with 3 select lines and two enable pins: active-low $\overline{EN_0}$ and active-high EN_1), we can cascade **four 8-to-1 MUXes** to construct the 32×1 MUX.

- **Lower Order Select Lines (C, D, E):** These represent the 3 Least Significant Bits (LSBs) and are connected directly to the selector bits S_2, S_1, S_0 of all four MUXes simultaneously. They choose 1 out of 8 inputs within an enabled MUX.
- **Higher Order Select Lines (A, B):** These represent the 2 Most Significant Bits (MSBs) and act as chip-select signals. They ensure that exactly one of the four MUXes is activated at any given time.
- **Output Stage:** Since only one MUX is active at a time (and disabled MUXes output Logic '0'), their outputs can be combined using a single **4-input OR gate** to produce the final 32×1 MUX output.

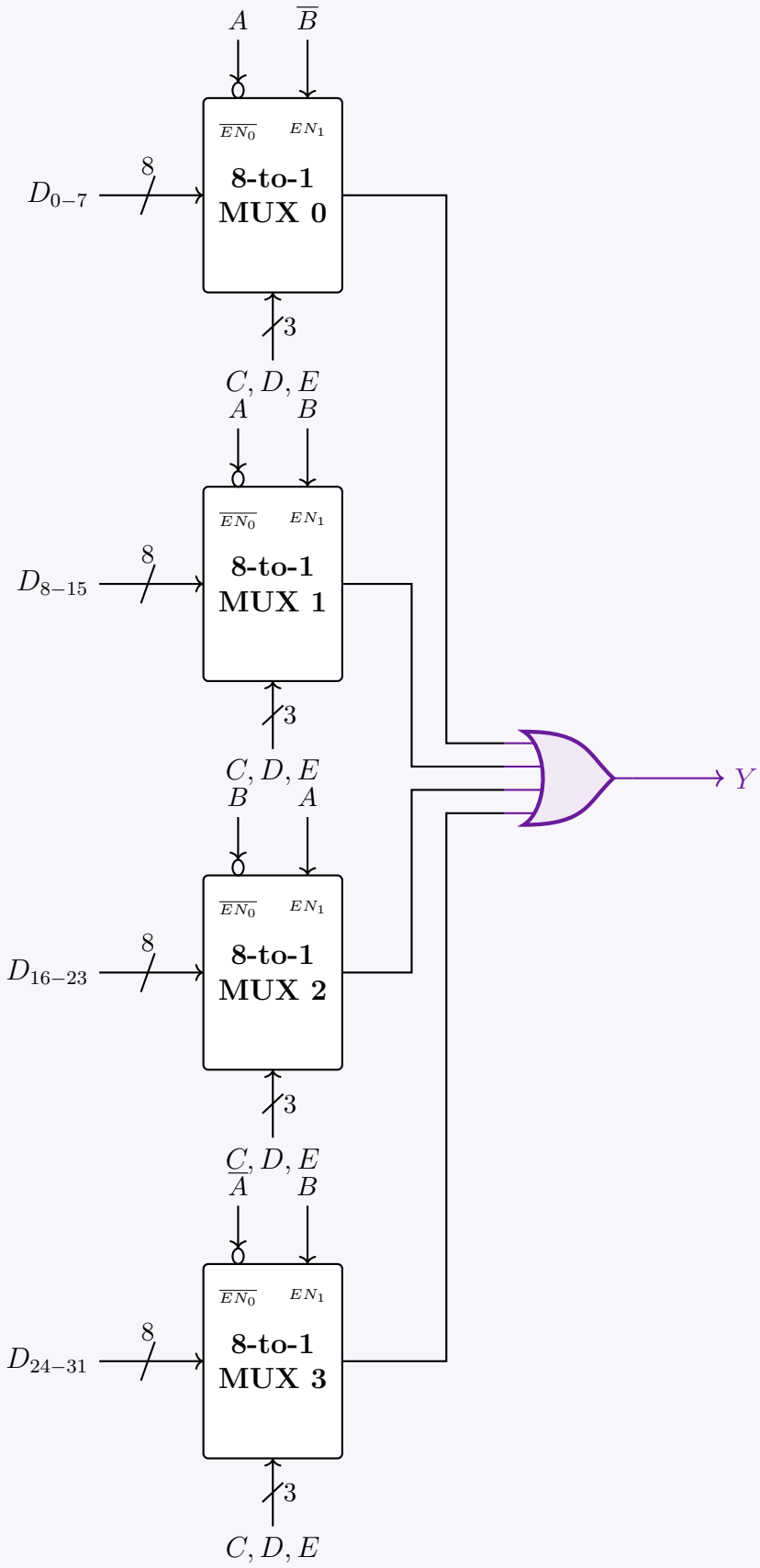
2. Enable Logic Design (Minimizing Gate Count)

Instead of using a separate 2-to-4 decoder for the MSBs, we can cleverly apply A, B, and their complements directly to the built-in $\overline{EN_0}$ and EN_1 pins of the MUXes. A MUX is active only when $\overline{EN_0} = 0$ **AND** $EN_1 = 1$.

MUX Block	Data Range	A	B	$\overline{EN_0}$ Connection	EN_1 Connection	Activation Logic Check
MUX 0	$D_0 - D_7$	0	0	A (which is 0)	$\overline{B}$ (which is 1)	Active when $A = 0, B = 0$
MUX 1	$D_8 - D_{15}$	0	1	A (which is 0)	B (which is 1)	Active when $A = 0, B = 1$
MUX 2	$D_{16} - D_{23}$	1	0	B (which is 0)	A (which is 1)	Active when $A = 1, B = 0$
MUX 3	$D_{24} - D_{31}$	1	1	$\overline{A}$ (which is 0)	B (which is 1)	Active when $A = 1, B = 1$

By utilizing the internal enables, the extra decoding logic is reduced to strictly **two NOT gates** (to generate $\overline{A}$ and $\overline{B}$).

3. Circuit Diagram



Q3. Design a full adder using NOT gates and two 4 to 1 multiplexers. **(12 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Answer

1. Problem Analysis and Truth Table

A Full Adder adds three 1-bit binary numbers (A, B, and C_{in}) and outputs a Sum (S) and a Carry (C_{out}). To implement a 3-variable Boolean function using a 4-to-1 multiplexer (which has 2 select lines), we map the two most significant variables (A and B) to the select lines S_1 and S_0. The remaining variable (C_{in}) and constants $(0, 1)$ will be applied to the data input lines (I_0, I_1, I_2, I_3).
The truth table for the Full Adder is grouped by the combinations of A and B to determine the relationship between C_{in} and the outputs.

Select Lines		Input	Outputs		MUX Input Evaluation	
A (S_1)	B (S_0)	C_{in}	Sum (S)	$Carry$ (C_{out})	Sum MUX Input	C_{out} MUX Input
0	0	0	0	0	$I_0 = C_{in}$	$I_0 = 0$
0	0	1	1	0		
0	1	0	1	0	$I_1 = \overline{C_{in}}$	$I_1 = C_{in}$
0	1	1	0	1		
1	0	0	1	0	$I_2 = \overline{C_{in}}$	$I_2 = C_{in}$
1	0	1	0	1		
1	1	0	0	1	$I_3 = C_{in}$	$I_3 = 1$
1	1	1	1	1		

2. Multiplexer Implementation Strategy

Based on the grouped truth table, we extract the boolean expressions for the MUX data inputs:
For the Sum (S) MUX:

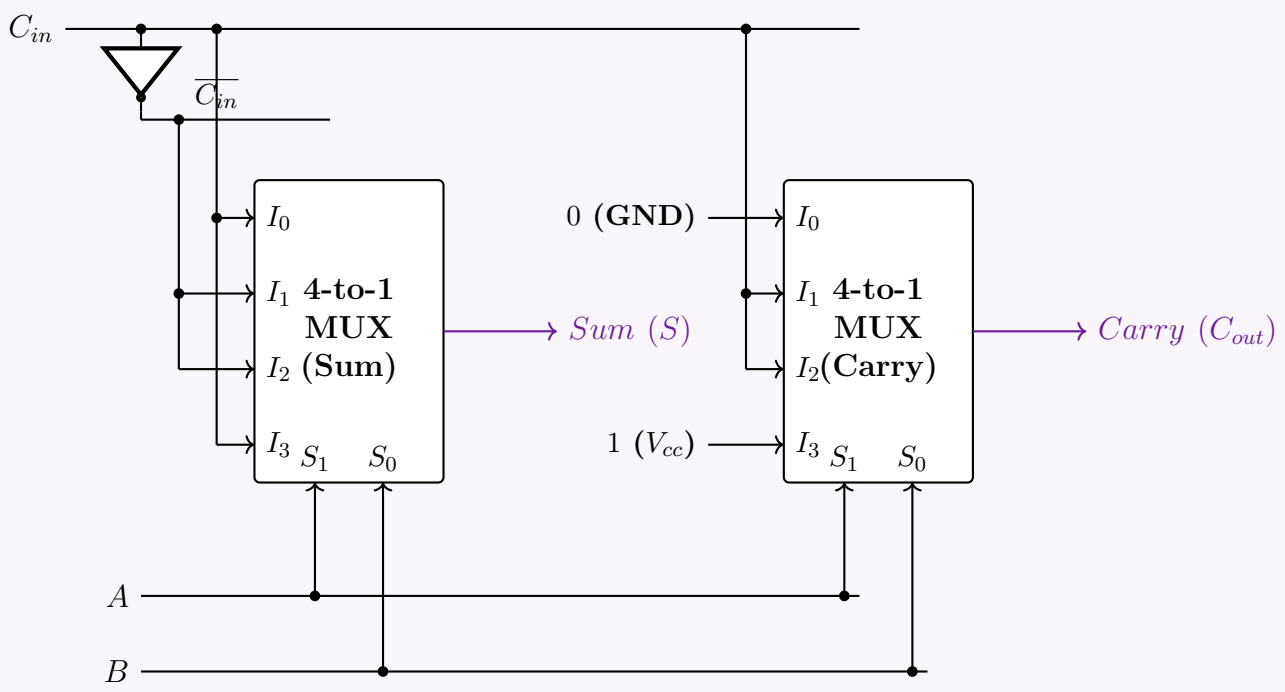
$$I_0 = C_{in}$$
$$I_1 = \overline{C_{in}}$$
$$I_2 = \overline{C_{in}}$$
$$I_3 = C_{in}$$

For the Carry (C_{out}) MUX:

$$I_0 = 0 \text{ (Logic 0/GND)}$$
$$I_1 = C_{in}$$
$$I_2 = C_{in}$$
$$I_3 = 1 \text{ (Logic 1/}V_{cc}\text{)}$$

We will require a single **NOT gate** to generate the $\overline{C_{in}}$ signal for the Sum MUX. Both MUXes will share the A and B inputs on their select lines.

3. Circuit Diagram



Q4. Implement the following Boolean function with a 4×1 multiplexer and external gates:

$$F(A, B, C, D) = \Sigma\,(1, 2, 4, 7, 8, 9, 10, 11, 13, 15)$$

(13 marks) [CSE 205 | L-2/T-1 | 2022–23]

Answer

1. Problem Analysis and Implementation Strategy

We are required to implement a 4-variable Boolean function $F(A, B, C, D) = \Sigma\,(1, 2, 4, 7, 8, 9, 10, 11, 13, 15)$ using a 4×1 Multiplexer. A 4×1 MUX has 2 select lines and 4 data inputs. To implement a 4-variable function, we will apply the **variable mapping method**:

- We connect the two Most Significant Bits (MSBs), A and B, to the select lines: $S_1 = A$ and $S_0 = B$.
- The remaining variables, C and D, will be used to determine the logic expressions applied to the four data input lines (I_0, I_1, I_2, I_3).

2. Multiplexer Implementation Table

We construct the implementation table by listing all 16 minterms of the 4 variables. The minterms are arranged in 4 columns corresponding to the 4 combinations of the select lines (A, B). The rows correspond to the 4 combinations of the data variables (C, D). We circle/highlight the minterms present in the given function.

Variables		I_0	I_1	I_2	I_3
C	D	$(A=0, B=0)$	$(A=0, B=1)$	$(A=1, B=0)$	$(A=1, B=1)$
0	0	0	**4**	**8**	12
0	1	**1**	5	**9**	**13**
1	0	**2**	6	**10**	14
1	1	3	**7**	**11**	**15**
SOP Expression		$\overline{C}D + C\overline{D}$	$\overline{C\overline{D}} + CD$	$\overline{C\overline{D}} + \overline{C}D + C\overline{D} + CD$	$\overline{C}D + CD$
MUX Input		$\mathbf{C \oplus D}$	$\mathbf{\overline{C \oplus D}}$	**1**	**D**

3. Boolean Derivations for MUX Inputs

Based on the minterms present in each column, we derive the minimized Boolean expressions for the MUX inputs:

$$I_0 = \Sigma(1,2) = \overline{C}D + C\overline{D} = \mathbf{C \oplus D} \quad \text{(XOR operation)}$$

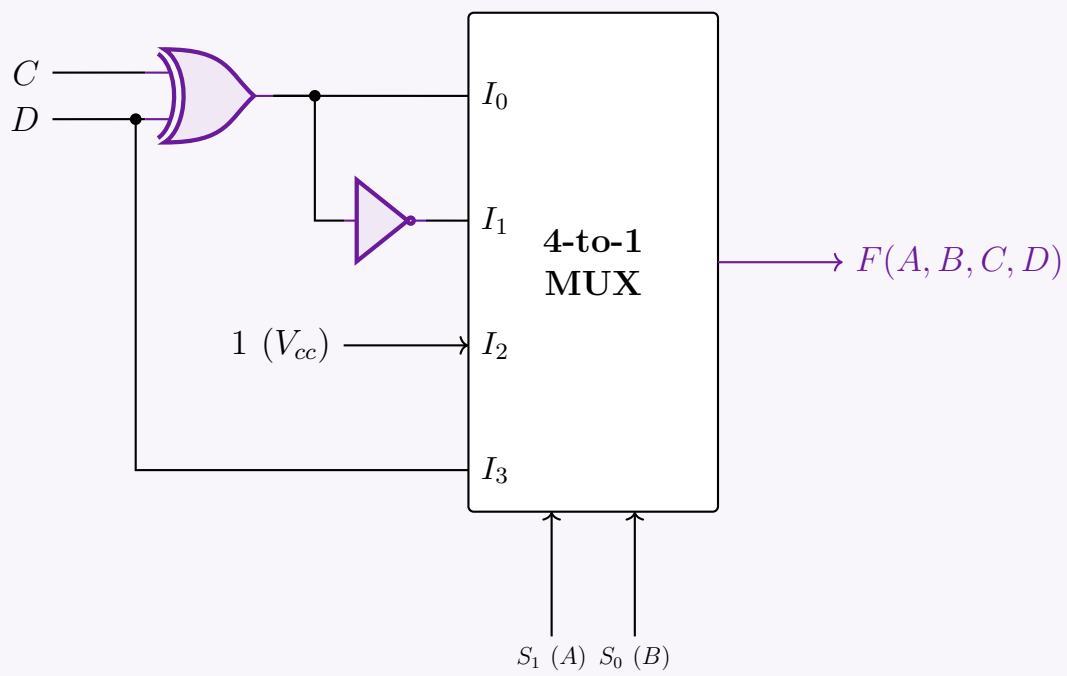
$$I_1 = \Sigma(4,7) = \overline{C\overline{D}} + CD = \mathbf{C \odot D} = \mathbf{\overline{C \oplus D}} \quad \text{(XNOR operation)}$$

$$I_2 = \Sigma(8,9,10,11) = 1 \quad \text{(All minterms present, connects to Logic 1)}$$

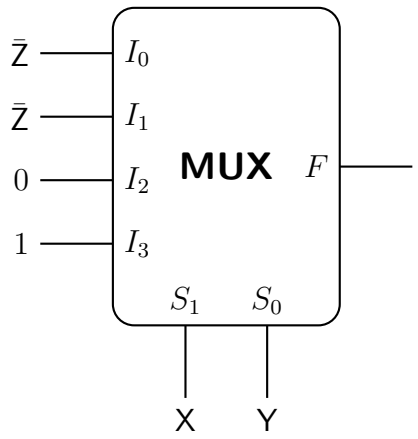
$$\begin{aligned} I_3 &= \Sigma(13,15) = \overline{C}D + CD \\ &= D(\overline{C} + C) \quad \text{(Distributive Law)} \\ &= D(1) = \mathbf{D} \quad \text{(Complement Law and Identity Law)} \end{aligned}$$

We can generate I_1 simply by passing the output of the XOR gate (I_0) through a NOT gate.

4. Logic Circuit Implementation



Q5. What will be the output function $F(X, Y, Z)$ in the product of maxterms form in the given 4×1 MUX circuit? Assume X as the most significant bit and Z as the least significant bit. (The MUX has inputs $Z', Z, 0, 1$ and selection lines S_1S_0 connected to X, Y.)



(12 marks) **[CSE 205 | L-2/T-1 | 2021–22]**

Answer

1. Problem Analysis & Characteristic Equation

The logic function implemented by a 4×1 Multiplexer is defined by its standard characteristic equation:

$$F(S_1, S_0) = \overline{S_1 S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$

Based on the provided problem description, we map the variables to the MUX ports as follows:

- **Selection Lines:** $S_1 = X$ (MSB) and $S_0 = Y$
- **Data Inputs:** $I_0 = \overline{Z}$, $I_1 = Z$, $I_2 = 0$, and $I_3 = 1$

Substituting these mappings into the characteristic equation yields the Boolean function in terms of X, Y, and Z:

$$\begin{aligned} F(X, Y, Z) &= \overline{XY}(\overline{Z}) + \overline{X}Y(Z) + X\overline{Y}(0) + XY(1) \\ &= \overline{XYZ} + \overline{X}YZ + XY \quad \text{(Applying Zero Property and Identity Law)} \end{aligned}$$

2. Canonical Sum of Products (SOP) Expansion

To find the maxterms reliably, we first determine the active minterms by expanding the function into its canonical Sum of Products (SOP) form. The term XY is missing the variable Z, so we expand it using the Boolean rule $Z + \overline{Z} = 1$:

$$\begin{aligned} F(X, Y, Z) &= \overline{XYZ} + \overline{X}YZ + XY(Z + \overline{Z}) \\ &= \overline{XYZ} + \overline{X}YZ + XYZ + XY\overline{Z} \end{aligned}$$

Next, we translate each product term into its corresponding binary combination and minterm notation (m_i):

- $\overline{XYZ} \rightarrow 000_2 \rightarrow m_0$
- $\overline{X}YZ \rightarrow 011_2 \rightarrow m_3$
- $XY\overline{Z} \rightarrow 110_2 \rightarrow m_6$
- $XYZ \rightarrow 111_2 \rightarrow m_7$

Thus, the canonical SOP form is:

$$F(X, Y, Z) = \Sigma\, m(0, 3, 6, 7)$$

3. Conversion to Product of Maxterms (POS)

For any Boolean function of n variables, the maxterms (M_j) represent the input combinations for which the function evaluates to logic 0. The set of maxterms is the exact complement of the set of minterms with respect to the universal set of indices 0 to $2^n - 1$. Since $n = 3$ (for variables X, Y, Z), the indices range from 0 to 7. Extracting the missing indices from our minterm set gives the maxterm set:

$$\text{Maxterms} = \{0, 1, 2, 3, 4, 5, 6, 7\} \setminus \{0, 3, 6, 7\} = \{1, 2, 4, 5\}$$

Therefore, the output function in Product of Maxterms (POS) form is:

$$\mathbf{F(X, Y, Z) = \Pi\, M(1, 2, 4, 5)}$$

4. Truth Table Verification

To guarantee mathematical flawlessness, we verify the output logic across all possible combinations of X, Y, Z.

X (S_1)	Y (S_0)	Z	**Active MUX Input**	F	**Classification**
0	0	0	$I_0 = \overline{Z}$	**1**	Minterm m_0
0	0	1		**0**	**Maxterm M_1**
0	1	0	$I_1 = Z$	**0**	**Maxterm M_2**
0	1	1		**1**	Minterm m_3
1	0	0	$I_2 = 0$	**0**	**Maxterm M_4**
1	0	1		**0**	**Maxterm M_5**
1	1	0	$I_3 = 1$	**1**	Minterm m_6
1	1	1		**1**	Minterm m_7

The truth table visually confirms that the function generates a '0' exclusively at indices $1, 2, 4$, and 5, perfectly validating the analytical POS derivation.

Q6. Math teacher Mina Rahman sent sixteen question choices $Q_0, \ldots, Q_{15}$ to her students with a 2-bit binary code C_1C_0. If $C_1C_0 = 11$, Q_0 is selected; $C_1C_0 = 10$, Q_1 is selected; …and so on. The code is activated by active-low $\overline{EN_1}$ and active-high EN_2. Design the multiplexing circuit. **(15 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Answer

1. Problem Analysis & Clarification

The problem states that there are sixteen question choices $(Q_0, \ldots, Q_{15})$ but provides only a 2-bit binary selection code (C_1C_0). In digital logic, an n-bit selection code can uniquely address exactly 2^n inputs. Therefore, a 2-bit code (C_1C_0) yields $2^2 = 4$ unique combinations.
To create a mathematically sound multiplexing circuit based on the specific selection sequence provided in the problem, we will design a **4-to-1 Multiplexer (MUX)** addressing the four specified choices (Q_0, Q_1, Q_2, Q_3). *Note: If a 16-to-1 selection was strictly required, the system would necessitate a 4-bit selection code $(C_3C_2C_1C_0)$.*

2. Selection Logic & Variable Mapping

Based on the defined behavior, the selection sequence is inverted compared to standard binary weighting. We map the variables as follows:

- **Selection Lines:** $S_1 = C_1$ (MSB) and $S_0 = C_0$ (LSB).
- **Enable Logic:** The MUX is active only when $\overline{EN_1} = 0$ (active-low) and $EN_2 = 1$ (active-high).
- **Data Input Mapping:**
  - When $C_1C_0 = 11$ (Decimal 3), Q_0 is selected $\implies I_3 = Q_0$
  - When $C_1C_0 = 10$ (Decimal 2), Q_1 is selected $\implies I_2 = Q_1$
  - When $C_1C_0 = 01$ (Decimal 1), Q_2 is selected $\implies I_1 = Q_2$
  - When $C_1C_0 = 00$ (Decimal 0), Q_3 is selected $\implies I_0 = Q_3$

3. Truth Table and Boolean Expression

Let the global Enable signal be E. The circuit operates correctly only when $E = 1$.

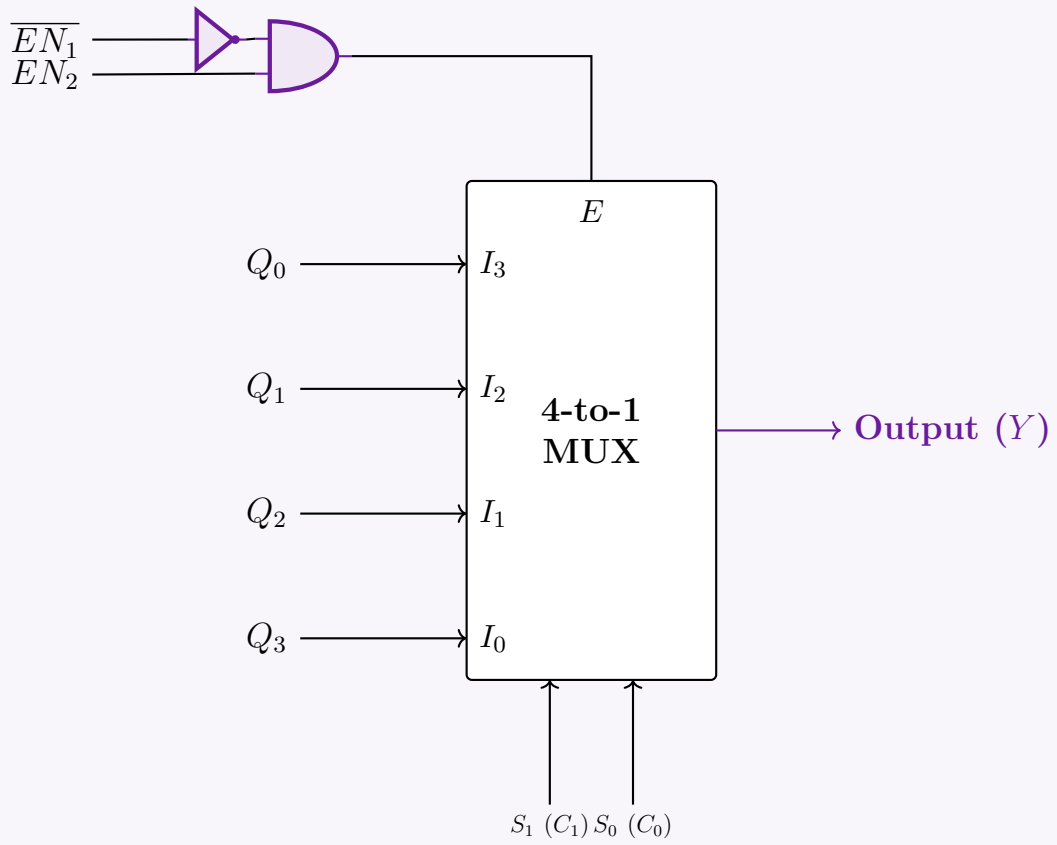
$$E = (\overline{EN_1})' \cdot EN_2$$

Enable Inputs $\overline{EN_1}$	EN_2	**Selection Lines** C_1 (S_1)	C_0 (S_0)	**Internal Enable (E)**	**Output** Y
1	X	X	X	0	0 (High-Z/Inactive)
X	0	X	X	0	0 (High-Z/Inactive)
0	1	0	0	1	Q_3
0	1	0	1	1	Q_2
0	1	1	0	1	Q_1
0	1	1	1	1	Q_0

The characteristic equation for the active multiplexer is:

$$Y = E \cdot \left(\overline{S_1S_0}I_0 + \overline{S_1}S_0I_1 + S_1\overline{S_0}I_2 + S_1S_0I_3\right)$$
$$Y = \left((\overline{EN_1})' \cdot EN_2\right) \cdot \left(\overline{C_1C_0}Q_3 + \overline{C_1}C_0Q_2 + C_1\overline{C_0}Q_1 + C_1C_0Q_0\right)$$

4. Logic Circuit Implementation



Q7. There are 16 electronic turnstile gates in a cricket stadium. The gates are numbered from 0 to 15 denoted by four bits $T_3T_2T_1T_0$. You have to design a digital ticketing system which uses a switch, and the switch has four input lines I_0, I_1, I_2, I_3. These lines can be selected by S_0, S_1. The switch is activated by two active low enables EN_1' and EN_2'. (The selection policy of the switch is as follows. If $S_1S_0 = 00$, I_0 is selected; if $S_1S_0 = 01$, I_1 is selected; if $S_1S_0 = 10$, I_2 is selected; ... so on.). You can have entry to the stadium if you get the ticket for any of the following gate numbers $(1, 3, 4, 6, 7, 9, 12, 15)$. Design the ticketing circuit using the switch. **(15 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Answer

1. Problem Formalization and Variable Mapping

The stadium gates are represented by a 4-bit number $T_3T_2T_1T_0$, where T_3 is the Most Significant Bit (MSB) and T_0 is the Least Significant Bit (LSB). The ticketing system allows entry for a specific set of gate numbers, which can be expressed as a Boolean function in canonical Sum of Products (SOP) form:

$$F(T_3, T_2, T_1, T_0) = \Sigma\, m(1, 3, 4, 6, 7, 9, 12, 15)$$

We are required to implement this function using a switch that acts as a **4-to-1 Multiplexer (MUX)**. The MUX has:

- **Selection Lines:** S_1, S_0. We map the most significant variables to these lines: $S_1 = T_3$ and $S_0 = T_2$.
- **Data Input Lines:** I_0, I_1, I_2, I_3. The logic for these will be derived using the remaining variables T_1 and T_0.
- **Enable Lines:** Two active-low enables, $\overline{EN_1}$ and $\overline{EN_2}$. The MUX is active only when both are connected to Logic 0 (Ground).

2. Multiplexer Implementation Table

We distribute the 16 minterms across 4 columns corresponding to the 4 combinations of the select lines (T_3, T_2). We then highlight the minterms present in the entry function to determine the boolean expression for each MUX input based on T_1 and T_0.

Variables		I_0	I_1	I_2	I_3
T_1	T_0	$(T_3 = 0, T_2 = 0)$	$(T_3 = 0, T_2 = 1)$	$(T_3 = 1, T_2 = 0)$	$(T_3 = 1, T_2 = 1)$
0	0	0	**4**	8	**12**
0	1	**1**	5	**9**	13
1	0	2	**6**	10	14
1	1	**3**	**7**	11	**15**

3. Boolean Derivations for MUX Inputs

Based on the active minterms in each column, we extract and simplify the Boolean expressions for I_0, I_1, I_2, and I_3:

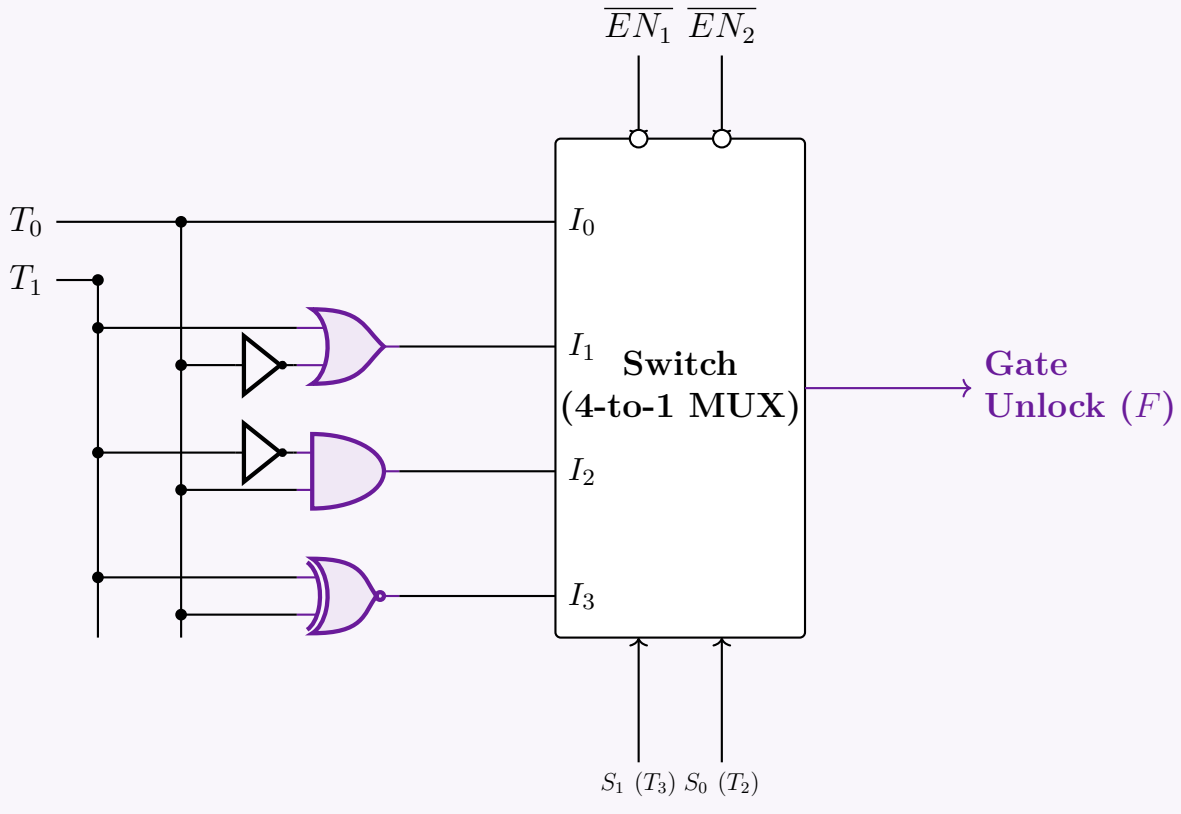
$$
\begin{aligned}
\mathbf{I_0} &= \Sigma(1,3) = \overline{T_1}T_0 + T_1T_0 \\
&= T_0(\overline{T_1} + T_1) && \text{(Distributive Law)} \\
&= T_0(1) = \mathbf{T_0} && \text{(Complement and Identity Laws)}
\end{aligned}
$$

$$
\begin{aligned}
\mathbf{I_1} &= \Sigma(4,6,7) = \overline{T_1T_0} + T_1\overline{T_0} + T_1T_0 \\
&= \overline{T_0}(\overline{T_1} + T_1) + T_1T_0 && \text{(Distributive Law)} \\
&= \overline{T_0} + T_1T_0 && \text{(Complement Law)} \\
&= (\overline{T_0} + T_1)(\overline{T_0} + T_0) && \text{(Absorption / Distributive Law: } A + \overline{A}B = A + B\text{)} \\
&= \mathbf{T_1} + \overline{\mathbf{T_0}} && \text{(OR operation)}
\end{aligned}
$$

$$
\mathbf{I_2} = \Sigma(9) = \overline{\mathbf{T_1}}\mathbf{T_0} \qquad \text{(AND operation, minimal form)}
$$

$$
\begin{aligned}
\mathbf{I_3} &= \Sigma(12,15) = \overline{T_1T_0} + T_1T_0 \\
&= \mathbf{T_1} \odot \mathbf{T_0} && \text{(XNOR operation)}
\end{aligned}
$$

4. Logic Circuit Implementation



Q8. Implement $F(A,B,C,D) = \Sigma\,(1,2,4,10,12,13,14,15)$ with a 4×1 multiplexer and external gates. Connect inputs A and B to the selection lines. Express F as a function of C and D for each of the four cases $AB = 00, 01, 10, 11$ to obtain the data-line requirements. **(17 marks)** **[CSE 205 | L-2/T-1 | 2019–20]**

Answer

1. Problem Analysis and Implementation Strategy

We are tasked with implementing a 4-variable Boolean function $F(A,B,C,D) = \Sigma\,(1,2,4,10,12,13,14,15)$ using a 4×1 multiplexer. A 4×1 MUX has 2 select lines and 4 data inputs. The problem specifies connecting inputs A and B to the select lines:

- **Selection Lines:** $S_1 = A$ (MSB) and $S_0 = B$ (LSB).
- **Data Lines (I_0, I_1, I_2, I_3):** The remaining variables, C and D, will dictate the logic expressions applied to the input lines for each of the four combinations of AB.

2. Multiplexer Implementation Table

We map the 16 minterms of the 4 variables across 4 columns corresponding to the states of AB. We highlight the minterms specified in the function to determine the residual logic required from variables C and D.

Variables		I_0	I_1	I_2	I_3
C	D	$(A=0,B=0)$	$(A=0,B=1)$	$(A=1,B=0)$	$(A=1,B=1)$
0	0	0	**4**	8	**12**
0	1	**1**	5	9	**13**
1	0	**2**	6	**10**	**14**
1	1	3	7	11	**15**

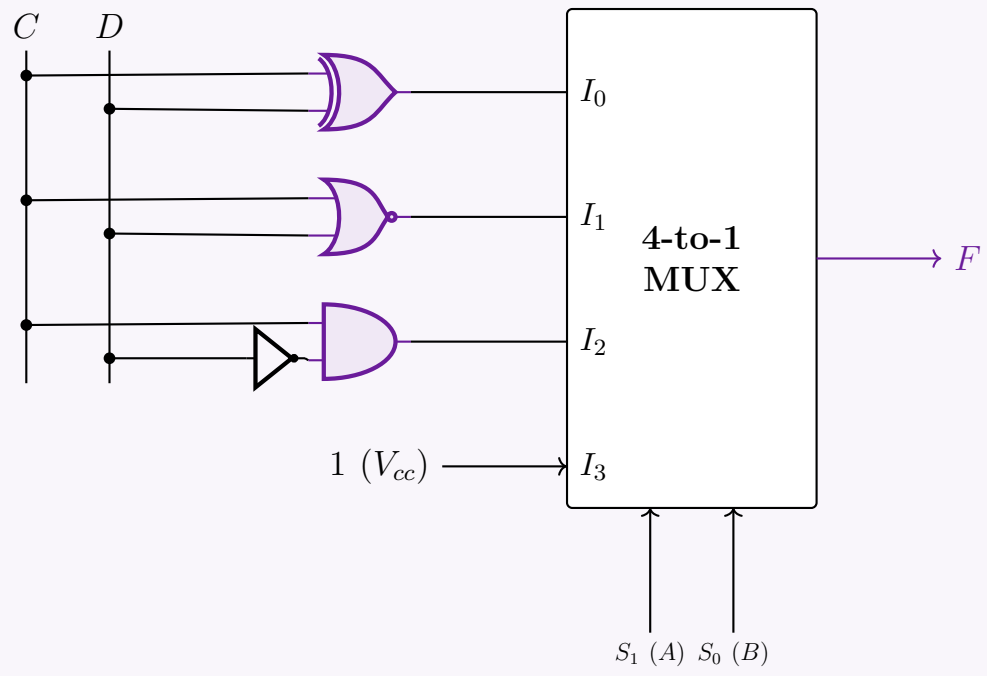
3. Boolean Derivations for Data Lines

By analyzing the active minterms in each column as a function of C and D, we systematically derive the minimal Boolean expression for each MUX input:

$$\begin{aligned}
\mathbf{I_0}\ (\mathbf{AB=00}) &= \Sigma(1,2) = \overline{C}D + C\overline{D} \\
&= \mathbf{C \oplus D} && \text{(Definition of XOR operation)} \\
\mathbf{I_1}\ (\mathbf{AB=01}) &= \Sigma(4) = \overline{C\,D} \\
&= \overline{\mathbf{C+D}} && \text{(De Morgan's Law - NOR operation)} \\
\mathbf{I_2}\ (\mathbf{AB=10}) &= \Sigma(10) = \mathbf{C\overline{D}} && \text{(AND operation with inverted D)} \\
\mathbf{I_3}\ (\mathbf{AB=11}) &= \Sigma(12,13,14,15) = \overline{C\,D} + \overline{C}D + C\overline{D} + CD \\
&= \overline{C}(\overline{D}+D) + C(\overline{D}+D) && \text{(Distributive Law)} \\
&= \overline{C}(1) + C(1) && \text{(Complement Law: } X+\overline{X}=1\text{)} \\
&= \overline{C} + C = \mathbf{1} && \text{(Identity and Complement Laws)}
\end{aligned}$$

4. Logic Circuit Implementation

Using the derived input expressions, we construct the circuit with standard logic gates feeding into the 4×1 MUX.



Q9. Using a 4×1 line MUX, design a logic circuit for $f(w,x,y,z) = \Pi\,(0,2,5,6,7,8,9,10)$, where the MUX has selector bits S_1S_0, input lines $I_0 \ldots I_3$, active-low enable $\overline{EN_1}$ and active-high enable EN_2. **(15 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Answer

1. Problem Analysis and Canonical Conversion

The given Boolean function is defined in Product of Maxterms (POS) form:

$$f(w,x,y,z) = \Pi\,(0,2,5,6,7,8,9,10)$$

To implement this using a Multiplexer (MUX), it is more systematic to express the function in its canonical Sum of Minterms (SOP) form. For a 4-variable function, the universal set of indices is 0 to $2^4 - 1 = 15$. The minterms are the indices missing from the maxterm list:

$$\begin{aligned}
\text{Minterms} &= \{0,1,\ldots,15\} \setminus \{0,2,5,6,7,8,9,10\} \\
&= \{1,3,4,11,12,13,14,15\}
\end{aligned}$$

Thus, the equivalent SOP expression is:

$$f(w, x, y, z) = \Sigma\,(1, 3, 4, 11, 12, 13, 14, 15)$$

2. Multiplexer Variable Mapping

A 4×1 multiplexer requires 2 selection lines (S_1, S_0) to address its 4 input lines (I_0, I_1, I_2, I_3). We map the most significant variables to the selection lines:

- **Selection Lines:** $S_1 = w$ and $S_0 = x$.
- **Data Lines:** The logic for inputs I_0, I_1, I_2, I_3 will be determined as functions of the remaining variables, y and z.

3. Implementation Table

We distribute the 16 possible minterms across 4 columns according to the combinations of w and x. We then highlight the active minterms belonging to the function to extract the logic expression for each MUX input.

Variables		I_0	I_1	I_2	I_3
y	z	$(w = 0, x = 0)$	$(w = 0, x = 1)$	$(w = 1, x = 0)$	$(w = 1, x = 1)$
0	0	0	**4**	8	**12**
0	1	**1**	5	9	**13**
1	0	2	6	10	**14**
1	1	**3**	7	**11**	**15**

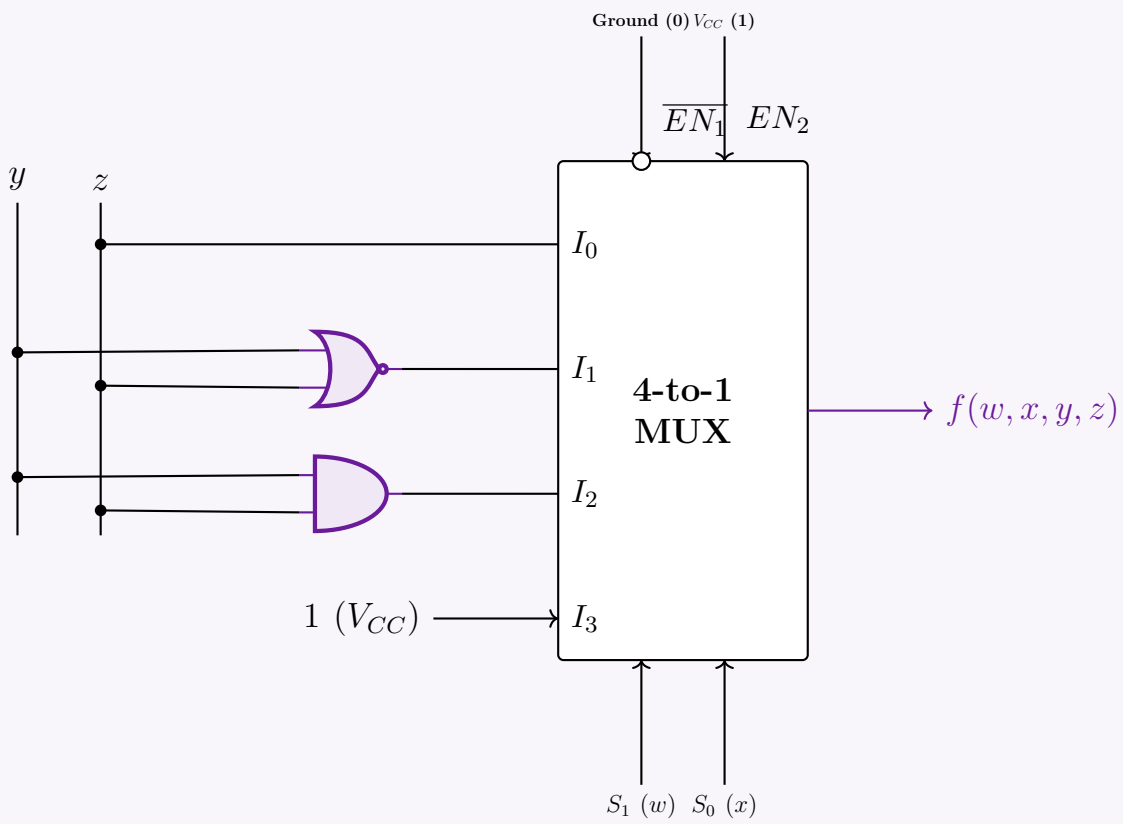
4. Boolean Derivations for Input Lines

We formulate the minimum Boolean expression for each input column based on the active minterms:

$$\begin{aligned}
\mathbf{I_0}\ (\mathbf{wx = 00}) &= \Sigma(1, 3) = \overline{y}z + yz \\
&= z(\overline{y} + y) && \text{(Distributive Law)} \\
&= z(1) = \mathbf{z} && \text{(Complement and Identity Laws)} \\
\mathbf{I_1}\ (\mathbf{wx = 01}) &= \Sigma(4) = \mathbf{\overline{yz}} \\
&= \mathbf{\overline{y + z}} && \text{(De Morgan's Law - NOR operation)} \\
\mathbf{I_2}\ (\mathbf{wx = 10}) &= \Sigma(11) = \mathbf{yz} && \text{(AND operation)} \\
\mathbf{I_3}\ (\mathbf{wx = 11}) &= \Sigma(12, 13, 14, 15) = \overline{yz} + \overline{y}z + y\overline{z} + yz \\
&= \overline{y}(\overline{z} + z) + y(\overline{z} + z) && \text{(Distributive Law)} \\
&= \overline{y}(1) + y(1) && \text{(Complement Law: } A + \overline{A} = 1\text{)} \\
&= \overline{y} + y = \mathbf{1} && \text{(Identity and Complement Laws)}
\end{aligned}$$

5. Logic Circuit Implementation

The circuit includes the MUX enables: $\overline{EN_1}$ (active-low) must be tied to Ground (0), and EN_2 (active-high) must be tied to V_{CC} (1) for normal operation.



Q10. Design a 1-bit Full Subtractor using a 4×1 multiplexer. The MUX has selector bits S_1S_0, input lines I_0, I_1, I_2, I_3, and active-low enable $\overline{EN}$. **(12 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Answer

1. Truth Table of a 1-Bit Full Subtractor

A 1-bit Full Subtractor subtracts a subtrahend (Y) and a borrow-in (B_{in}) from a minuend (X). It produces two outputs: Difference (D) and Borrow-out (B_{out}). The truth table defining this behavior is:

Inputs			**Outputs**	
X (Minuend)	Y (Subtrahend)	B_{in} (Borrow In)	D **(Difference)**	B_{out} **(Borrow Out)**
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2. Multiplexer Configuration Strategy

Since a Full Subtractor has two independent output functions (D and B_{out}), we require **two** 4×1 multiplexers.

- **Selection Lines:** We assign the variables X and Y to the MUX selector bits: $S_1 = X$ and $S_0 = Y$.
- **Data Lines (I_0, I_1, I_2, I_3):** The inputs for each MUX will be evaluated as a function of the remaining variable, B_{in}.
- **Enable Line:** The problem specifies an active-low enable ($\overline{EN}$). For the MUX to function, this line must be tied to Logic 0 (Ground).

3. Boolean Derivation for Data Lines

We determine the input data lines algebraically by fixing the selection lines X and Y, and expressing the output solely in terms of B_{in}.

A. Difference Output (D)

The standard Boolean expression for the Difference is $D = X \oplus Y \oplus B_{in}$. Let's evaluate this for all four selector combinations:

$$\begin{aligned}
\mathbf{I_0}\ (\mathbf{XY = 00}): \quad & D = 0 \oplus 0 \oplus B_{in} \\
& = 0 \oplus B_{in} = \mathbf{B_{in}} && \text{(Identity Law of XOR)} \\
\mathbf{I_1}\ (\mathbf{XY = 01}): \quad & D = 0 \oplus 1 \oplus B_{in} \\
& = 1 \oplus B_{in} = \mathbf{\overline{B_{in}}} && \text{(Inversion Law of XOR)} \\
\mathbf{I_2}\ (\mathbf{XY = 10}): \quad & D = 1 \oplus 0 \oplus B_{in} \\
& = 1 \oplus B_{in} = \mathbf{\overline{B_{in}}} && \text{(Inversion Law of XOR)} \\
\mathbf{I_3}\ (\mathbf{XY = 11}): \quad & D = 1 \oplus 1 \oplus B_{in} \\
& = 0 \oplus B_{in} = \mathbf{B_{in}} && \text{(Identity Law of XOR)}
\end{aligned}$$

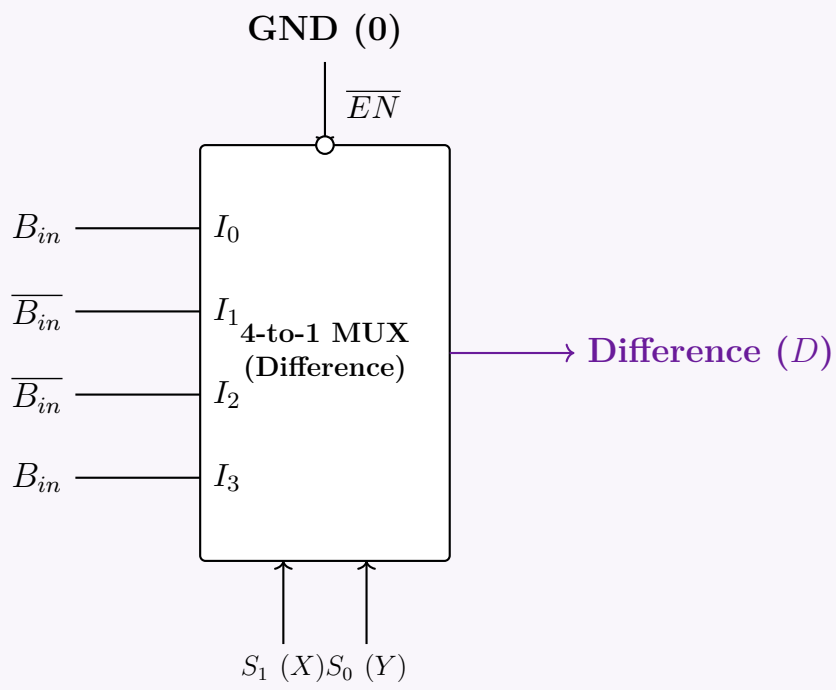
B. Borrow-Out Output (B_{out})

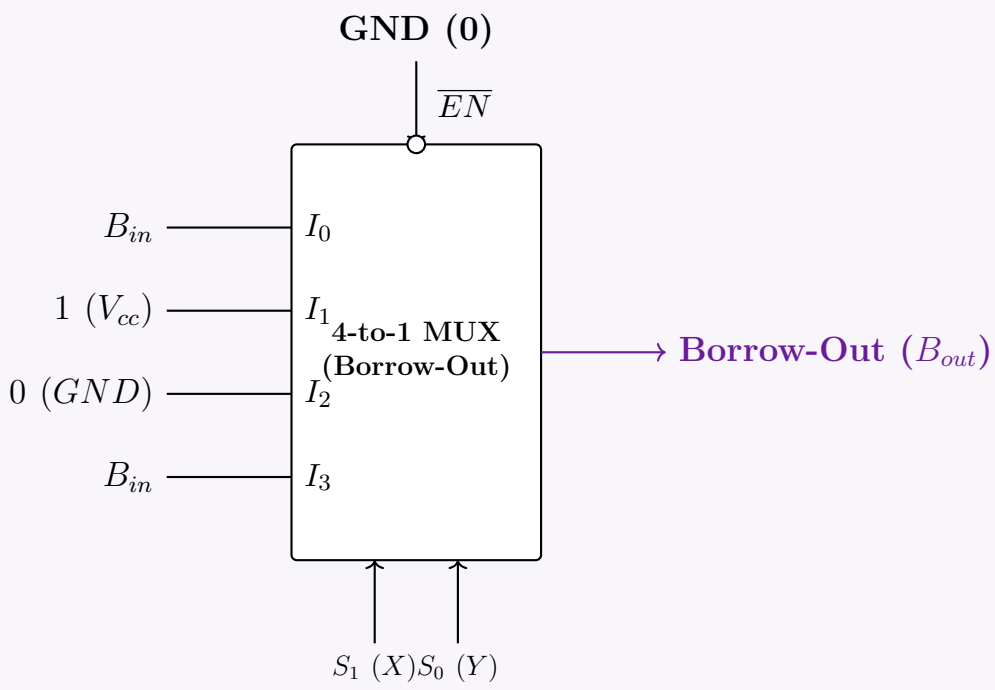
The standard minimized Boolean expression for Borrow-out is $B_{out} = \overline{X}Y + \overline{X}B_{in} + YB_{in}$.

$$\begin{aligned}
\mathbf{I_0}\ (\mathbf{XY = 00}): \quad & B_{out} = \overline{0}(0) + \overline{0}B_{in} + 0(B_{in}) \\
& = 0 + 1 \cdot B_{in} + 0 = \mathbf{B_{in}} && \text{(Identity and Annulment Laws)} \\
\mathbf{I_1}\ (\mathbf{XY = 01}): \quad & B_{out} = \overline{0}(1) + \overline{0}B_{in} + 1(B_{in}) \\
& = 1 + B_{in} + B_{in} = \mathbf{1} && \text{(Annulment Law: } 1 + A = 1) \\
\mathbf{I_2}\ (\mathbf{XY = 10}): \quad & B_{out} = \overline{1}(0) + \overline{1}B_{in} + 0(B_{in}) \\
& = 0 + 0 \cdot B_{in} + 0 = \mathbf{0} && \text{(Annulment Law: } 0 \cdot A = 0) \\
\mathbf{I_3}\ (\mathbf{XY = 11}): \quad & B_{out} = \overline{1}(1) + \overline{1}B_{in} + 1(B_{in}) \\
& = 0 \cdot 1 + 0 \cdot B_{in} + B_{in} = \mathbf{B_{in}} && \text{(Identity and Annulment Laws)}
\end{aligned}$$

4. Logic Circuit Implementation

Using the derived data lines, we construct the twin 4×1 MUX circuit for the Full Subtractor.





Q11. Using an 8×1 line MUX, design a logic circuit for $f(w, x, y, z) = \text{POS}(0, 2, 5, 6, 7, 8, 9, 10)$, where the MUX has selector bits $S_2S_1S_0$, input lines $I_0 \dots I_7$, and active-low enable $\overline{EN}$. (If $S_2S_1S_0 = 000$, I_0 is selected; …and so on.) **(15 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Answer

Step 1: Convert the Function to Sum of Products (SOP) Form

The given boolean function is expressed as a Product of Sums (POS), characterized by its maxterms. Since there are 4 variables (w, x, y, z), the total number of fundamental combinations is $2^4 = 16$, corresponding to indices 0 to 15.

We can easily switch from POS to SOP (Sum of Products) by finding the complementary indices, as a function contains either the minterm or the maxterm for every possible input combination:

$$\begin{aligned} f(w, x, y, z) &= \prod M(0, 2, 5, 6, 7, 8, 9, 10) \\ &= \sum m(1, 3, 4, 11, 12, 13, 14, 15) \end{aligned}$$

Step 2: Assign Variables to Multiplexer Control Lines

We must implement this 4-variable function using an 8×1 Multiplexer. An 8×1 MUX has 3 selector lines (S_2, S_1, S_0). We partition our 4 variables such that the most significant variables are tied to the selectors, and the least significant variable acts as the data input to the MUX.

- **Selector Lines:** Connect $w \to S_2$, $x \to S_1$, and $y \to S_0$.
- **Data Input Variable:** The remaining variable is z. Thus, the data inputs $I_0, I_1, \dots, I_7$ will be derived as functions of z (i.e., $0, 1, z,$ or $\overline{z}$).

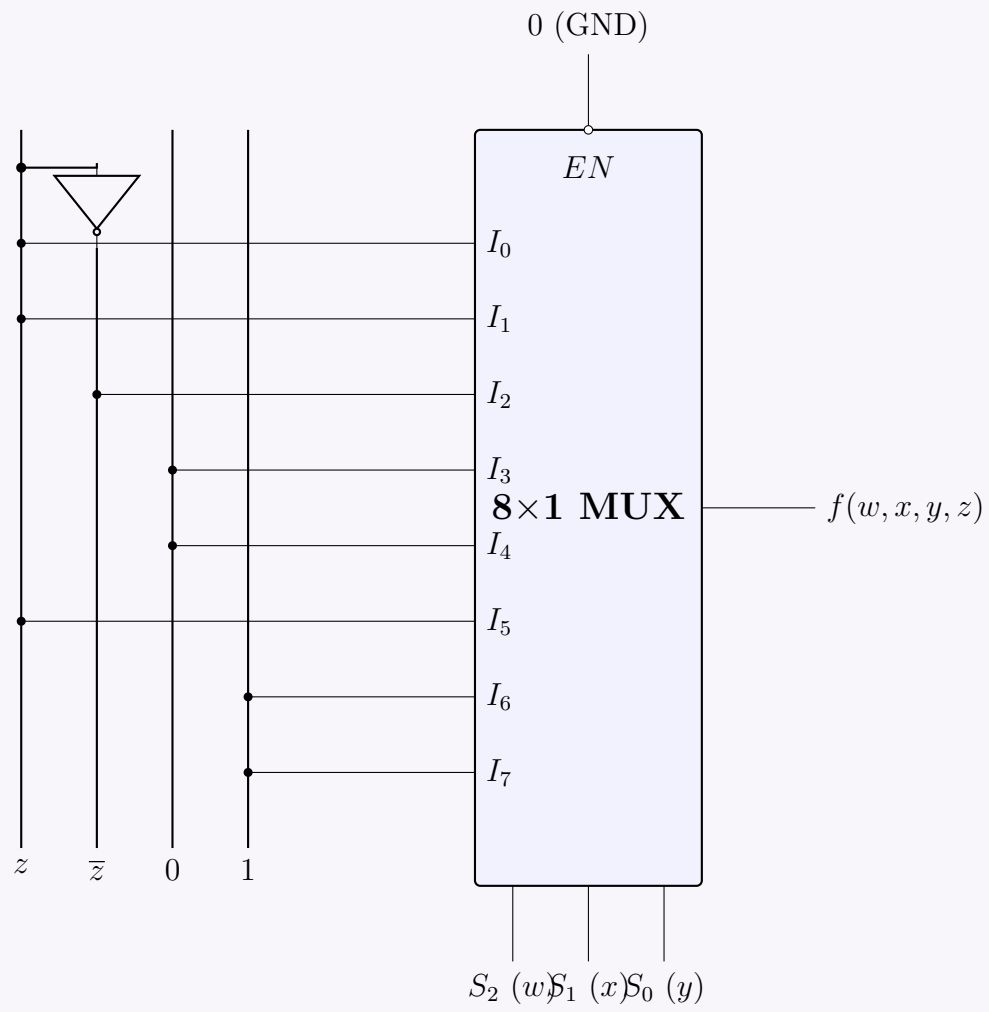
Step 3: Construct the Multiplexer Implementation Table

We create an implementation table mapping the 16 possible minterms into the 8 data inputs based on the state of z. Each data line I_k controls a pair of minterms: one where $z = 0$ (even index) and one where $z = 1$ (odd index). We circle or bold the minterms present in our SOP expression to determine the logical expression for each I_k.

$S_2(w)$	$S_1(x)$	$S_0(y)$	MUX Input	$z = 0$	$z = 1$	Logic Function
0	0	0	I_0	0	**1**	$I_0 = z$
0	0	1	I_1	2	**3**	$I_1 = z$
0	1	0	I_2	**4**	5	$I_2 = \overline{z}$
0	1	1	I_3	6	7	$I_3 = 0$
1	0	0	I_4	8	9	$I_4 = 0$
1	0	1	I_5	10	**11**	$I_5 = z$
1	1	0	I_6	**12**	**13**	$I_6 = 1$
1	1	1	I_7	**14**	**15**	$I_7 = 1$

Step 4: Circuit Diagram

The MUX has an active-low enable $\overline{EN}$. For the multiplexer to function normally, this pin must be tied to a logic low state (0 or GND).



Q12. Design a 1-bit Full Adder using two 4×1 multiplexers. The MUX has selector bits S_1S_0, input lines I_0, I_1, I_2, I_3, and active-low enable $\overline{EN}$. (If $S_1S_0 = 00$, I_0 is selected; $01 \rightarrow I_1$; $10 \rightarrow I_2$; $11 \rightarrow I_3$.) **(12 marks)** **[CSE 205 | L-2/T-1 | 2017–18]**

Answer

Step 1: Truth Table of a 1-bit Full Adder

A 1-bit full adder computes the sum of three bits: A, B, and C_{in}. It produces two outputs: Sum and C_{out} (Carry-out). The truth table for these operations is defined as follows:

Inputs			**Outputs**	
A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Step 2: Assign Variables to Multiplexer Selectors

We are tasked with implementing the two output functions (Sum and C_{out}) using two 4×1 multiplexers. A 4×1 MUX requires 2 selector lines (S_1, S_0). We will map the inputs A and B to the selector lines of both multiplexers:

- $S_1 = A$
- $S_0 = B$

The remaining input, C_{in}, will be used to derive the data inputs I_0, I_1, I_2, I_3 for each MUX.

Step 3: Deriving MUX Inputs

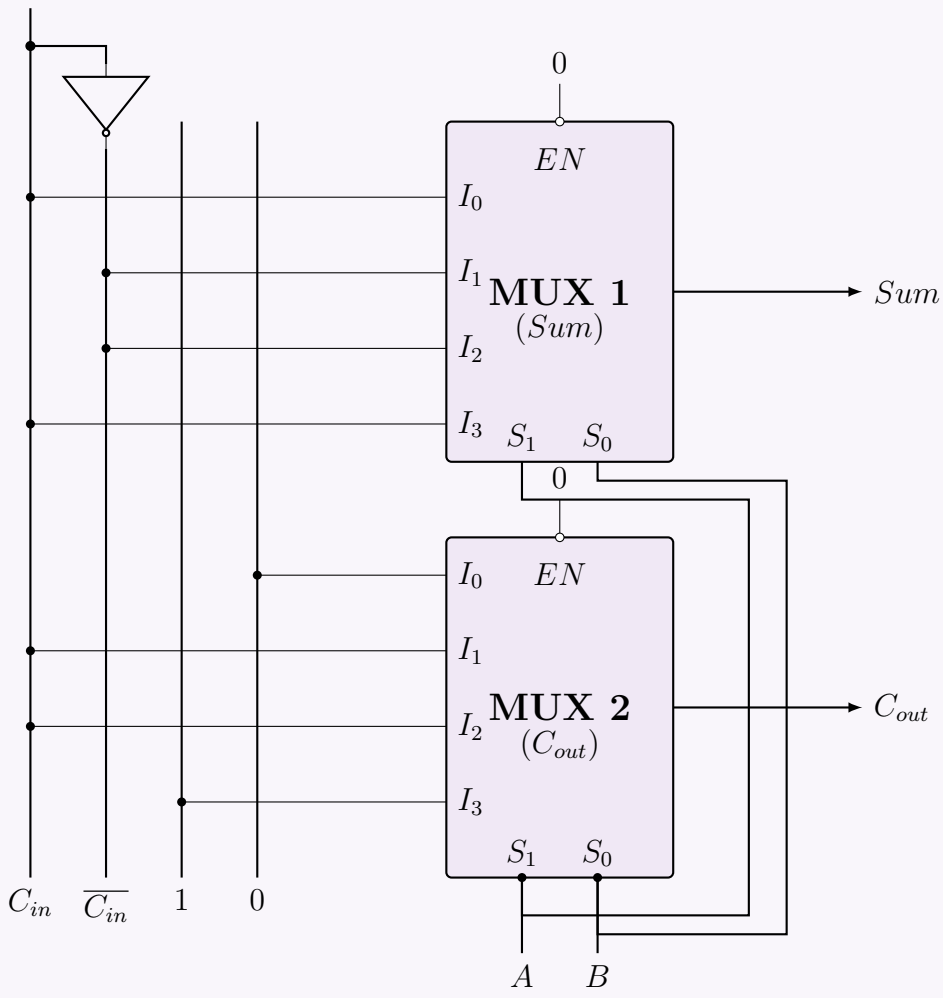
We evaluate the relationship between the outputs and C_{in} for every combination of A and B.
For the Sum Multiplexer (MUX 1):

$$
\begin{aligned}
A = 0, B = 0 \ (S_1S_0 = 00) &\implies Sum = C_{in} && \therefore I_0 = C_{in} \\
A = 0, B = 1 \ (S_1S_0 = 01) &\implies Sum = \overline{C_{in}} && \therefore I_1 = \overline{C_{in}} \\
A = 1, B = 0 \ (S_1S_0 = 10) &\implies Sum = \overline{C_{in}} && \therefore I_2 = \overline{C_{in}} \\
A = 1, B = 1 \ (S_1S_0 = 11) &\implies Sum = C_{in} && \therefore I_3 = C_{in}
\end{aligned}
$$

For the Carry-out Multiplexer (MUX 2):

$$\begin{aligned}
A=0, B=0\ (S_1S_0=00) &\implies C_{out}=0 && \therefore I_0=0\\
A=0, B=1\ (S_1S_0=01) &\implies C_{out}=C_{in} && \therefore I_1=C_{in}\\
A=1, B=0\ (S_1S_0=10) &\implies C_{out}=C_{in} && \therefore I_2=C_{in}\\
A=1, B=1\ (S_1S_0=11) &\implies C_{out}=1 && \therefore I_3=1
\end{aligned}$$

Step 4: Circuit Diagram
Both multiplexers have an active-low enable $\overline{EN}$. This pin must be tied to a logic low state (0 or GND) for normal operation. The data rails provide $0, 1, C_{in}$, and $\overline{C_{in}}$.



Q13. Using only 4×1 line, 2-bit multiplexers, design $f(w, x, y, z) = \pi\,(2, 4, 5, 7, 11, 12, 13)$. (Use minimum gates if required.) **(12 marks)**
[CSE 205 | L-2/T-1 | 2016–17]

Answer

Step 1: Convert the Function to Sum of Products (SOP) Form
The given boolean function is provided in Product of Sums (POS) form, specifying the maxterms. For a 4-variable function (w, x, y, z), the domain ranges from 0 to 15. We convert this to Sum of Products (SOP) by identifying the missing indices, which correspond to the minterms:

$$\begin{aligned}
f(w, x, y, z) &= \prod M(2, 4, 5, 7, 11, 12, 13)\\
&= \sum m(0, 1, 3, 6, 8, 9, 10, 14, 15)
\end{aligned}$$

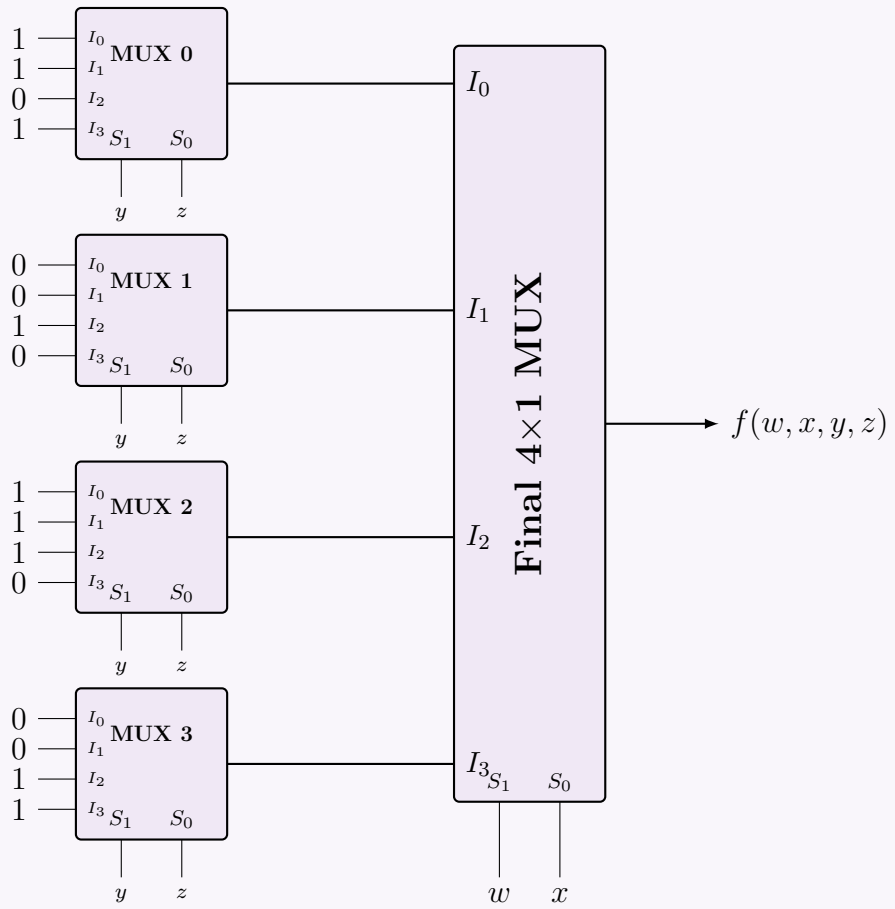
Step 2: Multiplexer Tree Design Strategy (Shannon's Expansion)
The prompt restricts us to using **only** 4×1 line multiplexers. A single 4×1 MUX has 2 selector lines and 4 data inputs, which inherently evaluates any 2-variable logic function by tying its inputs directly to 0 or 1.
Because we have a 4-variable function and cannot use external logic gates (AND, OR, NOT), we must construct a **Multiplexer Tree** to form an equivalent 16×1 multiplexer.

- **Level 2 (Output Stage):** A single 4×1 MUX controlled by the two Most Significant Bits (MSBs), w and x.
- **Level 1 (Input Stage):** Four 4×1 MUXes controlled by the two Least Significant Bits (LSBs), y and z. The outputs of these four MUXes feed directly into the Output Stage MUX.

Step 3: Deriving the MUX Data Inputs
We evaluate the function for all 16 combinations. The bits w and x select which Level 1 MUX is active, while y and z select the specific input ($0, 1, 2$, or 3) inside that Level 1 MUX.

Level 2 Selectors		**Minterm Values for y, z (S_1, S_0)**				**Level 1 MUX**
$w\,(S_1)$	$x\,(S_0)$	$00\,(I_0)$	$01\,(I_1)$	$10\,(I_2)$	$11\,(I_3)$	**Assignment**
0	0	$m_0 = \mathbf{1}$	$m_1 = \mathbf{1}$	$m_2 = 0$	$m_3 = \mathbf{1}$	**MUX 0** inputs: $1, 1, 0, 1$
0	1	$m_4 = 0$	$m_5 = 0$	$m_6 = \mathbf{1}$	$m_7 = 0$	**MUX 1** inputs: $0, 0, 1, 0$
1	0	$m_8 = \mathbf{1}$	$m_9 = \mathbf{1}$	$m_{10} = \mathbf{1}$	$m_{11} = 0$	**MUX 2** inputs: $1, 1, 1, 0$
1	1	$m_{12} = 0$	$m_{13} = 0$	$m_{14} = \mathbf{1}$	$m_{15} = \mathbf{1}$	**MUX 3** inputs: $0, 0, 1, 1$

Step 4: Circuit Diagram
Below is the complete MUX tree. Notice that no logic gates are required; all primary inputs to the Level 1 multiplexers are tied strictly to logic high (1) or logic low (0).



Q14. Design and implement the circuit diagram of a 1-bit full subtractor circuit using 2×1 MUXs only. You cannot use any other gates. **(12 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Answer

Step 1: Truth Table of a 1-bit Full Subtractor
A full subtractor performs the operation $A - B - B_{in}$, producing a Difference (D) and a Borrow-out (B_{out}).

Inputs			**Outputs**	
A (Minuend)	B (Subtrahend)	B_{in} (Borrow-in)	D (Difference)	B_{out} (Borrow-out)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Step 2: Mathematical Derivation using Shannon's Expansion
To implement these functions using **only** 2×1 multiplexers, we use Shannon's Expansion Theorem to factor the expressions around the variables A and B, since a 2×1 MUX natively implements the logic $Y = \overline{S}I_0 + SI_1$.

1. Difference (D):

$$D(A, B, B_{in}) = \overline{A} \cdot \left[\overline{B}(B_{in}) + B(\overline{B_{in}})\right] + A \cdot \left[\overline{B}(\overline{B_{in}}) + B(B_{in})\right]$$

This directly implies a 2-level MUX tree:

- **MUX 1** ($A = 0$ branch, Selector=B): $I_0 = B_{in}$, $I_1 = \overline{B_{in}}$
- **MUX 2** ($A = 1$ branch, Selector=B): $I_0 = \overline{B_{in}}$, $I_1 = B_{in}$
- **MUX 3** (Output stage, Selector=A): $I_0 = \text{MUX } 1_{out}$, $I_1 = \text{MUX } 2_{out}$

2. Borrow-out (B_{out}):

$$B_{out}(A, B, B_{in}) = \overline{A} \cdot \left[\overline{B}(B_{in}) + B(1)\right] + A \cdot \left[\overline{B}(0) + B(B_{in})\right]$$

Similarly, this gives us:

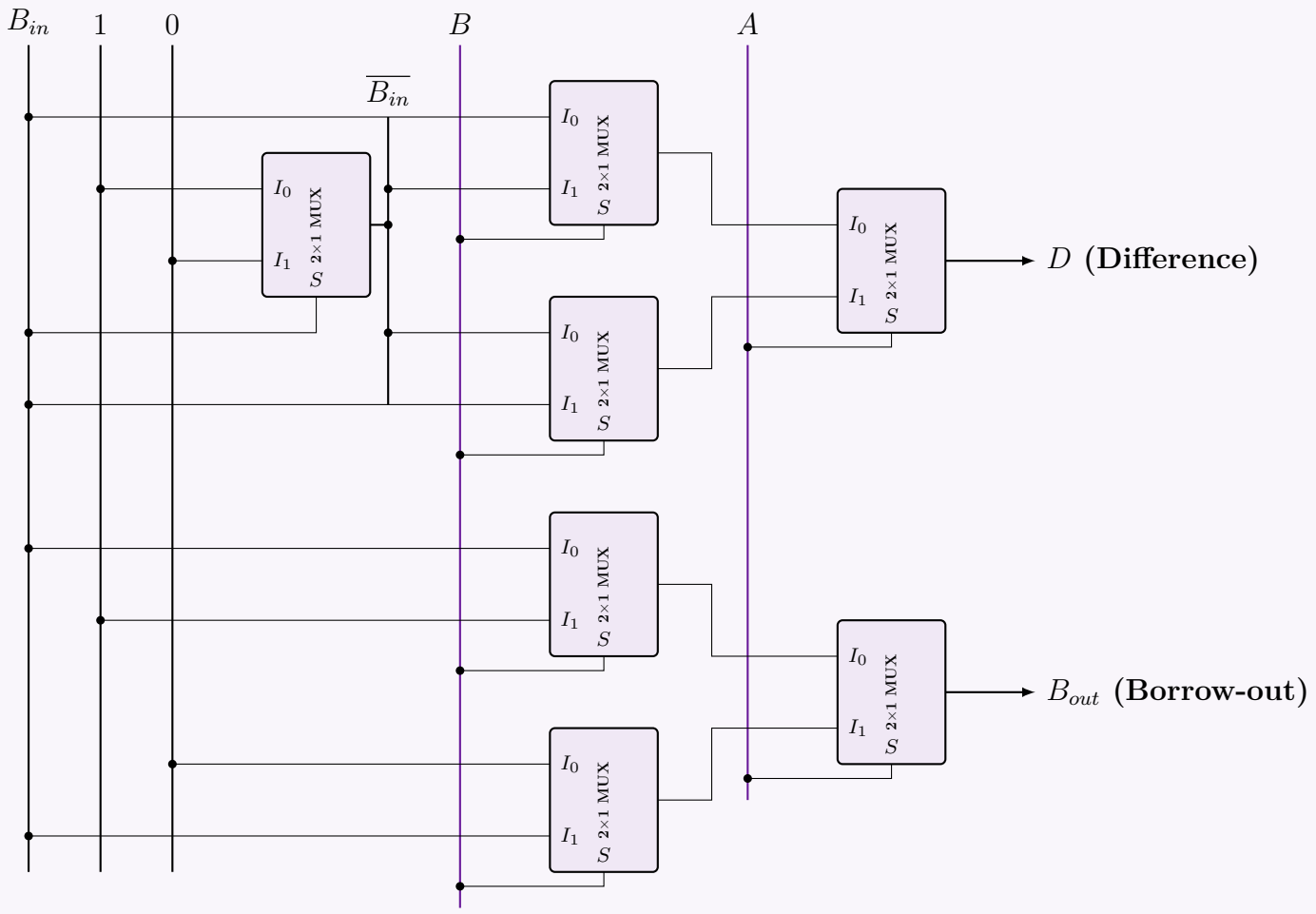
- **MUX 4** ($A = 0$ branch, Selector=B): $I_0 = B_{in}$, $I_1 = 1$
- **MUX 5** ($A = 1$ branch, Selector=B): $I_0 = 0$, $I_1 = B_{in}$
- **MUX 6** (Output stage, Selector=A): $I_0 = \text{MUX } 4_{out}$, $I_1 = \text{MUX } 5_{out}$

3. NOT Gate for $\overline{B_{in}}$: Because we cannot use standard logic gates, we must generate $\overline{B_{in}}$ using a 2×1 MUX:

$$\overline{B_{in}} = \overline{B_{in}} \cdot (1) + B_{in} \cdot (0)$$

- **MUX 0** (Inverter, Selector=B_{in}): $I_0 = 1$, $I_1 = 0$

Step 3: Circuit Implementation



Q15. Consider the Boolean function $f(w, x, y, z) = \Pi\,(2, 4, 5, 7, 11, 12, 13)$. Implement the function using a 4×1 MUX and other basic gates if necessary. **(10 marks)** **[CSE 205 | L-2/T-1 | 2015–16]**

Answer

Step 1: Convert the Function to Sum of Products (SOP) Form
The given Boolean function is provided in Product of Sums (POS) form, specifying the maxterms. For a 4-variable function (w, x, y, z), the domain ranges from 0 to 15. We first convert this to the Sum of Products (SOP) form by identifying the missing indices, which correspond to the minterms:

$$\begin{aligned} f(w, x, y, z) &= \prod M(2, 4, 5, 7, 11, 12, 13) \\ &= \sum m(0, 1, 3, 6, 8, 9, 10, 14, 15) \end{aligned}$$

Step 2: Assign Variables to the Multiplexer Selectors
We are required to implement the function using a 4×1 multiplexer. A 4×1 MUX requires exactly 2 selector lines (S_1, S_0). We will assign the two Most Significant Bits (MSBs) to the selector lines:

- $S_1 = w$
- $S_0 = x$

This implies that the multiplexer's data inputs (I_0, I_1, I_2, I_3) will be derived as Boolean expressions in terms of the remaining variables, y and z.

Step 3: Deriving MUX Data Inputs
We evaluate the minterms corresponding to each selector combination to find the expressions for I_0, I_1, I_2, I_3.
1. For $w = 0, x = 0$ (Selects I_0): This branch covers minterms m_0, m_1, m_2, m_3. The minterms present in the function are m_0, m_1, m_3.

$$\begin{aligned} I_0(y,z) &= \bar{y}\bar{z} + \bar{y}z + yz \\ &= \bar{y}(\bar{z} + z) + yz && \text{(Factoring out } \bar{y}) \\ &= \bar{y}(1) + yz && \text{(Complement Law: } \bar{z} + z = 1) \\ &= \bar{y} + yz \\ &= (\bar{y} + y)(\bar{y} + z) && \text{(Distributive Law)} \\ &= \bar{y} + z \end{aligned}$$

2. For $w = 0, x = 1$ (Selects I_1): This branch covers minterms m_4, m_5, m_6, m_7. The only minterm present is m_6.

$$I_1(y,z) = y\bar{z}$$

3. For $w = 1, x = 0$ (Selects I_2): This branch covers minterms m_8, m_9, m_{10}, m_{11}. The minterms present are m_8, m_9, m_{10}.

$$\begin{aligned} I_2(y,z) &= \bar{y}\bar{z} + \bar{y}z + y\bar{z} \\ &= \bar{y}(\bar{z} + z) + y\bar{z} \\ &= \bar{y} + y\bar{z} \\ &= (\bar{y} + y)(\bar{y} + \bar{z}) && \text{(Distributive Law)} \\ &= \bar{y} + \bar{z} \\ &= \overline{yz} && \text{(De Morgan's Law)} \end{aligned}$$

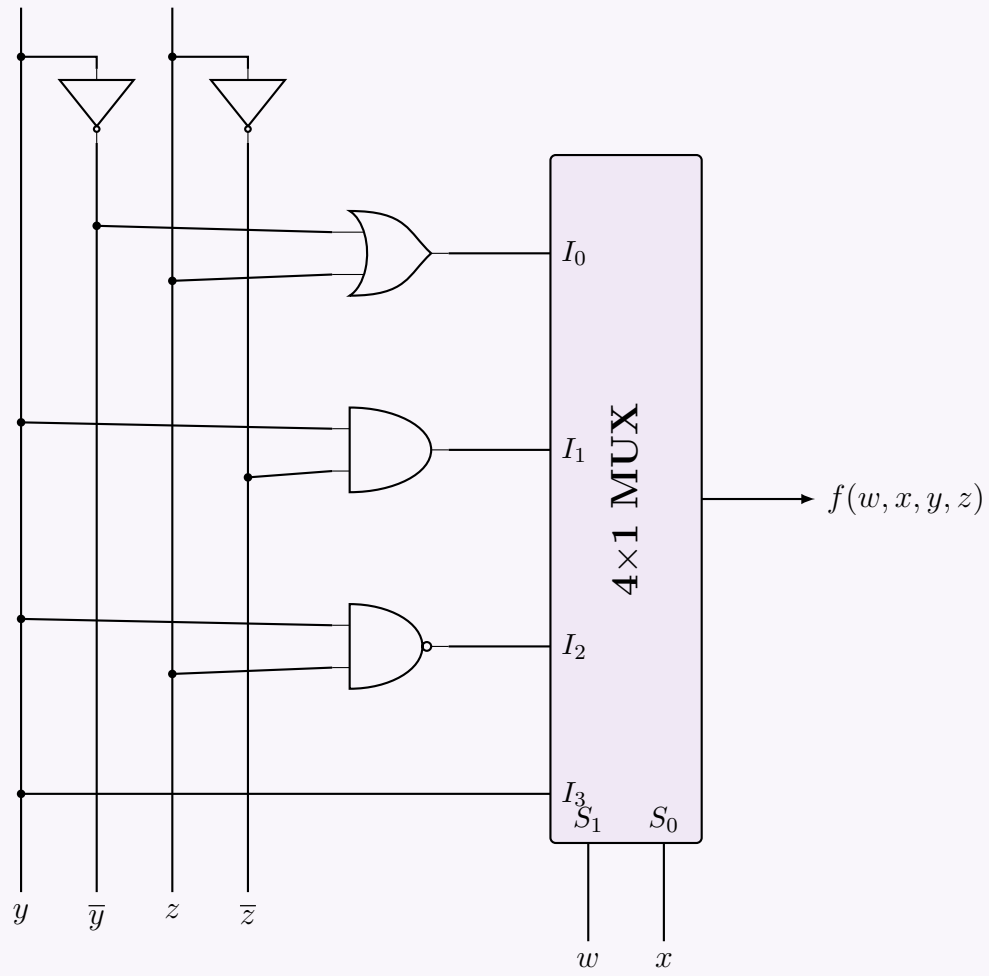
4. For $w = 1, x = 1$ (Selects I_3): This branch covers minterms $m_{12}, m_{13}, m_{14}, m_{15}$. The minterms present are m_{14}, m_{15}.

$$\begin{aligned} I_3(y,z) &= y\bar{z} + yz \\ &= y(\bar{z} + z) \\ &= y \end{aligned}$$

Summary of Inputs:

w (S_1)	x (S_0)	**Active Minterms** (y, z)	**Input Equation**
0	0	m_0, m_1, m_3	$I_0 = \bar{y} + z$
0	1	m_6	$I_1 = y\bar{z}$
1	0	m_8, m_9, m_{10}	$I_2 = \overline{yz}$
1	1	m_{14}, m_{15}	$I_3 = y$

Step 4: Circuit Diagram
The logical expressions for the inputs can be implemented using standard basic gates: an OR gate for I_0, an AND gate for I_1, and a NAND gate for I_2 (implementing $\bar{y} + \bar{z}$ directly via De Morgan's theorem). Inverters are used to generate $\bar{y}$ and $\bar{z}$.



Q16. Show the truth table for a three-state buffer. Construct a 2-to-1 line MUX with three-state buffers. **(3+5 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Answer

1. Truth Table for a Three-State Buffer

A three-state (or tri-state) buffer acts as a standard buffer but includes a third operational state: **High-Impedance** (Z). It is controlled by an Enable line (E).

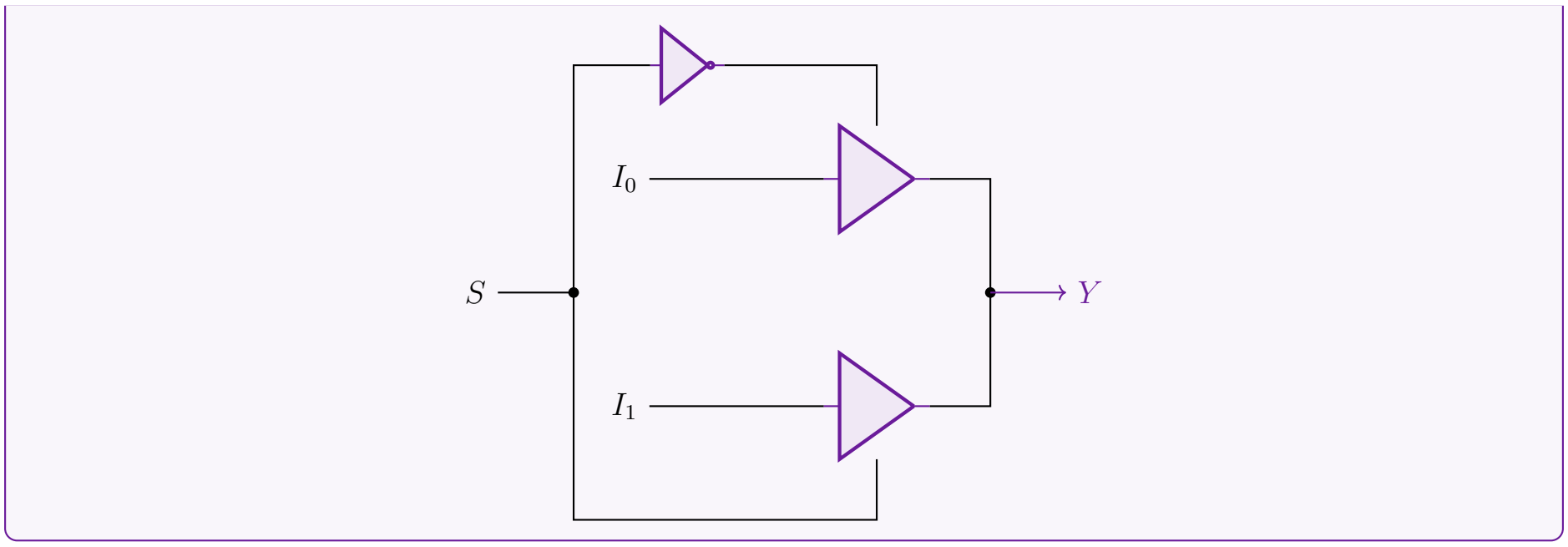
- When $E = 1$ (Active), the buffer acts normally, passing the input A directly to the output Y.
- When $E = 0$ (Inactive), the output is disconnected from the input, placing the output in a High-Impedance state (Z), effectively acting as an open circuit.

Inputs		**Output**
Enable (E)	**Data** (A)	Y
0	X (Don't Care)	**Z** (High-Impedance)
1	0	**0**
1	1	**1**

2. Construction of a 2-to-1 MUX using Three-State Buffers

A 2-to-1 Multiplexer routes one of two input signals (I_0 or I_1) to a single output (Y) based on the value of a Select line (S). Using three-state buffers, we can construct this by connecting the outputs of two buffers together. To avoid bus contention (short circuits), we must ensure that **only one buffer is enabled at any given time**.

- **When $S = 0$:** The inverted signal $\overline{S} = 1$ enables the top buffer. Input I_0 is passed to Y. The bottom buffer is in state Z.
- **When $S = 1$:** The direct signal $S = 1$ enables the bottom buffer. Input I_1 is passed to Y. The top buffer is in state Z.



Q17. An 8×1 multiplexer has inputs A, B, C, D where inputs A, B, and C are connected to selection inputs S_2, S_1, and S_0 respectively. The data inputs I_0 through I_7 are: $I_1 = I_2 = 0$; $I_3 = I_7 = 1$; $I_4 = I_5 = D$; $I_0 = I_6 = D'$. Determine the Boolean function that the multiplexer implements. **(15 marks)** **[CSE 205 | L-2/T-1 | 2013–14]**

Answer

1. Characteristic Equation of the 8×1 Multiplexer

An 8×1 multiplexer outputs a Boolean function determined by the logical sum of the products of its selection line minterms and their corresponding data inputs. The general output equation F is given by:

$$F(S_2, S_1, S_0) = \sum_{k=0}^{7} m_k \cdot I_k$$

where m_k represents the k-th minterm of the selection variables.
Given the selection inputs $S_2 = A$, $S_1 = B$, and $S_0 = C$, we can map the given data inputs I_0 through I_7 as follows:

Selection Lines			**Data Input**	**Assigned Value**
A (S_2)	B (S_1)	C (S_0)	I_k	$F(A, B, C, D)$
0	0	0	I_0	D'
0	0	1	I_1	0
0	1	0	I_2	0
0	1	1	I_3	1
1	0	0	I_4	D
1	0	1	I_5	D
1	1	0	I_6	D'
1	1	1	I_7	1

2. Algebraic Formulation

Substituting the given data inputs into the characteristic equation:

$$\begin{aligned} F(A, B, C, D) &= A'B'C'(D') + A'B'C(0) + A'BC'(0) + A'BC(1) \\ &\quad + AB'C'(D) + AB'C(D) + ABC'(D') + ABC(1) \\ &= A'B'C'D' + 0 + 0 + A'BC + AB'C'D + AB'CD + ABC'D' + ABC \\ &= A'B'C'D' + A'BC + AB'C'D + AB'CD + ABC'D' + ABC \end{aligned}$$

3. Conversion to Canonical Sum of Minterms (SOP)

To find the standard minterms Σm, we expand the terms missing the variable D by multiplying them by $(D + D') = 1$ (Complement Law):

$$A'BC = A'BC(D + D') = A'BCD + A'BCD' \qquad \text{(Minterms } 7, 6\text{)}$$
$$ABC = ABC(D + D') = ABCD + ABCD' \qquad \text{(Minterms } 15, 14\text{)}$$

The remaining terms are already standard minterms:

- $A'B'C'D' \rightarrow m_0$

- $AB'C'D \rightarrow m_9$
- $AB'CD \rightarrow m_{11}$
- $ABC'D' \rightarrow m_{12}$

Thus, the canonical Boolean function implemented by the multiplexer is:

$$\mathbf{F(A,B,C,D) = \Sigma m(0,6,7,9,11,12,14,15)}$$

4. Logic Minimization using Karnaugh Map

We map the active minterms onto a 4-variable K-map to determine the simplified Boolean expression.

AB \ CD	00	01	11	10
00	1	0	0	0
01	0	0	1	1
11	1	0	1	1
10	0	1	1	0

Group Extractions:

- **Red / 2×2 Block (m_6, m_7, m_{14}, m_{15}):** Variables A and D change, leaving **BC**.
- **Green / Horizontal Pair (m_9, m_{11}):** Variable C changes, leaving **AB′D**.
- **Blue / Edge Pair (m_{12}, m_{14}):** Variable C changes, leaving **ABD′**.
- **Yellow / Isolated Cell (m_0):** No variables change, leaving **A′B′C′D′**.

Final Minimal Boolean Function:

$$\mathbf{F(A,B,C,D) = BC + AB'D + ABD' + A'B'C'D'}$$

Q18. Construct a 16×1 multiplexer with two 8×1 and one 2×1 multiplexers. **(5 marks)** **[CSE 205 | L-2/T-1 | 2013–14]**

Answer

1. Design Strategy and Logic Formulation

A 16×1 multiplexer routes one of 16 data inputs (I_0 to I_{15}) to a single output (Y) based on a 4-bit selection code (S_3, S_2, S_1, S_0), where S_3 is the Most Significant Bit (MSB).
Since we have 8×1 multiplexers available (which accept 3 selection bits and 8 data inputs), we can partition the 16 inputs into two groups of 8:

- **Lower Half (I_0 to I_7):** Selected when the MSB $S_3 = 0$.
- **Upper Half (I_8 to I_{15}):** Selected when the MSB $S_3 = 1$.

Configuration Step-by-Step:

(a) **First Stage (8×1 Multiplexers):** The three Least Significant Bits (LSBs) S_2, S_1, S_0 are applied simultaneously to the selection lines of both 8×1 multiplexers.

$$F_1 = \Sigma_{k=0}^{7} m_k(S_2, S_1, S_0) \cdot I_k \qquad \text{(Output of top MUX)}$$
$$F_2 = \Sigma_{k=0}^{7} m_k(S_2, S_1, S_0) \cdot I_{k+8} \qquad \text{(Output of bottom MUX)}$$

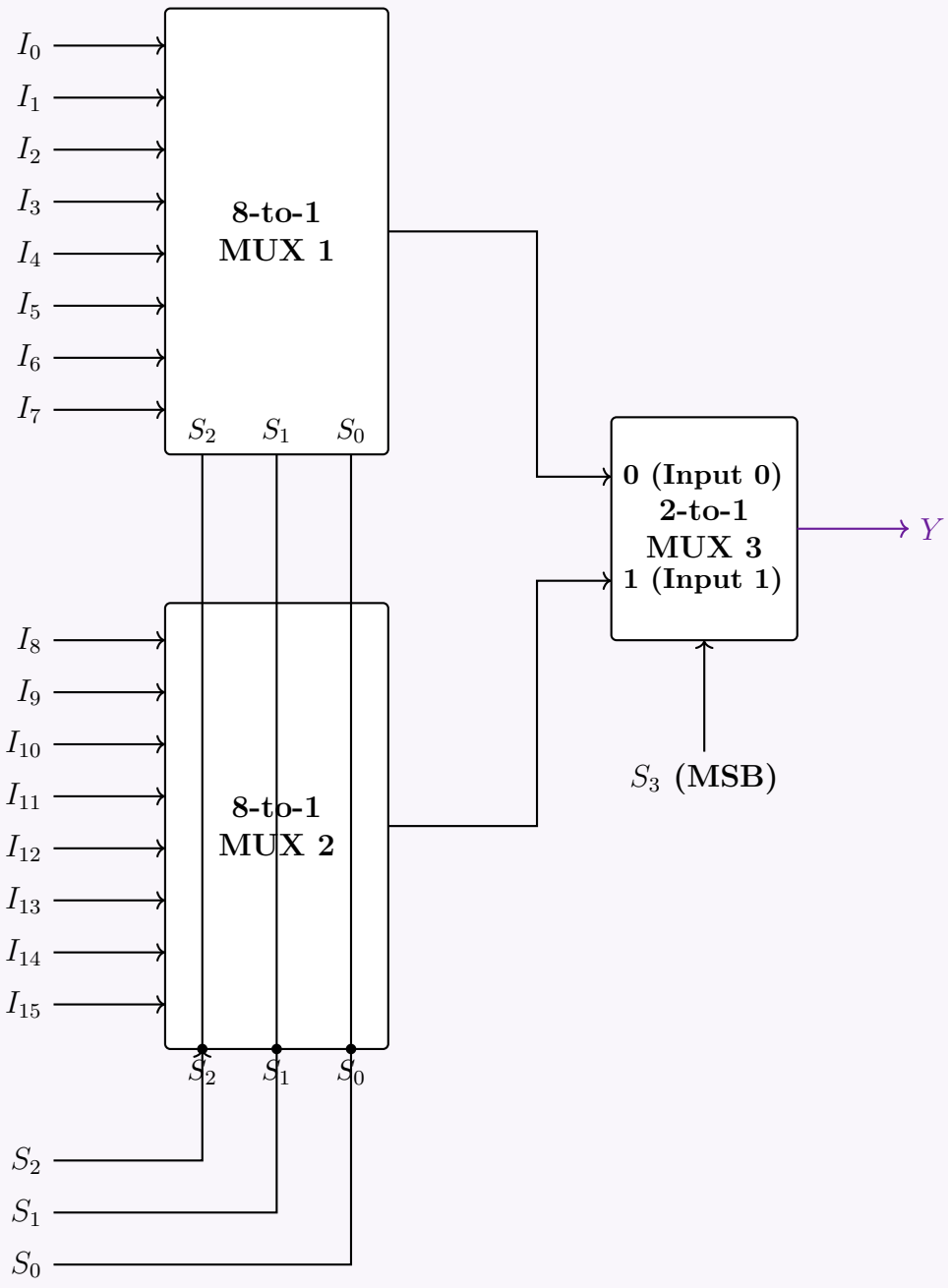
(b) **Second Stage (2×1 Multiplexer):** The outputs F_1 and F_2 serve as the data inputs to the 2×1 multiplexer. The MSB S_3 acts as the selection line for this final multiplexer to toggle between the lower and upper data banks.

$$Y = \overline{S_3} \cdot F_1 + S_3 \cdot F_2 \qquad \text{(Multiplexer logical expansion)}$$

2. Truth Table Summary

Selection Lines				Active MUX in 1st Stage	Selected Output (Y)
S_3	S_2	S_1	S_0		
0	0	0	0	MUX 1 (Top)	I_0
⋮	⋮	⋮	⋮		⋮
0	1	1	1		I_7
1	0	0	0	MUX 2 (Bottom)	I_8
⋮	⋮	⋮	⋮		⋮
1	1	1	1		I_{15}

3. Logic Circuit Implementation



Q19. Implement $F = \Sigma\,(3, 5, 7, 10, 11, 13, 14, 15)$ using a 16-to-1 MUX. **(10 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Answer

1. MUX Implementation Strategy

A 16-to-1 Multiplexer has 4 selection lines (S_3, S_2, S_1, S_0) and 16 data inputs (I_0 to I_{15}). To implement a 4-variable Boolean function $F(A, B, C, D)$ directly using a 16-to-1 MUX, we follow a straightforward mapping technique:

- Connect the 4 Boolean variables directly to the 4 selection lines: $S_3 = A$, $S_2 = B$, $S_1 = C$, and $S_0 = D$.
- Connect the data inputs I_i corresponding to the active minterms (present in the function) to **Logic 1** (V_{cc}).
- Connect the data inputs I_i corresponding to the absent minterms to **Logic 0** (Ground).

Given the function:

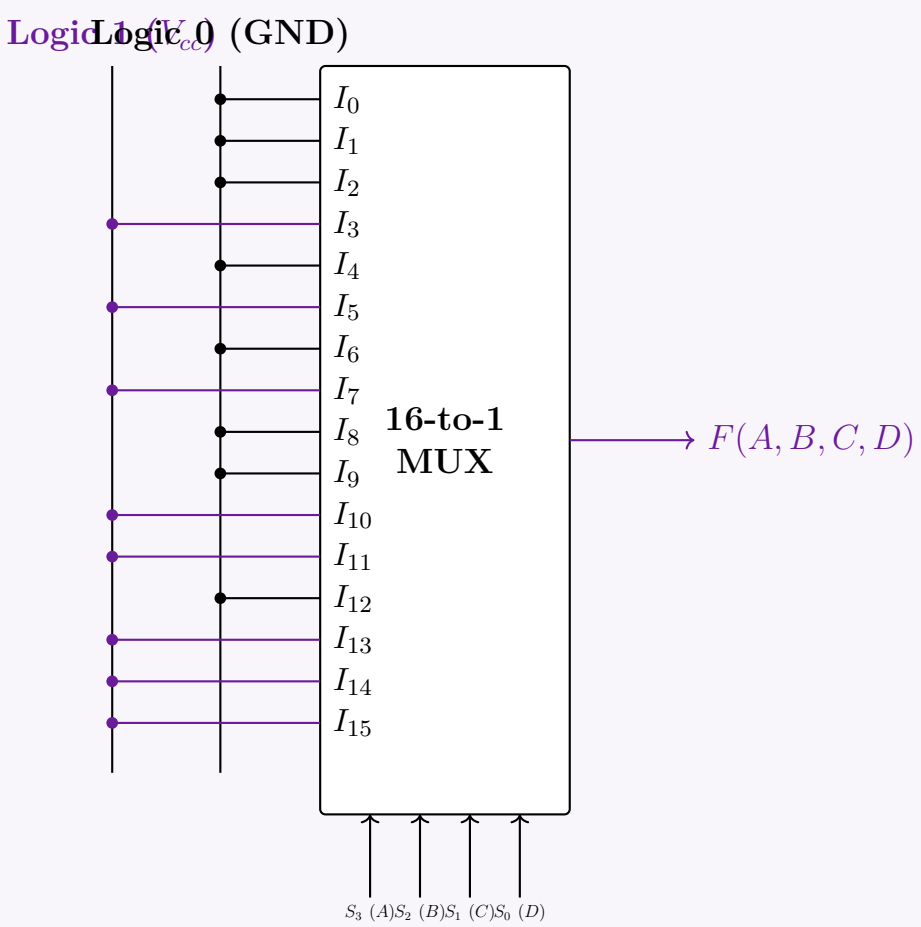
$$F(A, B, C, D) = \Sigma\,(3, 5, 7, 10, 11, 13, 14, 15)$$

The active minterms are $m_3, m_5, m_7, m_{10}, m_{11}, m_{13}, m_{14}, m_{15}$.

2. Input Mapping Table

Minterm	$A(S_3)$	$B(S_2)$	$C(S_1)$	$D(S_0)$	Data Input	Assigned Value
m_0, m_1, m_2	Various combinations				I_0, I_1, I_2	**0** (GND)
m_3	0	0	1	1	I_3	**1** (V_{cc})
m_4	0	1	0	0	I_4	**0** (GND)
m_5	0	1	0	1	I_5	**1** (V_{cc})
m_6	0	1	1	0	I_6	**0** (GND)
m_7	0	1	1	1	I_7	**1** (V_{cc})
m_8, m_9	Various combinations				I_8, I_9	**0** (GND)
m_{10}, m_{11}	Various combinations				I_{10}, I_{11}	**1** (V_{cc})
m_{12}	1	1	0	0	I_{12}	**0** (GND)
m_{13}, m_{14}, m_{15}	Various combinations				I_{13}, I_{14}, I_{15}	**1** (V_{cc})

3. Circuit Diagram



Q20. Can the function $F = \Sigma\,(3, 5, 7, 10, 11, 13, 14, 15)$ be implemented using only one 4×1 MUX? If yes, show how; if not, explain why. **(10 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

Answer

Conclusion: Yes, the function $F(A, B, C, D) = \Sigma\,(3, 5, 7, 10, 11, 13, 14, 15)$ **can** be implemented using strictly one 4×1 multiplexer (MUX) and absolutely no additional logic gates. This is achieved by strategically choosing variables C and D as the selection lines.

1. Theoretical Foundation

A 4×1 MUX implements any logic function conforming to its characteristic equation:

$$Y = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3$$

where S_1, S_0 are the select lines and I_0, I_1, I_2, I_3 are the data inputs.
To implement a 4-variable function using only one 4×1 MUX (which has 2 select lines), we must map two variables to the select lines. The remaining two variables will dictate the logic required at the data inputs. To avoid using external logic gates (such as AND/OR/NOT gates), the data inputs must resolve entirely to constants (0 or 1) or single input variables (uncomplemented or complemented).

2. Mathematical Derivation (Shannon's Expansion)

We apply Shannon's Expansion Theorem to factor out the variables C and D, setting them as our select variables ($S_1 = C$, $S_0 = D$).

First, let us expand the given function into its canonical sum of minterms:

$$\begin{aligned} F(A,B,C,D) &= \sum m(3,5,7,10,11,13,14,15) \\ &= m_3 + m_5 + m_7 + m_{10} + m_{11} + m_{13} + m_{14} + m_{15} \\ &= A'B'CD + A'BC'D + A'BCD + AB'CD' + AB'CD + ABC'D + ABCD' + ABCD \end{aligned}$$

Next, we group the terms based on the four possible combinations of C and D:

$$\begin{aligned} F = {} & C'D'(0) && \text{(No minterms end in } C=0, D=0) \\ & + C'D\,(A'B + AB) && \text{(From } m_5 \text{ and } m_{13}) \\ & + CD'\,(AB' + AB) && \text{(From } m_{10} \text{ and } m_{14}) \\ & + CD\,(A'B' + A'B + AB' + AB) && \text{(From } m_3, m_7, m_{11}, m_{15}) \end{aligned}$$

Now, we systematically simplify the expressions for each MUX input using Boolean algebra:

- **For $C'D'$ (I_0):**
  The coefficient is 0.
  $\therefore$ **$I_0 = 0$**
- **For $C'D$ (I_1):**

$$\begin{aligned} I_1 &= A'B + AB \\ &= B(A' + A) && \text{(Distributive Law)} \\ &= B(1) && \text{(Complement Law: } A' + A = 1) \\ \therefore \mathbf{I_1} &= \mathbf{B} && \text{(Identity Law)} \end{aligned}$$

- **For CD' (I_2):**

$$\begin{aligned} I_2 &= AB' + AB \\ &= A(B' + B) && \text{(Distributive Law)} \\ &= A(1) && \text{(Complement Law: } B' + B = 1) \\ \therefore \mathbf{I_2} &= \mathbf{A} && \text{(Identity Law)} \end{aligned}$$

- **For CD (I_3):**

$$\begin{aligned} I_3 &= A'B' + A'B + AB' + AB \\ &= A'(B' + B) + A(B' + B) && \text{(Distributive Law)} \\ &= A'(1) + A(1) && \text{(Complement Law)} \\ &= A' + A && \text{(Identity Law)} \\ \therefore \mathbf{I_3} &= \mathbf{1} && \text{(Complement Law)} \end{aligned}$$

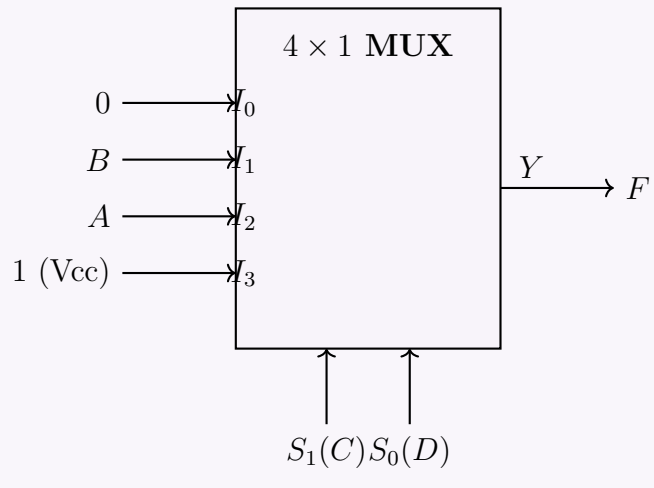
Since all derived inputs (0, B, A, and 1) can be obtained via direct wiring without any logic gates, the implementation is valid.

3. Implementation Table

The following table maps the state of the select lines (C, D) to the corresponding minterms and the necessary data inputs.

S_1 (C)	S_0 (D)	**Data Input**	**Relevant Minterms in F**	**Simplified Input**
0	0	I_0	None	**0**
0	1	I_1	$m_5(0101), m_{13}(1101)$	**B**
1	0	I_2	$m_{10}(1010), m_{14}(1110)$	**A**
1	1	I_3	$m_3(0011), m_7(0111), m_{11}(1011), m_{15}(1111)$	**1**

4. Circuit Diagram



Note: If we had chosen A and B as the select lines instead, the inputs I_0 through I_3 would have required complex expressions in terms of C and D (e.g., $I_3 = C + D$), which would necessitate additional logic gates. The choice of C and D perfectly avoids this constraint.

Q21. Implement the following function F using one 4×1 multiplexer and basic gates:

$$F(A, B, C, D) = \Sigma\,(1, 3, 4, 11, 12, 13, 14, 15)$$

(15 marks) [CSE 205 | L-2/T-1 | 2011–12]

Answer

Conclusion: The function $F(A, B, C, D) = \Sigma\,(1, 3, 4, 11, 12, 13, 14, 15)$ is implemented using a 4×1 multiplexer. Variables A and B are assigned to select lines S_1 and S_0, while variables C and D generate the data inputs I_0, I_1, I_2, and I_3.

1. Theoretical Foundation & Expansion

The 4×1 MUX output is given by:

$$Y = \overline{A}\overline{B}I_0 + \overline{A}BI_1 + A\overline{B}I_2 + ABI_3$$

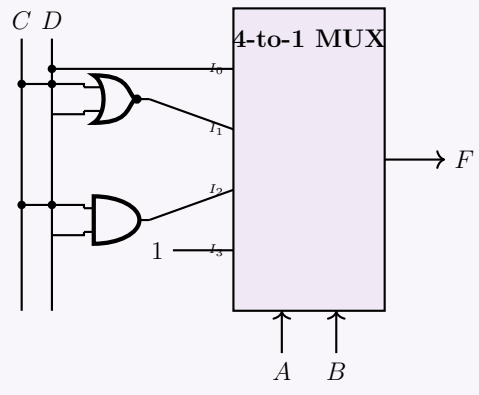
Factoring the function $F(A, B, C, D)$ based on A and B:

$$F = \overline{A}\overline{B}(m_1 + m_3) + \overline{A}B(m_4) + A\overline{B}(m_{11}) + AB(m_{12} + m_{13} + m_{14} + m_{15})$$
$$F = \overline{A}\overline{B}(\overline{C}D + CD) + \overline{A}B(\overline{C}D' \cdot 1) + A\overline{B}(CD \cdot 1) + AB(1)$$

2. Derivation of Data Inputs

- I_0 $(A = 0, B = 0)$: $I_0 = \overline{C}D + CD = D(\overline{C} + C) = D$
- I_1 $(A = 0, B = 1)$: $I_1 = \overline{C}D' = (C + D)'$ (NOR gate)
- I_2 $(A = 1, B = 0)$: $I_2 = CD$ (AND gate)
- I_3 $(A = 1, B = 1)$: $I_3 = 1$ (Tie to V_{cc})

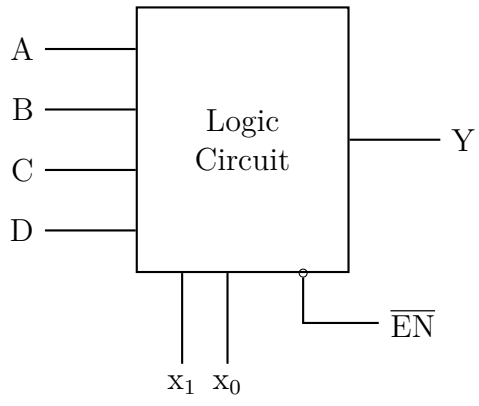
3. Logic Circuit Implementation



Custom Combinational Circuit Design

Q1. Using a 4-bit binary adder and XOR gates, design a circuit which takes two 4-bit BCD inputs A and B and returns: $A + 1$ if input is 00; A' if input is 01; $A' + 1$ if input is 10; A if input is 11. **(13 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Q2. Design a Binary-to-Excess-3 converter using a logic circuit block with inputs A, B, C, D, control lines x_1, x_0, and active-low $\overline{EN}$. Output Y: if $x_1x_0 = 00$, $A \to Y$; if 01, $B \to Y$; if 10, $C \to Y$; if 11, $D \to Y$.



(8 marks) [CSE 205 | L-2/T-1 | 2016–17]

Q3. Design a combinational circuit with 3 inputs x, y, z and 3 outputs A, B, C. When the binary input is 0, 1, 2, or 3, the binary output is two greater than the input. When the binary input is 4, 5, 6, or 7, the binary output is three less than the input. **(18 marks)**
[CSE 205 | L-2/T-1 | 2014–15]

Magnitude Comparator

Q1. Design a logic circuit which finds the maximum value between two 3-bit numbers. Explain the operation using the two numbers 7 and 3. **(10 marks)** **[CSE 205 | L-2/T-1 | 2020–21]**

Q2. Design a logic circuit which returns the smaller value between two 4-bit binary numbers A and B. Show the operation with $A = 7$, $B = 9$. **(12 marks)** **[CSE 205 | L-2/T-1 | 2018–19]**

Q3. Design a digital circuit with logic gates which takes two 3-bit binary numbers X and Y and returns a 3-bit binary number Z equal to the maximum between X and Y. (X can be expressed as $X_2X_1X_0$, Y can be expressed as $Y_2XY_1Y_0$; and Z can be expressed as $Z_2Z_1Z_0$). **(10 marks)** **[CSE 205 | L-2/T-1 | 2017–18]**

Q4. Design and explain a 2-bit magnitude comparator. **(10 marks)** **[CSE 205 | L-2/T-1 | 2016–17]**

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Flip-Flops, Latches & Race-Around Problems

SR, JK, D, T Flip-Flops, Excitation Tables, Race Conditions

S5 Flip-Flops, Latches & Race-Around Problems

SR Latch & SR/JK Undefined Condition

Q1. Discuss why the condition $S = R = 1$ leads to an unstable condition for an SR latch. **(7 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Q2. Consider the following transition table:

$y \setminus x_1x_2$	00	01	11	10
0	(0)	(0)	1	(0)
1	0	(1)	(1)	(1)

Design the circuit with NAND SR latches and gates. **(10 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Q3. Write the excitation table for a SR latch constructed with NOR gates. **(4 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Q4. What do you understand by mechanical switch contact bounce? Describe how the $\overline{S}\,\overline{R}$ latch can eliminate mechanical switch contact bounce. **(10 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Q5. Explain the undefined condition in SR flip-flops. How is this problem solved in JK flip-flops? **(4+4 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Q6. Show the logic diagram and function table of an SR latch with NAND gates. **(4 marks)** [CSE 205 | L-2/T-1 | 2011–12]

JK Flip-Flop Characteristics

Q1. What is the problem in the construction of a JK flip-flop? How can we overcome this problem? **(6 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Q2. Derive the characteristic table and the characteristic equation for a JK flip-flop. **(4 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Master-Slave D Flip-Flop

Q1. A clocked sequential circuit uses a master-slave configuration.

(a) Explain how signal propagation through the master and slave stages affects the timing behavior of the circuit.

(b) Analyze and justify the possibility of race-around under non-ideal clock conditions (finite rise and fall time).

(12 marks) [CSE 205 | L-2/T-1 | 2024–25]

Q2. Draw the block diagram of a negative edge triggered Master-Slave D flip-flop using D latches. **(10 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q3. Draw the circuit diagram of a master-slave JK flip-flop with asynchronous preset and clear using only NOR gates. **(5 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Q4. Draw the circuit diagram of a master-slave JK Flip-Flop using only NAND gates. **(8 marks)** [CSE 205 | L-2/T-1 | 2018–19]

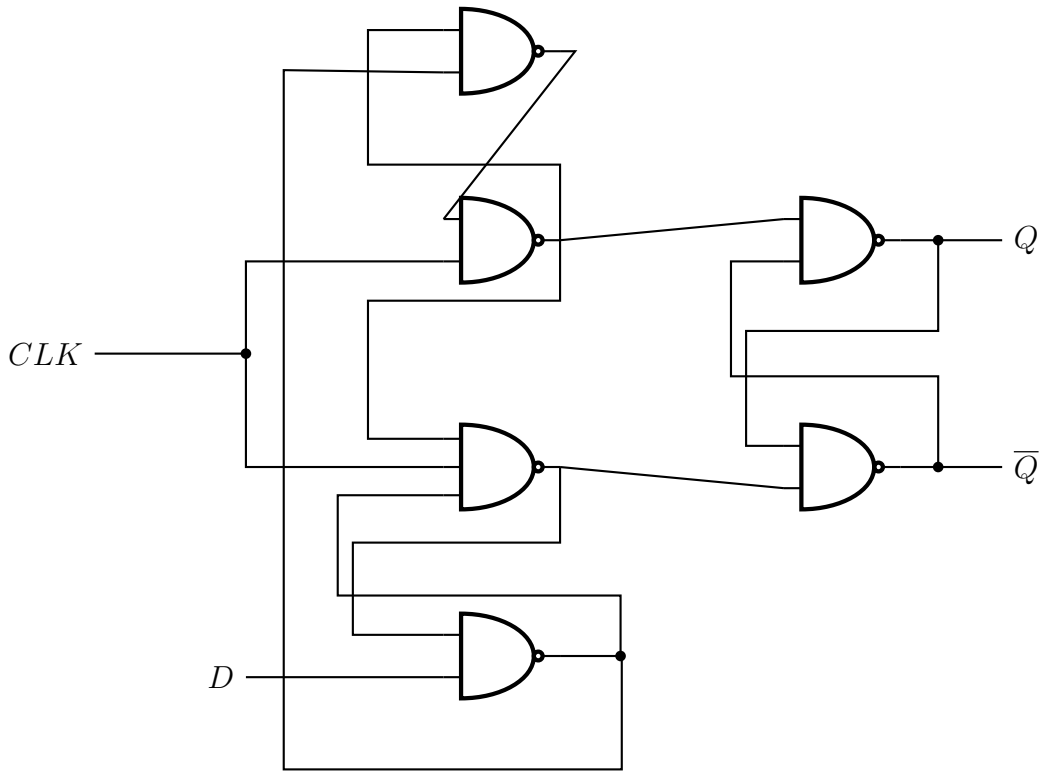
Q5. Why do we use master-slave flip-flops? **(5 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Q6. Draw the block diagram and explain the operation of a Master-Slave negative edge triggered D flip-flop. **(12 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Q7. Draw and explain the operation of a Master-Slave D flip-flop. **(12 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Sequential Circuit Timing Analysis

Q1. Consider a sequential circuit. Let the current values be: CLK $= 1$, $D = 1$, $Q = 1$, $Q' = 0$. If the value of D changes from 1 to 0 after the "Hold Time" ends and CLK remains unchanged at 1, what will be the values of Q and Q'?



(6 marks) [CSE 205 | L-2/T-1 | 2015–16]

Latch vs Flip-Flop

Q1. Write the differences between a latch and a flip-flop. **(5 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Flip-Flop Conversion

Q1. Design a T flip-flop using a JK flip-flop and verify its operation using state transitions. **(10 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Q2. Construct a JK flip-flop using a T flip-flop and basic gates. **(6 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Q3. Construct a JK flip-flop from a T flip-flop and necessary gates. Show all the steps of this construction process. **(11 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q4. Design a D flip-flop using a T flip-flop and gates. **(5 marks)** [CSE 205 | L-2/T-1 | 2020–21]

↪ *Similar: 2019–20*

Q5. Design a JK Flip-Flop using a T Flip-Flop and basic gates. **(10 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Q6. Design a SR Flip-Flop using a D Flip-Flop and a 4×1 MUX. **(10 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Q7. Convert: (i) a JK flip-flop into a T flip-flop, and (ii) a D flip-flop into a JK flip-flop. **(8 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Q8. Convert a T flip-flop into a JK flip-flop and vice versa. **(4+4 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Q9. Convert a D flip-flop to a T flip-flop. Use necessary gates. **(4 marks)** [CSE 205 | L-2/T-1 | 2011–12]

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

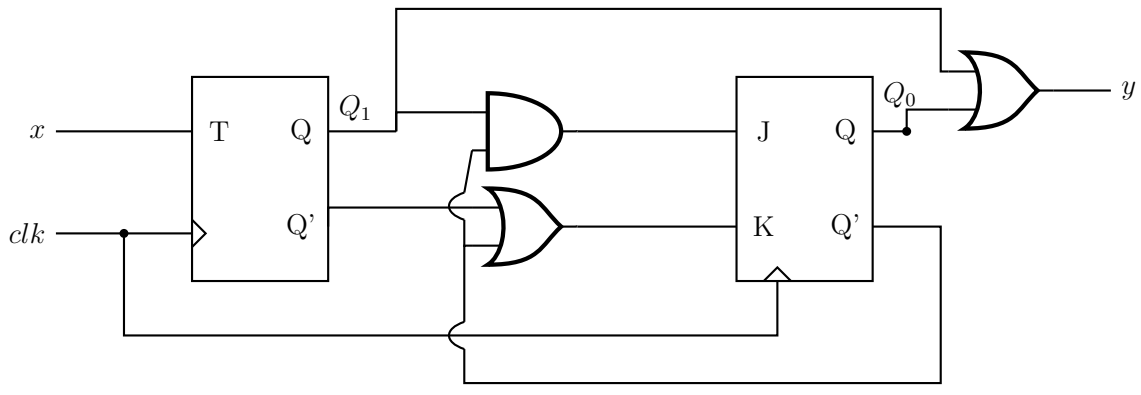
Sequential Circuits, State Diagrams & FSM

Mealy & Moore Machines, State Tables, State Minimization, Pulse/Fundamental Mode

S6 Sequential Circuits, State Diagrams & FSM

State Diagram Derivation from Circuit

Q1. Analyze the sequential circuit in the figure below. Derive the transition table, excitation table and state table.



(12 marks) [CSE 205 | L-2/T-1 | 2023–24]

Q2. A sequential circuit has two flip-flops A and B, two inputs x_1 and x_2, and one output z. The flip-flop input equations and circuit output equations are:

$$J_A = y_A x_1 + y_B x_2, \qquad K_A = y_A y_B + x_1 x_2$$
$$J_B = y_A x_1, \qquad K_B = y_A y_B x_1 + y_B x_2$$
$$z = y_A x_1 x_2 + y_B x_1 x_2$$

(a) Draw the logic diagram of the circuit.

(b) Derive the transition table, excitation table and state table.

(c) Draw the state diagram.

(20 marks) [CSE 205 | L-2/T-1 | 2021–22]

Q3. A sequential circuit has two flip-flops A and B, two inputs x_1 and x_2, and one output z. The flip-flop input equations and circuit output equations are:

$$J_A = y_A x_1 + y_B x_2, \qquad K_A = y_A y_B + x_1 x_2$$
$$J_B = y_A x_1, \qquad K_B = y_A y_B x_1 + y_B x_2$$
$$z = y_A x_1 x_2 + y_B x_1 x_2$$

For the above circuit:

(a) Draw the logic diagram of the circuit.

(b) Derive the transition table, excitation table, and state table.

(c) Draw the state diagram.

(25 marks) [CSE 205 | L-2/T-1 | 2019–20]

Q4. A sequential circuit has two JK Flip-Flops A and B, two inputs x and y, and one output z. The flip-flop input equations and circuit output equation are:

$$J_A = Ax + By, \qquad K_A = AB + xy$$
$$J_B = Ax, \qquad K_B = ABx + By, \qquad z = Axy + Bxy$$

(a) Draw the logic diagram of the circuit.

(b) Derive the state table.

(c) Draw the state diagram.

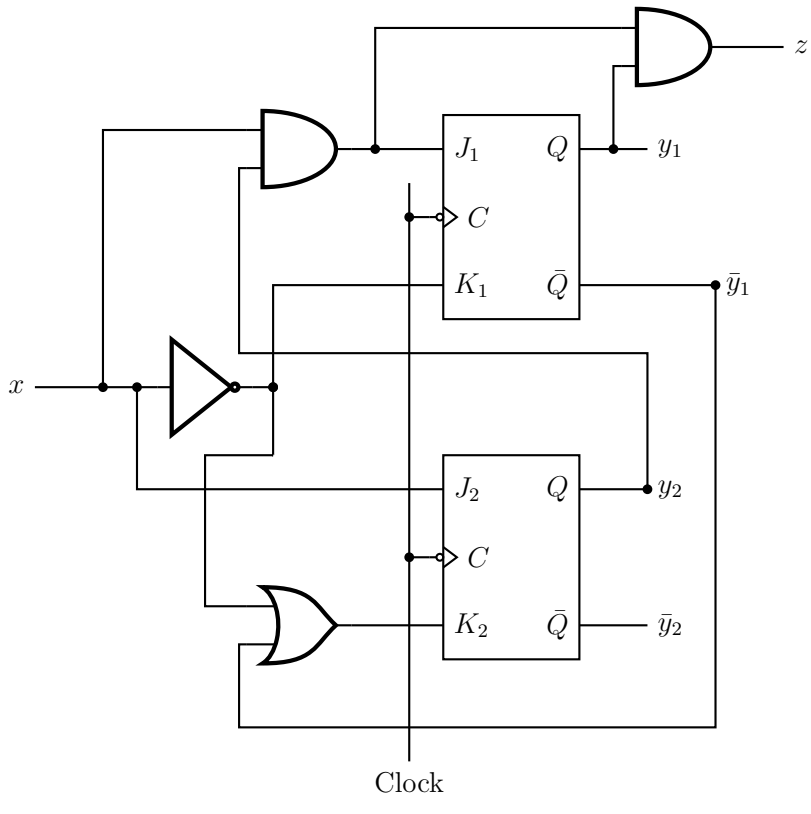
(4+7+4 marks) [CSE 205 | L-2/T-1 | 2016–17]

Q5. A sequential circuit has two JK Flip-Flops A and B, two inputs x and y, and one output z. The Flip-Flop input equations and circuit output equation are:

$$J_A = Bx + B'y', \quad K_A = B'xy', \quad J_B = A'x, \quad K_B = A + xy', \quad z = Ax'y' + Bx'y'$$

Draw the logic diagram of the circuit, tabulate the state table and derive the state equations for A and B. **(15 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Q6. Derive the state diagram from the following sequential circuit. **(15 marks)** [CSE 205 | L-2/T-1 | 2012–13]

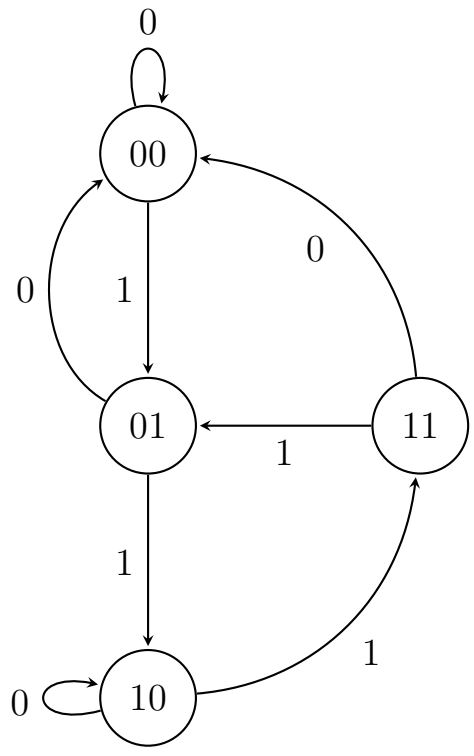


Q1. Design the circuit corresponding to the given state diagram using T flip-flops:

(a) Write the state table.

(b) Find the input equations for the T flip-flops.

(c) Draw the circuit diagram.



(21 marks) [CSE 205 | L-2/T-1 | 2013–14]

Q2. Design a one-input, one-output serial 2's complementer using D flip-flops. The circuit accepts a string of bits from the input and generates the 2's complement at the output. The circuit can be reset asynchronously to start and end the operation. Include: (i) State diagram, (ii) State table, (iii) Simplified logic equations, (iv) Logic diagram. **(15 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Q3. Design a control unit with a synchronous sequential circuit for a vending machine that accepts 5 Tk, 10 Tk and 20 Tk notes and releases a soft drink bottle worth 20 Tk. The machine releases a drink whenever the deposit amount equals or exceeds 20 Tk and keeps the change. The maximum deposit at any time is 15 Tk. The machine releases the deposit whenever the 'CHANGE' input is given. Outputs: Release Drink Z_1 and Release Change Z_2. Inputs X_1X_2: Tk5=00, Tk10=01, Tk20=10, CHANGE=11. Use JK Flip-Flops. **(20 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Input	X_1X_2
Tk 5	00
Tk 10	01
Tk 20	10
CHANGE	11

Sequence Detector

Q1. Develop a sequential circuit that detects a sequence of three or more consecutive 1's in a string of bits coming through an input line. Use D flip-flops. Show the state diagram, the state table and simplified logic equations. You do not need to show the logic diagram. **(12 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q2. Design a sequence recognizer using T flip-flops that recognizes a 4-bit **non-overlapping** binary sequence X from an input stream of bits, where:

$$\text{Sequence } X = \text{4-bit binary representation of your roll number} \pmod{16}$$

(25 marks) [CSE 205 | L-2/T-1 | 2019–20]

Q3. Design a sequence recognizer that recognizes the sequence **1101** from an input sequence (overlapping allowed). Design the circuit using SR Flip-Flops and draw the circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Q4. Design a clocked sequential circuit that recognizes the input sequence **1010** (including overlap) such that for input $x = 01010100011010010100$ the output is $z = 00001010000001000010$. Design the circuit using T Flip-Flops and basic gates. **(20 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Q5. Draw the state diagram in Moore model of a synchronous sequential circuit that results in an output of 1 whenever the sequence 10011 occurs. The circuit is required to recognize overlapping sequences. **(8 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Q6. Design a sequential circuit that detects a sequence of three or more consecutive 1s in a string of bits coming through an input line. Use D flip-flops. The design must include the following:

(a) State diagram
(b) State table
(c) Simplified logic equations
(d) Logic

(15 marks) [CSE 205 | L-2/T-1 | 2011–12]

Mealy & Moore Model Comparison & Block Diagrams

Q1. Draw the block diagram of Mealy and Moore state machines. State the difference between two models with a simple example. **(10 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Q2. Transform the Mealy machine shown below into a corresponding Moore machine.

PS	NS, Z	
	$x = 0$	$x = 1$
A	B, 1	C, 0
B	C, 1	D, 0
C	D, 0	E, 1
D	E, 0	A, 1
E	A, 1	B, 1

(6 marks) [CSE 205 | L-2/T-1 | 2023–24]

Q3. Differentiate between Mealy and Moore machines with block diagrams. **(10 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q4. Transform the following Mealy machine into a corresponding Moore machine.

PS	NS, Z	
	x = 0	x = 1
A	C, 0	B, 0
B	A, 1	D, 0
C	B, 1	A, 1
D	D, 1	C, 0

(5 marks) [CSE 205 | L-2/T-1 | 2021–22]

Q5. Obtain the Mealy equivalent state table for the following Moore machine.

PS	NS		Z
	x = 0	x = 1	
A	A	B	0
B	C	B	0
C	C	D	1
D	A	D	0

(7 marks) [CSE 205 | L-2/T-1 | 2018–19]

Q6. Define (i) a Mealy machine and (ii) a Moore machine. **(5 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Q7. Draw the block diagram of Mealy and Moore state machines. **(8 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Q8. Draw the block diagrams of Mealy and Moore finite state machines. **(6 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Q9. What are the advantages and disadvantages of using the Mealy model and the Moore model for sequential circuit design? **(6 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Q10. Differentiate Mealy and Moore models. **(4 marks)** [CSE 205 | L-2/T-1 | 2011–12]

State Minimization (Implication Table)

Q1. For the state-table of a completely specified circuit (PS: A–H), find the equivalence partitions and write the state-table of the minimal machine. Name the state that are equivalent .

PS	NS, Z $(x=0)$	NS, Z $(x=1)$
A	B,1	H,1
B	F,1	D,1
C	D,0	E,1
D	C,0	F,1
E	D,1	C,1
F	C,1	C,1
G	C,1	D,1
H	C,0	A,1

(10 marks) [CSE 205 | L-2/T-1 | 2020–21]

Q2. For the state-table of a completely specified circuit, find the equivalent partitions and write the state-table of the minimal machine. Name the equivalent states.

PS	NS, Z		
	I	J	K
A	A,0	B,1	E,1
B	B,0	A,1	F,1
C	A,1	D,0	E,0
D	F,0	C,1	A,0
E	A,0	D,1	E,1
F	B,0	D,1	F,1

(8 marks) [CSE 205 | L-2/T-1 | 2018–19]

Q3. For the state-table of a completely specified circuit, find the equivalence partitions and write the state-table of the minimal machine. Name the equivalent states.

x_1x_2	NS,Z			
PS	00	01	11	10
A	D, 0	D, 0	F, 0	A, 0
B	C, 1	D, 0	E, 1	F, 0
C	C, 1	D, 0	E, 1	A, 0
D	D, 0	B, 0	A, 0	F, 0
E	C, 1	F, 0	E, 1	A, 0
F	D, 0	D, 0	A, 0	F, 0
G	G, 0	G, 0	A, 0	A, 0
H	B, 1	D, 0	E, 1	A, 0

(10 marks) [CSE 205 | L-2/T-1 | 2017–18]

Q4. You are given the following state table of a synchronous sequential circuit.

Present State	Next State		Output	
	$x=0$	$x=1$	$x=0$	$x=1$
a	a	c	0	0
b	d	a	1	0
c	f	f	0	0
d	e	b	1	0
e	g	g	1	0
f	c	c	0	0
g	b	h	1	0
h	h	c	0	0

(i) Find the reduced state table using an implication table.

(ii) Draw the equivalent minimized state diagram.

(10 marks) **[CSE 205 | L-2/T-1 | 2016–17]**

Q5. You are given a state table of a synchronous sequential circuit. Find the reduced state table using the implication table.

Present State	Next State		Output	
	x=0	x=1	x=0	x=1
a	d	b	0	0
b	e	a	0	0
c	g	f	0	1
d	a	d	1	0
e	a	d	1	0
f	c	b	0	0
g	a	e	1	0

(12 marks) **[CSE 205 | L-2/T-1 | 2015–16]**

Q6. Find a critical race-free state assignment for a reduced flow table (states a–d, inputs 00/01/11/10) using the Multiple-row method.

	00	01	11	10
a	(a)	c	(a)	d
b	a	(b)	c	(b)
c	(c)	(c)	(c)	d
d	(d)	b	a	(d)

(10 marks) **[CSE 205 | L-2/T-1 | 2014–15]**

Q7. Using an implication table, reduce the following state table to a minimum number of states and write the reduced state table. **(15 marks)** **[CSE 205 | L-2/T-1 | 2013–14]**

Present State	Next State		Output	
	$x=0$	$x=1$	$x=0$	$x=1$
A	F	B	0	0
B	D	C	0	0
C	F	E	0	0
D	G	A	1	0
E	D	C	0	0
F	F	B	1	1
G	G	H	0	1
H	G	A	1	0

Q8. Using an implication table, reduce the following state table to a minimum number of states and write the reduced state table. **(17 marks)** **[CSE 205 | L-2/T-1 | 2012–13]**

	X	
	0	**1**
A	E/0	D/0
B	A/1	F/0
C	C/0	A/1
D	B/0	A/0
E	D/1	C/0
F	C/0	D/1
G	H/1	G/1
H	C/1	B/1

Q9. For the state-table of a completely specified circuit, find the equivalence partitions and write the state-table of the minimal machine. Name the states that are equivalent. *[Diagram: state table with present states a–h and next states/outputs for $x = 0$ and $x = 1$ to be inserted.]* **(12+3 marks)** **[CSE 205 | L-2/T-1 | Uncertain]**

Incompletely Specified Machine Minimization

Q1. For the state table of a completely specified asynchronous sequential circuit, find out the equivalent partitions, implication table. Formulate the state table of the minimal machine. Find out the states those are equivalent.

x1x2	NS, z			
PS	x1x2=00	x1x2=01	x1x2=11	x1x2=10
A	D/0	D/0	F/0	A/0
B	C/1	D/0	E/1	F/0
C	C/1	D/0	E/1	A/0
D	D/0	B/0	A/0	F/0
E	C/1	F/0	E/1	A/0
F	D/0	D/0	A/0	F/0
G	G/0	G/0	A/0	A/0
H	B/1	D/0	E/1	A/0

(PS = Present state, NS = next state, x1, x2 are input, z is output)

(12 marks) **[CSE 205 | L-2/T-1 | 2024–25]**

Q2. For the incompletely specified machine shown below, derive the implication table and determine the maximal compatibles and the maximal incompatibles. Find the upper bound and the lower bound of the number of states of the minimal machine. Give the state table of the minimal machine.

PS	NS, Z	
	$x = 0$	$x = 1$
A	B, 0	C, 0
B	D, 0	–, –
C	F, 0	E, 1
D	B, 0	G, 0
E	F, 0	C, 1
F	E, 0	D, 1
G	F, 0	–, –

(23 marks) **[CSE 205 | L-2/T-1 | 2023–24]**

Q3. Reduce the number of states of the following state table using implication table. Show the reduced state table.

State	Next State		Output	
	Input X = 0	Input X = 1	Input X = 0	Input X = 1
A	A	B	0	0
B	C	D	0	1
C	A	B	0	0
D	E	D	0	1
E	A	D	0	1

(11 marks) **[CSE 205 | L-2/T-1 | 2022–23]**

Q4. For the incompletely specified machine shown in the figure, derive the implication table and determine the maximal compatibles and the maximal incompatibles. Find the upper bound and the lower bound of the number of states of the minimal machine. Give the state table of the minimal machine.

PS	NS, Z		
	I_1	I_2	I_3
A	C, 0	E, 1	-
B	C, 0	E, -	-
C	B, -	C, 0	A, -
D	B, 0	C, -	E, -
E	-	E, 0	A, -

(15 marks) **[CSE 205 | L-2/T-1 | 2021–22]**

Q5. For the incompletely specified machine below, complete the implication table and determine the maximal compatibles and maximal incompatibles. Find the upper and lower bounds on the number of states of the minimal machine. Give the state table of the minimal machine.

PS	NS, Z_1Z_2			
	00	01	11	10
A	A, 00	E, 01	-	A, 01
B	-	C, 10	B, 00	D, 11
C	A, 00	C, 10	-	-
D	A, 00	-	-	D, 11
E	-	E, 01	F, 00	-
F	-	G, 10	F, 00	G, 11
G	A, 00	-	-	G, 11

(25 marks) **[CSE 205 | L-2/T-1 | 2020–21]**

Q6. For the incompletely specified machine shown below, derive the implication table and determine the maximal compatibles and the maximal incompatibles. Find the upper bound and the lower bound of the minimal machine and determine the minimal machine.

PS	NS, Z	
	x = 0	x = 1
A	A, -	B, 1
B	G, -	D, 0
C	B, 1	B, -
D	A, 1	B, -
E	C, -	A, -
F	F, -	C, -
G	G, -	G, -

(25 marks) **[CSE 205 | L-2/T-1 | 2019–20]**

Q7. For the incompletely specified machine (Figure 7(a)), complete the implication table and determine the maximal compatibles and maximal incompatibles. Find the upper and lower bounds. Give the state table of the minimal machine.

PS	NS, Z	
	x = 0	x = 1
A	B, 0	C, 1
B	D, 0	C, 1
C	A, 0	E, 0
D	-	F, 1
E	G, 1	F, 0
F	B, 0	-
G	D, 0	E, 0

(20 marks) **[CSE 205 | L-2/T-1 | 2018–19]**

Q8. For the incompletely specified machine , complete the implication table and determine the maximal compatibles and maximal incompatibles. Find the upper and lower bounds on the number of states of the minimal machine. Give the state table of the minimal machine.

PS	NS,Z	
	x = 0	x = 1
A	A, -	B, 1
B	G, -	D, 0
C	B, 1	B, -
D	A, 1	B, -
E	C, -	A, -
F	F, -	C, -
G	G, -	G, -

(15 marks) **[CSE 205 | L-2/T-1 | 2017–18]**

Redundant States

Q1. What are the problems that may occur when redundant states are present in the state diagram? **(6 marks)** **[CSE 205 | L-2/T-1 | 2012–13, 2013–14]**

Output Assignment in Flow Table

Q1. Write down the procedure to assign the output to unstable states in a flow table. **(7 marks)** **[CSE 205 | L-2/T-1 | 2011–12]**

Fundamental Mode Design

Q1. Design an asynchronous circuit functioning in fundamental mode. The circuit has three inputs a, b, and c, and one output U. $U = 1$ only if all inputs are equal to '1', and they went to '1' in the order a, b, then c. U returns to 0 when all inputs are returned to '0'. Derive the primitive flow table, reduced flow table and the minimum number of states required to have a race-free state assignment, and find a possible state assignment. **(23 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Q2. At a junction of a single-track railroad and a road, traffic lights are to be installed. The lights are to be controlled by switches that are pressed or released by the trains. When a train approaches the junction from either direction and is within 1500 feet from it, the lights are to change from green to red and remain red until the train is 1500 feet past the junction. You may assume that the length of a train is smaller than 3000 feet. Determine a primitive flow table and reduce it. **(20 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Q3. Design a two-input (x_1, x_2), one-output (z) fundamental mode circuit that operates as follows. The output changes from 0 to 1 only on the first x_1 input change that follows an x_2 input change. A $1 \rightarrow 0$ output change occurs only when x_1 changes from 1 to 0 while $x_2 = 1$. Determine the primitive flow table, reduced flow table, race-free state assignment, K-maps and circuit diagram. **(25 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Q4. Design a fundamental mode circuit to function as an electrical lock. The lock has two switch inputs X_1 and X_2. Design the circuit so that the lock-open signal $Z = 1$ is produced only after the following conditions have been satisfied:

(a) Begin with $X_1 = X_2 = 0$.

(b) While $X_2 = 0$, X_1 is turned on and then turned off.

(c) Then while X_1 remains off, X_2 is turned on to open the lock ($Z = 1$).

(d) For any deviation in the sequence of the above transitions, the lock remains closed ($Z = 0$).

Determine the primitive flow table, reduced flow table, race-free state assignment, K-maps and circuit diagram. **(25 marks)** [CSE 205 | L-2/T-1 | 2019–20]

Q5. Construct a primitive flow table for a fundamental mode circuit where the output $z = 0$ when the two inputs x_1 and x_2 are equal. $z = 1$ results when $x_1 = 0$ and x_2 changes from 0 to 1, and when $x_1 = 1$ and x_2 changes from 1 to 0. No other input change causes any output change. Determine the reduced flow table containing the minimum number of rows. **(10+10 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Q6. Design a circuit for a digital lock using T latches. The lock has two push-button inputs A and B and produces a single pulse Z_1 for each activation. The two switches are mechanically interlocked so that simultaneous pulses are not possible. Features: (i) opening sequence is AABABA; (ii) three B pulses give absolute reset; (iii) any incorrect use of A produces output Z_2 to ring a bell. **(20 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Q7. Construct a primitive flow table for a fundamental mode circuit with two inputs (x_1, x_2) and two outputs (z_1, z_2). Specifications: when $x_1x_2 = 00$, $z_1z_2 = 00$. Output 10 is produced following input sequence $00 \rightarrow 01 \rightarrow 11$ and remains until x_1x_2 returns to 00. Output 01 is produced following sequence $00 \rightarrow 10 \rightarrow 11$ and remains until 00 input returns the output to 00. Determine the minimum row-reduced flow table. **(10+10 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Q8. What does it mean when an asynchronous sequential circuit is assumed to operate in the fundamental mode? **(4 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Q9. Derive the primitive flow table for a negative-edge triggered T flip-flop. **(12 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Q10. A specification is given below for a gated latch circuit with two inputs G and D and an output Q. Assume fundamental mode operation.

"Binary information present at the D input is transferred to the Q output when G is equal to 1. The Q output will follow the D input as long as G = 1. When G goes to 0, the information that was present at the D input at the time the transition occurred is retained at the Q output."

Derive the primitive flow table and then obtain the reduced flow table. **(6+6 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Q11. For a given primitive flow table (states a–h, inputs 00/01/11/10), perform the following:

(a) Find all compatible pairs by means of an implication table.

(b) Find the maximal compatibles by means of a merger diagram.

(c) Find a minimal set of compatibles that covers all the states and is closed.

Excitation Function & Transition Table

Q1. An asynchronous sequential circuit is described by the excitation function $Y = x_1x_2' + (x_1 + x_2')y$ and the output function $z = y$.

(a) Draw the logic diagram of the circuit.

(b) Derive the transition table and output map.

(c) Obtain a two-state flow table.

(12 marks) [CSE 205 | L-2/T-1 | 2022–23]

Q2. An asynchronous sequential circuit has two internal states and one output. The excitation and output functions are:

$$Y_1 = x_1x_2 + x_1y_2' + x_2'y_1$$
$$Y_2 = x_2 + x_1y_1'y_2 + x_1'y_1$$
$$Z = x_2 + y_1$$

Draw the logic diagram of the circuit and derive the transition table. **(15 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Q3. An asynchronous sequential circuit is described by the following excitation function:

$$Y = X_1X_2' + (X_1 + X_2')\,y$$

Draw the logic diagram of the circuit and derive the transition table. **(10 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Hazards in Asynchronous Circuits

Q1. What is a static-1 hazard? Give an example of a static-1 hazard in an asynchronous sequential circuit. **(5 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Serial Parity Generation

Q1. Design a serial parity generation circuit. The circuit receives a sequence of bits and determines whether the sequence contains an even or odd number of ones. The circuit output p should be 0 for even parity and 1 for odd parity. **(10 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Q2. Design a serial parity generation circuit. The circuit receives a sequence of bits and determines whether the sequence contains an even or odd number of ones. The output p should be 0 for even parity and 1 for odd parity. **(8 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Q3. For the following primitive flow table shown in Figure , answer the following questions:

(i) Find all compatible pairs by means of an implication table.

(ii) Find the maximal compatibles by means of a merger diagram.

(iii) Find a minimal set of compatibles that covers all the states and is closed.

	00	01	11	10
a	(a), 0	b, –	–, –	e, –
b	a, –	(b), 0	c, –	–, –
c	–, –	d, –	(c), 0	h, –
d	a, –	(d), 1	–, –	–, –
e	a, –	–, –	f, –	(e), 0
f	–, –	g, –	(f), 0	h, –
g	a, –	(g), 0	–, –	–, –
h	a, –	–, –	–, –	(h), 0

(10+5+5 marks) [CSE 205 | L-2/T-1 | 2014–15]

Race Around Condition

Q1. What is a race in fundamental mode circuits? Explain critical and non-critical races with examples. **(12 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q2. For the given reduced flow table, find a valid assignment without any critical race and complete the modified flow table.

	00	01	11	10
A	(A),0	D,0	B,-	(A),0
B	A,-	-	C,-	-
C	A,-	(C),1	(C),1	(C),1
D	A,0	(D),0	(D),0	C,-

(10 marks) **[CSE 205 | L-2/T-1 | 2021–22]**

Q3. For the given reduced flow table, find a valid assignment without any critical race and complete the modified flow table.

x_1x_2	00	01	11	10
a	ⓐ/0	b/-	ⓐ/1	b/-
b	a/-	ⓑ/0	c/-	ⓑ/0
c	a/-	ⓒ/1	ⓒ/0	b/-

(10 marks) **[CSE 205 | L-2/T-1 | 2020–21]**

Q4. For the given reduced flow table, find a valid assignment without any critical race and complete the modified flow table.

x_1x_2	00	01	11	10
a	(a)/0	d/0	(a)/1	c/0
b	(b)/0	c/-	(b)/0	d/-
c	b/0	(c)/1	a/1	(c)/0
d	a/0	(d)/0	b/0	(d)/1

(8 marks) **[CSE 205 | L-2/T-1 | 2018–19]**

Q5. For the given reduced flow table, find a valid assignment without any critical race and complete the modified flow table.

x_1x_2	00	01	11	10
a	(a)/0	d/0	(a)/1	c/0
b	(b)/0	c/-	(b)/0	d/-
c	b/0	(c)/1	a/1	(c)/0
d	a/0	(d)/0	b/0	(d)/1

(10 marks) **[CSE 205 | L-2/T-1 | 2017–18]**

Q6. What is race condition? Explain different types of races with examples. **(10 marks)** **[CSE 205 | L-2/T-1 | 2016–17]**

Q7. Investigate the transition table (state variables y_1y_2, inputs x_1x_2) and determine all race conditions and whether they are critical or noncritical. Determine also whether there are any cycles.

	x_1x_2			
y_1y_2	00	01	11	10
00	10	(00)	11	10
01	(01)	00	10	10
11	01	00	(11)	(11)
10	11	00	(10)	(10)

(10 marks) **[CSE 205 | L-2/T-1 | 2014–15]**

Q8. What is a race in fundamental mode circuits? Explain critical and non-critical races with an example. **(4+8 marks)** **[CSE 205 | L-2/T-1 | 2011–12]**

Pulse Mode Logic

Q1. Design an automatic vending machine, where the candy bars inside the machine cost 3 Tk and the machine accepts 1 Tk and 2 Tk only. The system accepts the bills sequentially. The circuit produces a level output that delivers the candy whenever the amount received by the machine is 3 Tk or more. The excess amount is kept deposited for the next candy. Design a pulse mode circuit for the vending machine using S-R flip-flops. Provide the circuit diagram as well. **(25 marks)** **[CSE 205 | L-2/T-1 | 2024–25]**

Q2. Design a pulse mode sequential circuit that satisfies the following requirements. The circuit has two input lines x_1 and x_2 along with one level output line z. An output transition from 0 to 1 is produced only after the occurrence of the last x_2 pulse in the sequence x_1-x_2-x_1-x_2. The output is reset from 1 to 0 only after the first x_1 pulse that occurs following the 0 to 1 output transition. The circuit allows overlapping sequences. Develop the pulse mode circuit using JK flip-flops. Draw the circuit diagram. **(23 marks)** **[CSE 205 | L-2/T-1 | 2023–24]**

Q3. Design an automatic vending machine. The candy bars inside the machine cost 15 Tk, and the machine accepts 5 Tk and 10 Tk only. An electromechanical system accepts the money sequentially. The circuit produces a pulse output that delivers the candy whenever the amount received by the machine is 15 Tk or more. The excess amount is kept deposited for the next candy. Design a pulse mode circuit for the vending machine using S-R latches. **(20 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q4. Design a circuit for a digit lock using T latches. The lock has two input push-button switches A, B and it outputs a single pulse (Z_1) for each activation. The two switches are interlocked mechanically so that simultaneous pulses are not possible. The lock should have the following features: (i) The opening sequence is AABABA. (ii) Three B pulses should give an absolute reset. (iii) Any incorrect use of the A switch will cause an output (Z_2) to ring a bell to warn that the lock is being tampered with. **(20 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Q5. Design an automatic vending machine. Candy bars cost 3 Tk; the machine accepts 1 Tk and 2 Tk coins sequentially. The circuit produces a level output delivering the candy whenever the amount received equals or exceeds 3 Tk. Excess amount is kept for the next candy. Design a pulse mode circuit using SR Flip-Flops. Draw the circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Q6. Write down the restrictions on the duration of input for pulse mode asynchronous circuits. **(4 marks)** [CSE 205 | L-2/T-1 | 2011–12]

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Registers & Counters

Shift Registers, Asynchronous & Synchronous Counters, Ring Counters

S7 Registers & Counters

Ripple (Asynchronous) Counter

Q1. Design the Modulo-100 ripple down counter with JK flip-flops and basic gates. **(6 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Q2. Show the logic diagram of a four-bit binary ripple (asynchronous) countdown counter using T flip-flops that trigger on the negative edge of the clock. **(12 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q3. A MOD-13 counter counts from 0000 to 1100. Design the MOD-13 ripple up/down counter with positive edge-triggered JK flip-flops and basic gates. **(5 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Q4. Design a BCD ripple down counter using D flip-flops. **(5 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Q5. Design a BCD ripple down counter using D flip-flops. **(7 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Q6. Design a 4-bit modulo-12 asynchronous up counter using JK Flip-Flops. **(5 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Q7. A MOD-12 counter counts from 0000 to 1011. Design the MOD-12 ripple counter with positive edge-triggered JK Flip-Flops and basic gates. **(10 marks)** [CSE 205 | L-2/T-1 | 2016–17]

↪ *Similar: 2019–20*

Q8. Design a 4-bit Ripple counter using D Flip-Flops and briefly explain its operation. **(10 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Q9. (a) Design a modulo-6 3-bit asynchronous (ripple) up counter with positive edge-triggered JK flip-flops.

(b) Show the timing diagram for the above designed counter.

(c) What are the problems of the above counter?

(8+4+3 marks) [CSE 205 | L-2/T-1 | 2012–13]

Q10. Draw and explain a four-bit binary count-up ripple counter with T flip-flops. **(12 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Synchronous Counter Design

Q1. A ring counter is initialized incorrectly.

(a) Analyze the resulting sequence of states.

(b) Propose a circuit modification to ensure automatic correction.

(12 marks) [CSE 205 | L-2/T-1 | 2024–25]

Q2. Using JK flip-flops, design a synchronous counter that counts in the following sequence: $1, 2, 5, 7, 1, 2, \ldots$ Analyze the final circuit to ensure that if the circuit enters any of the unused states, after a finite number of clock cycles it comes to a used state. **(13 marks)** [CSE 205 | L-2/T-1 | 2022–23]

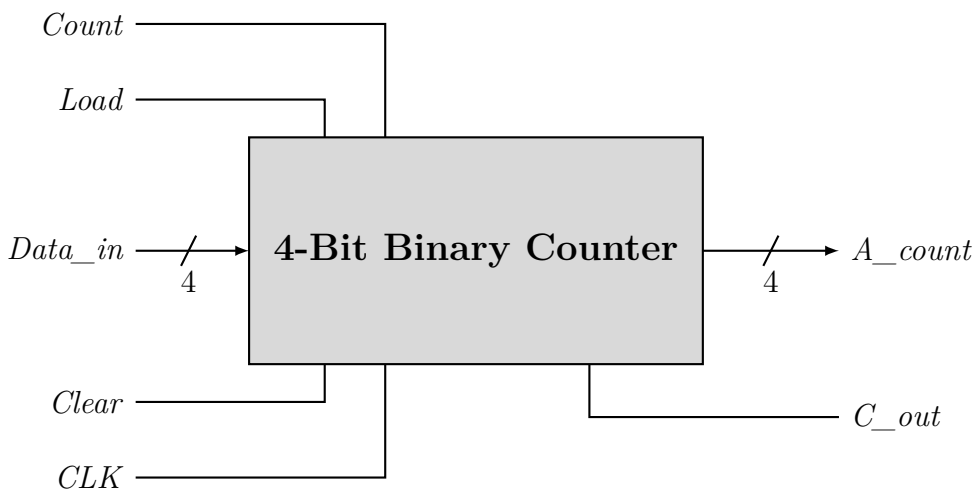
Q3. Design a synchronous counter that counts in the following sequence: $0 \to 2 \to 4 \to 7 \to 5 \to 3 \to 0 \to 2 \to 4 \to \ldots$ Design the counter using one JK flip-flop, one T flip-flop and one D flip-flop. Draw the circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Q4. Design a synchronous counter that counts in the sequence $0 \to 2 \to 4 \to 7 \to 5 \to 3 \to 0 \to 2 \to 4 \to \cdots$ Design the counter using SR Flip-Flops and draw the circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Q5. Design a synchronous counter that counts in the following sequence: $0 \to 1 \to 2 \to 5 \to 4 \to 6 \to 0 \to 1 \to 2 \to \cdots$ Design the counter using SR Flip-Flops and draw the circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Q6. Design a synchronous counter with the sequence: $1 \to 4 \to 5 \to 7 \to 2 \to 1$. You need not take any measure for the unused states. **(10 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Q7. Show two ways to design a Modulo-6 counter using a 4-bit binary counter and necessary basic gates. The Modulo-6 counter repeatedly goes through $0, 1, 2, 3, 4, 5$.



(12 marks) [CSE 205 | L-2/T-1 | 2015–16]

Q8. Design a 3-bit binary up counter that counts $1 \to 2 \to 5 \to 7 \to 1 \cdots$, where any invalid state goes to the immediate next valid state in the sequence. Use T Flip-Flops. Show the state diagram, state table, function equations, minimization (if necessary) and the circuit diagram. **(15 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Q9. Design a 3-bit synchronous binary up counter that counts $1 \to 2 \to 4 \to 7 \to 1 \to \cdots$, where any invalid state goes to the immediate next state in the sequence. Use JK flip-flops. Show the state table, state diagram, function minimization (if necessary), and circuit diagram. **(20 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Q10. Design a synchronous counter with the following binary sequence: $0, 1, 3, 7, 6, 4$ and repeat. Use T flip-flops. **(15 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Synchronous Gray Code Counter

Q1. Design a 3-bit synchronous Gray Code Counter using JK Flip-Flops. Show the state diagram, state table, function equations and the circuit diagram. **(15 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Q2. Develop a synchronous 3-bit counter with a Gray code sequence using JK flip-flops.
(Hint: Gray code count sequence for a 2-bit counter is: $00, 01, 11, 10, 00, 01, \ldots$*)* **(18 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Switch-Tail Ring Counter

Q1. Design a 5-bit ring counter using a shift register and illustrate its operation using a timing diagram. **(10 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Q2. Draw the logic diagram of a four-stage switch-tail ring counter. **(5 marks)** [CSE 205 | L-2/T-1 | 2011–12]

Johnson Counter vs Ring Counter

Q1. Write a short note about a ring counter. **(5 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q2. Distinguish between a ring counter and a Johnson counter. Design a 5-bit ring counter and a 5-bit Johnson counter using shift registers and illustrate their operation using timing diagrams. **(10 marks)** [CSE 205 | L-2/T-1 | 2020–21]

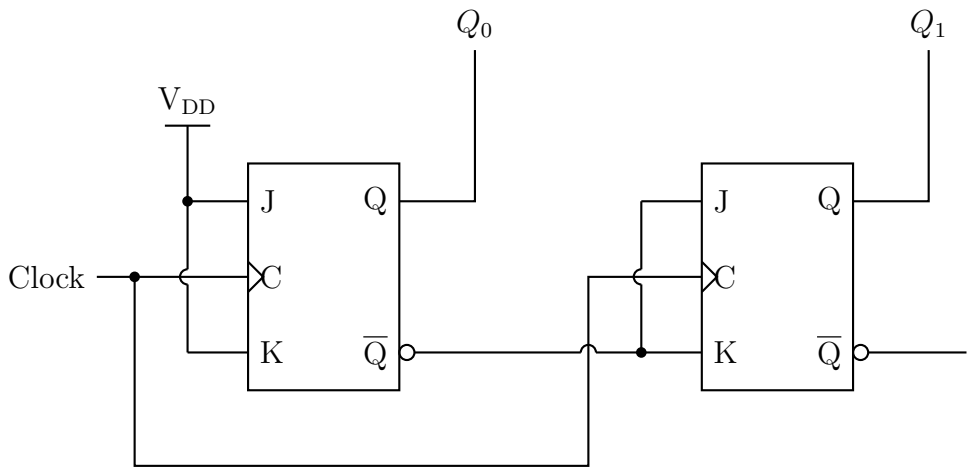
↪ *Similar: 2019–20*

Q3. Design a 4-bit Johnson counter using a shift register and illustrate its operation using a timing diagram. **(7 marks)** [CSE 205 | L-2/T-1 | 2018–19]

Q4. Design a 4-bit Johnson Counter. What is the disadvantage of a Johnson Counter? What is the difference between a Johnson Counter and a Ring Counter? **(10 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Counter Direction Analysis

Q1. A counter circuit has V_{DD} assigned to Logic 1; the current values of LSB Q_0 and MSB Q_1 are 0 and 1 respectively. Find the direction (up or down) in which the circuit counts when the next clock pulse occurs. What would have to be altered to make the circuit count in the other direction?



(7 marks) [CSE 205 | L-2/T-1 | 2015–16]

Clock Generator / Divider

Q1. Given a 100 MHz clock signal, derive a circuit using D flip-flops to generate 50 MHz and 25 MHz clock signals. Draw a timing diagram for all three clock signals. **(10 marks)** [CSE 205 | L-2/T-1 | 2017–18]

Universal Shift Register

Q1. Design a 4-bit universal shift register using multiplexers and D flip-flops. **(13 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Q2. Compare the hardware efficiency and flexibility of universal shift registers with dedicated shift registers in complex digital systems. **(7 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Q3. Use a 2-to-4 decoder, NAND gates and edge triggered D flip-flops to design a 4-bit shift register module that has the following functions:

S_1	S_0	Mode
0	0	Shift right (all 4 bits)
0	1	Shift left (all 4 bits)
1	0	Synchronous common clear
1	1	Synchronous parallel load

(10 marks) [CSE 205 | L-2/T-1 | 2021–22]

Q4. Draw the logic diagram of a 3-bit universal shift register with mode selection inputs s_2, s_1, s_0. The register operates with eight modes: (000) No change, (001) Invert all bits, (010) Set all bits to 0, (011) Set all bits to 1, (100) Parallel load, (101) Shift left, (110) Shift right, (111) No change. **(15 marks)** [CSE 205 | L-2/T-1 | 2016–17]

Q5. Draw the logic diagram of a two-bit register with two D flip-flops and two 4×1 multiplexers with mode selection inputs s_1 and s_0. The register operates with four modes: (00) No change, (01) Complement the two outputs, (10) Clear register to 0 (synchronous), (11) Load parallel data. **(12 marks)** [CSE 205 | L-2/T-1 | 2015–16]

Q6. Draw the circuit diagram of a 4-bit universal shift register using JK flip-flops. **(12 marks)** [CSE 205 | L-2/T-1 | 2012–13]

Serial Adder & Serial Subtractor

Q1. A 4-bit shift register is used for serial data transmission.

(a) Analyze the timing relationship between input data, clock and output.

(b) Explain how errors may propagate in serial communication using shift registers.

(16 marks) [CSE 205 | L-2/T-1 | 2024–25]

Q2. Design a sequential circuit to copy the content of shift register A to shift register B. Use a block diagram to represent the shift register in the diagram. **(12 marks)** [CSE 205 | L-2/T-1 | 2022–23]

Q3. Design a serial subtractor that will perform the operation $A - B$, where $A = a_{n-1} \ldots a_1 a_0$ and $B = b_{n-1} \ldots b_1 b_0$. The operands are applied to the serial subtractor sequentially, beginning with bits a_0 and b_0. Use JK flip-flops in your design. **(10 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Q4. Design a serial adder that computes the sum of two 4-bit binary numbers A and B stored in individual shift registers. The result of the addition is stored in the shift register representing A. **(10 marks)** **[CSE 205 | L-2/T-1 | 2017–18]**

Q5. Design a serial adder that computes the sum of two 4-bit binary numbers A and B stored in individual shift registers. The result of the addition is stored in the shift register representing A. Use the sequential logic design procedure. **(10 marks)** **[CSE 205 | L-2/T-1 | 2016–17]**

Q6. Design a serial subtractor that subtracts the content of register B from the content of register A, where the contents of A and B are stored in individual shift registers. The result of the subtraction is stored in the shift register representing A. (Both A and B are 4-bit binary numbers.) **(10 marks)** **[CSE 205 | L-2/T-1 | 2013–14]**

Q7. Draw the logic diagram of a serial adder. Show the values of inputs and outputs of all components (e.g., flip-flop, register) used in the design after every clock pulse while adding the following two numbers: 0101 and 0011. **(6+6 marks)** **[CSE 205 | L-2/T-1 | 2011–12, 2012–13]**

BUET · CSE 205 · Digital Logic Design

Digital Logic Design

Memory & Programmable Logic

RAM, ROM, Address Multiplexing, PLA Design

S8 Memory & Programmable Logic

ROM Design & Expansion

Q1. You have several 32×4 ROMs in the laboratory. Each ROM can be enabled/disabled using a 1/0 signal respectively. How would you implement a 128×4 ROM using the available ROMs? You may use any additional circuitry if needed. **(10 marks)** [CSE 205 | L-2/T-1 | 2013–14]

PLA Design

Q1. Design a PLA to implement the following functions with the minimization of the number of product terms:

$$F_1(x, y, z) = \Sigma\,(1, 3, 5, 6), \qquad F_2(x, y, z) = \Sigma\,(0, 1, 6, 7)$$
$$F_3(x, y, z) = \Sigma\,(3, 5), \qquad F_4(x, y, z) = \Sigma\,(1, 2, 4, 5, 7)$$

(15 marks) [CSE 205 | L-2/T-1 | 2024–25]

Q2. Tabulate the PLA programming table for the two Boolean functions listed below. Minimize the numbers of product terms:

$$F_1(A, B, C) = \Sigma\,(0, 1, 2, 4)$$
$$F_2(A, B, C) = \Sigma\,(0, 5, 6, 7)$$

(15 marks) [CSE 205 | L-2/T-1 | 2022–23]

Q3. A combinational circuit is defined by the following two functions:

$$F_1(A, B, C) = \Sigma\,(3, 5, 6, 7)$$
$$F_2(A, B, C) = \Sigma\,(0, 2, 4, 7)$$

The circuit needs to be implemented with a PLA having three inputs, four product terms, and two outputs. Show the PLA programming table. **(15 marks)** [CSE 205 | L-2/T-1 | 2021–22]

Q4. Design a digital security lock using PLA which takes BCD as input and opens after converting the input to Excess-3 code. Use the minimum number of product terms. (Show the fuse map.) **(15 marks)** [CSE 205 | L-2/T-1 | 2020–21]

Q5. Design a circuit for the following functions using a Programmable Logic Array (PLA):

$$A(x, y, z) = \Sigma\,(0, 1, 2, 5, 7)$$
$$B(x, y, z) = \Sigma\,(0, 1, 3, 6, 7)$$

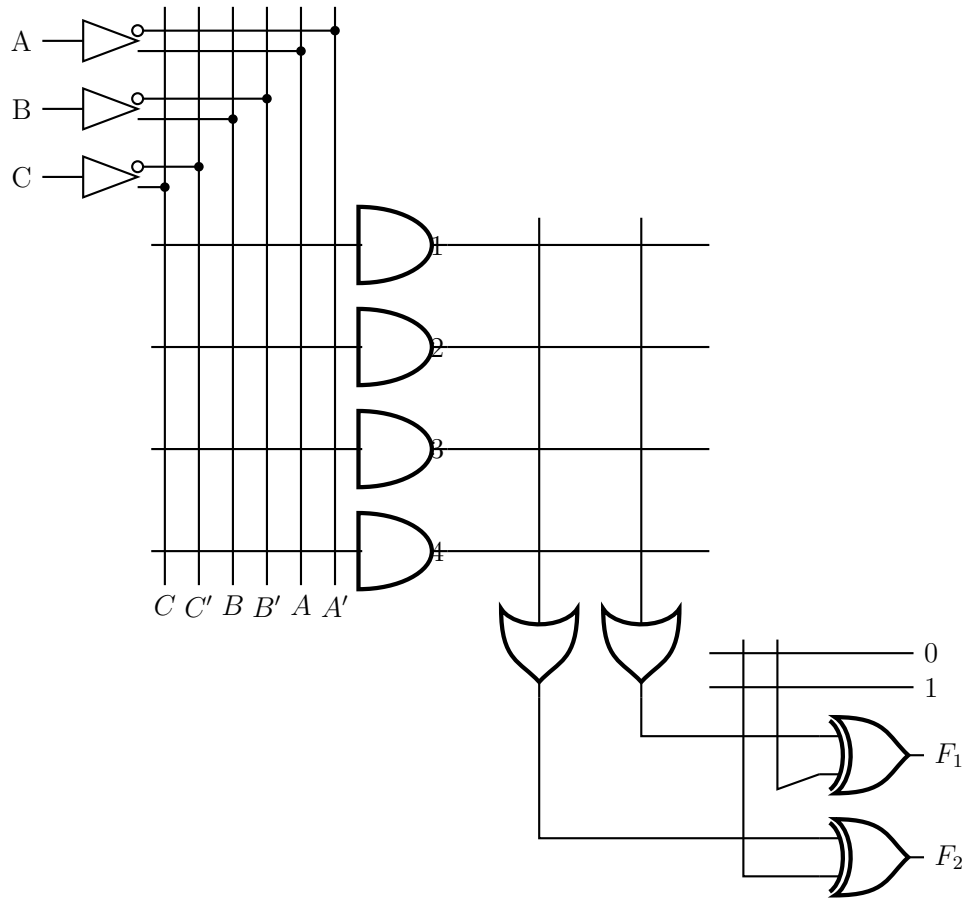
(10 marks) [CSE 205 | L-2/T-1 | 2019–20]

Q6. Design a security lock using PLA which will be opened using the following combinations:

$$A(x, y, z) = \Sigma\,(1, 3, 5, 6), \quad B(x, y, z) = \Sigma\,(0, 1, 6, 7)$$
$$C(x, y, z) = \Sigma\,(3, 5), \quad D(x, y, z) = \Sigma\,(1, 3, 4, 5, 7)$$

(12 marks) [CSE 205 | L-2/T-1 | 2018–19]

Q7. For a given Programmable Logic Array (PLA), find the function expressions for F_1 and F_2 and show the PLA programming table.



(13 marks) [CSE 205 | L-2/T-1 | 2015–16]

Q8. Write the difference between ROM and PLA. A combinational circuit is defined by the following functions:

$$F_1(A, B, C) = \Sigma\,(0, 1, 2, 4)$$
$$F_2(A, B, C) = \Sigma\,(0, 5, 6, 7)$$

Implement the circuit with a PLA having three inputs, four product terms and two outputs. **(5+10 marks)** [CSE 205 | L-2/T-1 | 2014–15]

Q9. Derive the PLA program table for a combinational circuit that squares a 3-bit binary number. Minimize the number of product terms. **(25 marks)** [CSE 205 | L-2/T-1 | 2013–14]

Q10. Implement the following three Boolean functions with a PLA:

$$F_1 = \Sigma\,(0, 1, 2, 4)$$
$$F_2 = \Sigma\,(0, 5, 6, 7)$$
$$F_3 = \Sigma\,(0, 3, 5, 7)$$

(15 marks) [CSE 205 | L-2/T-1 | 2011–12]

RAM Design

Q1. If you use two dimensional decoding for addressing a 1K word memory, can you optimize the cost? Justify your answer. **(8 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Synchronous Sequential Circuit Design

Q1. Design an automatic vending machine, when the candy bars inside the machine cost 3 Tk and the machine accepts 1 Tk and 2 Tk only. The system accepts the bills sequentially. The circuit produces a level output that delivers the candy whenever the amount received by the machine is 3 Tk or more. The excess amount is kept deposited for the next candy. Design a synchronous sequential circuit for the vending machine using S-R flip-flops. Provide the circuit diagram as well. **(25 marks)** [CSE 205 | L-2/T-1 | 2024–25]

Q2. Design a simplified traffic-light controller that switches traffic lights on a crossing where a north-south (NS) street intersects an east-west (EW) street. The input to the controller is the WALK button pushed by pedestrians who want to cross the street. The outputs are two signals NS and EW that control the traffic lights in NS and EW directions. When NS or EW are 0, the red light is on and when they are 1, the green light is on. When there are no pedestrians, NS = 0 and EW = 1 for 1 minute, followed by NS = 1 and EW = 0 for 1 minute and so on. When a WALK button is pushed, NS and EW both come 1 for a minute when the present minute expires. After that the NS and EW signals continue alternating. For the traffic-light controller: (i) Develop a state diagram and state/output table. (ii) Find the transition and excitation tables using S-R flipflops. (iii) Draw a circuit diagram. **(23 marks)** [CSE 205 | L-2/T-1 | 2023–24]

Q3. Design a divide-by-three counter which counts the number of '1's from an input sequence of binary stream. This counter outputs one 1 for every three 1's seen as input (not necessarily in succession). After outputting a 1, it starts counting all over again. Design the counter using D flip-flops. **(12 marks)** **[CSE 205 | L-2/T-1 | 2023–24]**